Explore more resources
Altera® Design Hub

FPGA AI Suite Handbook

Updated for FPGA AI Suite: **2025.3.1**

Online Version **863373**
Send Feedback **2025.12.08**

Contents

1. FPGA AI Suite Handbook....7

2. What is the FPGA AI Suite?....8

2.1. FPGA AI Suite Components....9
2.2. The FPGA AI Suite Tool Flow....9
2.3. The FPGA AI Suite User Flow....11
2.4. The FPGA AI Suite IP....13
2.4.1. The FPGA AI Suite IP Overlay Architecture....15
2.4.2. Supported Models....25
2.4.3. Model Performance....26
2.5. The FPGA AI Suite Compiler....32
2.6. The FPGA AI Suite Design Examples....32
2.6.1. PCIe-Attach Design Example....34
2.6.2. Open FPGA Stack (OFS) for PCIe-Attach Design Examples....36
2.6.3. Hostless DDR-Free Design Example....36
2.6.4. Hostless JTAG Design Example....37
2.6.5. SoC Design Example....37
2.6.6. Design Example Components....39

3. Design Considerations Before You Begin....57

3.1. Consider an FPGA AI Suite Design Example as Starting Point....57
3.2. Determine Your Performance Bottlenecks....60
3.3. Select Your FPGA Device....60

4. Installing FPGA AI Suite....61

4.1. Using the FPGA AI Suite Docker* Image....63
4.1.1. FPGA AI Suite Docker Image Requirements....64
4.1.2. Running the FPGA AI Suite Docker Container....65
4.1.3. Containerized FPGA AI Suite Docker Image Quick-Start Tutorial....66
4.2. Installing the FPGA AI Suite Compiler and IP Generation Tools....67
4.2.1. Supported FPGA Families....68
4.2.2. FPGA AI Suite Operating System Prerequisites....68
4.2.3. Installing FPGA AI Suite....73
4.2.4. Installing OpenVINO Toolkit....79
4.2.5. Installing Quartus Prime Pro Edition Software....84
4.2.6. Setting Required Environment Variables....85
4.2.7. Finalizing Your FPGA AI Suite Installation....86
4.2.8. Downloading Precompiled Bitstreams and SD Card Images....86
4.3. Installing FPGA AI Suite Design Example Prerequisites....86
4.3.1. Creating a Working Directory....87
4.3.2. PCIe Design Example Prerequisites....87
4.3.3. OFS for PCIe-Attach Design Example Prerequisites....93
4.3.4. Hostless DDR-Free Design Example Prerequisites....96
4.3.5. Hostless JTAG Design Example Prerequisites....96
4.3.6. SoC Design Example Prerequisites....99

5. FPGA AI Suite Tutorial and Design Example Demonstration Applications....130

5.1. FPGA AI Suite Quick Start Tutorial....130

5.1.1. Preparing OpenVINO Model Zoo for the PCIe Design Example....130
5.1.2. Preparing a Model for the PCIe Design Example....130
5.1.3. Running the Graph Compiler....132
5.1.4. Preparing an Image Set....132
5.1.5. Programming the FPGA Device....134
5.1.6. Performing Inference on the PCIe Design Example....135
5.1.7. Building an FPGA Bitstream for the PCIe Design Example....137
5.1.8. Building the Example FPGA Bitstreams....138
5.1.9. Preparing a ResNet50 v1 Model....138
5.1.10. Performing Inference on the Inflated 3D (I3D) Graph....138
5.1.11. Performing Inference on YOLOv3 and Calculating Accuracy Metrics....140
5.1.12. Performing Inference Without an FPGA Board....142
5.2. FPGA AI Suite Design Example Demonstration Applications....143
5.2.1. Running the PCIe-Attach Design Example Demonstration Applications....143
5.2.2. Running the Hostless DDR-Free Design Example....156
5.2.3. Running the JTAG Design Example Demonstration Application....159
5.2.4. Running the SoC Design Example Demonstration Applications....163

6. Creating an Architecture File for the FPGA AI Suite IP....167
6.1. Predefined FPGA AI Suite Architecture Files....167
6.1.1. Architecture Description File Format for Instance Parameterization....168
6.2. FPGA AI Suite Architecture File Breakdown....169
6.2.1. FPGA AI Suite IP Supported Layers and Hyperparameter Ranges....171
6.2.2. Architecture Description File Parameters....173
6.3. FPGA AI Suite IP Parameterization....189

7. Compiling Your Model with the FPGA AI Suite Compiler....193
7.1. Compiling a Graph....193
7.2. Estimating Graph Performance....195
7.3. Estimating the Area and Power of an Architecture....196
7.4. The FPGA AI Suite Compiler Report....197
7.4.1. Partitioning Table Report....198
7.4.2. Model Analyzer Reports....198
7.4.3. Visualizing How FPGA AI Suite Implements Your Graph....200
7.5. FPGA AI Suite Compiler Command Line Options....203
7.5.1. Inputs (`dla_compiler` Command Options)....204
7.5.2. Outputs (`dla_compiler` Command Options)....205
7.5.3. Reporting (`dla_compiler` Command Options)....206
7.5.4. Compilation Options (`dla_compiler` Command Options)....208
7.5.5. Architecture Options (`dla_compiler` Command Options)....210
7.5.6. Architecture Optimizer Options (`dla_compiler` Command Options)....211
7.5.7. Analyzer Tool Options (`dla_compiler` Command Options)....213
7.5.8. Miscellaneous Options (`dla_compiler` Command Options)....215
7.6. Compiler Inputs and Outputs....215
7.6.1. FPGA AI Suite Input File Formats....216
7.6.2. FPGA AI Suite Compiler Graph Export Formats....216

8. Generating the FPGA AI Suite IP for Integration into an FPGA Design....218
8.1. IP Generation Utility Execution Flows....218
8.2. IP Generation Utility Inputs....220

8.3. IP Generation Utility Outputs..... 220
8.4. IP Generation Utility Command Line Options..... 221
8.4.1. The `--flow create_ip` Flow..... 223
8.4.2. The `--flow add_arch` Flow..... 224
8.4.3. The `--flow list` Flow..... 224
8.4.4. The `--flow remove_arch` Flow..... 225

9. Optimizing Your FPGA AI Suite IP..... 226
9.1. Folding Input..... 226
9.2. Parallelizing Inference Using FPGA AI Suite with Multiple Lanes and Multiple Instances 227
9.3. Transforming the Layout of Input Data..... 230
9.3.1. Full Input Layout Transform..... 230
9.3.2. Lightweight Layout Transform..... 235
9.4. Make Precision vs. Performance Trade-offs for Your FPGA AI Suite IP..... 240
9.4.1. Block Floating Point (BFP)..... 240
9.4.2. Improving Layer Accuracy by using Mixed Precision..... 241
9.4.3. Using the Mixed Precision Feature..... 243
9.5. FPGA AI Suite IP Supported Layers and Hyperparameter Ranges..... 248
9.6. FPGA AI Suite IP Parameterization..... 250
9.7. Generating an Optimized Architecture..... 253
9.7.1. Generating an Architecture for Highest Performance..... 253
9.7.2. Generating an Architecture Optimized for a Frame Rate Target Value..... 255

10. Integrating FPGA AI Suite IP into an FPGA Design..... 256
10.1. FPGA AI Suite IP Directory Structure..... 256
10.2. Interfacing FPGA AI Suite IP an FPGA Design for a Typical System..... 256
10.3. FPGA AI Suite IP Interface..... 257
10.3.1. Clock and Reset..... 258
10.3.2. AXI Interfaces..... 258
10.3.3. AXI Streaming Interface..... 259
10.3.4. CSR Map and Descriptor Queue..... 264
10.3.5. Interfacing the FPGA AI Suite IP to Avalon Memory Map (AVMM)..... 271
10.4. Instantiating the FPGA AI Suite IP in Platform Designer..... 271
10.4.1. Resource Utilization of FPGA AI Suite IP..... 273

11. Using FPGA AI Suite as a PCIe-Attach Platform..... 275
11.1. PCIe-Attach Design Example System Architecture..... 276
11.1.1. PCIe-Attach System Overview..... 276
11.1.2. PCIe-Attach Design Example Hardware..... 276
11.2. OFS PCIe-Attach Design Example Components..... 280
11.2.1. OFS PCIe-Attach Hardware Components..... 280
11.2.2. OFS PCIe-Attach Software Components..... 282

12. Using FPGA AI Suite in Hostless DDR-Free Mode..... 283
12.1. Generating Artifacts for Hostless DDR-Free Operation..... 283
12.2. Hostless DDR-Free Design Example System Architecture..... 284
12.2.1. Hostless DDR-Free Design Example System Overview..... 284
12.2.2. Hostless DDR-Free Design Example Hardware..... 285
12.3. Hostless DDR-Free Design Example Quartus Prime System Console..... 287
12.3.1. Hostless DDR-Free Design Example Quartus Prime System Console Script Options..... 288

12.3.2. Hostless DDR-Free Design Example Inference Functionality....288
12.3.3. Hostless DDR-Free Design Example System Reset....289
12.3.4. Hostless DDR-Free Design Example Input Data Conversion....289
12.3.5. Measuring Performance in the Hostless DDR-Free Design Example....290
12.4. Hostless DDR-Free Design Example JTAG to Avalon MM Host Register Map....291
12.5. Changing the ML Graph in a Hostless DDR-Free Architecture....291
12.5.1. Updating Hostless DDR-Free MIF Files Through the CSR....292

13. Using the FPGA AI Suite in Hostless JTAG Mode....294
13.1. Hostless JTAG Design Example Components....294
13.1.1. Hostless JTAG Hardware Components....294
13.1.2. Hostless JTAG Software Components....295

14. Using FPGA AI Suite as an Embedded Platform....297
14.1. FPGA AI Suite SoC Design Example Inference Sequence Overview....297
14.2. Memory-to-Memory (M2M) Variant Design....299
14.2.1. The mSGDMA FPGA IP....299
14.2.2. RAM considerations....300
14.3. Streaming-to-Memory (S2M) Variant Design....300
14.3.1. Streaming Enablement for FPGA AI Suite....301
14.3.2. Nios V Subsystem....302
14.3.3. Streaming System Operation....302
14.3.4. Resolving Input Rate Mismatches Between the FPGA AI Suite IP and the Streaming Input....303
14.3.5. The Layout Transform IP as an Application-Specific Block in the SoC Design Example....303
14.4. Top Level....307
14.4.1. Clock Domains....308
14.5. The SoC Design Example Platform Designer System....308
14.5.1. The **dla_0** Platform Designer Layer (`dla.qsys`)....309
14.5.2. The **hps_0** Platform Designer Layer (`hps.qys`)....309
14.6. Fabric EMIF Design Component....310
14.7. PLL Configuration....310
14.8. Yocto Build and Runtime Linux Environment....310
14.8.1. Yocto Recipe: `recipes-core/images/coredla-image.bb`....311
14.8.2. Yocto Recipe: `recipes-bsp/u-boot/u-boot-socfpga_%.bbappend`....311
14.8.3. Yocto Recipe: `recipes-drivers/msgdma-userio/msgdma-userio.bb`.311
14.8.4. Yocto Recipe: `recipes-drivers/uio-devices/uio-devices.bb`....311
14.8.5. Yocto Recipe: `recipes-kernel/linux/linux-socfpga-lts_%.bbappend`....312
14.8.6. Yocto Recipe: `recipes-support/devmem2/devmem2_2.0.bb`....312
14.8.7. Yocto Recipe: `wic`....312
14.9. SoC Design Example MMD Layer Hardware Interaction Library....313
14.9.1. MMD Layer Hardware Interaction Library Class `mmd_device`....313
14.9.2. MMD Layer Hardware Interaction Library Class `uio_device`....314
14.9.3. MMD Layer Hardware Interaction Library Class `dma_device`....314
14.10. FPGA AI Suite SoC Design Example Run Process....314
14.10.1. Exporting Trained Graphs from Source Frameworks....314
14.10.2. Compiling Exported Graphs Through the FPGA AI Suite....314
14.11. FPGA AI Suite SoC Design Example Build Process....315

14.11.1. Building the Quartus Prime Project....316
14.11.2. Building the Bootable SD Card Image (.wic)....319

15. Using FPGA AI Suite in Video Applications....323
15.1. Nios Subsystem....324
15.1.1. Stream Controller Communication Protocol....324
15.1.2. Buffer Flow in Streaming Mode using Nios V Software Scheduler....325
15.2. Building the Stream Controller Module....328
15.3. Building the Streaming Demonstration Applications....328
15.4. Running the Streaming Demonstration....329
15.4.1. The streaming_inference_app Application....329
15.4.2. The image_streaming_app Application....330

16. Using the FPGA AI Suite IP with High Bandwidth Memory on Stratix 10 MX and Agilex 7 M-Series Devices....333
16.1. Current FPGA AI Suite IP Implementation With DDR....333
16.2. HBM Connection Considerations....334
16.2.1. Memory Size and Bandwidth Considerations....334
16.2.2. Memory Data Width Considerations....336
16.2.3. Conclusion....337

17. Developing Software Applications with the FPGA AI Suite....339
17.1. Understanding the FPGA AI Suite Runtime Software Stack....339
17.2. Using the FPGA AI Suite Software Emulation....340
17.3. Running Inference on the FPGA AI Suite IP Without the OpenVINO Runtime....342
17.3.1. Files Generated by the FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility....342
17.3.2. Building the FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility....343
17.3.3. Running the FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility....343
17.3.4. FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility Example Application....345

A. FPGA AI Suite Handbook Archives....346

B. FPGA AI Suite Handbook Document Revision History....347
B.1. FPGA AI Suite Getting Started Guide Document Revision History....347
B.2. FPGA AI Suite Compiler Reference Manual Document Revision History....349
B.3. FPGA AI Suite IP Reference Manual Document Revision History....351
B.4. FPGA AI Suite Example Designs User Guide Revision History....354
B.4.1. FPGA AI Suite PCIe-based Design Example User Guide Document Revision History....355
B.4.2. FPGA AI Suite SoC Design Example User Guide Document Revision History....356
B.5. AN 1008: Using the FPGA AI Suite Docker Image Document Revision History....357
B.6. AN 1020: Using the FPGA AI Suite IP with High Bandwidth Memory on Stratix 10 MX and Agilex 7 M-Series Devices Document Revision History....358

1. FPGA AI Suite Handbook

The FPGA AI Suite is a suite of tools and design examples to help you get AI inference from your pretrained models running on Altera® FPGA devices, either in concert with a CPU or in a hostless configuration.

The *FPGA AI Suite Handbook* provides the following information:

- An overview of the FPGA AI Suite and its components such as the FPGA AI Suite IP and the FPGA AI Suite compiler.
- Installation instructions and a list of prerequisites
- Tutorials and walkthroughs that take you through the process of running inference, including performance and area estimation
- How to customize your FPGA AI Suite IP by changing parameters settings.
- A description of the FPGA AI Suite Compiler and its use modes.
- Details about the compiler command options and the format of compile inputs and outputs.
- How to generate RTL and a Quartus® Prime Platform Designer IP index (`.ipx`) file with the FPGA AI Suite IP generation utility so that you can integrate your FPGA AI Suite IP with a larger FPGA design.
- A description of the design and implementation of design examples that you can use to accelerate AI inference using the FPGA AI Suite, Intel® Distribution of OpenVINO™ toolkit, and various development boards (depending on the design example).

Intel Distribution of OpenVINO toolkit Requirement

To use the FPGA AI Suite, you must be familiar with the Intel Distribution of OpenVINO toolkit.

FPGA AI Suite Version 2025.3.1 requires the Intel Distribution of OpenVINO toolkit Version 2024.6 LTS. For OpenVINO documentation, refer to https://docs.openvino.ai/2024/documentation.html.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

2. What is the FPGA AI Suite?

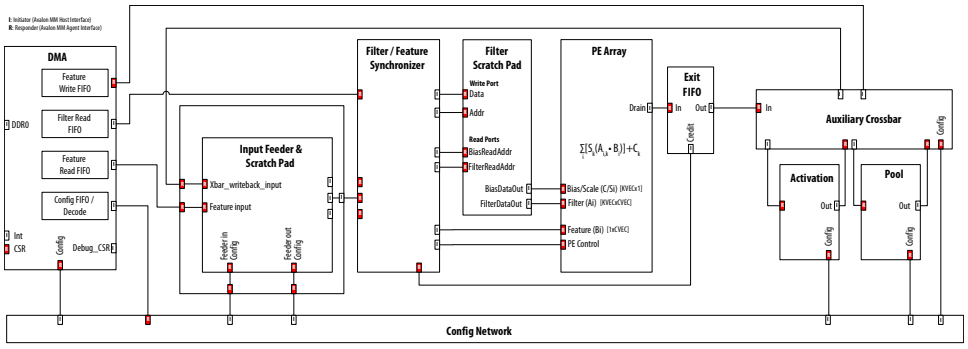
The FPGA AI Suite comprises a set of tools that take a pretrained AI model and some user-provided configuration information and generates the following items:

- Hardware description language (HDL) source that targets a specific FPGA device
- C/C++ emulation code
- Inference runtime for host control path.

The user-provided configuration information takes the form of parameters that configure the FPGA AI Suite IP, which is an IP with an FPGA overlay architecture. You can use the IP parameters to configure the overlay to your ML model as well optimizing the performance for your needs. The overlay architecture is a collection of optimized RTL blocks that forms a complete inference pipeline: tensor processing units, memory controllers, data movers, and interconnect logic—all working together as a cohesive system.

An architecture generator takes your parameters and the FPGA overlay to generate custom RTL specifically optimized for the target ML model and FPGA device.

Figure 1. Example Block Diagram of Deployed FPGA AI Suite IP



The IP has two important parts: the inference engine and the stream buffer unit. The inference engine is a highly parallel processor array that handles convolutions, pooling, activations, and all other neural network operations at hardware speed. The stream buffer unit manages data flow between external memory and the processing elements to keep the FPGA datapath as full as possible without bottlenecks.

You can create your own IP configuration files (often called *architecture files* or *.arch files*) or you can use one of the provided example preconfigured architecture files that target a combination of ML model and FPGA device family.

The FPGA AI Suite also includes evaluation and prototyping platforms (the FPGA AI Suite *design examples*) that you can use as a starting point based on your deployment methodology.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

2.1. FPGA AI Suite Components

The FPGA AI Suite consists of several components to enable AI inference on Altera FPGAs.

The FPGA AI Suite consists of the following components:

- **IP**

  For details, refer to The FPGA AI Suite IP on page 13.

  For a list of models supported by the FPGA AI Suite IP, refer to Supported Models on page 25.
- **Compiler** (`dla_compiler` command)

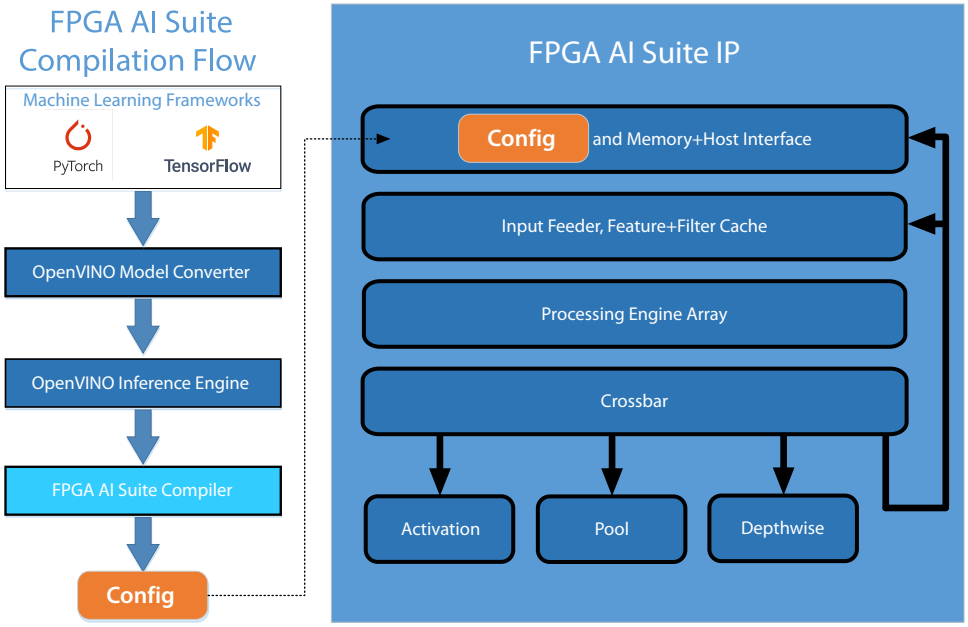
  For details, refer to The FPGA AI Suite Compiler on page 32.
- **IP generation tool**

  For details, refer to Generating the FPGA AI Suite IP for Integration into an FPGA Design on page 218.
- **Design examples**

  For details, refer to The FPGA AI Suite Design Examples on page 32

Figure 2. Connection Between FPGA AI Suite Compiler and FPGA AI Suite IP

The following diagram illustrates the connection between two primary components of the FPGA AI Suite – the compiler and the IP:



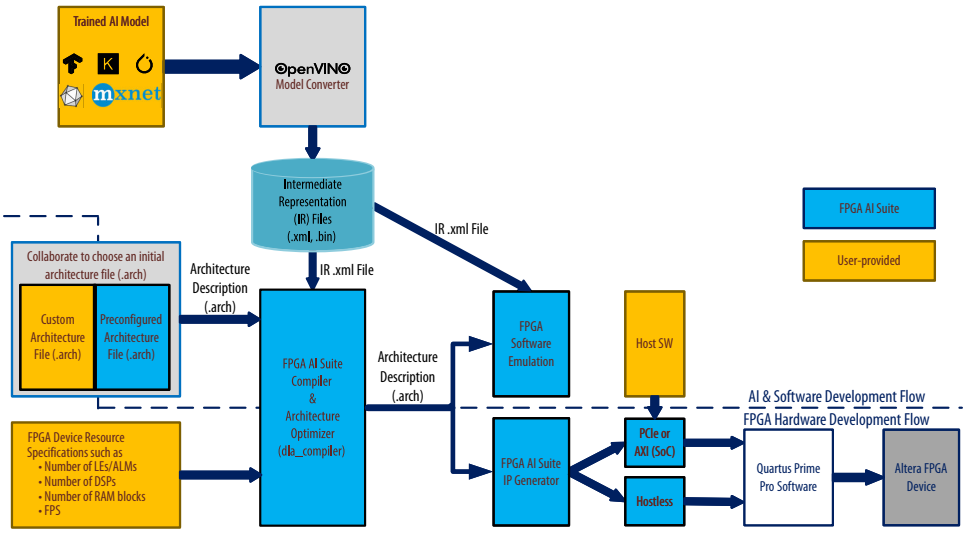
2.2. The FPGA AI Suite Tool Flow

The FPGA AI Suite takes a trained ML model and a user-defined architecture file and then analyzes the ML model structure, maps operations to FPGA hardware resources, quantizes weights and activations for optimal precision, and then generates an IP that

you can integrate into an FPGA system. The FPGA AI Suite also supports a C/C++ interface (via OpenVINO) to bit-accurate emulation. You can use this emulation for pre-hardware development of the runtime stack and pre-hardware validation of the ML model accuracy.

The following diagram illustrates the FPGA AI Suite tool flow:

Figure 3. FPGA AI Suite Tool Flow



Moving from ML Model to IR

The FPGA AI Suite tool flow starts with your trained ML models from TensorFlow, PyTorch, Keras, MXNet, or ONNX. These models are fed to the OpenVINO Model Converter to generate an intermediate representation (IR) of your model. The IR consists of a `.xml` file that describes the model topology and a `.bin` file that contains the model weights. These IR files are the main input into the FPGA AI Suite tool flow.

Collaborate Early to Choose an Initial Architecture

Your AI developers and FPGA hardware engineers collaborate early to determine a starting architecture description (`.arch`) file. They can choose one of the predefined architecture description files provided with the FPGA AI Suite, or they can take a more complex route and create a custom architecture file.

Enabling Parallel Software and Hardware Development Flows

After you have the IR for your ML model and an initial architecture file, the FPGA AI Suite compiler (`dla_compiler`) Architecture Optimizer takes the IR data and enables two parallel optimization paths:

- AI & Software Development Flow

  The Architecture Optimizer output feeds into FPGA software emulation. This emulation enables functional validation and performance profiling before any hardware synthesis, which is critical for rapid iteration. The software flow targets Linux* hosts—either through PCIe interface, JTAG-Avalon® interface, or an AXI interface (for an HPS host on an SoC FPGA device). Different deployment targets have different runtime stacks.

- FPGA Hardware Development Flow

  Use the Architecture Optimizer to update the architecture file with FPGA resource specifications. The optimizer consumes FPGA resource specifications (ALMs, DSPs, RAM blocks, FPS targets) and generates an optimized `.arch` file. The FPGA AI Suite IP Generator consumes the `.arch` file and produces a customized IP core. Example designs across a variety of platforms (PCIe-attach, SoC, and hostless) integrate the custom IP to perform machine learning inference.

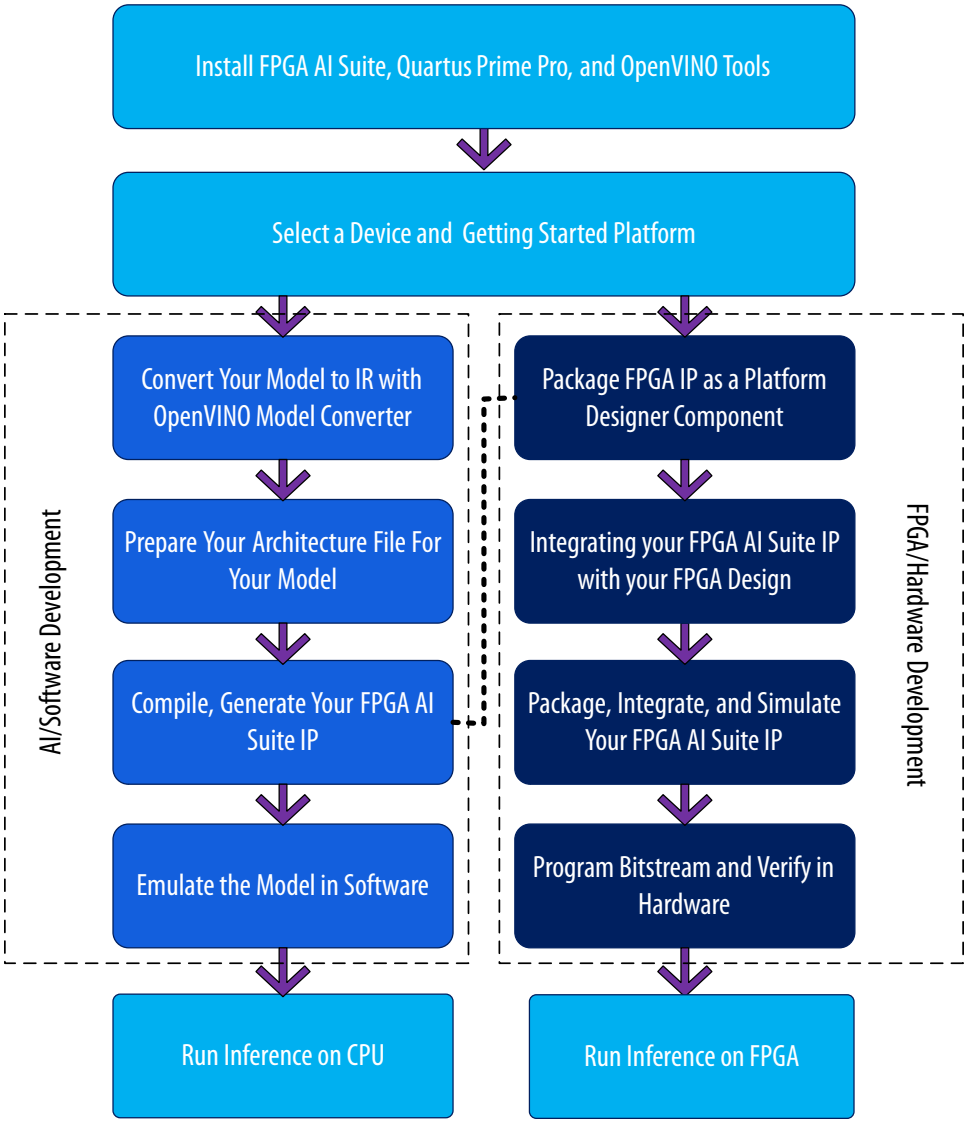
Final Hardware Implementation

- Both development paths converge at Quartus Prime Pro Edition. The software combines the validated architecture with the generated IP, performing synthesis, place-and-route, and timing closure to produce the final FPGA bitstream.

2.3. The FPGA AI Suite User Flow

While the tool flow showed the technical pipeline, this user flow shows how AI/ Software developers and FPGA engineers collaborate to transform ML models into deployed accelerators.

Figure 4. FPGA AI Suite User Flow



Both groups start together with the same software stack: FPGA AI Suite, Quartus Prime Pro Edition, and OpenVINO toolkit. The two groups also decide on the platform they are developing: PCIe*-attached for data center, embedded HPS for edge, 4K video processing, or DDR-free for ultra-low latency. This shared decision anchors the entire deployment strategy.

After that, the AI/software development work and the FPGA hardware development work can move and iterate their parts of the application roughly in parallel:

- AI/software development work

  This work involves optimizing neural network graphs with OpenVINO Model Converter, defining architecture files to enhance hardware performance, and compiling the design with the FPGA AI Suite compiler into custom inference IP. The design can then be iteratively validated through software emulation to ensure a robust architecture before proceeding to FPGA hardware development.

- FPGA hardware development work

  Starting with the first iteration of the custom inference IP, the FPGA engineers work to integrate the IP into the overall system design. This process includes connecting the IP to existing system components through Visual Designer Studio or Platform Designer, performing RTL simulations, and running Quartus compilation. The FPGA Hardware Engineers must ensure timing closure, optimize resource utilization, and successfully generate the final bitstream.

Architecture definition requires both ML insights and hardware constraints. IP parameterization needs performance targets from AI engineers and resource budgets from FPGA engineers. And that final inference validation depends on both teams to verify functionality—software engineers on CPU, FPGA engineers on silicon.

You can use software emulation results to determine if further architecture optimization is needed. Your hardware resource usage results can inform model quantization decisions. Use performance benchmarks to drive parameter tuning. With each iteration of the parallel development cycles, the ML model- FPGA hardware fit tightens.

The parallel-track helps eliminate development bottlenecks. AI engineers do not need to wait for synthesis to validate models, and FPGA engineers do not need ML expertise to integrate IP. Yet collaboration points ensure optimal co-design. It's structured flexibility that enables rapid deployment while maintaining development rigor.

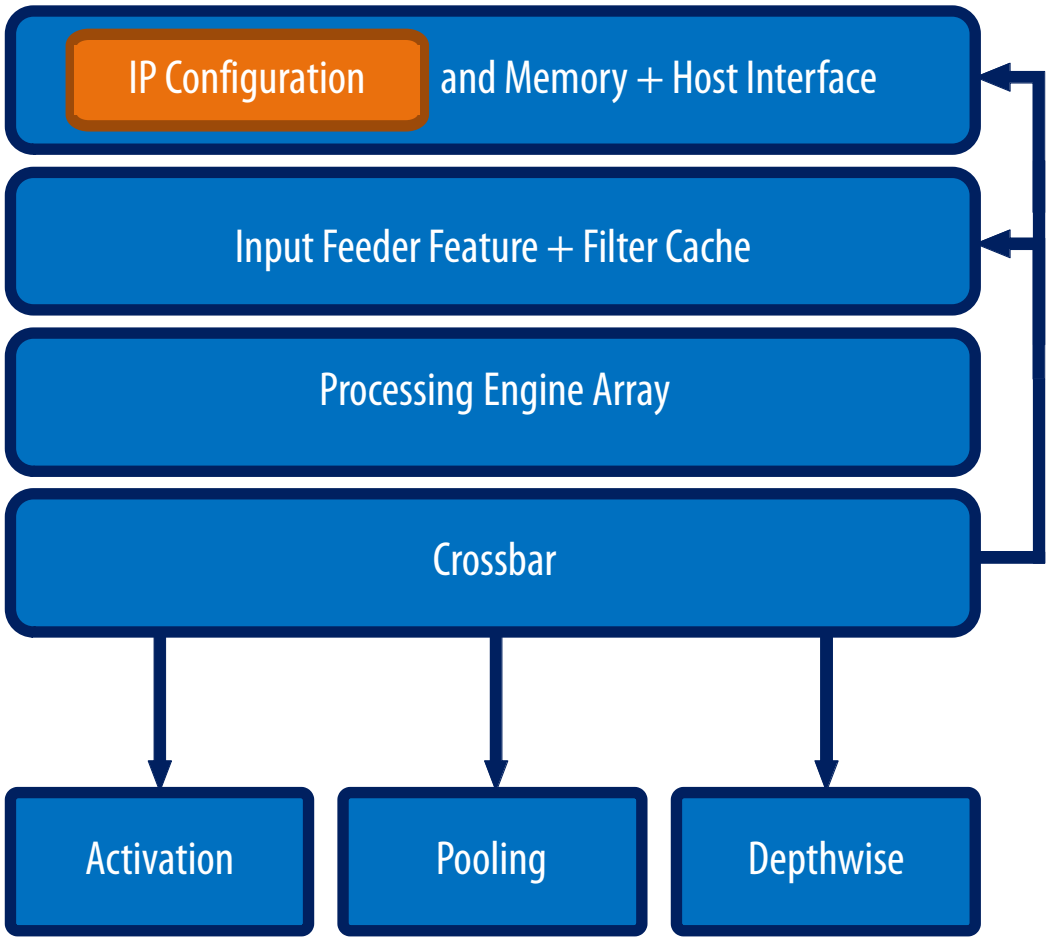
2.4. The FPGA AI Suite IP

The FPGA AI Suite IP is an RTL-instantiable configurable IP with AXI interfaces that you can instantiate into a generic embedded FPGA system.

The IP is configured through parameters defined in an *architecture description file* (`.arch`), often called an *architecture file*. The architecture description file, along with the OpenVINO intermediate representation of your trained model, is compiled by the FPGA AI Suite compiler into configuration instructions for the IP.

Figure 5. High-Level Architecture of the FPGA AI Suite IP

The following diagram shows a high-level architecture of the FPGA AI Suite IP:

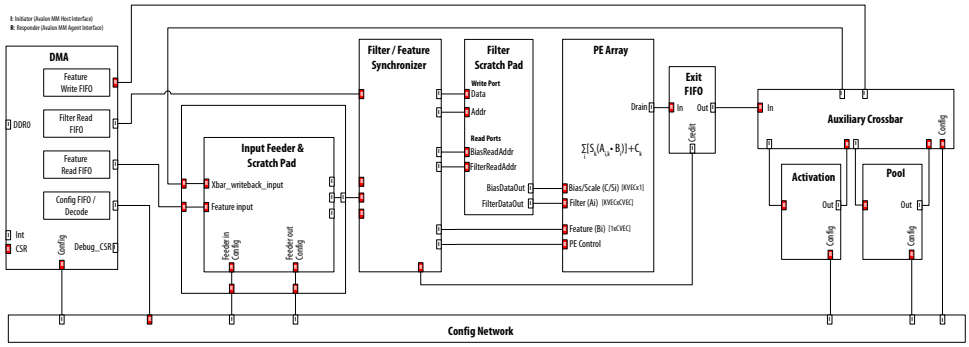


The primary parameters defined in an Architecture Description File cover the following properties:

- PE array vectorization
- Scratch pad sizing
- External memory bus bandwidth
- Types/vectorization of auxiliary layer blocks

The following diagram is an architecture diagram for a specific instantiation of the FPGA AI Suite IP. The blocks connected to the crossbar in this diagram are examples. The selection of blocks connected to the crossbar are determined by compile time parameters.

Figure 6. Architecture of an Example Instantiation of the FPGA AI Suite IP



Two teams are typically involved in the implementation of an AI feature:

- An AI/software development team responsible for developing and delivering an AI model.
- An FPGA hardware development team responsible for integrating the FPGA AI Suite IP and runtime together into a system.

Defining the IP architecture straddles the boundary between these two teams. The ML team must develop an AI model that meets the target performance in some parameterization of the configurable IP. The FPGA team must ensure it fits onto the FPGA and closes timing.

Although responsibility for defining the parameterization of the configurable architecture can lie with either team (but is a joint responsibility), it can often be easier for the ML team to define the architecture.

The team responsible for defining the IP parameterization can use the FPGA AI Suite compiler (`dla_compiler` command) area and performance estimator tools to guide their decisions. For more information about using the compiler, refer to Compiling Your Model with the FPGA AI Suite Compiler on page 193.

In addition to the FPGA team and the ML team, another team is likely responsible for the software integration on the host. Depending on the system details, this software is likely responsible for interfacing with OpenVINO and communicating (via the BSP) with the FPGA AI Suite IP. This software is likely based on the runtime system that is included with the FPGA AI Suite design examples.

2.4.1. The FPGA AI Suite IP Overlay Architecture

Machine learning (ML) tasks differ significantly in their throughput and resource utilization requirements, and the characteristics of ML graphs vary accordingly. As a result, no universally optimal FPGA AI Suite IP configuration exists that can achieve optimal performance in every scenario. Your model is unique to your application, and you need to specify for the FPGA AI Suite compiler an architecture description file to generate the overlay IP.

The FPGA AI Suite IP functions as an FPGA overlay, analogous to a soft processor, where modules can be added or removed to meet specific design requirements. Its configurability facilitates various trade-offs between inference performance (throughput and latency) and FPGA resource utilization (area). Configuration options are defined within Architecture Description Files (Arch files).

An Arch file is a text-based protobuf file with the .arch extension, describing the parameters of the FPGA AI Suite IP. To create an architecture file after understanding the following concepts of it, refer to Creating an Architecture File for the FPGA AI Suite IP on page 167.

Conceptually, FPGA AI Suite Overlay IP has two sets of components: control logic and datapath components.

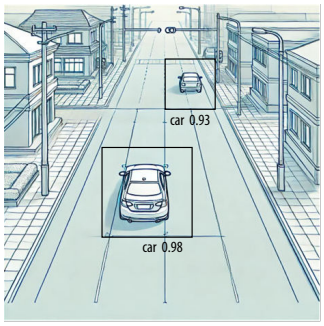
- The datapath components do heavy lifting—they accept and buffer incoming data, perform computations, and output inference results. The datapath components expose a series of parameters for customization.
  - ☐ The datapath components are shown as blocks in Architecture of an Example Instantiation of the FPGA AI Suite IP in The FPGA AI Suite IP on page 13.
  - ☐ The datapath components are the major optimization and customization venue.
- The control logic orchestrates the data movement, storage, and computation in the datapath. The control logic in the overlay architecture receives instruction from the Config Network.
  - ☐ The control logic provides little configurability for customization.

The datapath components of the overlay architecture can be precisely customized using Architecture Description Files (.arch). This file allows adjustments of critical parameters that affect the organization and data movement and storage.

For instance, the most important concept is the processing element (PE) parallelism, as it controls how many computational operations can be executed simultaneously, directly influencing inference speed. Since other components are built around the PE, they frequently draw reference to the concepts introduced in the PE. Hence, a comprehensive understanding of the concepts in PE is essential to obtain a successful design using the FPGA AI Suite.

2.4.1.1. How Does the FPGA AI Suite Overlay Architecture Work?

In a typical deep learning application such as object detection on urban streets, the process begins with selecting a pre-trained deep learning model designed for real-time inference. These models are structured as computational models: a directed network of operations such as convolution, matrix multiplication, normalization, and activation functions. At an abstract level, the model functions as a mapping from input space to output space, denoted as a function F(x). In this context, the input x represents image or video data, and the output F(x) consists of bounding boxes indicating object locations along with associated confidence scores.



$car = ObjectDetection(image)$ — An end-to-end ML Task

$output = F(input)$ — A generalization of ML Tasks

$y = F(x)$ — An abstraction

$F(x)$ — We call this function the neuron network

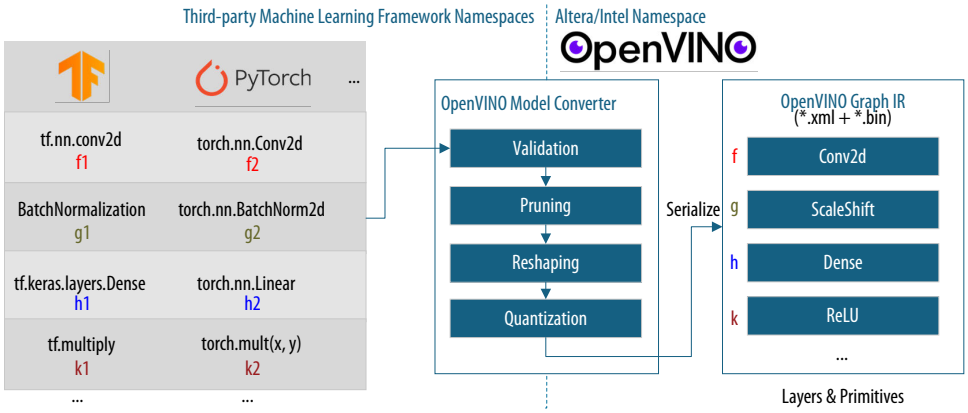
$y = (f \circ g \circ h \circ \ldots \circ k)(x)$ — It is also a composition of many other functions

Deep learning models used for tasks such as object detection are typically composed of multiple layers, each representing a mathematical function with associated parameters. Trainable parameters—such as weights and biases—are adjusted during the training phase to improve model accuracy, while hyperparameters—such as convolutional filter size—remain fixed. These layers commonly include convolutional operations for feature extraction, spatial reduction mechanisms like pooling or striding, normalization layers for training stability, activation functions to introduce non-linearity, and output layers for generating predictions such as bounding boxes and class probabilities. When structured effectively, these components enable the model to perform complex, high-level tasks in a single inference pass.

Models obtained from public repositories are generally trained using modern machine learning frameworks. While implementations vary across frameworks, the underlying mathematical principles remain consistent. To facilitate deployment across heterogeneous hardware platforms—including CPUs, GPUs, and FPGAs—the OpenVINO toolkit converts these models into a unified Intermediate Representation (IR) using its Model Converter, enabling efficient and portable execution.

The IR comprises two critical files:

- an XML file detailing the model's network topology, with OpenVINO layer operations and parameters represented as nodes (including their hyperparameters), and data flows represented as edges.
- a BIN file containing the trained model's weights and biases.



The OpenVINO Intermediate Representation (IR) provides a standardized and optimized model format for efficient execution across heterogeneous hardware platforms, including CPUs and FPGAs. Within the FPGA AI Suite toolchain, the IR serves as the input to the front-end compiler, replacing the need to directly handle models from various third-party frameworks. Supported layers and hyperparameter ranges for the FPGA AI Suite Overlay IP are documented in the corresponding reference tables.

The FPGA AI Suite Compiler processes IR files to generate a Runtime Configuration Binary. This binary encodes the control logic required by the Overlay IP to manage data movement, coordinate execution, and perform inference efficiently.

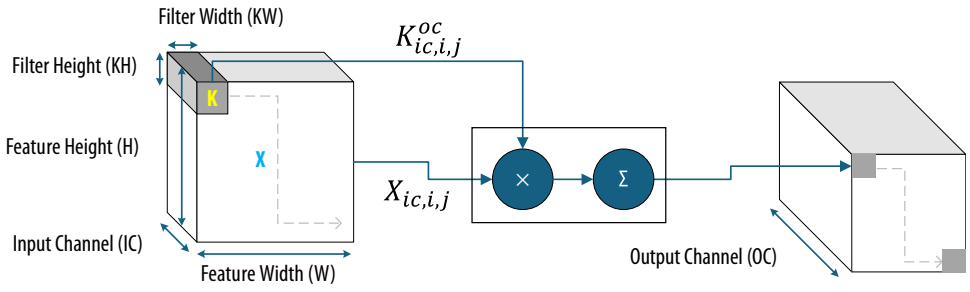
Due to the variability in model architectures and layer complexity, a single instantiation of the Overlay IP may not be optimal for all models. Customization is enabled through an Architecture Description File (.ARCH), a human-readable configuration file that defines architectural parameters. This allows the overlay to be tailored for specific models or shared across multiple models without re-running Quartus synthesis.

The Overlay IP supports multiple system topologies, including configurations with high-performance host CPUs, embedded processors, or standalone operation. It also accommodates various data access patterns, such as direct streaming or external DDR memory access. For example, in edge vision applications, video frames can be streamed directly into the overlay, with inference results output in real time and intermediate data retained on-chip to minimize memory bandwidth usage.

2.4.1.2. Parallelism in the FPGA AI Suite IP

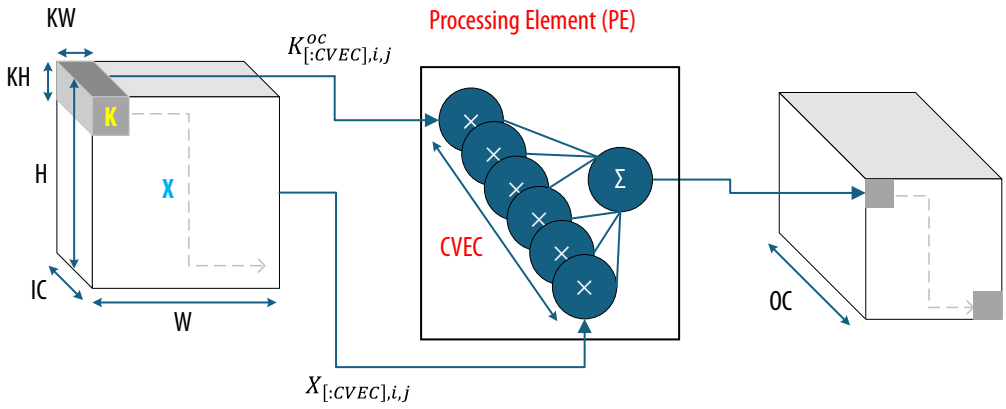
This section outlines a step-by-step optimization process designed to convey a high-level understanding of computation parallelization within the FPGA AI Suite IP. The discussion includes the parameterization of the IP within the Architecture Description File (.arch file) and the corresponding implications on computation.

Considering a convolutional layer within an ML graph performing the operation, the input to this convolution is referred to as the input feature, the convolution kernel as a filter, and the output from the convolution as the output feature. Initially, in a naïve convolution approach, as the filter traverses the input feature, a single product computation is performed at each position and accumulated sequentially.



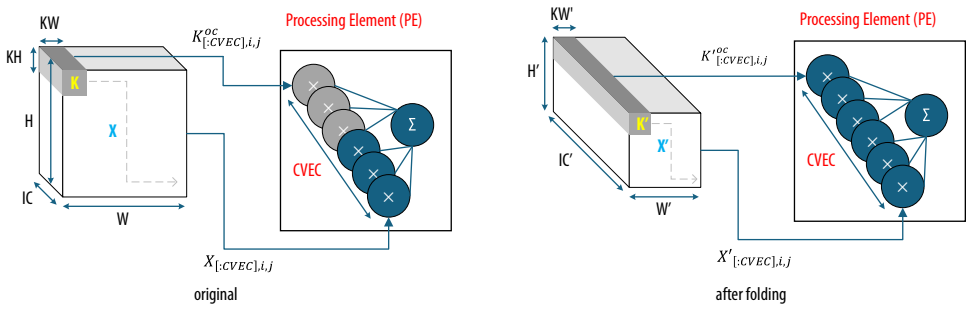
Once the entire input feature has been processed for a given filter, a single result within the output feature is produced. This method, computing only one product at a time, underutilizes available FPGA hardware resources.

The first optimization step involves increasing parallel computation capacity by unrolling computations along the input channel dimension. By duplicating dot-product computations *c_vector* (sometimes referred to as *CVEC*) times, multiple products can be calculated simultaneously. Each duplicated dot-product engine is termed a processing element (PE). Exploiting the FPGA's spatial hardware architecture in this manner significantly enhances performance and trades-off the usage of more DSPs. Additional details can be found in Parameter: c_vector.



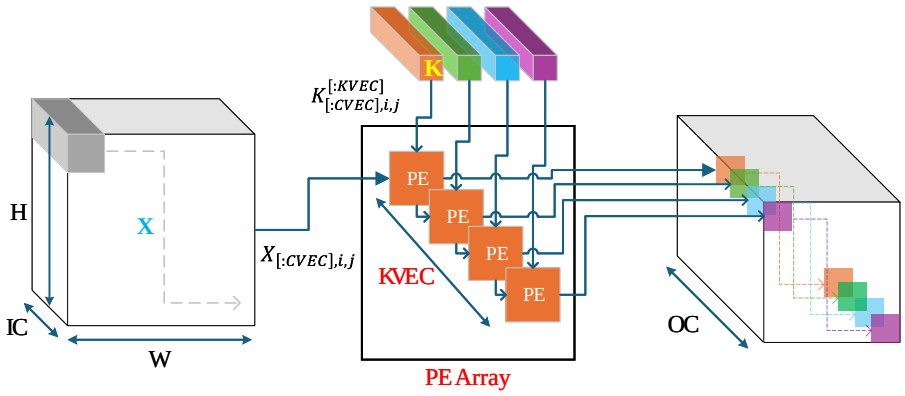
For computer vision tasks, input tensors to the ML graph are usually 8-bit pixels, or are organized in a more mathematically intuitive way matching the convolution layer, which is different in which the PE consumes them. Since the PE computes dot-product for *c_vector* number of FP16 feature elements at a time, for it to efficiently process input features, layout transform is responsible for converting the input features in memory to a desired shape and precision for the PE. Layout transform can be thought of as an optimized address lookup and datatype conversion for input features in memory so that feature elements can arrive at PE ready for dot-product. One important parallelism enabled by the layout transform is folding.

As the size of the dot-product increases across the input channel of the input feature, to fully utilize the PE, the input channel is more desirable to be close to the value of *c_vector*. For image-based tasks, typical input channels may only include three channels (Red, Green, Blue), whereas width and height dimensions are generally larger. To saturate the PE, the input feature may be reshaped so that the input channel is increased to match with or exceed the *c_vector* value. In the conceptual illustration for folding below, the original input feature on the left before folding, X, has a shallow input channel. The size of dot-product in the PE is larger than the number of input channels. Without folding, PEs may remain partially idle due to insufficient input channel data (underutilized dot-product in gray as the input is padded with zeros). By enabling folding, width and height dimensions are folded into the input channel dimension, resulting in an adjusted input to the PE with an expanded input channel dimension that eliminated as much zero-padding as possible. As a result, the folded input feature, X', has a larger input channel that gives a better utilization of the dot-products in PE.



For additional information on layout transform, refer to Parameter: enable_layout_transform and Folding Input on page 226. For a variant of the layout transform feature and comparison between the Full Layout Transform, refer to the Transforming the Layout of Input Data on page 230.

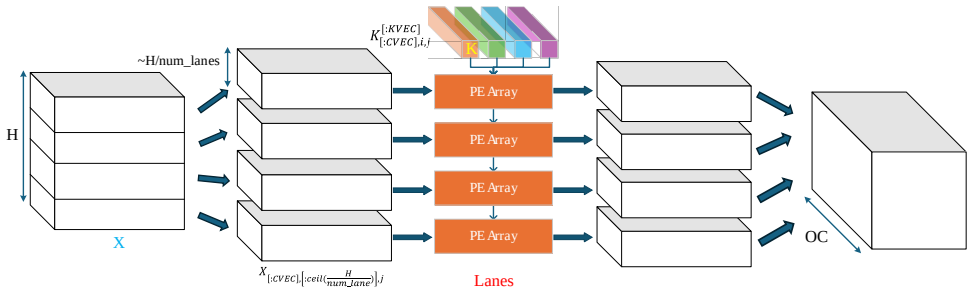
The second optimization further capitalizes on FPGA spatial architecture by unrolling computations across the output channel dimension. Rather than computing a single dot product for one output channel, the PE itself is replicated, with each PE calculating the dot product for a distinct output channel. The input feature sequentially passes through each PE, with dot products computed using the respective filters specific to each output channel. Collectively, these PEs form a PE Array. This parallelism introduces a new parameter, *k_vector*, which is the total number of parallel PEs in the PE Array. For more details about this parameter, refer to Parameter: k_vector.



Given that input features typically have substantial width and height dimensions, a third optimization involves parallelizing computations along the height dimension (that is, dimensions represented as BxCx**H**xW). Architectures employing parallelism along the height dimension are referred to as multilane architectures.

Multilane architectures partition input features into segments along the height dimension, enabling parallel convolution computation within multiple lanes, significantly increasing throughput. Each lane comprises duplicated auxiliary modules (aux_modules) and *k_vector* number of PEs, computing convolution on height segments in parallel while sharing filter data among lanes. The segmented input features and filters traverse the 2D PE arrays in a systolic manner, and the computed

dot products are aggregated to produce the output feature. The *num_lanes* parameter governs the quantity of lanes instantiated, theoretically enhancing throughput proportionally.



Although multilane architectures can substantially boost performance, careful consideration is necessary due to increased FPGA resource utilization and memory bandwidth demands. Specifically, DSP resource usage scales directly with *num_lanes*, and logic gate requirements grow correspondingly. Additionally, DDR bandwidth consumption rises with multilane architecture since more input features are being consumed by the PE arrays in parallel. Also, ML graphs with limited height dimensions or few convolutional layers might not gain performance benefits and could experience unnecessary resource overhead.

Multilane-enabled architectures within the architecture directory are identifiable by filenames ending with `_Multilane` (for example, `AGX7_Performance_Multilane.arch`). For comprehensive details on multilane configurations, see Parameter: num_lanes.

The material in this publication will help you to effectively balance performance against resource utilization, experiment with different lane configurations, and utilize the model analyzer tool.

Related Information

- Parameter Group: Global Parameters on page 173
- Optimizing Your FPGA AI Suite IP on page 226
- Model Analyzer Reports on page 198

2.4.1.2.1. Comparing ML Architectures using the Compiler Report

The parallelism of processing elements (PEs) affects throughput and resource usage. The following table shows the PE throughput as an assessment of the PE performance with DDR transfer overhead discounted.

PE throughput and resources usage increase as the level of parallelism increases.

The values in this table are based on the AGX7_Performance architecture targeting a ResNet50 graph. The values were obtained from the FPGA AI Suite compiler report.

	c_vector=4 k_vector=4	c_vector=16 k_vector=16	c_vector=16 k_vector=32	num_lanes=4
Architecture	Minimal parallelism	Unrolled (16x) dot-product	Unrolled (2x) PEs	Unrolled (4x) PE Arrays
PE Throughput	2fps	28fps	54fps	178fps
Area	ALMs: 29475 DSPs: 26 M20Ks: 296	ALMs: 32310 DSPs: 122 M20Ks: 685	ALMs: 37436 DSPs: 202 M20Ks: 786	ALMs: 122944 DSPs: 768 M20Ks: 900

Related Information

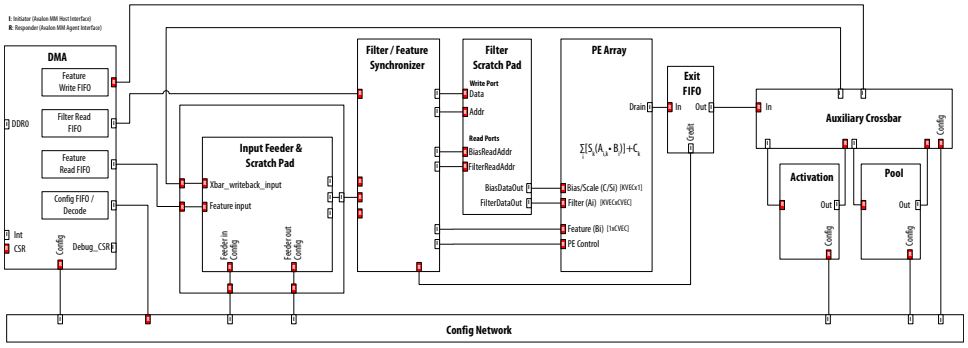
The FPGA AI Suite Compiler Report on page 197

2.4.1.3. FPGA AI Suite IP Datapath Component Organization

The FPGA AI Suite IP provides components in the datapath that can be customized via an architecture (`.arch`) file.

Figure 7. FPGA AI Suite IP Block Diagram

The following diagram shows the connections and I/O of these components:



- Processing element array

  Many of the concepts described below are related to the Processing Element. Refer to the detailed discussion in the Parallelism in the FPGA AI Suite IP on page 18 before continuing.

- Input feeder

  For details, refer to Input Feeder on page 23.

- PE array filter Interleave

  For details, refer to PE Array Filter Interleave on page 23

- Scratchpad sizing

  Scratchpad memory holds filters and supplies them to the PE array in parallel.
- Crossbar

  Crossbar acts as a cache for the output feature exits from the PE array. The size of the buffer array in the crossbar is proportional to the output size of the PE array.
- Architecture Precision

  For details, refer to Architecture Precision on page 24

To learn more about Architecture File parameters in an example architecture file,refer to Creating an Architecture File for the FPGA AI Suite IP on page 167. To explore advanced optimization techniques provided by the Architecture File, refer to Optimizing Your FPGA AI Suite IP on page 226.

Input Feeder

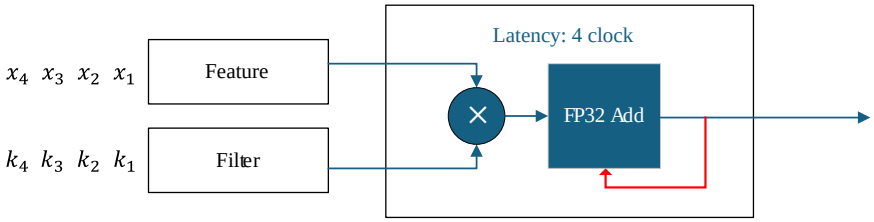
Input features that come from DMA are stored temporarily in a static cache, which is the stream buffer. Stream buffer depth controls the input feature size that can be stored on-chip. Stream buffer width is proportional to the PE array input size, c_vector and k_vector, as it needs to sufficiently supply enough input features to the PEs.

The stream buffer size can be changed by the architecture optimizer. Changing the size of the stream affects the performance and area: M20K is reversely proportional to the buffer size since they are used for the buffer; FPS lowers as the buffer size, but smaller graph is less impacted; DDR bandwidth: reduce the on-chip buffering meaning spilling to DDR more.

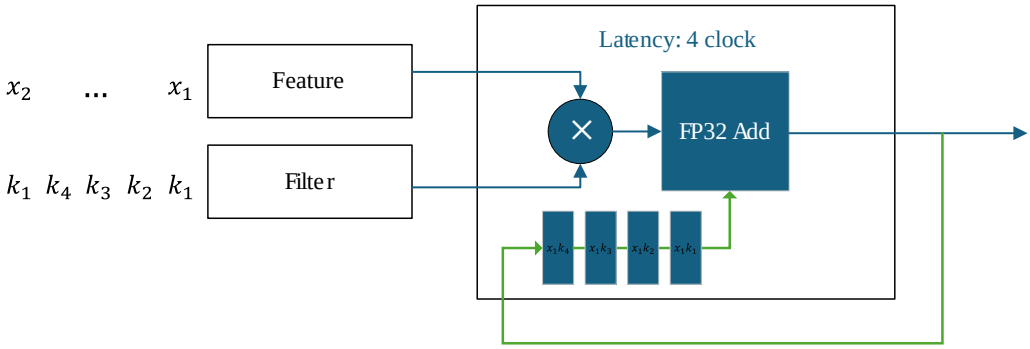
PE Array Filter Interleave

DSPs inside the PE array have a fixed latency, L, in computing multiply-and-add. This latency would lower the occupancy due to loop-carry dependency of PE if the PE had naively computed dot-product of consecutive features and filters.

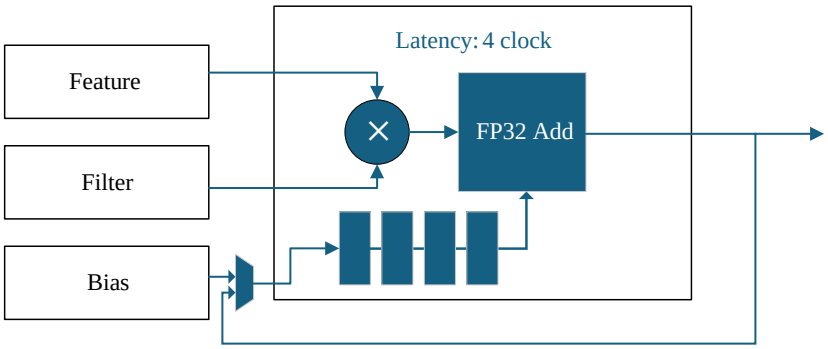
- □



In the naïve case illustrated above, without any interleaving, consecutive features and filters are multiplied and accumulated together. In this case, to compute the next partial sum, $x_t \cdot k_t$, the previous partial sum, $x_{t-1} \cdot k_{t-1}$, needs to be available immediately at the FP32 add block. This is impossible due to the latency of the hardware. Consequently, this occurs a loop-carry dependency show in the red feedback path of 4 clock cycles.



By interleaving filters (notice that different filters are multiplied with the same feature) and inserting pipeline registers of depth equal to the latency and interleaving filters, the PE computes partial sums for L different output features. In this way, the next partial sum does not depend on the result of the previous. Therefore, the penalty on the feedback path (shown in green) is eased. Because the PE is fully pipelined in this way, the occupancy of the PE is optimized, which increases throughout.



Inserting bias into the dot-product computation is enabled by this architecture of the PE. By multiplexing the input to the pipeline registers, the addition can choose between biases or the partial sums.

For more information on setting the interleaving parameters, refer to PE Array Parameters: num_interleaved_features, num_interleaved_filters.

Architecture Precision

The precision for filter weights. This parameter affects DDR traffic and on-chip buffer area usage, and the size of IP.

The `dla_compiler` tool maps each layer of your graph to the IP block that performs the computation for a layer during inference. The precision at which each block operates is as follows:

- The accumulator operates at fp32 precision.
- The accumulator bias value (if present) is fp16 precision.

- The auxiliary blocks (activations, pooling, and depthwise convolution) operate at fp16 precision.
- The feature inputs to the FPGA AI Suite IP must be provided at fp16 precision:
  - ☐ External AXI Bus Interface Parameters

    Defines the AXI bus width. Changing this parameter affects the DDR bandwidth and fine-tuning this parameter is the responsibility of the FPGA team.
  - ☐ Types/vectorization of auxiliary layer blocks

    Enables an auxiliary module, typically, an activation module to support execution of that activation function on FPGA. If an activation module is turned off, host CPU needs to execute that activation layer, descendant or parent layers may be assigned to CPU.

2.4.2. Supported Models

The FPGA AI Suite supports the following pretrained models from OpenVINO Model Zoo:

Table 1. Models Supported by FPGA AI Suite 2025.3.1

OpenVINO Model Zoo path	Model	Framework
`public/mobilenet-v1-1.0-224-tf`	MobileNet V1	TensorFlow
`public/mobilenet-v2-pytorch`	MobileNet V2	PyTorch
`public/mobilenet-v2-1.4-224`	MobileNet V2	TensorFlow
`public/mobilenet-v3-large-1.0-224-tf`	MobileNet V3	TensorFlow
`public/resnet-50-tf`	ResNet-50	TensorFlow
`intel/unet-camvid-onnx-0001`	UNet	PyTorch
`public/yolo-v3-tf`	YOLO v3	TensorFlow
`public/yolo-v3-tiny-tf`	TinyYOLO v3	TensorFlow
	Yolo v8 (all heads)	PyTorch
`public/squeezenet1.1`	SqueezeNet v1.1	Caffe
`public/i3d_rgb_tf`	Inflated 3D (I3D)	TensorFlow
	Multilayer Perceptrons (MLPs)	

After installation, the location `$COREDLA_ROOT/example_graphs/MLP/` contains an example MLP graph.

In addition, the FPGA AI Suite supports certain signed INT8 symmetric quantized models that use neural network compression framework (NNCF) flows.

Customized models that are similar to the supported models or are derived from the above models are not supported, although in some cases they might work without modification.

The supplied example architectures (or *IP Configurations*) support all of the above models, except for the `Small_NoSoftmax` and `Small_Softmax` architectures that support only ResNet-50, MobileNet V1, and MobileNet V2.

2.4.2.1. MobileNet V2 differences between Caffe and TensorFlow models

There are two inverted bottlenecks (group of expand, depthwise, projection) in which TensorFlow has already gone down to 14x14 while Caffe is still at 28x28. This is the only place where the structure of the graph differs. TensorFlow also uses ReLU6, implemented with a clamp in the FPGA AI Suite IP, while Caffe uses ReLU.

2.4.3. Model Performance

The performance estimator tool assumes the following f_{MAX} values for FPGA devices. For information about the performance estimator too, refer to Estimating Graph Performance on page 195.

- Agilex™ 3: 350 MHz
- Agilex 5: 350 MHz
- Agilex 7: 400 MHz
- Arria® 10: 265 MHz

These assumptions are reasonable and conservative for the standard speed bin. As shown by the results in this section, the achieved f_{MAX} of the design example typically exceeds these assumptions.

The performance results for the designs that follow were achieved using the `dla_build_example_design.py` script that is included with the FPGA AI Suite. The script uses a standard (-2) speed bin with a single seed and uses high-effort compiler settings.

The runtime hosts used for determining the performance results are as follows:

- **Agilex 7 runtime host:** SUSE Linux Enterprise Server 15 host on an Intel Xeon® processor E5-1650 @ 3.5 GHz.

This design uses a dedicated DDR interface for the IP. The batch size is 1. Performance varies based on the clock speed, the DDR latency and bandwidth.

The `dla_build_example_design.py` script includes the following two `.qsf` lines to enable non-default Quartus Prime options during design compilation:

```
set_global_assignment -name ALLOW_SHIFT_REGISTER_MERGING_ACROSS_HIERARCHIES
ALWAYS
set_global_assignment -name DISABLE_REGISTER_MERGING_ACROSS_HIERARCHIES OFF
```

The architectures in the tables that follow are in the `$COREDLA_ROOT/example_architectures/` directory. Review the README file in that directory for information about each architecture.

The **IP Throughput** column in the tables that follow shows the performance for the portion of the graph that runs on the FPGA device. In many cases, the entire graph runs on the FPGA device. The IP Throughput is representative of performance if the IP is used in a hostless configuration.

The **IP+host Throughput** column in the tables that follow shows the performance including the host. The IP+host performance may be lower than IP-only performance if the host is unable to stream data to the FPGA device quickly enough, or if the host is limited by some of the processing associated with the graph (for example, the host performs NMS for the YOLOv3 graph). Achievable IP+host performance depends on the speed and loading of the host and the FPGA AI Suite IP.

Details - FPGA AI Suite 2025.3.1

Architecture	f_{MAX}	ALMs	DSPs	M20Ks	Registers	Board
AGX7_FP16_Generic	600 MHz	33.0 k	186	516	90 k	DE10(1)
AGX7_FP16_Performance	600 MHz	101.5 k	1162	1530	299 k	DE10(1)
AGX7_Small_NoSoftmax	610 MHz	17.0 k	80	300	53 k	DE10(1)
AGX7_Small_Softmax	608 MHz	18.6 k	90	308	57 k	DE10(1)
AGX7_Generic	591 MHz	38.4 k	202	782	109 k	DE10(1)
AGX7_Performance	533 MHz	66.2 k	650	1273	172 k	DE10(1)
AGX7_Performance_Giant	480 MHz	128.4 k	1546	2360	360 k	DE10(1)
AGX7_Streaming_Ddrfree_Resnet18	541 MHz	74.4k	296	8179	177 k	AG7I_DK(2)
AGX5_Performance	308 MHz	79.8 k	266	1151	195 k	AG5_MOD(3)

public/mobilenet-v1-1.0-224

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_FP16_Generic	33.0 k	186	2268	171	171	71.2	89.5
AGX7_FP16_Performance	101.5 k	1162	9223	579	520	71.2	89.5
AGX7_Small_NoSoftmax	17.0 k	80	2804	169	169	70.9	89.6
AGX7_Small_Softmax	18.6 k	90	2797	169	168	70.9	89.5
AGX7_Generic	38.4 k	202	3259	251	251	70.9	89.5
AGX7_Performance	66.2 k	650	8245	525	294	70.9	89.5
AGX7_Performance_Giant	128.4 k	1546	14476	1398	444	71.0	89.6

(1) Terasic* DE10-Agilex Development Board (DE10-Agilex-B2E2)

(2) Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)

(3) Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)

(*) **DDR** is estimated minimum average read + write (that is, read + write require at least this much bandwidth on average). Peak bandwidth is higher.

public/mobilenet-v2

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_FP16_Generic	33.0 k	186	3682	149	148	71.8	89.6
AGX7_FP16_Performance	101.5 k	1162	7112	381	371	71.8	89.6
AGX7_Small_NoSoftmax	17.0 k	80	4684	144	139	71.6	89.6
AGX7_Small_Softmax	18.6 k	90	4673	143	139	71.8	89.4
AGX7_Generic	38.4 k	202	3522	263	250	65.6	86.8
AGX7_Performance	66.2 k	650	6831	327	269	71.7	89.4
AGX7_Performance_Giant	128.4 k	1546	10368	1046	458	71.8	89.4

public/mobilenet-v2-1.4-224

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_FP16_Generic	33.0 k	186	4133	123	123	74.8	91.9
AGX7_FP16_Performance	101.5 k	1162	9000	299	293	74.8	91.9
AGX7_Generic	38.4 k	202	4181	151	146	74.7	91.8
AGX7_Performance	66.2 k	650	8341	277	237	74.7	91.8
AGX7_Performance_Giant	128.4 k	1546	13095	829	477	74.7	91.7

public/mobilenet-v3-large-1.0-224-tf

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_FP16_Generic	33.0 k	186	3846	172	169	75.8	92.1
AGX7_FP16_Performance	101.5 k	1162	11461	245	238	75.8	92.1
AGX7_Generic	38.4 k	202	4513	180	172	72.3	90.7
AGX7_Performance	66.2 k	650	10838	236	224	72.1	90.5
AGX7_Performance_Giant	128.4 k	1546	14525	353	284	72.6	90.6

public/resnet-50-tf

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_FP16_Generic	33.0 k	186	3009	32	32	76.8	92.9
AGX7_FP16_Performance	101.5 k	1162	11701	166	164	76.8	92.9
AGX7_Small_NoSoftmax	17.0 k	80	5961	28	28	77.0	92.9
AGX7_Small_Softmax	18.6 k	90	5947	28	28	77.1	92.9

continued...

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_Generic	38.4 k	202	4154	60	60	77.1	92.9
AGX7_Performance	66.2 k	650	11029	156	144	76.9	92.9
AGX7_Performance_Giant	128.4 k	1546	13313	224	218	76.9	92.8

Resnet50 v1 (Caffe)

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_FP16_Generic	33.0 k	186	2825	38	38	74.4	91.4
AGX7_FP16_Performance	101.5 k	1162	12025	193	195	74.4	91.4
AGX7_Small_NoSoftmax	17.0 k	80	4178	37	37	74.1	91.4
AGX7_Small_Softmax	18.6 k	90	4167	37	37	74.2	91.3
AGX7_Generic	38.4 k	202	4432	72	72	74.2	91.3
AGX7_Performance	66.2 k	650	11531	185	181	74.0	91.4
AGX7_Performance_Giant	128.4 k	1546	14363	257	233	74.1	91.4

intel/unet-camvid-onnx-0001

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]
AGX7_FP16_Generic	33.0 k	186	826	1.09
AGX7_FP16_Performance	101.5 k	1162	4543	7.55
AGX7_Small_NoSoftmax	17. k	80	1141	1.10
AGX7_Small_Softmax	18.6 k	90	1138	1.10
AGX7_Generic	38.4 k	202	1305	2.12
AGX7_Performance	66.2 k	650	4016	6.83
AGX7_Performance_Giant	128.4 k	1546	5430	10.69

public/yolo-v3-tf

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Detection mAP @0.5	Detection mAP @0.5:0.95
AGX7_FP16_Generic	33.0 k	186	1431	4.3	4.1	62.27	31.58
AGX7_FP16_Performance	101.5 k	1162	6340	27.9	27.9	62.25	31.58
AGX7_Generic	38.4 k	202	1878	8.1	7.6	62.28	31.49
AGX7_Performance	66.2 k	650	5806	25.4	10.5	62.22	31.47
AGX7_Performance_Giant	128.4 k	1546	8585	38.4	26.9	62.25	31.46

public/yolo-v3-tiny-tf

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Detection mAP @0.5	Detection mAP @0.5:0.95
AGX7_FP16_Generic	33.0 k	186	1185	41.6	35.4	35.79	14.77
AGX7_FP16_Performance	101.5 k	1162	4552	116.6	115.3	35.81	14.78
AGX7_Generic	38.4 k	202	2314	81.7	64.1	35.76	14.74
AGX7_Performance	66.2 k	650	4187	107.2	34.7	35.73	14.72
AGX7_Performance_Giant	128.4 k	1546	5451	100.8	54.2	35.81	14.75

public/yolo-v8-nano detection

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Detection mAP @0.5	Detection mAP @0.5:0.95
AGX7_FP16_Performance	101.5 k	1162	6780	93	91	51.13	36.51
AGX7_Generic	38.4 k	202	2412	49	39	51.05	36.46
AGX7_Performance	66.2 k	650	6416	92	31	51.05	36.45

public/yolo-v8-nano classification

Architecture	ALMs	DSPs	DDR(*) [MB/s]	Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_FP16_Performance	101.5 k	1162	9718	1301	67.88	87.76
AGX7_Generic	38.4 k	202	5429	932	67.86	87.74
AGX7_Performance	66.2 k	650	9715	1296	67.76	87.82

public/squeezenet1.1

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]	IP+host Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_FP16_Generic	33.0 k	186	634	219	220	58.5	81.1
AGX7_FP16_Performance	101.5 k	1162	4598	924	774	58.5	81.1
AGX7_Small_NoSoftmax	17.0 k	80	929	221	221	58.5	81.0
AGX7_Small_Softmax	18.6 k	90	926	220	220	58.5	81.0
AGX7_Generic	38.4 k	202	1704	529	529	58.5	81.0
AGX7_Performance	66.2 k	650	4253	852	265	58.4	81.0
AGX7_Performance_Giant	128.4 k	1546	5213	864	451	58.3	81.1

public/i3d_rgb_tf

Architecture	ALMs	DSPs	DDR(*) [MB/s]	Throughput [fps]	Top-1 [%]	Top-5 [%]
AGX7_FP16_Generic	33.0 k	186	442	0.61	65.79	82.89
AGX7_FP16_Performance	101.5 k	1162	2545	4.11	66.22	82.89
AGX7_Small_NoSoftmax	17.0 k	80	491	0.58	65.35	82.89
AGX7_Small_Softmax	18.6 k	90	490	0.58	65.57	83.11
AGX7_Generic	38.4 k	202	733	1.34	65.57	83.11
AGX7_Performance	66.2 k	650	2286	3.69	65.13	83.11
AGX7_Performance_Giant	128.4 k	1546	2713	4.22	65.79	82.89

ResNet50 V1

Architecture	ALMs	DSPs	DDR(*) [MB/s]	IP Throughput [fps]
AGX5_Performance	79.8 k	266	4657	74.78

ResNet18 (PyTorch)

The table that follows presents performance measurements for a DDR-free streaming architecture built for the ResNet-18 PyTorch. This analysis focuses on two metrics: core IP throughput and IP throughput:

- Core IP throughput

  This metric isolates the performance of the core FPGA AI Suite IP, excluding the input and output streamer. It highlights the computational efficiency of the processing elements within the architecture

- IP throughput

  This metric captures the overall inference performance of the overlay IP, including the input and output streaming components.

Architecture	ALMs	DSPs	M20ks	IP Throughput [fps]
AGX7_Streaming_Ddrfree_Resnet18	74.4 k	296	8179	173

2.4.3.1. Throughput on the MobileNetV1 model (and other very fast models)

Due to the high system throughput, the MobileNetV1 performance with large IP instances is strongly dependent on the host.

2.4.3.2. DDR-Free Streaming Performance

DDR-free mode relies solely on on-chip memory for data storage. This mode is particularly beneficial for applications where minimizing latency and maximizing throughput are critical, as it eliminates the need for data transfer between on-chip and off-chip memory.

However, DDR-free mode imposes certain constraints:

- Memory Constraints

  The architecture file must have sufficient on-chip memory to accommodate all graph parameters in the filter scratchpad. There must also be sufficient on-chip memory to store all intermediate surfaces in the stream buffer.

- High Coupling to Graph

  Loading a new graph into a DDR-free instance of the FPGA AI Suite IP requires briefly pausing inference operations while the new graph is loaded via the CSR interface. By contrast, the IP can switch between graphs without any additional delay when using external DDR memory.

For more information about DDR-free operation, refer to Using FPGA AI Suite in Hostless DDR-Free Mode on page 283.

For performance values, refer to *ResNet18 (PyTorch)* in Model Performance on page 26.

2.5. The FPGA AI Suite Compiler

The FPGA AI Suite compiler (the `dla_compiler` command) is built on the OpenVINO framework. The compiler for a specific release of the FPGA AI Suite depends on a specific OpenVINO version.

In the FPGA AI Suite, the compiler does the following tasks:

- **Generate architectures**

  The compiler generates an IP parameterization that is optimized for a given machine learning (ML) model or set of models, while attempting to fit the FPGA AI Suite IP block into a given resource footprint.

- **Estimate IP performance**

  The compiler estimates of the performance of the AI IP block for a given parameterization.

- **Estimate IP FPGA resource consumption**

  The compiler estimates of the FPGA resources (ALMs, M20k blocks, and DSPs) required for a given IP parameterization.

- **Create an ahead-of-time compiled graph**

  The compiler creates a compiled version of an ML model that includes the following items:

  ☐ Instructions necessary to control the IP

  ☐ Model weights

  This compiled model is suitable for use by the design examples or in a production deployment.

2.6. The FPGA AI Suite Design Examples

The FPGA AI Suite provides design examples that demonstrate real-world deployment scenarios. Each design showcases a different integration strategy and targets specific FPGA development platforms.

You can find more information about the FPGA AI Suite design examples in later sections of this publication and also at the Altera FPGA Developer Site.

Table 2. FPGA AI Suite Design Examples Descriptions

Design Example	Description
PCIe-attach design example	Demonstrates OpenVINO toolkit integration with FPGA AI Suite for host-attached inference acceleration. The implementation showcases high-bandwidth PCIe data transfer, host-to-accelerator communication protocols, and efficient memory management strategies. Target platform: • Terasic DE10-Agilex Development Board (DE10-Agilex-B2E2)
OFS PCIe-attach design example	Takes advantage of Open FPGA Stack (OFS) standardized interfaces for production-ready deployments, featuring robust driver integration, scalable multicard support, and data center-optimized workflows. Target platforms: • Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1) • Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES) • FPGA SmartNIC N6001-PL Platform (without Ethernet controller)
Hostless DDR-Free design examples	Demonstrates hostless DDR-free operation of the FPGA AI Suite IP. Graph filters, bias, and FPGA AI Suite IP configurations are stored in internal memory on the FPGA device. The designs eliminate external memory dependencies by storing model weights, biases, and configurations entirely in on-chip memory. Ideal for ultra-low latency, deterministic inference in resource-constrained environments. Target platform: • Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)
Hostless JTAG design example	Demonstrates the step-by-step sequence of configuring FPGA AI Suite IP and starting inference by writing into CSRs directly via JTAG. This design exposes the complete FPGA AI Suite IP configuration sequence through direct CSR manipulation via JTAG, which is valuable for understanding the hardware initialization flow and developing custom control interfaces. Target platforms: • Agilex 3 FPGA C-Series Development Kit (DK-A3Y135BM16AEA) • Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
SoC design example	Demonstrates how OpenVINO toolkit and the FPGA AI Suite can support a CPU-offload architecture in integrated SoC environments. The design example showcases HPS-to-FPGA communication, shared memory architectures, and embedded Linux integration with the OpenVINO runtime. Target platforms: • Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1) • Agilex 7 FPGA I-Series Transceiver-SoC Development Kit (DK-SI-AGI027FC) • Arria 10 SX SoC FPGA Development Kit (DK-SOC-10AS066S)

Table 3. FPGA AI Suite Design Examples Properties Overview

The following table provides an overview of the some of the main properties of the design examples.

Design Example Type	Target FPGA Device	Host	Memory	Stream[4]	Design Example Identifier[5]	Supported Development Kit
PCIe-Attached	Agilex 5	External host processor	DDR	M2M	agx5e_modular_ofs_pcie	Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
	Agilex 7				agx7_de10_pcie	Terasic DE10-Agilex Development Board (DE10-Agilex-B2E2)

continued...

Design Example Type	Target FPGA Device	Host	Memory	Stream[4]	Design Example Identifier[5]	Supported Development Kit
					agx7_iseries_ofs_pcie	Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)
					agx7_n6001_ofs_pcie	FPGA SmartNIC N6001-PL Platform (without Ethernet controller)
Hostless DDR-Free	Agilex 7	Hostless[6]	DDR-Free	Direct	agx7_iseries_ddrfree	Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)
Hostless JTAG-Attached	Agilex 3		DDR	M2M	agx3c_jtag	Agilex 3 FPGA C-Series Development Kit (DK-A3Y135BM16AEA)
	Agilex 5		DDR	M2M	agx5e_modular_jtag	Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
SoC	Agilex 5	On-device HPS	DDR	M2M and S2M	agx5_soc_m2m agx5_soc_s2m	Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
	Agilex 7				agx7_soc_m2m agx7_soc_s2m	Agilex 7 FPGA I-Series Transceiver-SoC Development Kit (DK-SI-AGI027FC)
	Arria 10				a10_soc_m2m a10_soc_s2m	Arria 10 SX SoC FPGA Development Kit (DK-SOC-10AS066S)

2.6.1. PCIe-Attach Design Example

The FPGA AI Suite PCIe-attach design example demonstrates how the Intel Distribution of OpenVINO toolkit and the FPGA AI Suite support the look-aside deep learning acceleration model.

---

(4) For the **Stream** column, the entries are defined as follows:

M2M FPGA AI Suite runtime software running on the external host transfers the image (or data) to the FPGA DDR memory.

S2M Streaming input data is copied to FPGA on-device memory. The FPGA AI Suite runtime runs on the FPGA device (HPS or RTL state machine). The runtime is used only to coordinate the data transfer from FPGA DDR memory into the FPGA AI Suite IP.

Direct Data is streamed directly in and out of the FPGA on-chip memory.

(5) For the **Design Example Identifier** column, these entries are the value to use with the FPGA AI Suite Design Example Utility (`dla_build_example_design.py`) command to build the design example

The PCIe-attach design example is implemented with the following components:

- FPGA AI Suite IP
- Intel Distribution of OpenVINO toolkit
- Terasic DE10-Agilex Development Board
- Sample hardware and software systems that illustrate the use of these components

This design example includes prebuilt FPGA bitstreams that correspond to preoptimized architecture files. However, the design example build scripts let you choose from a variety of architecture files and build (or rebuild) your own bitstreams, provided that you have a license permitting bitstream generation.

This design is provided with the FPGA AI Suite as an example showing how to incorporate the IP into a design. This design is not intended for unaltered use in production scenarios. Any potential production application that uses portions of this design example must review them for both robustness and security.

The following sections in this document describe the steps to build and execute the design:

- Building the FPGA AI Suite PCIe Design Example Runtime on page 91
- Running the PCIe-Attach Design Example Demonstration Applications on page 143

The following sections in this document describe design decisions and architectural details about the design:

- Design Example Components on page 39

Use this document to help you understand how to create a PCIe design example with the targeted FPGA AI Suite architecture and number of instances and compiling the design.

---

(4) For the **Stream** column, the entries are defined as follows:

M2M FPGA AI Suite runtime software running on the external host transfers the image (or data) to the FPGA DDR memory.

S2M Streaming input data is copied to FPGA on-device memory. The FPGA AI Suite runtime runs on the FPGA device (HPS or RTL state machine). The runtime is used only to coordinate the data transfer from FPGA DDR memory into the FPGA AI Suite IP.

Direct Data is streamed directly in and out of the FPGA on-chip memory.

(5) For the **Design Example Identifier** column, these entries are the value to use with the FPGA AI Suite Design Example Utility (`dla_build_example_design.py`) command to build the design example

(6) Device is accessible through the JTAG System Console host.

2.6.2. Open FPGA Stack (OFS) for PCIe-Attach Design Examples

The FPGA AI Suite Open FPGA Stack (OFS) for PCIe-attach design examples demonstrate the design and implementation for accelerating AI inference using the FPGA AI Suite, Intel Distribution of OpenVINO toolkit, and boards that support PCIe Attach OFS:

- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
- Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)
- FPGA SmartNIC N6001-PL Platform (without Ethernet controller)

  Tip: N6001-PL SmartNIC boards are available through ODM partners. For more information, including ordering information, refer to the SmartNIC N6000-PL product brief.

The following section in this document describe the steps to build and execute the design:

- OFS for PCIe-Attach Design Example Prerequisites on page 93

The following section in this document describe design decisions and architectural details about the design:

- OFS PCIe-Attach Design Example Components on page 280

2.6.3. Hostless DDR-Free Design Example

The FPGA AI Suite provides a design example to demonstrate hostless and DDR-free operation of the FPGA AI Suite IP. Graph filters, bias, and FPGA AI Suite IP configurations are stored in on-chip memory on the FPGA device instead of DDR memory on the board.

The DDR-free design example demonstrates how FPGA AI Suite supports the following features:

- DDR-free operation
- Hostless operation (that is, running on the devices without the FPGA AI Suite runtime)
- Streaming of input features
- Streaming of inference results

The DDR-Free design example is implemented with the following components:

- FPGA AI Suite IP
- Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)
- Sample hardware and software systems that illustrate the use of these components

For more details about DDR-free operation, refer to Using FPGA AI Suite in Hostless DDR-Free Mode on page 283.

The design example build scripts in FPGA AI Suite Design Example Utility on page 39 let you choose from a variety of architecture files and build your own bitstreams, provided that you have a license permitting bitstream generation.

This design is provided with the FPGA AI Suite as an example showing how to incorporate the FPGA AI Suite IP into a DDR-Free design. This design is not intended for unaltered use in production scenarios. Any potential production application that uses portions of this design example must be reviewed for both robustness and security.

The following sections in this document describe the steps to build and execute the design:

- Hostless DDR-Free Design Example Prerequisites on page 96
- Running the Hostless DDR-Free Design Example on page 156

The related links that follow link to the parts of this document that describe design decisions and architectural details about the design.

Related Information

- Hostless DDR-Free Design Example System Architecture on page 284
- Hostless DDR-Free Design Example Quartus Prime System Console on page 287
- Hostless DDR-Free Design Example JTAG to Avalon MM Host Register Map on page 291
- Changing the ML Graph in a Hostless DDR-Free Architecture on page 291

2.6.4. Hostless JTAG Design Example

The hostless JTAG design example demonstrates how to instantiate one instance of the FPGA AI Suite IP on an Agilex 3 C-Series or Agilex 5 E-Series device.

It places configurations and data on external DDR memory and allows an external host to interact with the memory and FPGA AI Suite IP via JTAG.

This design example targets the following development kits:

- Agilex 3 FPGA C-Series Development Kit (DK-A3Y135BM16AEA)
- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)

The following sections in this document describe the steps to build and execute the design:

- Running the JTAG Design Example Demonstration Application on page 159

The following sections in this document describe design decisions and architectural details about the design:

- Hostless JTAG Design Example Components on page 294

2.6.5. SoC Design Example

The FPGA AI Suite SoC design example shows how the Intel Distribution of OpenVINO toolkit and the FPGA AI Suite support the CPU-offload deep learning acceleration model in an embedded system

The SoC design examples are implemented with the following components:

- FPGA AI Suite IP
- Intel Distribution of OpenVINO toolkit
- The community-supported OpenVINO ARM plugin
- Sample hardware and software systems that illustrate the use of these components
- Arm*-Linux build scripts built using Yocto frameworks for the hard processor systems (HPSs) on the following FPGA SoC devices:
  - ☐ Agilex 5 E-Series SoC
  - ☐ Agilex 7 I-Series SoC
  - ☐ Arria 10 SX SoC

For an easier initial experience, these design examples include prebuilt FPGA bitstreams and a Linux-compiled system image that correspond to pre-optimized FPGA AI Suite architecture files.

You can copy this disk-image to an SD card and insert the card into a supported FPGA development kit. Additionally, you can use the design example scripts to choose from a variety of architecture files and build (or rebuild) your own bitstreams, subject to IP licensing limitations.

The following section in this document describes the steps to build and execute the design:

- FPGA AI Suite SoC Design Example Run Process on page 314

The following sections in this document describe design decisions and architectural details about the design:

- FPGA AI Suite SoC Design Example Build Process on page 315
- Using FPGA AI Suite as an Embedded Platform on page 297
- Additional FPGA AI Suite SoC Design Example Software Components on page 55
- Using FPGA AI Suite in Video Applications on page 323

Use those sections to help you understand how to create a SoC design example with the targeted FPGA AI Suite architecture.

SoC Design Example Execution Models

The SoC design example has two execution models:

- A memory-to-memory (M2M) execution model, which provides a `dla_benchmark` interface to the inference engine, similar to the PCIe design examples.

  For design details, refer to Memory-to-Memory (M2M) Variant Design on page 299.
- A streaming-to-memory (S2M) execution model that demonstrates a streaming data source

  For simplicity in this design example, the streaming data source is the SoC ARM CPU itself, which streams to a layout transform on the FPGA, but this design illustrates one of the suggested system architectures for any streaming source.

  For design details, refer to Streaming-to-Memory (S2M) Variant Design on page 300.

The design example is typically compiled for the S2M execution model, which supports both the M2M and S2M modes. A reduced functionality bitstream is also included as a compilation option, which supports only the M2M execution model.

The SoC design example has been optimized for simplicity, to create a flexible foundation that you can use to build more complex SoC designs.

2.6.6. Design Example Components

2.6.6.1. FPGA AI Suite Design Example Utility

The main entry point into the design example build system is the FPGA AI Suite design example utility (`dla_build_example_design`). This utility coordinates different aspects of building a design example, from the initial project configuration to extracting area metrics after a compile completes. With this utility, you can create bitstreams from custom AI Suite IP architecture files and with a configurable number of instances.

To use the FPGA AI Suite design example build utility, ensure that your local development environment has been setup according to the steps in Installing the FPGA AI Suite Compiler and IP Generation Tools on page 67.

2.6.6.1.1. The `dla_build_example_design` Command

The FPGA AI Suite design example utility (`dla_build_example_design`) configures and compiles the bitstreams for the FPGA AI Suite design examples. The command has the following basic syntax:

- Linux systems:

```
dla_build_example_design.py [--logfile logfile] [--debug] [action]
```

- Windows* systems:

```
dla_build_example_design [--logfile logfile] [--debug] [action]
```

where [*action*] is one of the following actions:

Table 4. Design Example Utility Actions

Action	Description
`list`	List the available design examples.
`build`	Build a design example.
`qor`	Generate QoR reports.
`quartus-compile`	Run a Quartus Prime compile.
`scripts`	Managed the build support scripts.

Some of the command actions have different additional required and optional parameters. Use the command help to see a list of available options for the command and its actions.

By default, the `dla_build_example_design.py` command always instructs the `dla_create_ip` command to create licensed IP. If no license can be found, inference-limited, unlicensed RTL is generated. The build log indicates if the IP is licensed or unlicensed. For more information about licensed and unlicensed IP, refer to The `--unlicenced`/`--licenced` Options.

Getting `dla_build_example_design.py` Command Help

For general help with the command and to see the list of available actions, run the following command:

```
dla_build_example_design.py --help
```

For help with a command actions and to the list of available options and arguments required for an action, run the following command:

```
dla_build_example_design.py action --help
```

Command Logging and Debugging

By default, the FPGA AI Suite design example utility logs its output to a set location in the (default) `build_platform` directory. Override this location by specifying the `--logfile` *`logfile`* option.

For extra debugging output, specify the `--debug` option.

Both the logging and debugging options must be specified before the command action.

2.6.6.1.2. Listing Available FPGA AI Suite Design Examples

To list the available FPGA AI Suite design examples, run the following command:

```
dla_build_example_design.py list
```

This command shows the design example identifiers (used with the `build` action of the design example utility) along with a short description of the design example and its target Quartus Prime version.

A list of the design examples and their identifiers is also available in FPGA AI Suite Design Examples Properties Overview.

2.6.6.1.3. Building FPGA AI Suite Design Examples

To build FPGA AI Suite design examples, specify the `build` option of the design example utility.

In the most simple case of building a design example for an existing architecture, you can build a design example with the following command:

```
dla_build_example_design.py build \
  <design_example_identifier> \
  <architecture file>
```

For example, use the following command to build the Agilex 7 PCIe design example that targets the DE10-Agilex-B2E2 board using the `AGX7_Generic` architecture:

```
dla_build_example_design.py build \
  agx7_de10_pcie \
  $COREDLA_ROOT/example_architectures/AGX7_Generic.arch
```

The default build directory is '`build_platform`'. The utility creates the folder if it does not exist. To specify a different build directory, specify the `--output-dir` *<directory>* option:

```
dla_build_example_design.py build \
  --output-dir <directory> \
  <design_example_identifier> \
  <architecture file>
```

Where the *<design_example_identifier>* values are listed in the "FPGA AI Suite Design Examples Properties Overview" table in The FPGA AI Suite Design Examples on page 32.

By default, the utility also prevents the build directory from being overwritten. You can override this behavior with the `--force` option.

After the build is complete, the build directory has the following files and folders:

- `coredla_ip/`

  This folder contains the RTL for the configured FPGA AI Suite IP.

- `hw/`

  This folder contains the Quartus Prime or Open FPGA Stack (OFS) project files. It also includes its own self-contained copy of the contents of the `coredla_ip/` folder

- `.build.json`

  This contents of this file (sometimes referred to as the "build context" file) allow the build to be split into multiple steps.

- Reports

  The build directory will contain any log files generated by the build utility (such as `build.log)` and the QoR summary that is generated by a successful compilation.

- Bitstreams

  This build directory will contain the bitstreams to program the target FPGA device as follows:

  - ☐ For OFS-based designs, `.gbs` files.
  - ☐ For other designs, `.sof` and `.rbf` files.

Staging FPGA AI Suite Design Example Builds

The FPGA AI Suite design example utility supports staged builds via the `--skip-compile` option. When you specify this option, the utility creates the build folder and prepares the Quartus Prime or Open FPGA Stack (OFS) project but does not run compilation.

With a staged design example build flow, you must run the utility a few times to generate the same outputs as the regular build flow. For a complete build with a staged build flow, you would run the following commands:

```
dla_build_example_design.py build --skip-compile \
  <design_example_identifier> \
  <architecture file>

dla_build_example_design.py quartus-compile build_platform

# Optional: Generate QoR summary
dla_build_example_design.py qor build_platform
```

Information about the build configuration, namely what was passed into the `dla_build_example_design.py build` command, is stored inside of the build context (`.build.json`) file.

You can run the `dla_build_example_design.py quartus-compile` and `dla_build_example_design.py qor` commands multiple times. An example of when running these commands multiple times can be useful is when you have to recompile the bitstream after removing an unnecessary component from the design example.

You can also directly call the design compilation script. An FPGA AI Suite design example uses the `build_project.sh` script. The script must be executed in a Bash-compatible shell. For example:

```
bash build_project.sh
```

The current working directory must be the `hw/` directory.

2.6.6.2. Example Architecture Bitstream Files

The FPGA AI Suite provides example Architecture Files and bitstreams for the design examples. The files are distributed as separate download packages. After you install one or more of the packages, the files are located in the FPGA AI Suite installation directory.

Related Information

Downloading Precompiled Bitstreams and SD Card Images on page 86

2.6.6.3. Design Example Software Components

The design examples contain a sample software stack for the runtime flow.

For a typical design example, the following components comprise the runtime stack:

- OpenVINO Toolkit (Inference Engine, Heterogeneous Plugin)
- FPGA AI Suite runtime plugin
- Vendor-provided FPGA board driver or OPAE driver (depending on the design example)

The design example contains the source files and Makefiles to build the FPGA AI Suite runtime plugin. The OpenVINO component (and OPAE components, where used) is external and must be manually preinstalled.

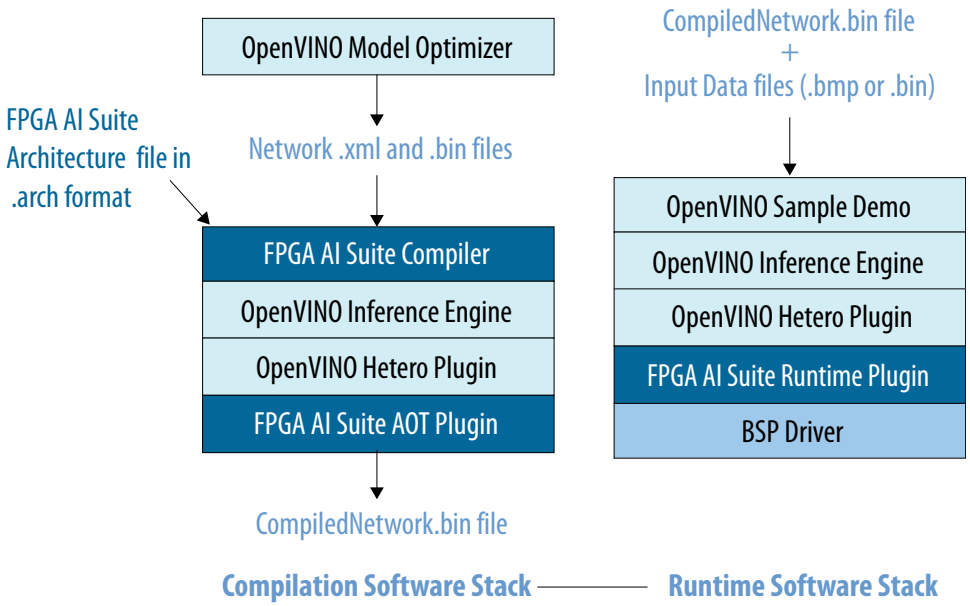
A separate flow compiles the AI network graph using the FPGA AI Suite compiler, as shown in figure *Software Stacks for FPGA AI Suite Inference* that follows as the Compilation Software Stack.

The compilation flow output is a single binary file called `CompiledNetwork.bin` that contains the compiled network partitions for FPGA and CPU devices along with the network weights. The network is compiled for a specific FPGA AI Suite architecture and batch size. This binary is created on-disk only when using the Ahead-Of-Time flow; when the JIT flow is used, the compiled object stays in-memory only.

An Architecture File describes the FPGA AI Suite IP architecture to the compiler. You must specify the same Architecture File to the FPGA AI Suite compiler and to the FPGA AI Suite design example utility (`dla_build_example_design.py`).

The runtime flow accepts the `CompiledNetwork.bin` file as the input network along with the image data files.

Figure 8. Software Stacks for FPGA AI Suite Inference



The runtime stack cannot program the FPGA with a bitstream. To build a bitstream and program the FPGA devices:

1. Compile the design example.
2. Program the device with the bitstream.

Instructions for these steps are provided in the sections for each design example.

To run inference through the OpenVINO Toolkit on the FPGA, set the OpenVINO device configuration flag (used by the heterogeneous Plugin) to `FPGA` or `HETERO:FPGA,CPU`.

2.6.6.3.1. OpenVINO FPGA Runtime Overview

The purpose of the runtime front end is as follows:

- Provide input to the FPGA AI Suite IP
- Consume output from the FPGA AI Suite IP
- Control the FPGA AI Suite IP

Typically, this front-end layer provides the following items:

- The `.arch` file that was used to configure the FPGA AI Suite on the FPGA.
- The ML model (possibly precompiled into an Ahead-of-Time `.bin` file by the FPGA AI Suite compiler (`dla_compiler`).
- A target device that is passed to OpenVINO

  The target device may instruct OpenVINO to use the HETERO plugin, which allows a graph to be partitioned onto multiple devices.

One of the directories provided in the installation of the FPGA AI Suite is the `runtime/` directory. In this directory, the FPGA AI Suite provides the source code to build a selection of OpenVINO applications. The `runtime/` directory also includes the `dla_benchmark` command line utility that you can use to generate inference requests and benchmark the inference speed.

The following applications use the OpenVINO API. They support the OpenVINO HETERO plugin, which allows portions of the graph to fall-back onto the CPU for unsupported graph layers.

- `dla_benchmark` (adapted from OpenVINO benchmark_app)
- `classification_sample_async`
- `object_detection_demo_yolov3_async`
- `segmentation_demo`

Each of these applications serve as a runtime executable for the FPGA AI Suite. You might want to write your own OpenVINO-based front ends to wrap the FPGA plugin. For information about writing your own OpenVINO-based front ends, refer to the OpenVINO documentation.

Some of the responsibilities of the OpenVINO FPGA plugin are as follows:

- Inference Execution
  - ☐ Mapping inference requests to an IP instance and internal buffers
  - ☐ Executing inference requests via the IP, managing synchronization and all data transfer between host and device.
- Input / Output Data Transform
  - ☐ Converting the memory layout of input/output data
  - ☐ Converting the numeric precision of input/output data

2.6.6.3.2. OpenVINO FPGA Runtime Plugin

The FPGA runtime plugin uses the OpenVINO Inference Engine Plugin API.

The OpenVINO Plugin architecture is described in the *OpenVINO Plugin Developer Guide*.

The source files are located under `runtime/plugin`. The three main components of the runtime plugin are the Plugin class, the Executable Network class, and the Inference Request class. The primary responsibilities for each class are as follows:

Plugin class

- Initializes the runtime plugin with an FPGA AI Suite architecture file which you set as an OpenVINO configuration key (refer to Running the Ported OpenVINO Demonstration Applications in the PCIe Design Example on page 155).
- Contains `QueryNetwork` function that analyzes network layers and returns a list of layers that the specified architecture supports. This function allows network execution to be distributed between FPGA and other devices and is enabled with the `HETERO` mode.
- Creates an executable network instance in one of the following ways:
  - ☐ Just-in-time (JIT) flow: Compiles a network such that the compiled network is compatible with the hardware corresponding to the FPGA AI Suite architecture file, and then loads the compiled network onto the FPGA device.
  - ☐ Ahead-of-time (AOT) flow: Imports a precompiled network (exported by FPGA AI Suite compiler) and loads it onto the FPGA device.

Executable Network Class

- Represents an FPGA AI Suite compiled network
- Loads the compiled model and config data for the network onto the FPGA device that has already been programmed with an FPGA AI Suite bitstream. For two instances of FPGA AI Suite, the Executable Network class loads the network onto both instances, allowing them to perform parallel batch inference.
- Stores input/output processing information.
- Creates infer request instances for pipelining multiple batch execution.

Infer Request class

- Runs a single batch inference serially.
- Executes five stages in one inference job – input layout transformation on CPU, input transfer to DDR, FPGA AI Suite FPGA execution, output transfer from DDR, output layout transformation on CPU.
- In asynchronous mode, executes the stages on multiple threads that are shared across all inference request instances so that multiple batch jobs are pipelined, and the FPGA is always active.

2.6.6.3.3. FPGA AI Suite Runtime

The FPGA AI Suite runtime implements lower-level classes and functions that interact with the memory-mapped device (MMD). The MMD is responsible for communicating requests to the driver, and the driver connects to the BSP, and ultimately to the FPGA AI Suite IP instance or instances.

The runtime source files are located under `runtime/coredla_device`. The three most important classes in the runtime are the Device class, the GraphJob class, and the BatchJob class.

Device class

- Acquires a handle to the MMD for performing operations by calling `aocl_mmd_open`.
- Initializes a DDR memory allocator with the size of 1 DDR bank for each FPGA AI Suite IP instance on the device.
- Implements and registers a callback function on the MMD DMA (host to FPGA) thread to launch FPGA AI Suite IP for batch=1 after the batch input data is transferred from host to DDR.
- Implements and registers a callback function (interrupt service routine) on the MMD kernel interrupt thread to service interrupts from hardware after one batch job completes.
- Provides the `CreateGraphJob` function to create a GraphJob object for each FPGA AI Suite IP instance on the device.
- Provides the `WaitForDla(`*`instance id`*`)` function to wait for a batch inference job to complete on a given instance. Returns instantly if the number of batch jobs finished (that is, the number of jobs processed by interrupt service routine) is greater than number of batch jobs waited for this instance. Otherwise, the function waits until interrupt service routine notifies. Before returning, this function increments the number of batch jobs that have been waited for this instance.

GraphJob class

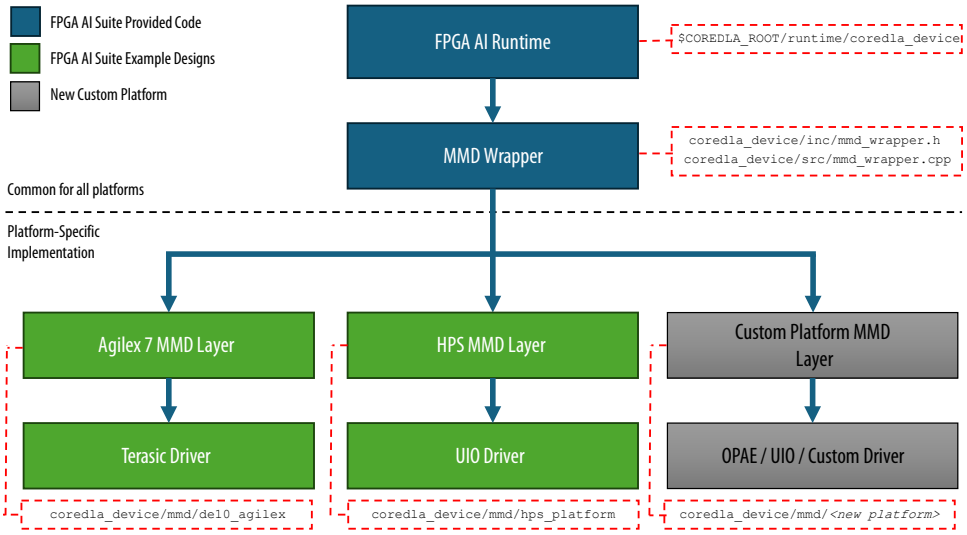
- Represents a compiled network that is loaded onto one instance of the FPGA AI Suite IP on an FPGA device.
- Allocates buffers in DDR memory to transfer configuration, filter, and bias data.
- Creates BatchJob objects for a given number of pipelines and allocates input and output buffers for each pipeline in DDR.

BatchJob class

- Represents a single batch inference job.
- Stores the DDR addresses for batch input and output data.
- Provides `LoadInputFeatureToDdr` function to transfer input data to DDR and start inference for this batch asynchronously.
- Provides `ReadOutputFeatureFromDdr` function to transfer output data from DDR. Must be called after inference for this batch is completed.

2.6.6.3.4. FPGA AI Suite Custom Platform

Figure 9. Overview of FPGA AI Suite MMD Runtime



The interface to the user-space portion of the BSP drivers is centralized in the `MmdWrapper` class, which can be found in the file `$COREDLA_ROOT/runtime/coredla_device/inc/mmd_wrapper.h`. This file is a wrapper around the MMD API.

The FPGA AI Suite runtime uses this wrapper so that the runtime can be reused on all platforms. When porting the runtime to a new board, you must ensure that each of the member functions in `MmdWrapper` calls into a board-specific implementation function. You must also modify the runtime build process and adjacent code.

Any implementation of a runtime for the FPGA AI Suite must support the following features via the MMD Wrapper:

- Open the device
- Register an interrupt service routine
- Read/write 32-bit register values in the IP control-and-status register (CSR)
- Transfer bulk data between the host and device

2.6.6.3.5. Memory-Mapped Device (MMD) Driver

The FPGA AI Suite runtime MMD software uses a driver to access and interact with the FPGA device. To integrate the FPGA AI Suite IP into your design on your platform, the MMD layer must interface with the hardware using the appropriate drivers (such as OPAE, UIO, or a custom driver). For example, the PCIe design example uses the drivers provided by the OpenCL board support package (BSP) for the Terasic DE10-Agilex Development Board.

If your board vendor provides a BSP, you can use the MMD Wrapper to interface the BSP with the FPGA AI Suite IP. Review the following sections for examples of adapting a vendor-provided BSP to use with the FPGA AI Suite IP:

- Terasic DE10-Agilex Development Board BSP Example on page 53
- Agilex 7 PCIe-Attach OFS-based BSP Example on page 54

You can create a custom BSP for your board, but that process can be complex and can require more work.

The FPGA AI Suite runtime MMD software uses a driver to access and interact with the FPGA device. This driver is supplied as part of the board vendor BSP or, for OFS-based boards, the OPAE driver.

The source files for the MMD driver are provided in `runtime/coredla_device/mmd`.

The source files contain classes for managing and accessing the FPGA device by using driver-supplied functions for reading/writing to CSR, reading/writing to DDR, and handling kernel interrupts.

Obtaining BSP Drivers

Contact your FPGA board vendor for information about the BSP for your FPGA board.

Obtaining the OPAE Drivers

Contact your FPGA board vendor for information about the OPAE driver for your FPGA board.

For the FPGA AI Suite OFS for PCIe attach design example, the OPAE driver is installed when you follow the steps in OFS for PCIe-Attach Design Example Prerequisites on page 93.

2.6.6.3.6. FPGA AI Suite Runtime MMD API

This section describes board-level functions that are defined in the `mmd_wrapper.cpp` file. Your implementation of the functions in the `mmd_wrapper.cpp` file for your specific board may differ. For examples of these functions, refer to the provided MMD implementations under `$COREDLA_ROOT/runtime/coredla_device/mmd/`.

The `mmd_wrapper.cpp` file contains the following MMD functions that are adapted from the Open FPGA Stack (OFS) accelerator support package (ASP) functions of the same name. For more information about these functions, refer to the OFS AFS Memory Mapped Device Layer documentation.

- `aocl_mmd_get_offline_info`
- `aocl_mmd_open`
- `aocl_mmd_close`
- `aocl_mmd_set_interrupt_handler`

Although several of the functions in the FPGA AI Suite MMD API share names and intended behavior with OpenCL MMD API functions, you do not need to use an OpenCL BSP. The naming convention is maintained for historical reasons only.

The `mmd_wrapper.cpp` file contains the following functions provided only with the FPGA AI Suite:

- `dla_mmd_get_max_num_instances`
- `dla_mmd_get_ddr_size_per_instance`
- `dla_mmd_get_coredla_clock_freq`

- dla_mmd_get_ddr_clock_freq
- dla_mmd_csr_read
- dla_mmd_csr_write
- dla_mmd_ddr_read
- dla_mmd_ddr_write
- dla_is_stream_controller_valid
- dla_mmd_stream_controller_read
- dla_mmd_stream_controller_write

The dla_mmd_get_max_num_instances Function

Returns the maximum number of FPGA AI Suite IP instances that can be instantiated on the platform. In the FPGA AI Suite PCIe design examples, this number of IP instances that can be instantiated is the same as the number of external memory interfaces (for example, DDR memories).

Syntax

```
int dla_mmd_get_max_num_instances()
```

The dla_mmd_get_ddr_size_per_instance Function

Returns the maximum amount of external memory available to each FPGA AI Suite IP instance.

Syntax

```
uint64_t dla_mmd_get_ddr_size_per_instance()
```

The dla_mmd_get_coredla_clock_freq Function

Given the device handle, return the FPGA AI Suite IP PLL clock frequency in MHz. Return a negative value if there is an error.

In the PCIe design example, this value is determined by allowing a set amount of wall clock time to elapse between reads of counters onboard the IP.

Syntax

```
double dla_mmd_get_coredla_clock_freq(int handle)
```

The dla_mmd_get_ddr_clock_freq Function

Section:

Returns the DDR clock frequency, in Mhz. Check the documentation from your board vendor to determine this value.

Syntax

```
double dla_mmd_get_ddr_clock_freq()
```

The `dla_mmd_csr_read` Function

Performs a control status register (CSR) read for a given instance of the FPGA AI Suite IP at a given address. The result is stored in the `data` directory.

Syntax

```
int dla_mmd_csr_read(int handle, int instance, uint64_t addr, uint32_t *data)
```

The `dla_mmd_csr_write` Function

Performs a control status register (CSR) write for a given instance of the FPGA AI Suite IP at a given address.

Syntax

```
int dla_mmd_csr_write(int handle, int instance, uint64_t addr, const uint32_t
*data)
```

The `dla_mmd_ddr_read` Function

Performs an external memory read for a given instance of the FPGA AI Suite IP at a given address. The result is stored in the `data` directory.

Syntax

```
int dla_mmd_ddr_read(int handle, int instance, uint64_t addr, uint64_t length,
void *data)
```

The `dla_mmd_ddr_write` Function

Performs an external memory write for a given instance of the FPGA AI Suite IP at a given address.

Syntax

```
int dla_mmd_ddr_write(int handle, int instance, uint64_t addr, uint64_t length,
const void *data)
```

The `dla_is_stream_controller_valid` Function

Optional. Required if `STREAM_CONTROLLER_ACCESS` is defined.

Queries the streaming controller device to see if it is valid.

Syntax

```
bool dla_is_stream_controller_valid(int handle, int instance)
```

For more information about the stream controller module, refer to Stream Controller Communication Protocol on page 324.

The `dla_mmd_stream_controller_read` Function

Optional. Required if STREAM_CONTROLLER_ACCESS is defined.

Reads an incoming message from the streaming controller.

Syntax

```
int dla_mmd_stream_controller_read(int handle, int instance, uint64_t addr,
uint64_t length, void* data)
```

For more information about the streaming controller device, refer to Stream Controller Communication Protocol on page 324.

The `dla_mmd_stream_controller_write` Function

Optional. Required if `STREAM_CONTROLLER_ACCESS` is defined.

Writes an outgoing message from the streaming controller.

Syntax

```
int dla_mmd_stream_controller_write(int handle, int instance, uint64_t addr,
uint64_t length, const void* data)
```

For more information about the streaming controller device, refer to Stream Controller Communication Protocol on page 324.

2.6.6.3.7. Board Support Package (BSP) Overview

Every FPGA platform consists of the FPGA fabric and the hard IP that surrounds it. For example, an FPGA platform might provide an external memory interface as hard IP to provide access to external DDR memory. Soft logic that is synthesized on the FPGA fabric needs to be able to communicate with the hard IP blocks, and the implementation details are typically platform-specific.

A board support package (BSP) typically consists of two parts:

- A software component that runs in the host operating system.

  This component includes the MMD and operating system driver for the board.
- A hardware component that is programming into the FPGA fabric.

  This component consists of soft logic that enables the use of the FPGA peripheral hard IP blocks around the FPGA fabric. This component acts as the bridge between the FPGA AI Suite IP block in the FPGA fabric and the hard IP blocks.

Depending on your board and board vendor, you can have the following options for obtaining a BSP:

- If your board supports the Open FPGA Stack (OFS), you can use (and adapt, if necessary) an OFS reference design or FPGA interface manager (FIM).

  For some boards, there are precompiled FIM reference designs available.
- Obtain a BSP directly from your board vendor. You board vendor might have multiple BSPs available for you board.
- Create your own BSP.

For a BSP to be compatible with FPGA AI Suite, the BSP must provide the following capabilities:

- Enable the FPGA AI Suite IP and the host to interface with the external memory interface IP.
- Enable the FPGA AI Suite IP to interface with the runtime (for example, PCIe IP for the PCIe design example, or the HPS-to-FPGA AXI bridge for the SoC design example).
- Enable the FPGA AI Suite IP to send interrupts to the runtime. If your BSP does not support this capability, you must use polling to determine when an inference is complete.
- Enable the host to access the FPGA AI Suite IP CSR.

The BSPs available for the boards supported by the FPGA AI Suite design example support these capabilities.

Related Information

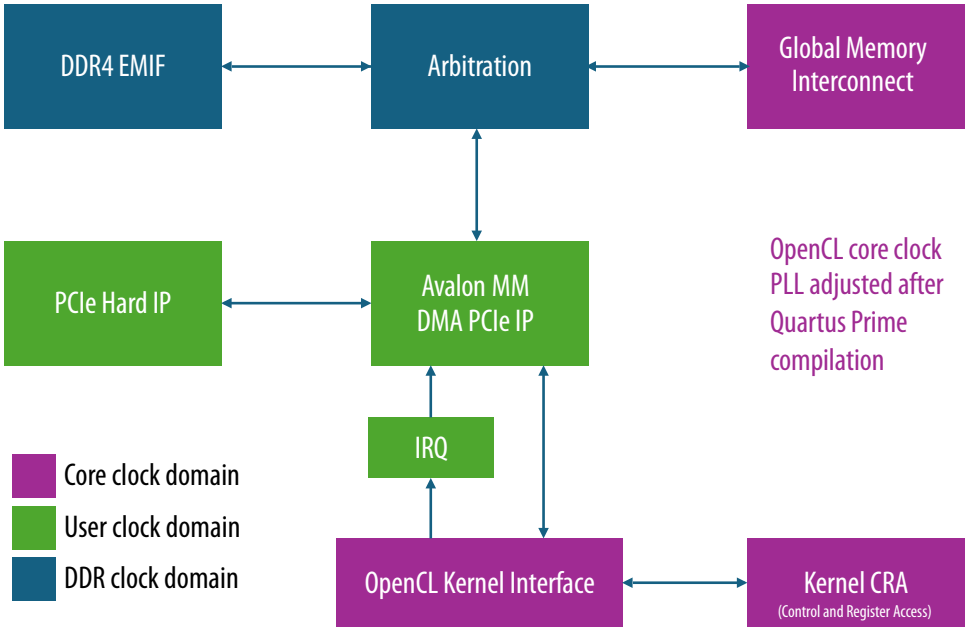
Open FPGA Stack (OFS) documentation

Terasic DE10-Agilex Development Board BSP Example

For the Agilex 7 PCIe design example on the Terasic DE10-Agilex Development Board, the BSP provided by Terasic is adapted to work with the FPGA AI Suite IP. The Terasic-provided BSP is OpenCL™-based.

The following diagram shows the high-level interactions between the FPGA interface IPs on the platform, and the a custom OpenCL kernel. The different colors in the diagram indicate different clock domains.

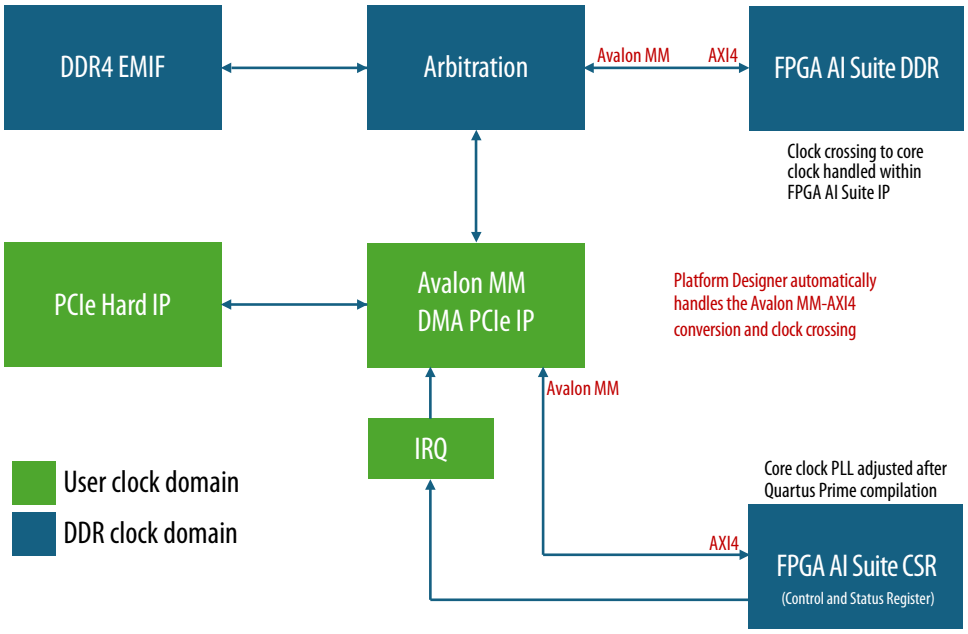
Figure 10. Terasic BSP with OpenCL Kernel



The PCIe hard IP can read/write to the DDR4 external memory interface (EMIF) via the DMA and the Arbitrator. Additional logic is provided to handle interrupts from the custom IP and propagate them back to the host through the PCIe interface.

The following diagram hows how the Terasic DE10-Agilex Development Board BSP can be adapted to support the FPGA AI Suite IP.

Figure 11. Terasic BSP With FPGA AI Suite IP



Platform Designer automatically adds clock-domain crossings between Avalon memory-mapped interfaces and AXI4 interfaces, making the integration with the BSP easier.

For a custom platform, consider following a similar approach of modifying the BSP provided by the vendor to integrate in the FPGA AI Suite IP.

Agilex 7 PCIe-Attach OFS-based BSP Example

For OFS-based devices, the BSP consists of a platform-specific FPGA interface manager (FIM) and a platform-agnostic accelerator functional unit (AFU).

The FPGA AI Suite OFS for PCIe attach design example supports Agilex 7 PCIe Attach OFS.

You can obtain the source files needed to build a Agilex 7 PCIe Attach FIM or obtain prebuilt FIMs for some boards from *OFS Agilex 7 PCIe Attach FPGA Development Directory* in GitHub.

The AFU wraps the FPGA AI Suite IP and must meet the following general requirements:

- The AFU must include an instance of the FPGA AI Suite IP.
- The AFU must support host access (for example, via DMA) to external memory that is shared with the FPGA AI Suite IP.
- The AFU must propagate interrupts from the FPGA AI Suite IP to the host.
- The AFU Must support host access to the FPGA AI Suite IP CSR memory.

If you are creating your own FPGA AI Suite AFU, consider starting with an AFU design example that implements some of the required functionality. Some examples designs and what they are offer are as follows:

- For an example of enabling direct memory access so the host can access DDR memory, review the direct memory access (DMA) AFU example on GitHub
- For an example of interrupt handling, review the oneAPI Accelerator Support Package (ASP) on GitHub.
- For an example MMD implementation, review the oneAPI Accelerator Support Package (ASP) on GitHub.

Related Information

- Direct memory access (DMA) AFU example on GitHub
- oneAPI accelerator support package (ASP) on GitHub
- Agilex 7 PCIe Attach OFS documentation
- Agilex 7 PCIe Attach OFS Workload Development Guide

2.6.6.3.8. Additional FPGA AI Suite SoC Design Example Software Components

The FPGA AI Suite SoC design example contains a software environment for the runtime flow.

The software environment for the supported FPGA development kits consists of the following components:

- Yocto build and runtime Linux environment
- Intel Distribution of OpenVINO toolkit Version 2024.6 LTS (Inference Engine, Heterogeneous plugin)
- OpenVINO Arm CPU plugin
- FPGA AI Suite runtime plugin
- MMD hardware library

The FPGA AI Suite SoC design example contains the source files, Makefiles, and scripts to cross compile all the software for the supported FPGA development kit. The Yocto SDK provides the cross compiler, and is the first component that must be built.

The machine learning network graph is compiled separately using the OpenVINO Model Converter and the FPGA AI Suite compiler (`dla_compiler`) command. When you compile the graph for the FPGA AI Suite SoC design example, ensure that you specify the `--foutput-format=open_vino_hetero` and `-o` *<path_to_file>*`/CompiledNetwork.bin` options.

The AOT file from the FPGA AI Suite compiler contains the compiled network partitions for FPGA and CPU devices along with the network weights. The network is compiled for a specific FPGA AI Suite architecture and batch size.

The SoC flow does not support the Just-In-Time (JIT) flow because Arm libraries are not available for the FPGA AI Suite compiler.

An architecture file (`.arch`) describes the FPGA AI Suite IP architecture to the compiler. You must specify the same architecture file to the FPGA AI Suite compiler and to the FPGA AI Suite design example build script (`dla_build_example_design.py`).

The runtime stack cannot program the FPGA with a bitstream. The bitstream must be built into the SD card (`.wic`) image that is used to program the flash card, as described in (Optional) Creating an SD Card Image (.wic) on page 101 and Writing the SoC Design Example SD Card Image (.wic) to an SD Card on page 109.

The runtime inference on the SoC FPGA device uses the OpenVINO Arm CPU plugin. To enable fallback to the OpenVINO Arm CPU plugin for graph layers that are not supported on the FPGA, the device flag must be set to `HETERO:FPGA,CPU` during the AOT compile step and when you run the `dla_benchmark` command.

In some cases, a layer might be supported by the FPGA even though the OpenVINO Arm CPU plugin does not support the layer. This support is handled by the HETERO plugin and the layer is executed on the FPGA as expected. As an example, 3D convolution layers are not supported by the OpenVINO Arm CPU plugin but still work properly provided that the `.arch` file used for the FPGA AI Suite IP configuration has enabled support for 3D convolutions.

Related Information

- Running the Graph Compiler on page 132
- Compiling a Graph on page 193
- Architecture Description File Parameters on page 173
- Compilation Options (dla_compiler Command Options) on page 208

3. Design Considerations Before You Begin

Developing an AI inference solution that uses Altera FPGA devices requires careful architectural planning that spans from your trained model to the final deployment of your AI solution.

These considerations ensure that you meet your system performance, power, resource utilization, cost, and feature requirements. They also help guide optimization efforts and help you to select the most appropriate development path for your application.

3.1. Consider an FPGA AI Suite Design Example as Starting Point

The FPGA AI Suite provides several design examples that demonstrate how to integrate the FPGA AI Suite IP with real hardware platforms. These examples serve as a foundation to evaluate features, prototype workflows, and understand runtime interactions.

To choose the most appropriate platform, evaluate your application architecture and system-level constraints by asking yourself the following questions:

- Do you want to offload inference to the FPGA from a host CPU (look-aside)?
- Do you require standalone (hostless) operation without DDR memory?
- Is your system embedded, with an onboard ARM core (SoC)?
- Are you targeting PCIe-attached cards, or SoCs with integrated HPS?
- Do you want to use prebuilt bitstreams or build custom ones with Quartus?

Review the following available platforms and associated design examples, along with the key hardware and software components involved to help you decide if an FPGA AI Suite design example is a good starting point for your design:

- PCIe Host on page 57
- Hostless DDR-Free on page 58
- Hostless JTAG on page 59
- SoC Host on page 59
- AI Video (SoC Host) on page 59
- Custom Platform on page 59

PCIe Host

Use Case: Server or workstation offloading inference to a PCIe-attached FPGA.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

Recommended boards:

- Terasic DE10-Agilex Development Board (DE10-Agilex-B2E2)
- Open FPGA Stack (OFS)-based boards:
  - ☐ Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
  - ☐ Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)
  - ☐ FPGA SmartNIC N6001-PL Platform (without Ethernet controller)

Key Characteristics:

- Supports look-aside model with host-driven control.
- Integrates with the Intel Distribution of OpenVINO toolkit (x86 host).
- Build scripts support architecture selection and optional bitstream regeneration.
- Designed for performance benchmarking and throughput-optimized inference.

Hostless DDR-Free

Use Case: Fully autonomous AI inference on FPGA, with no host processor and no external DDR memory.

Recommended boards:

- Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)

Key Characteristics:

- Inputs, weights, and configurations stored in on-chip RAM or MIF.
- No external DDR or runtime required.
- Data streaming and results are handled via direct hardware interfaces.
- Suitable for ultra-low-latency and minimal-footprint deployments.

DDR-Free Scenarios

DDR-Free architecture trades on-chip memory blocks with filter data access efficiency. Using DDR-Free architecture is beneficial in the following scenarios:

- When DDR bandwidth bottlenecks the system performance but on-chip memory is sufficient, storing filter and configuration data on-chip lessen the needs for data transfer between the on-chip memory and external memory.
- When the graph is reasonably-sized, its weights and biases fit on-chip memory, and a low latency or a high throughput is critical to your application. In this case, storing them to on-chip memory accelerates the access time and reduces per-layer latency.
- When the multilane feature is enabled, PE arrays consume data more frequently in parallel. Storing filter data on-chip and letting them be shared across lanes result in the least extra resource overhead instead of fetching from external memory.

DDR-Free Constraints

DDR-free mode imposes certain constraints:

- Memory Constraints: The architecture file must have sufficient on-chip memory to accommodate all graph parameters in the filter scratchpad. There must also be sufficient on-chip memory to store all intermediate surfaces in the stream buffer.

Hostless JTAG

Use Case: Direct control of FPGA-based inference through JTAG interface, typically in lab environments or tightly controlled edge systems.

Recommended boards:

- Agilex 3 FPGA C-Series Development Kit (DK-A3Y135BM16AEA)
- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)

Key Characteristics:

- External DDR is used for weights/features.
- Host communicates over JTAG to FPGA.
- Good for development, bring-up, or research workflows.

SoC Host

Use Case: Embedded AI inference using the FPGA hard processor system (HPS).

Recommended boards:

- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
- Agilex 7 FPGA I-Series Transceiver-SoC Development Kit (DK-SI-AGI027FC)
- Arria 10 SX SoC FPGA Development Kit (DK-SOC-10AS066S)

Key Characteristics:

- Supports CPU-offload model using OpenVINO ARM plugin on Linux.
- Uses Yocto-based builds to generate bootable SD card images.
- Offers two execution modes:
  - **M2M (Memory-to-Memory)**: Benchmark-style execution.
  - **S2M (Streaming-to-Memory)**: Demonstrates live streaming from CPU to FPGA.

AI Video (SoC Host)

Use Case: Embedded AI inference on video inputs using the FPGA hard processor system (HPS).

Recommended boards:

- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)

Key Characteristics:

Custom Platform

Use Case: Support for non-standard or production-specific platforms that are not directly covered by default design examples.

Examples:

- Carrier boards with custom pin maps or I/O.
- PCIe add-in cards with proprietary form factors.
- Edge or embedded systems with bespoke power or memory configurations.

Key Considerations:

- **Platform Definition:** Use a custom Open FPGA Stack (OFS) Platform Interface Manager (PIM) or modify an existing BSP.
- **Interface Integration:** Ensure compatibility with chosen I/O (PCIe, JTAG, streaming interfaces).
- **Bitstream Generation:** You must have a valid FPGA AI Suite license that allows custom compilation.
- **Toolchain Compatibility:** Align Quartus, BSP, and runtime versions with FPGA AI Suite requirements.
- **Software Stack:** Modify or extend the OpenVINO integration layer (for host-side) or use lightweight inference APIs if hostless.

This structured platform breakdown helps ensure you're targeting the appropriate flow for your system constraints. Once your platform is selected, you can follow the design example's instructions to build, modify, or integrate the FPGA AI Suite into your deployment pipeline.

3.2. Determine Your Performance Bottlenecks

The biggest determinant of the performance, power, and cost of your AI inference solution is the FPGA device that you select. Determine what portions of your design should be executed in software vs. hardware (that is, using the FPGA logic resources). Profiling your application gives you a preliminary estimation of where the bottlenecks in your application are.

3.3. Select Your FPGA Device

The FPGA AI Suite is designed to support a range of FPGA device families for your applications. The following Altera FPGA device families are supported:

- Cyclone® 10 GX
- Agilex 3
- Agilex 5
- Agilex 7
- Arria 10
- Stratix® 10

Device selection depends on:

- Model complexity and input selection (area resources)
- Memory bandwidth requirements
- Interface needs (PCIe, Ethernet, MIPI, HDMI)
- Power and thermal budget

4. Installing FPGA AI Suite

The FPGA AI Suite has two main use modes: software-only or with-FPGA-hardware. Thanks to the bit-accurate software emulation provided by the FPGA AI Suite tools, you can complete most of your AI application development without needing FPGA hardware.

You need FPGA AI Suite installation with FPGA hardware only if you want to run inference on the FPGA (maybe as part of testing your application) or if you want to run one of the FPGA AI Suite design examples.

Software-Only FPGA AI Suite Installation

With a software-only FPGA AI Suite installation, you have the following possible configurations: FPGA AI Suite standalone or FPGA AI Suite with Quartus Prime Pro Edition.

FPGA AI Suite Standalone Installation

With a standalone installation, you can run the FPGA AI Suite compiler to estimate the performance of your ML graph within the FPGA AI Suite IP, estimate the FPGA area required by your FPGA AI Suite IP, and optimize the FPGA AI Suite IP architecture for the resources available on your target FPGA device.

You can also use the FPGA AI Suite software emulation to evaluate the accuracy of your machine learning model against the architecture file for your FPGA AI Suite IP.

For an FPGA AI Suite standalone installation, you have the following installation options:

- (Recommended) Using the FPGA AI Suite Docker* Image on page 63
- Installing the FPGA AI Suite Compiler and IP Generation Tools on page 67

FPGA AI Suite Installation with Quartus Prime Pro Edition

The addition of Quartus Prime Pro Edition to the software stack provides all of the capabilities of the FPGA AI Suite standalone installation and adds the following capabilities:

- Integrate your FPGA AI Suite IP into your FPGA design by using Visual Designer Studio or Platform Designer.
- Generate bitstreams for your target FPGA device.

For an FPGA AI Suite installation with Quartus Prime Pro Edition, you have the following installation options:

- (Recommended) Using the FPGA AI Suite Docker* Image on page 63

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

- Installing the FPGA AI Suite Compiler and IP Generation Tools on page 67

  When you install FPGA AI Suite as part of this installation flow, you have two options for your FPGA AI Suite installation with Quartus Prime Pro Edition:

  - ☐ (Recommended) Install FPGA AI Suite (and Quartus Prime Pro Edition) with the Quartus Prime installer.
  - ☐ Install FPGA AI Suite with Linux system package management tools.

FPGA AI Suite Installation with FPGA Hardware

An FPGA AI Suite installation with FPGA hardware supports the full AI development cycle for FPGA devices including running inference on the FPGA device.

The software installation for an FPGA AI Suite installation with FPGA hardware is considerably more complex because you must consider both the FPGA AI Suite software requirements and the software requirements for your FPGA hardware.

The FPGA AI Suite design examples provide a good starting point for trying an FPGA AI Suite installation with FPGA hardware. The design examples have associated precompiled bitstreams that you can use to skip some of the device compilation steps.

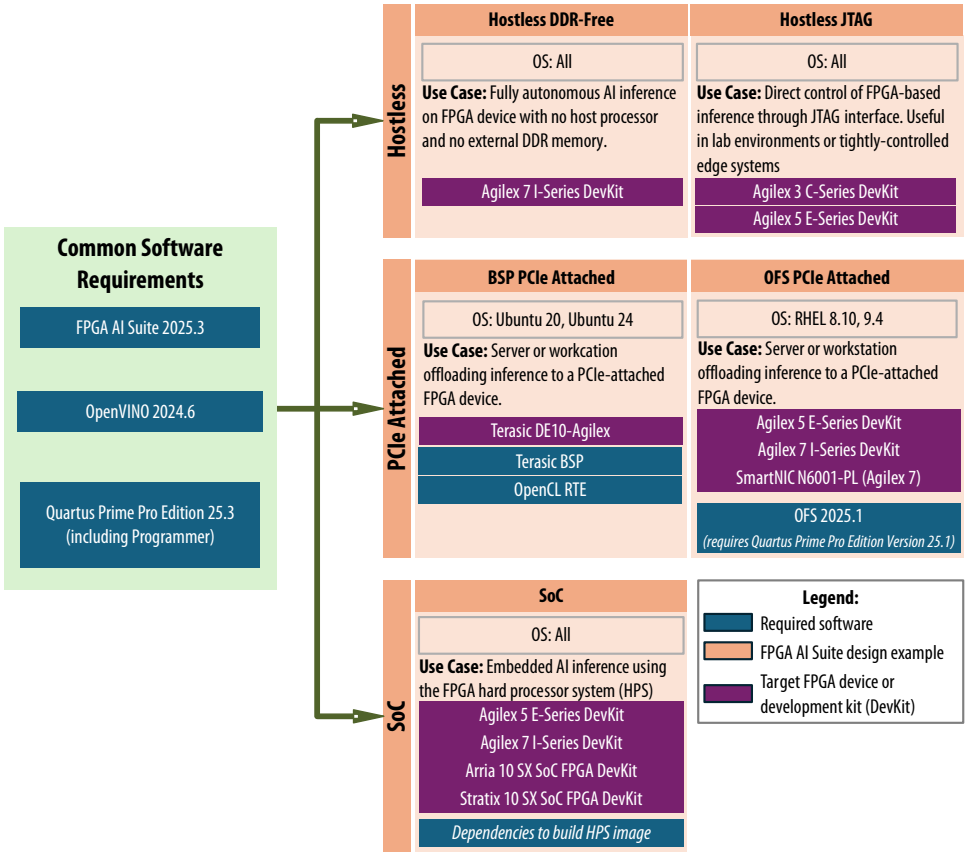
For software dependencies for the FPGA AI Suite design examples, refer to *Quartus Prime Pro Edition and Additional Software Requirements for FPGA AI Suite Design Example Boards* in Installing Quartus Prime Pro Edition Software on page 84.

For an FPGA AI Suite installation with FPGA hardware, complete these steps:

1. Installing the FPGA AI Suite Compiler and IP Generation Tools on page 67
2. Installing FPGA AI Suite Design Example Prerequisites on page 86

Figure 12. FPGA AI Suite Installation with Design Example Requirements Overview

The following diagram provides an brief overview of the FPGA AI Suite design examples and their hardware and software requirements:



Related Information

Downloading Precompiled Bitstreams and SD Card Images on page 86

4.1. Using the FPGA AI Suite Docker* Image

The FPGA AI Suite Docker* image is intended to run with a Docker client running on a Microsoft* Windows or Linux system. Docker clients on macOS* operating systems are not supported. This containerized version of FPGA AI Suite enables easy and quick access to the various tools in FPGA AI Suite.

Important: The FPGA AI Suite Docker image is updated only for major releases of FPGA AI Suite. For example, the most recent version of the FPGA AI Suite Docker image is the image for 2025.3. The FPGA AI Suite Docker image was not updated for Version 2025.3.1.

A larger part of the Docker image functionality does not require a physical FPGA board attached to the host machine. However, if your intention is to run hardware inference on an FPGA board, keep the following FPGA AI Suite Docker image limitations in mind:

- You cannot program an FPGA bitstream from within the container.
- The container cannot act as the host machine for the FPGA device.

All prerequisites to running inference, such as building FPGA bitstreams, are possible in the FPGA AI Suite Docker image. However, if you want to use an FPGA AI Suite design example within the container, you might need to install additional dependencies for the design example.

The quick-start tutorial demonstrates the ease of use and ease of deployment of a deep learning model to the edge. The tutorial presents the Memory-to-Memory (M2M) Variant Design on page 299 execution model that provides a `dla_benchmark` interface to the inference engine, similar to the PCIe design examples. It also introduces the Streaming-to-Memory (S2M) Variant Design on page 300 execution model. The S2M model uses the FPGA SoC ARM CPU as the streaming data source that streams data to a layout transform on the FPGA device. The S2M design example illustrates a recommended system architectures for any streaming source.

Related Information

SoC Design Example Prerequisites on page 99

4.1.1. FPGA AI Suite Docker Image Requirements

The FPGA AI Suite Docker images has the following hardware and software requirements:

Hardware Requirements

- Laptop or Desktop Computer
  - ☐ 16 GB RAM
  - ☐ 50 GB free storage

  The system must have an Internet connection without an active VPN or firewall. If you have a VPN or firewall, you might need to configure the Docker CLI to use proxies. For details, refer to https://docs.docker.com/engine/cli/proxy/.
- (Optional) An FPGA development kit supported by FPGA AI Suite. The development kit is required only if you want test running inference on the FPGA device. You do not need a development kit to run the FPGA AI Suite tools.

  Tip: Most of the steps in this document do not require FPGA hardware. Any steps that specifically require FPGA hardware like a development kit to complete are indicated in this document with an **[FPGA]** tag.

  The FPGA AI Suite Docker image supports the following development kits :
  - ☐ Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
  - ☐ Agilex 7 FPGA I-Series Transceiver-SoC Development Kit (DK-SI-AGI027FC)
  - ☐ Arria 10 SX SoC FPGA Development Kit (DK-SOC-10AS066S)

Software Requirements

- A Docker client, such as Docker Desktop or Docker Engine.

  You can get Docker Desktop at https://www.docker.com/products/docker-desktop/.

  For Docker Engine installation instructions, refer to https://docs.docker.com/engine/install/.

  Restriction: Docker clients on macOS operating systems are not supported.

- **[FPGA]** Quartus Prime Pro Edition Programmer and Tools Version 25.3.

  You can download this software from one of the following links:

  - ☐ https://www.altera.com/downloads/fpga-development-tools/quartus-prime-pro-edition-design-software-version-25-3-linux
  - ☐ https://www.altera.com/downloads/fpga-development-tools/quartus-prime-pro-edition-design-software-version-25-3-windows

  If you want to program the FPGA device, you must install Quartus Prime Pro Edition software on the host (not the Docker container).

4.1.2. Running the FPGA AI Suite Docker Container

If you have an FPGA development kit, ensure that the board is connected to power and that the UART/JTAG cables are connected to the host machine where you run the container.

The following FPGA AI Suite Docker images are available:

Table 5. Available FPGA AI Suite Docker Images

Image Name	Description and Pull Command	Use Case
FPGA AI Suite	An Ubuntu* 24 image with an FPGA AI Suite installation along with dependencies such as OpenVINO and system packages. Pull this package with the following command: `docker pull alterafpga/fpgaaisuite-v2025.3.1`	Use this image if you want run the FPGA AI Suite tools like the compiler or if you intend to perform emulator inference. You can also use this image if you intend to use the precompiled bitstreams.
FPGA AI Suite with Quartus Prime	An Ubuntu 24 image with an FPGA AI Suite installation along with dependencies such as OpenVINO and system packages and a Quartus Prime Pro Edition installation. Pull this package with the following command: `docker pull alterafpga/fpgaaisuite-quartus-v2025.3.1`	Use this image if you want run the FPGA AI Suite tools like the compiler and you need to compile bitstreams with Quartus Prime software.

Important: The Docker images have only the baseline dependencies available. Some of the FPGA AI Suite design examples have additional software dependencies that you can install into the container.

To run the FPGA AI Suite Docker image:

1. Start your Docker client, if it is not yet started.
2. Start a command prompt session.
3. In the command prompt session, run one of the following commands:

- For the FPGA AI Suite image, run the following command:

```
docker container run -it alterafpga/fpgaaisuite -v2025.3.1
```

- For the FPGA AI Suite with Quartus Prime image, run the following command:

```
docker container run -it --net=host alterafpga/fpgaaisuite-v2025.3.1
```

Tip: Use the `--name` option of the `docker container run` command to provide a custom name for your FPGA AI Suite container. For details, refer to the `docker container run` documentation at the DockerDocs web site.

When the container starts, you should see the following output:

```
[setupvars.sh] OpenVINO environment initialized
export COREDLA_VERSION=1
export COREDLA_ROOT='/opt/altera/fpga_ai_suite/ubuntu/dla'
```

To start using the FPGA AI Suite Docker container, try the Containerized FPGA AI Suite Docker Image Quick-Start Tutorial on page 66.

4.1.3. Containerized FPGA AI Suite Docker Image Quick-Start Tutorial

The FPGA AI Suite Docker images contain all the necessary dependencies to follow the FPGA AI Suite Quick Start Tutorial on page 130. However, there are some prerequisite steps and container-specific restrictions for following the tutorial in a containerized environment:

1. Setting up Python virtual environment for the OpenVINO Model Zoo.

   Before you can follow the instructions in Preparing OpenVINO Model Zoo for the PCIe Design Example on page 130, you must set up a Python virtual environment in your FPGA AI Suite Docker container.

   To set up the Python virtual environment in your FPGA AI Suite Docker container, run the following commands from within the container:

   ```
   mkdir ~/build-openvino-dev && cd ~/build-openvino-dev

   python3.12 -m venv openvino_env

   source openvino_env/bin/activate

   python -m pip install --upgrade pip

   pip install "openvino-dev[<frameworks>]==2024.6.0"
   ```

   Where *<frameworks>* is a comma-separated list of the deep learning frameworks that you want your OpenVINO development tools to support. The following values are supported: `caffe`, `kaldi`, `onnx`, `pytorch`, `tensorflow`, `tensorflow2`.

   For the Quick Start Tutorial, only `tensorflow` is needed.

   For more details about installing OpenVINO, refer to Installing OpenVINO Toolkit on page 79.

2. Setting Up Prerequisites for the Agilex 7 PCIe-Attach Design Example

The FPGA AI Suite Docker images do not support using the Docker container as the host machine for the FPGA, so running inference within the container for the Agilex 7 PCIe-Attach design example is unsupported. You can skip this step in the Quick Start Tutorial or set up the prerequisites of a different design example.

3. Programming the FPGA devices.

   Because the Docker container does not support programming the bitstream to an FPGA device from within the container, you cannot follow the instructions in Programming the FPGA Device on page 134.

   To program an FPGA device from within the Docker container:

   a. Move the `.sof` bitstream file from the container to your host machine with the `docker cp` command. For example,

   ```
   docker cp  container_id:/path/in/container/bitstream.sof \
              /destination/in/host/ \
              /bitstream.sof
   ```

   b. Program the FPGA device with a command similar to the following:

   ```
   quartus_pgm -c 1 -m jtag -o P;/path/to/bitstream.sof@1
   ```

4. Building an FPGA Bitstream for the PCIe design example

   To follow the instructions in Building an FPGA Bitstream for the PCIe Design Example on page 137, you must use the FPGA AI Suite with Quartus Prime Docker image. You must set the `LM_LICENSE_FILE` environment variable to point to your Quartus Prime license:

   ```
   export LM_LICENSE_FILE=...
   ```

   For the Agilex 7 PCIe design example, install the `patch` utility the container:

   ```
   apt install patch
   ```

5. If you are using the Terasic DE10-Agilex Development Board, you must install the Terasic Kernel Driver Software Prerequisites as outlined in Additional Software Prerequisites for the FPGA AI Suite PCIe Design Example for Agilex 7 Devices on page 89.

4.2. Installing the FPGA AI Suite Compiler and IP Generation Tools

Fastpath: You can avoid the need to perform the steps in this section by using the FPGA AI Suite Docker container. Use the installation instructions here only if you have more complex requirements or development environment restrictions. For more information about the FPGA AI Suite Docker container, refer to Using the FPGA AI Suite Docker* Image on page 63.

To install the FPGA AI Suite compiler and IP generation tools:

Tip: The following steps outline the installation process. You can follow the remaining parts of this section in sequence without returning to this topic to complete your installation.

1. Review Supported FPGA Families on page 68 to ensure that your target FPGA device is supported.
2. Ensure that your system meets the operating system requirements as outlined in FPGA AI Suite Operating System Prerequisites on page 68.
3. Install FPGA AI Suite.

   Tip: If you want to install FPGA AI Suite in a location other than the default directory, follow the instructions in Installing the FPGA AI Suite Into an Alternative Location on page 78.
4. Install OpenVINO Toolkit.
5. (Optional) Install Quartus Prime Pro Edition.

   Important: Quartus Prime Pro Edition is required for the FPGA AI Suite design examples. Otherwise, Quartus Prime Pro Edition is an optional component.
6. Set the required environment variables.
7. Finalize your installation by verifying that FPGA AI Suite commands run correctly.
8. (Optional) Download support packages with precompiled bitstreams and SoC SD Card images.

4.2.1. Supported FPGA Families

The FPGA AI Suite supports the following FPGA device families:

- Agilex 3
- Agilex 5
- Agilex 7
- Arria 10
- Cyclone 10 GX
- Stratix 10

The FPGA AI Suite is compatible with Quartus Prime Pro Edition Versions 24.3 - 25.3.

The FPGA AI Suite design examples require specific versions of Quartus Prime Pro Edition. For details, refer to Quartus Prime Pro Edition Requirements for FPGA AI Suite Design Example Boards

4.2.2. FPGA AI Suite Operating System Prerequisites

FPGA AI Suite supports installation into a containerized environment or into an operating system environment as follows:

Table 6. Supported Operating Environment

Environment	Notes
Docker Container	The FPGA AI Suite Docker container is intended to run with a Docker client running on a Microsoft Windows or Linux system. The FPGA AI Suite Docker container provides the fastest and simplest FPGA AI Suite installation and configuration to, so you can get your development started quickly. For information about using the FPGA AI Suite Docker container, refer to Using the FPGA AI Suite Docker* Image on page 63.
Ubuntu operating system: • Version 20.04 LTS(7) • Version 22.04 LTS • Version 24.04 LTS	Ubuntu operating systems are supported only natively. Ubuntu in a WSL 2 environment is not supported. For installation in a Windows environment, use either the FPGA AI Suite Docker container or a native Windows installation.
Red Hat* Enterprise Linux* operating system: • Version 8.10 • Version 9.4	—
Windows 11 operating system	Windows installations of FPGA AI Suite have the following limitations: • FPGA AI Suite can be installed only through the Quartus Prime installer. • Only the FPGA AI Suite compiler and IP generation tools are supported. • For FPGA AI Suite design examples, bitstream compilation is supported only for design examples that are purely Quartus Prime projects as follows: ☐ Hostless DDR-Free design example ☐ JTAG design example • The `dla_benchmark` command and the FPGA AI Suite OpenVINO runtime are not supported on Windows.

Tip: For most of your initial FPGA AI Suite development requirements, the Docker image is ideal. Install FPGA AI Suite on full operating system environments only if you have more complex requirements or development environment restrictions.

For the prerequisites for your FPGA AI Suite operating environment, refer to the following sections:

- Docker image requirements
- Red Hat Enterprise Linux Operating System Requirements on page 70
- Ubuntu Operating System Requirements on page 71
- Windows 11 Operating System Requirements on page 73

The FPGA AI Suite design examples might also have their own operating system requirements. For details, refer to Installing FPGA AI Suite Design Example Prerequisites on page 86.

(7) Standard support for Ubuntu Version 20.04 has ended. As a result, some repositories required for installation might not be available.

Important: The PCIe design examples require specific versions operating systems. This is required only to use the specific PCIe design example listed. For the operating system requirements for the PCIe design examples, refer to the following table:

Table 7. PCIe Design Example Operating System Requirements

Design Example	Operating System Requirement
PCIe design example for Agilex 7 devices	Ubuntu 20.04.5[7] or Ubuntu 24.02.2

For details about the operating system requirements for the PCIe design example, refer to PCIe Design Example Operating System Prerequisites on page 88.

4.2.2.1. Red Hat Enterprise Linux Operating System Requirements

FPGA AI Suite has the following requirements for Red Hat Enterprise Linux operating systems:

- You must have the following packages installed:

Table 8. Required Red Hat Enterprise Linux Packages

Red Hat Enterprise Linux Version	Required Packages
8.10	numactl-devel, cmake, git, curl, graphviz, gcc, gcc-c++, redhat-lsb, tbb-devel, gflags-devel, boost-devel, ninja-build
9.4	numactl-devel, cmake, git, curl, graphviz, gcc, gcc-c++, tbb-devel, gflags-devel, boost-devel, ninja-build *Tip:* If you receive an error message when installing the gflag-devel package similar to `Error: Unable to find a match: gflags-devel`, run the following command and then try installing the gflag-devel package again: `sudo dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm`

- You must have the CodeReady Linux Builder repository enabled.

  The FPGA AI Suite RPM package has dependencies that are in this repository. If you do not have this repository enabled, refer to Enabling the CodeReady Linux Builder Repository on page 70.

- CMake 3.31.6 or later (required by the FPGA AI Suite design examples).

  Check your version of CMake and ensure it is compatible. If you need to replace your system CMake version with version 3.31.6, refer to Replacing Your Operating System CMake Version on page 71.

- Python 3.12 (required by OpenVINO toolkit version that is required by FPGA AI Suite).

  Confirm that you have Python 3.12 on your system by running the `python3.12 -v` command. If this command returns an error, add Python 3.12 by following the instructions in Installing the Required Python Version on page 71.

Enabling the CodeReady Linux Builder Repository

The FPGA AI Suite RPM package has dependencies that are in the CodeReady Linux Builder Repository. Ensure that you have this repository enabled before you install FPGA AI Suite RPM package.

To enable the CodeReady Linux Builder repository, run the following command:

- Red Hat Enterprise Linux 8.10:

```
sudo subscription-manager repos --enable codeready-builder-for-rhel-8-x86_64-rpms
```

- Red Hat Enterprise Linux 9.4

```
sudo subscription-manager repos --enable codeready-builder-for-rhel-9-x86_64-rpms
```

Replacing Your Operating System CMake Version

To replace your operating system version of CMake with version 3.31.6, run the following commands:

```
cd ~/Downloads

wget https://github.com/Kitware/CMake/releases/download/v3.31.6/cmake-3.31.6-linux-x86_64.sh

sudo chmod 777 cmake-3.31.6-linux-x86_64.sh

sudo ./cmake-3.31.6-linux-x86_64.sh --prefix=/usr/local --exclude-subdir

./cmake-3.31.6-linux-x86_64.sh --prefix=$HOME/.local --exclude-subdir

export PATH=$HOME/.local/bin:$PATH

cmake –version

sudo rm -rf cmake-3.31.6-linux-x86_64.sh
```

Installing the Required Python Version

To install the required version of Python:

1. Install Python:

```
sudo dnf install python3.12
```

2. Verify your Python installation with the following command:

```
python3.12 -V
```

This command should return output indicating the version that you are using. For example:

```
Python 3.12.10
```

4.2.2.2. Ubuntu Operating System Requirements

FPGA AI Suite has the following requirements for Ubuntu operating systems:

- You must have the following packages installed:

Table 9. Required Ubuntu Packages

Ubuntu Version	Required Packages
20.04[8]	libnuma-dev, cmake, git, curl, graphviz, unzip, lsb, libtbb-dev, libgflags-dev, libboost-all-dev, ninja-build
22.04	libnuma-dev, cmake, git, curl, graphviz, unzip, lsb, libtbb-dev, libgflags-dev, libboost-all-dev, ninja-build
24.04	libnuma-dev, cmake, git, curl, graphviz, unzip, libtbb-dev, libgflags-dev, libboost-all-dev, ninja-build

- CMake 3.31.6 or later (required by the FPGA AI Suite design examples).

  Check your version of CMake and ensure it is compatible. If you need to update your version of CMake, refer to Updating CMake on page 72.
- Python 3.12 (required by OpenVINO toolkit version that is required by FPGA AI Suite).

  Confirm that you have Python 3.12 on your system by running the `python3.12 -v` command. If this command returns an error, add Python 3.12 by following the instructions in Installing the Required Python Version on page 73.

Updating CMake

To update your version of CMake to version 3.31.6:

1. If you do not have the `wget` package installed, install it with the following commands:

```
sudo apt update

sudo apt install wget
```

2. Download the CMake 3.31.6 packages for your operating system version by running the following commands:

   Ubuntu 20.04[8]

```
wget https://apt.kitware.com/ubuntu/pool/main/c/cmake/\
cmake_3.31.6-0kitware1ubuntu20.04.1_amd64.deb

wget https://apt.kitware.com/ubuntu/pool/main/c/cmake/\
cmake-data_3.31.6-0kitware1ubuntu20.04.1_all.deb
```

```
wget https://apt.kitware.com/ubuntu/pool/main/c/cmake/\
cmake_3.31.6-0kitware1ubuntu22.04.1_amd64.deb

wget https://apt.kitware.com/ubuntu/pool/main/c/cmake/\
cmake-data_3.31.6-0kitware1ubuntu22.04.1_all.deb
```

   Ubuntu 24.04

```
wget https://apt.kitware.com/ubuntu/pool/main/c/cmake/\
cmake_3.31.6-0kitware1ubuntu24.04.1_amd64.deb

wget https://apt.kitware.com/ubuntu/pool/main/c/cmake/\
cmake-data_3.31.6-0kitware1ubuntu24.04.1_all.deb
```

[8] Standard support for Ubuntu Version 20.04 has ended. As a result, some repositories required for installation might not be available.

3. Install the downloaded CMake packages with the following commands:

```
sudo dpkg -i cmake-data_3.31.6-0kitware1ubuntu*_all.deb

sudo dpkg -i cmake_3.31.6-0kitware1ubuntu*_amd64.deb
```

Installing the Required Python Version

To install the required version of Python:

1. Install Python 3.12:
   - Ubuntu 20.04[8]/Ubuntu 22.04

     ```
     sudo apt update && sudo apt upgrade -y

     sudo add-apt-repository ppa:deadsnakes/ppa -y

     sudo apt install python3.12 python3.12-venv libpython3.12 -y
     ```

   - Ubuntu 24.04

     ```
     sudo apt update && sudo apt upgrade -y

     sudo apt install python3.12 python3.12-venv libpython3.12 -y
     ```

2. Verify your Python installation with the following command:

   ```
   python3.12 -V
   ```

   This command should return output indicating the version that you are using. For example:

   ```
   Python 3.12.10
   ```

4.2.2.3. Windows 11 Operating System Requirements

To view the dot graph output from the FPGA AI Suite compiler, you must have the Graphviz graph visualization software installed. You can obtain the Graphviz software at the following URL: https://graphviz.org.

4.2.3. Installing FPGA AI Suite

You can install FPGA AI Suite in the following ways, depending on your operating system.

For most of your initial FPGA AI Suite development requirements, the Docker image is ideal. Install FPGA AI Suite on full operating system environments only if you have more complex requirements or development environment restrictions.

Table 10. Installation Options by Operating System

Operating System	Installation Options
Red Hat Enterprise Linux	• Using a Docker container. For details, refer to Using the FPGA AI Suite Docker* Image on page 63. • Using the Quartus Prime installer. For details, refer to Installing the FPGA AI Suite Using the Quartus Prime Installer on page 75. • Using the Linux package manager. For details, refer to Installing the FPGA AI Suite with System Package Management Tools on page 76.
Ubuntu Linux	• Using a Docker container. For details, refer to Using the FPGA AI Suite Docker* Image on page 63. • Using the Quartus Prime installer. For details, refer to Installing the FPGA AI Suite Using the Quartus Prime Installer on page 75. • Using the Linux package manager. For details, refer to Installing the FPGA AI Suite with System Package Management Tools on page 76.
Microsoft Windows	• Using a Docker container. For details, refer to Using the FPGA AI Suite Docker* Image on page 63. • Using the Quartus Prime installer. For details, refer to Installing the FPGA AI Suite Using the Quartus Prime Installer on page 75.

4.2.3.1. FPGA AI Suite Directory Structure

After you install the FPGA AI Suite you have the following directories in your *<ai_suite_root>*, which is the value assigned to the `COREDLA_ROOT` environment variable:

Table 11. FPGA AI Suite Directory Structure

Directory	Description
`$COREDLA_ROOT/`	Base directory
`$COREDLA_ROOT/bin`	Tool executables (`dla_compiler` command)
`$COREDLA_ROOT/lib`	Libraries (compiler API, OpenVINO plugins, etc)
`$COREDLA_ROOT/inc`	Public API header files
`$COREDLA_ROOT/util`	Additional runtime modules
`$COREDLA_ROOT/thirdparty`	Third-party packages used by the runtime
`$COREDLA_ROOT/area_model`	Data files
`$COREDLA_ROOT/demo`	Sample image files and bitstreams This directory exists only if you have downloaded the precompiled bitstreams and SD card images. For details, refer to Downloading Precompiled Bitstreams and SD Card Images on page 86.
`$COREDLA_ROOT/example_architectures`	Architecture files, each which defines a specific IP parameterization
`$COREDLA_ROOT/example_ip_cores`	Generated IP cores corresponding to the example architectures

continued...

Directory	Description
$COREDLA_ROOT/fpga	Source files used by the IP generator
$COREDLA_ROOT/licenses	License information for FPGA AI Suite and included libraries
$COREDLA_ROOT/platform	BSP source directory
$COREDLA_ROOT/runtime	Directory with OpenVINO demos and low-level runtime for FPGA interfacing
$COREDLA_ROOT/dla_plugin	Directory for OpenVINO heterogeneous compilation and inference integration with the FPGA

4.2.3.2. Installing the FPGA AI Suite Using the FPGA AI Suite Docker Image

The FPGA AI Suite Docker image is intended to run with a Docker client running on a Microsoft Windows or Linux system. Docker clients on macOS operating systems are not supported. This containerized version of FPGA AI Suite enables easy and quick access to the various tools in FPGA AI Suite.

Important: The FPGA AI Suite Docker image is updated only for major releases of FPGA AI Suite. For example, the most recent version of the FPGA AI Suite Docker image is the image for 2025.3. The FPGA AI Suite Docker image was not updated for Version 2025.3.1.

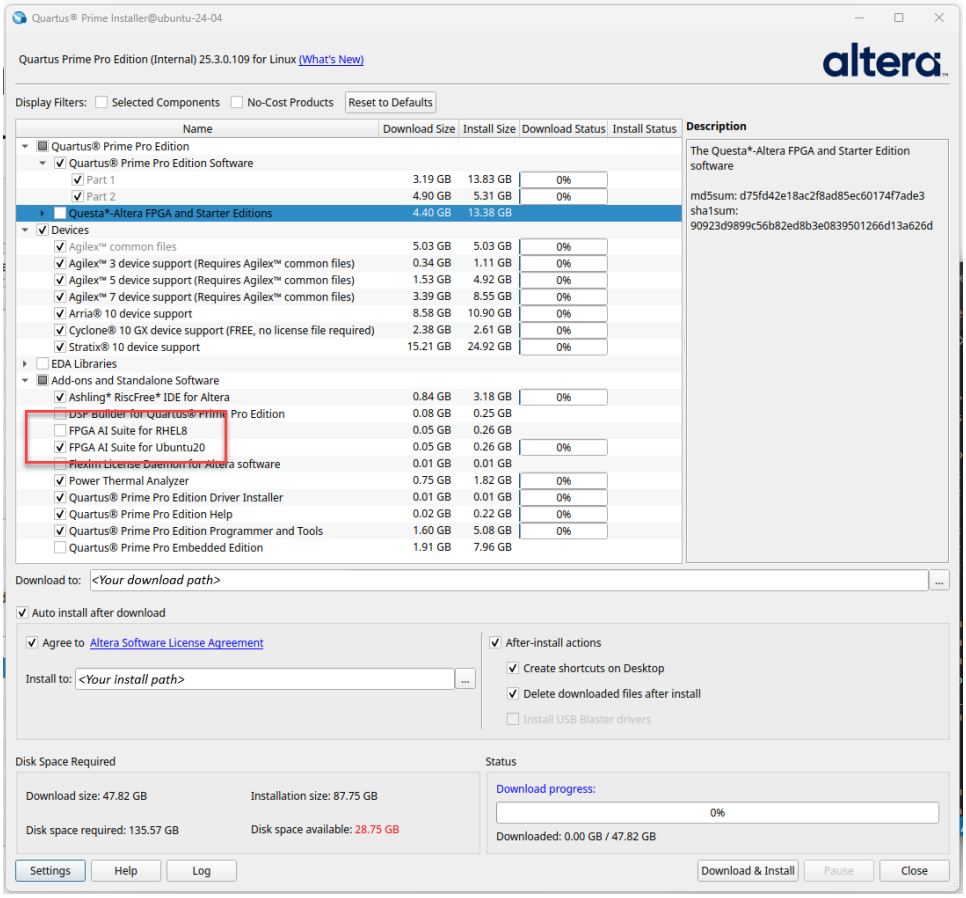
For more details about the FPGA AI Suite Docker image, including requirements, installation instructions, and how to run the container, refer to Using the FPGA AI Suite Docker* Image on page 63.

4.2.3.3. Installing the FPGA AI Suite Using the Quartus Prime Installer

The FPGA AI Suite is available through the Quartus Prime installer. To installFPGA AI Suite through the Quartus Prime installer, select the FPGA AI Suite checkbox in the installer interface. The FPGA AI Suite is installed in the Quartus Prime installation path specified in the installer. If you cannot see the checkbox for FPGA AI Suite, expand the "**Add-ons and Standalone Software** category.

Important: The Quartus Prime Installer provides only major releases of FPGA AI Suite. For example, the Quartus Prime Pro Edition installer for Versions 25.3 and 25.3.1 installer installs FPGA AI Suite Version 2025.3.

Figure 13. Quartus Prime Installer with FPGA AI Suite



If you are installing FPGA AI Suite on a Linux system, the Quartus Prime Installer provides installation options for the Linux distributions that FPGA AI Suite supports. Select the distribution that matches the operating system of your installation target system.

By default, the Quartus Prime installer installs the Quartus Prime software and the FPGA AI Suite into a location within the user's home directory.

Next Step

After installing FPGA AI Suite, continue your installation with the steps in Installing OpenVINO Toolkit on page 79.

4.2.3.4. Installing the FPGA AI Suite with System Package Management Tools

The FPGA AI Suite is provided as a package that you can install using your system package management tools. The default location for the installation is `/opt/altera`. If you want to install FPGA AI Suite in a different location, follow the instructions in Installing the FPGA AI Suite Into an Alternative Location on page 78.

To install the FPGA AI Suite with system package management tools, follow the instructions for your operating system:

- **Red Hat Enterprise Linux**: Follow the instructions in Installing FPGA AI Suite on Red Hat Enterprise Linux Operating Systems on page 77.
- **Ubuntu Linux**: Follow the instructions in Installing FPGA AI Suite on Ubuntu Operating Systems on page 77.

Installing FPGA AI Suite on Red Hat Enterprise Linux Operating Systems

For Red Hat Enterprise Linux, the FPGA AI Suite is provided as an RPM package that you can install with the `dnf` command. The `dnf` command automatically installs several dependencies.

To install the package into the default location in `/opt/altera/`:

1. Ensure that your system meets the operating system requirements in Red Hat Enterprise Linux Operating System Requirements on page 70.
2. Download the FPGA AI Suite 2025.3.1 installation package for your operating system from the following Altera Download Center URL:

   https://www.altera.com/downloads/add-on-development-tools/altera-fpga-ai-suite-current
3. Locate the downloaded FPGA AI Suite `.rpm` file and run the following command:

   ```
   sudo dnf install <full path to fpga-ai-suite.rpm>
   ```
4. (Optional) Add the FPGA AI Suite environment configuration script to your `.bashrc` file to automatically configure your FPGA AI Suite environment when you log on. Add the following line to your `.bashrc` file:

   ```
   source /opt/altera/fpga_ai_suite_2025.3/dla/setupvars.sh
   ```

Installing FPGA AI Suite on Ubuntu Operating Systems

For Ubuntu operating systems, the FPGA AI Suite is provided as a package that you can install with the `apt` command. The `apt` command automatically installs several dependencies.

To install the package into the default location in `/opt/altera/`:

1. Ensure that your system meets the operating system requirements in Ubuntu Operating System Requirements on page 71.
2. Download the FPGA AI Suite 2025.3.1 installation package for your operating system from the following AlteraAltera Download Center URL:

   https://www.altera.com/downloads/add-on-development-tools/altera-fpga-ai-suite-current

   Ubuntu 22.04/24.04:* Download the package file to `/tmp`.
3. Locate the downloaded FPGA AI Suite `.deb` file and run the following commands:

- Ubuntu 20.04[9]/22.04:

```
sudo apt install --reinstall libappstream4

sudo apt update

sudo apt install <full path to fpga-ai-suite.deb>
```

- Ubuntu 24.04:

```
sudo apt update

sudo apt install <full path to fpga-ai-suite.deb>
```

4. (Optional) Add the FPGA AI Suite environment configuration script to your `.bashrc` file to automatically configure your FPGA AI Suite environment when you log on. Add the following line to your `.bashrc` file:

```
source /opt/altera/fpga_ai_suite_2025.3/dla/setupvars.sh
```

Next Step

After installing FPGA AI Suite, continue your installation with the steps in Installing OpenVINO Toolkit on page 79.

4.2.3.4.1. Installing the FPGA AI Suite Into an Alternative Location

If you prefer to install the FPGA AI Suite into an alternative location, you can use the following commands:

- Red Hat Enterprise Linux 8.10:

```
sudo dnf install numactl-devel cmake git curl graphviz ninja-build\
          gcc gcc-c++ redhat-lsb tbb-devel gflags-devel boost-devel

rpm -i --prefix=<full path to alternative location> \
          <full path to fpga-ai-suite.rpm>
```

- Red Hat Enterprise Linux 9.4:

```
sudo dnf install numactl-devel cmake git curl graphviz gcc gcc-c++ \
          tbb-devel gflags-devel boost-devel

rpm -i --prefix=<full path to alternative location> \
          <full path to fpga-ai-suite.rpm>
```

- Ubuntu 20.04Standard support for Ubuntu Version 20.04 has ended. As a result, some repositories required for installation might not be available.[10]:

```
sudo apt update

sudo apt install libnuma-dev cmake git curl graphviz unzip libtbb-dev \
                 lsb libboost-all-dev libgflags-dev ninja-build

dpkg-deb -x <full path to fpga-ai-suite deb> /tmp/ai-suite
```

(9) Standard support for Ubuntu Version 20.04 has ended. As a result, some repositories required for installation might not be available.

(10) Standard support for Ubuntu Version 20.04 has ended. As a result, some repositories required for installation might not be available.

```
mv /tmp/ai-suite/opt/altera/fpga_ai_suite_2025.3.1 <full path to alternative
location>
```

- Ubuntu 22.04:

```
sudo apt update

sudo apt install libnuma-dev cmake git curl graphviz unzip libtbb-dev lsb \
                 libboost-all-dev libgflags-dev ninja-build

dpkg-deb -x <full path to fpga-ai-suite deb> /tmp/ai-suite

mv /tmp/ai-suite/opt/altera/fpga_ai_suite_2025.3.1 <full path to alternative
location>
```

- Ubuntu 24.04:

```
sudo apt update

sudo apt install libnuma-dev cmake git curl graphviz unzip libtbb-dev \
                 libboost-all-dev libgflags-dev ninja-build

dpkg-deb -x <full path to fpga-ai-suite deb> /tmp/ai-suite

mv /tmp/ai-suite/opt/altera/fpga_ai_suite_2025.3.1 <full path to alternative
location>
```

4.2.4. Installing OpenVINO Toolkit

The FPGA AI Suite requires the OpenVINO Toolkit 2024.6.

To prepare OpenVINO Toolkit for FPGA AI Suite:

1. Install the required Toolkit components:
   - **Linux operating systems:** Installing Required OpenVINO Toolkit Components on Linux Systems on page 80.
   - **Windows operating systems:** Installing Required OpenVINO Toolkit Components on Windows Systems on page 83.

The installer might issue an error message that indicates the OpenCL is not installed and is needed for GPU operation of OpenVINO. You can safely ignore this error message. The FPGA AI Suite does not require OpenVINO GPU support.

Required Python Version

OpenVINO Toolkit 2024.6 supports only Python 3.12.

Verify your Python version with the following command:

```
python3.12 -V
```

This command should return output indicating the version that you are using. For example:

```
Python 3.12.10
```

If you do not have the required Python version installed, follow the instructions in the operating system requirements for your environment:

- Red Hat Enterprise Linux Operating System Requirements on page 70
- Ubuntu Operating System Requirements on page 71

Installing Required OpenVINO Toolkit Components on Linux Systems

FPGA AI Suite requires you to install the OpenVINO Runtime and OpenVINO Development Tools separately.

The location where you install the OpenVINO Runtime is often referred to as *<openvino_installdir>* in FPGA AI Suite documentation. Typically, this location is `/opt/intel` or `/home/<user>/intel`.

To install the required OpenVINO Toolkit components:

1. Create a Python virtual environment and start it:
   a. Create a new directory and switch to it:
   ```
   mkdir ~/build-openvino-dev && cd ~/build-openvino-dev
   ```
   b. Create a Python virtual environment:
   ```
   python3.12 -m venv openvino_env

   source openvino_env/bin/activate
   ```
   c. Confirm that you are in the new Python virtual environment:
   ```
   python -V
   ```
   This command should return output indicating the version that you are using. For example:
   ```
   Python 3.12.10
   ```
2. Install the OpenVINO 2024.6 Runtime:
   a. If you do not have an `opt/intel` folder, create one as follows:
   ```
   sudo mkdir /opt/intel
   ```
   b. If you do not have a `~/Downloads` folder, create one as follows:
   ```
   mkdir ~/Downloads
   ```
   c. Go to your `Downloads` folder:
   ```
   cd ~/Downloads
   ```
   d. Download the OpenVINO 2024.6 Runtime archive file for your system, extract the files, and move them to the `/opt/intel` folder as follows:
   - **Red Hat Enterprise Linux***:
   ```
   curl -L https://storage.openvinotoolkit.org/\
     repositories/openvino/packages/2024.6/linux/\
     l_openvino_toolkit_rhel8_2024.6.0.17404.4c0f47d2335_x86_64.tgz \
     --output openvino_2024.6.0.tgz

   tar -xf openvino_2024.6.0.tgz
   ```

```
sudo mv \
  l_openvino_toolkit_rhel8_2024.6.0.17404.4c0f47d2335_x86_64 \
  /opt/intel/openvino_2024.6.0
```

- **Ubuntu 20**:

```
curl -L https://storage.openvinotoolkit.org/\
  repositories/openvino/packages/2024.6/linux/\
  l_openvino_toolkit_ubuntu20_2024.6.0.17404.4c0f47d2335_x86_64.tgz \
  --output openvino_2024.6.0.tgz

tar -xf openvino_2024.6.0.tgz

sudo mv \
  l_openvino_toolkit_ubuntu20_2024.6.0.17404.4c0f47d2335_x86_64 \
  /opt/intel/openvino_2024.6.0
```

- **Ubuntu 22**:

```
curl -L https://storage.openvinotoolkit.org/\
  repositories/openvino/packages/2024.6/linux/\
  l_openvino_toolkit_ubuntu22_2024.6.0.17404.4c0f47d2335_x86_64.tgz \
  --output openvino_2024.6.0.tgz

tar -xf openvino_2024.6.0.tgz

sudo mv \
  l_openvino_toolkit_ubuntu22_2024.6.0.17404.4c0f47d2335_x86_64 \
  /opt/intel/openvino_2024.6.0
```

- **Ubuntu 24**:

```
curl -L https://storage.openvinotoolkit.org/\
  repositories/openvino/packages/2024.6/linux/\
  l_openvino_toolkit_ubuntu24_2024.6.0.17404.4c0f47d2335_x86_64.tgz \
  --output openvino_2024.6.0.tgz

tar -xf openvino_2024.6.0.tgz

sudo mv \
  l_openvino_toolkit_ubuntu24_2024.6.0.17404.4c0f47d2335_x86_64 \
  /opt/intel/openvino_2024.6.0
```

e. Install system dependencies required by the OpenVINO Runtime by running the provided script:

- **Red Hat Enterprise Linux**:

```
cd /opt/intel/openvino_2024.6.0/install_dependencies/

sed -i -e 's|http://mirror.centos.org/centos/8-stream/AppStream|' \
'https://repo.almalinux.org/almalinux/8/AppStream|' \
./install_openvino_dependencies.sh

sed -i -e 's|yum install "\$iopt"|' \
'yum install \${iopt:+\$iopt}|' \
./install_openvino_dependencies.sh

sudo -E ./install_openvino_dependencies.sh
```

- **Ubuntu Linux**:

```
cd /opt/intel/openvino_2024.6.0/install_dependencies/

sudo -E ./install_openvino_dependencies.sh
```

f. Create a symbolic link:

```
cd /opt/intel

sudo ln -s openvino_2024.6.0 openvino_2024
```

Important: If you have already installed a previous release of OpenVINO 2024, a symbolic link to the `openvino_2024` folder might already exist. Unlink the previous link with the `sudo unlink openvino_2024` command, and then re-run the commands in this step.

g. Configure the OpenVINO Runtime environment variables with the following command:

```
source /opt/intel/openvino_2024/setupvars.sh
```

3. Install the OpenVINO Development Tools:

a. Upgrade the Python `pip` command:

```
python -m pip install --upgrade pip
```

b. Install the `openvino-dev` package:

```
pip install "openvino-dev[<frameworks>]==2024.6.0"
```

Where *<frameworks>* is a comma-separated list of the deep learning frameworks that you want your OpenVINO development tools to support. The following values are supported: `caffe`, `kaldi`, `onnx`, `pytorch`, `tensorflow`, `tensorflow2`.

For example, to install the OpenVINO development tools to support the Caffe, PyTorch*, and TensorFlow* frameworks, run the following command:

```
pip install "openvino-dev[caffe, pytorch, tensorflow]==2024.6.0"
```

Restriction: PyTorch 2.6.0 is not supported by the OpenVINO 2024.6.0 Open Model Zoo tools. Change your installed PyTorch version to 2.5.1 with the following commands:

```
pip freeze | grep torch

pip uninstall torch==2.6.0 torchvision==0.21.0

pip install torch==2.5.1 torchvision==0.20.1 \
    --index-url https://download.pytorch.org/whl/cu118
```

Tip: This command might produce the following error message:

```
ERROR: tensorflow 2.13.1 has requirement numpy<=1.24.3,>=1.22, but
you'll have numpy 1.24.4 which is incompatible
```

This error message is pip-specific indicating that some packages in the local Python environment do not conform to OpenVINO development tools requirements. You can safely ignore this warning message. The `openvino-dev` package is still successfully installed.

4. Install OpenCV as follows:

- On Red Hat Enterprise Linux systems, OpenCV is available from the CodeReady Linux Builder repository. You enabled the repository when you met the requirements outlined in Red Hat Enterprise Linux Operating System Requirements on page 70. Install OpenCV from the CodeReady Linux Build repository with the following commands:

```
sudo yum update

sudo yum install opencv opencv-devel

pip3 install opencv-python
```

- On Ubuntu Linux systems, OpenCV is available in the default repositories. Install OpenCV with the following commands:

```
sudo apt update

sudo apt install libopencv-dev python3-opencv

pip install opencv-python
```

If you need a version of OpenCV optimized for your hardware, you might need to build OpenCV from source. For instructions, refer to "Approach #2: Build OpenCV against specific version of OpenVINO" section of *OpenCV usage with OpenVINO*.

You must configure the OpenVINO environment each time you want to use the OpenVINO Toolkit. For convenience, you can add the configuration to your `.bashrc` file. For details, refer to Setting Required Environment Variables on page 85.

Installing Required OpenVINO Toolkit Components on Windows Systems

If you plan to run the FPGA AI Suite on a Windows operating system, install the required OpenVINO Toolkit components by following the instructions provided in ***Install OpenVINO Runtime on Windows from an Archive File*** at the OpenVINO documentation web site at the following URL:

https://docs.openvino.ai/2024/get-started/install-openvino/install-openvino-archive-windows.html

You must configure the OpenVINO environment each time you want to use the OpenVINO Toolkit. For convenience, you can add the configuration to your `.bashrc` file. For details, refer to Setting Required Environment Variables on page 85.

Next Step

After installing OpenVINO toolkit, continue your installation with the steps in Installing Quartus Prime Pro Edition Software on page 84.

Related Information

Installing Intel Distribution of OpenVINO toolkit

4.2.5. Installing Quartus Prime Pro Edition Software

The FPGA AI Suite is compatible with Quartus Prime Pro Edition for Linux Versions 24.3 - 25.3. FPGA AI Suite has limited support for Quartus Prime Pro Edition for Windows.

Tip: When designing a solution with the FPGA AI Suite, use the latest supported version of Quartus Prime Pro Edition.

Ensure that the `QUARTUS_ROOTDIR` environment variable is set correctly. For example, you can add the following to your `~/.bashrc` file to set the target directory appropriately:

```
export QUARTUS_ROOTDIR="<quartus_install_path>/quartus/"
```

Important: The FPGA boards required for the FPGA AI Suite design examples require specific versions of Quartus Prime Pro Edition. This requirement applies only to the design examples and does not apply to the FPGA AI Suite IP and compiler. For the Quartus Prime Pro Edition requirements for the design examples, refer to the following table:

Table 12. Quartus Prime Pro Edition and Additional Software Requirements for FPGA AI Suite Design Example Boards

Design Example Board	Quartus Prime Pro Edition Requirement	Additional Software Requirements
Agilex 3 FPGA C-Series Development Kit (DK-A3Y135BM16AEA)	Quartus Prime Pro Edition Version 25.3	Open FPGA Stack (OFS) 2025.1-1
Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)	OFS PCIe-Attach Design Example	
	Quartus Prime Pro Edition Version 25.3	Open FPGA Stack (OFS) 2025.1-1
	Hostless JTAG and SoC Design Examples	
	Quartus Prime Pro Edition Version 25.1	None
Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES) (PCIe-Attached)	Quartus Prime Pro Edition Version 24.3	Open FPGA Stack (OFS) 2025.1-1
Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES) (Hostless)	Quartus Prime Pro Edition Version 25.3	None
Agilex 7 FPGA I-Series Transceiver-SoC Development Kit (DK-SI-AGI027FC)	Quartus Prime Pro Edition Version 25.3	Ashling* RiscFree* IDE for Altera
Arria 10 SX SoC FPGA Development Kit (DK-SOC-10AS066S)	Quartus Prime Pro Edition Version 25.3	Ashling RiscFree IDE for Altera
FPGA SmartNIC N6001-PL Platform (without Ethernet controller) (PCIe-Attached)	Quartus Prime Pro Edition Version 25.1	Open FPGA Stack (OFS) 2025.1-1
Terasic DE10-Agilex Development Board (DE10-Agilex-B2E2)	Quartus Prime Pro Edition Version 22.4 or later. This document assumes that Version 25.3 is used.	None

Next Step

After installing Quartus Prime Pro Edition, continue your installation with Setting Required Environment Variables on page 85.

Related Information

Quartus Prime Pro Edition download site

4.2.6. Setting Required Environment Variables

Set the OpenVINO toolkit and FPGA AI Suite environment variables for your operating system as follows:

- Linux systems:

  Set the OpenVINO toolkit and FPGA AI Suite environment variables by running the following commands from a bash-style shell (do not use `/bin/dash` or `/bin/tcsh`) or by adding the commands to your `.bashrc` file. If you did not install OpenVINO toolkit into the default location, adjust the third command to use your OpenVINO toolkit location:

  ```
  source ~/build-openvino-dev/openvino_env/bin/activate

  unset python_version

  source /opt/intel/openvino_2024/setupvars.sh

  source /opt/altera/fpga_ai_suite_2025.3.1/dla/setupvars.sh
  ```

- Windows systems:

  Set the OpenVINO toolkit and FPGA AI Suite environment variables from a command prompt with the following commands. If you did not install OpenVINO toolkit into the default location, adjust the first command appropriately:

  ```
  "C:\Program Files (x86)\Intel\openvino_2024\setupvars.bat"

  "C:\altera_pro\2025.3.1\fpga_ai_suite\dla\setupvars.bat"
  ```

Tip: If you have multiple versions of FPGA AI Suite installed on your system, you can use `source` commands to switch between them. To switch, source the environment variable setup scripts for the version of FPGA AI Suite that you want to run. You must ensure that you also have the required OpenVINO toolkit version available.

For example, if you want to run FPGA AI Suite 2023.3, run the following commands:

```
source ~/build-openvino-dev/openvino_env/bin/activate

unset python_version

source /opt/intel/openvino_2022/setupvars.sh

source /opt/intel/fpga_ai_suite_2023.3/dla/setupvars.sh
```

In some older versions of FPGA AI Suite, the environment variable setting script is called `init_env.sh` and might be in a slightly different path (`dla/bin/init_env.sh`).

Next Step

After setting the required environment variables, complete your installation with the steps in Finalizing Your FPGA AI Suite Installation on page 86.

4.2.7. Finalizing Your FPGA AI Suite Installation

To finalize your FPGA AI Suite Installation, verify your installation by running a `dla_compiler` command.

You can verify your installation by running the following command:

```
dla_compiler --fanalyze-area --march $COREDLA_ROOT/*/AGX7_Generic.arch
```

The `dla_compiler` command prepares deep learning models for consumption by the FPGA AI Suite IP, produces performance estimates, and produces area (or FPGA resource usage) estimates. This example command produces an area estimate for the IP configuration stored in the file `AGX7_Generic.arch`.

If this command runs successfully, your FPGA AI Suite installation was successful.

4.2.8. Downloading Precompiled Bitstreams and SD Card Images

You can download precompiled bitstreams and SD card images for the FPGA AI Suite design examples from the **Bitstreams and SD Card Images** tab of the FPGA AI Suite download page at the following Altera Download Center URL:

https://www.altera.com/downloads/add-on-development-tools/altera-fpga-ai-suite-current

Download the appropriate package type for your installation (`.deb` or `.rpm`).

For the `.deb` and `.rpm` packages, the files are copied into `/opt/altera_fpga_ai_suite_2025.3.1/dla/demo`.

You can also get precompiled bitstreams for the FPGA AI Suite design examples from the following Altera FPGA Developer site URL:

https://github.com/altera-fpga/agilex-ed-ai-suite/releases

These files are not required to use FPGA AI Suite. The files are generated from the architectures that are available in the `example_architectures` directory of an FPGA AI Suite installation.

4.3. Installing FPGA AI Suite Design Example Prerequisites

The FPGA AI Suite design examples have additional hardware and software prerequisites. Ensure that your system meets these additional requirements before trying to run the design examples.

Prepackaged bitstreams are available for the design examples. If the prepackaged bitstreams are installed, they are installed in `demo/bitstreams/`.

The prepackaged bitstreams are available from the **Bitstreams and SD Card Images** tab of the FPGA AI Suite download page:

https://www.altera.com/downloads/add-on-development-tools/altera-fpga-ai-suite-current

To build design example bitstreams, you must have a license that permits bitstream generation for the IP, and have the correct version of Quartus Prime software installed. Use the `dla_build_example_design` utility to create a bitstream.

For details about this command, the steps it performs, and advanced command options, refer to The dla_build_example_design Command on page 39

4.3.1. Creating a Working Directory

All the FPGA AI Suite design examples require a writeable working directory that contains a copy of the example runtime and the demonstration files. Create this working directory before starting up with any of the FPGA AI Suite design examples.

If you want to use `~/coredla_work/` as your working directory, run the following commands:

```
mkdir ~/coredla_work

cd ~/coredla_work

source dla_init_local_directory.sh
```

The `dla_init_local_directory.sh` script does the following tasks:

- Sets `COREDLA_WORK` environment variable to point at working directory.
- Creates a locally writeable copy of the `runtime` directory.
- Creates a locally writeable copy of the `demo` directory.

To re-enter this work directory after opening a new shell, run the following commands:

```
source /opt/intel/openvino_2024/setupvars.sh

source /opt/altera/fpga_ai_suite_2025.3.1/dla/setupvars.sh

cd ~/coredla_work

source ./coredla_work.sh
```

4.3.2. PCIe Design Example Prerequisites

To install the FPGA AI Suite PCIe Design Example prerequisites:

1. Install the FPGA AI Suite compiler and IP generation tools.
2. Create a working directory.
3. Ensure that your system meets the PCIe design example hardware prerequisites.
4. Ensure that your system meets the PCIe design example operating system requirements.

5. Ensure that your system meets the PCIe design example software requirements.
6. For Terasic DE10-Agilex Development Board users, ensure that you meet the requirements outlined in Additional Software Prerequisites for the FPGA AI Suite PCIe Design Example for Agilex 7 Devices on page 89.
7. Build the FPGA AI Suite design example runtime.

4.3.2.1. PCIe Design Example Hardware Prerequisites

The minimum hardware requirement for the design examples are as follows:

- Desktop computer or server with at least 1 free PCIe slot
- Minimum RAM: 48GB
- Minimum storage: 100GB SSD

Supported Boards

The PCIe-based design examples requires the following board:

- Terasic DE10-Agilex Development Board

  This board is required by the PCIe design example for Agilex 7 devices.

4.3.2.2. PCIe Design Example Operating System Prerequisites

The operating system prerequisites for the PCIe design example depend on the target FPGA device.

PCIe Design Example for Agilex 7 Operating System Prerequisites

The PCIe design example for Agilex 7 devices requires Ubuntu 20.04 LTS[(11)] or 24.04 LTS.

For installation instructions, refer to https://ubuntu.com/tutorials/install-ubuntu-desktop.

4.3.2.3. PCIe Design Example Software Prerequisites

The PCIe design examples require the following software:

- CMake 3.31 or higher
- `libnuma-dev` package

The `apt` tool attempts to install these prerequisites automatically when you install the FPGA AI Suite following the instructions in Installing the FPGA AI Suite with System Package Management Tools on page 76.

Depending on the target board for the PCIe design example, you must meet additional software prerequisites:

- Additional Software Prerequisites for the FPGA AI Suite PCIe Design Example for Agilex 7 Devices on page 89

(11) Standard support for Ubuntu Version 20.04 has ended. As a result, some repositories required for installation might not be available.

4.3.2.3.1. Additional Software Prerequisites for the FPGA AI Suite PCIe Design Example for Agilex 7 Devices

The kernel driver for the Terasic BSP must be installed according to instructions provided by Terasic. Follow the instructions that follow, or contact your Terasic representative for additional details.

Important: If you follow the Terasic instructions, the Terasic instructions (on page 35) say to override the $LD_LIBRARY_PATH environment variable. Do not override the environment variable. Append paths to the $LD_LIBRARY_PATH environment variable instead.

A copy of the verified Terasic kernel driver is distributed as a separate file: `DE10_Agilex_revC_linux_BSP_21.2.zip`. Obtain this file from your Terasic representative.

Ubuntu Kernel Prerequisites

The kernel driver is tested on the Ubuntu 20.04 LTS and Ubuntu 24.04 LTS distributions with kernel version 5.15.0. You can confirm the kernel version of your Ubuntu system with the following command:

```
uname -r
```

Terasic Kernel Driver Software Prerequisites

To install the Terasic kernel driver, install the following software dependencies:

- Quartus Prime Pro Edition Programmer and Tools Version 22.4 or later.

  The instructions in this document assume that Quartus Prime Pro Edition Version 25.3 is used.

  Download the software from the **Individual Files** tab at the following URL: https://www.altera.com/downloads/fpga-development-tools/quartus-prime-pro-current.
- FPGA RTE for OpenCL Version 21.2.

  Download the software from **RTE** tab at the following URL: https://www.altera.com/downloads/add-development-tools/fpga-sdk-opencl-pro-edition-software-version-21-2.
- Altera FPGA Download Cable II (formerly USB-Blaster II) Driver for Linux.

  Configuration steps are available the following URL: https://www.intel.com/content/www/us/en/support/programmable/support-resources/download/dri-usb-b-lnx.html

After you have installed the Quartus Programmer, set up the required environment variable using the following command:

```
export PATH="$PATH:"<path to Quartus Programmer installation folder>/qprogrammer/quartus/bin
```

Your system must recognize the Altera FPGA Download Cable II to install the Terasic kernel driver and program your board.

To verify the connection of the Altera FPGA Download Cable II, issue the `jtagconfig` command. You should see the DE10-Agilex device listed. An example of the expected output is as follows:

```
$ jtagconfig
1) DE10-Agilex [1-11.1]
C24A0DD   AGFB014R24(A|B)
```

The list index number ("1" in the example) is also the cable number, which can be used to reprogram the board in Terasic Driver Installation on page 90.

Terasic Driver Installation

After you meet the Terasic kernel driver software prerequisites, complete the following steps:

1. Set up the environment with the following command:

   ```
   source <full path to opencl rte installation>/init_opencl.sh
   ```

2. Unzip the board support package in the current directory and set up the following environment variables (The value `$PWD` in the commands that follow must correspond to the directory in which the BSP packages was unzipped):

   ```
   tmp1=$PWD/DE10_Agilex_revC_linux_BSP_21.2/de10_agilex
   export AOCL_BOARD_PACKAGE_ROOT=$tmp1
   tmp1=$ALTERAOCLSDKROOT/host/linux64/lib
   tmp2=$AOCL_BOARD_PACKAGE_ROOT/linux64/lib
   export LD_LIBRARY_PATH=$tmp1:$tmp2:$LD_LIBRARY_PATH
   ```

3. Install the driver with the following command:

   ```
   aocl install
   ```

4. Rescan the PCIe bus or reboot the system to program the board. To verify that the driver bonds to the card, program the testing bitstream to the board with the following command:

   ```
   quartus_pgm -c <cable_number> -m jtag \
     -o "p;$AOCL_BOARD_PACKAGE_ROOT/bringup/B2E2_8GBx4/top.sof"
   ```

5. Perform a soft reboot of your host system for the system to recognize the Terasic DE10-Agilex Development Board:

   ```
   sudo reboot
   ```

   You can verify that the board is recognized by the host system with the following command:

   ```
   lspci | grep Altera
   ```

   If you do not see the board information when scanning the PCIe bus, try rebooting your system or reinstalling the board. In some cases, you might need to move the card to a different PCIe slot.

For more details and information about the Terasic kernel driver, refer to the Terasic OpenCL BSP Guide, *DE10-Agilex_OpenCL_21.2.pdf*, in the kernel driver package.

Ubuntu Kernel Update Requirements

If you update your Ubuntu kernel, you must reinstall the Terasic kernel driver.

Bitstream Generation Requirements

If you plan to use only the precompiled bitstreams supplied with the FPGA AI Suite, then no further steps are required.

If you plan to generate bitstreams corresponding to custom FPGA AI Suite IP architectures for the PCIe design example for Agilex 7 devices, then you must install Quartus Prime Pro Edition Version 25.3 and setup your `PATH` environment variable.

For Quartus Prime Pro Edition Version 25.3 prerequisites and installation instructions, refer to *Altera FPGA Software Installation and Licensing*.

Download the Quartus Prime Pro Edition software from the following URL: https://www.altera.com/downloads/fpga-development-tools/quartus-prime-pro-current

Bitstream generation works with Quartus Prime Pro Edition Version 25.3, even though the Terasic driver requires FPGA RTE for OpenCL Version 21.2.

You must also add the Platform Designer command (`qsys`) to your `PATH` environment variable. Add the following commands to your `.bashrc` file, and adjust the paths depending on your installation locations, if needed:

```
export PATH="$PATH:"~/altera_pro/25.3/qsys/bin/
export PATH="$PATH:"~/altera_pro/25.3/quartus/bin/
```

Reboot Requirements

Any time that you reboot your system after powering down, you must reprogram the board with the following command:

```
quartus_pgm -c <cable_number> -m jtag \
     -o "p;$AOCL_BOARD_PACKAGE_ROOT/bringup/B2E2_8GBx4/top.sof"
```

After you successfully install the board, continue by installing the compiler and IP generation tool as described in Installing the FPGA AI Suite Compiler and IP Generation Tools on page 67.

4.3.2.4. Building the FPGA AI Suite PCIe Design Example Runtime

The FPGA AI Suite PCIe-based Design Example `runtime` directory contains the source code for the OpenVINO plugins and customized versions of the following OpenVINO programs:

- `dla_benchmark`
- `classification_sample_async`
- `object_detection_demo_yolov3_async`
- `segmentation_demo`

The CMake tool manages the overall build flow to build the FPGA AI Suite runtime plugin.

4.3.2.4.1. FPGA AI Suite PCIe Design Example CMake Targets

The top level CMake build target is the FPGA AI Suite runtime plugin shared library, `libcoreDLARuntimePlugin.so`. It will not be built if the target is the software reference. Details on how to target one of the design example boards or the software emulation are specified in FPGA AI Suite PCIe Design Example Build Options on page 92. The source files used to build the `libcoreDLARuntimePlugin.so` target are located under the following directories:

- `runtime/plugin/src/`
- `runtime/coredla_device/src/`

The flow also builds additional targets as dependencies for the top-level target. The most significant additional targets are as follows:

- The Input and Output Layout Transform library, `libdliaPluginIOTransformations.a`. The sources for this target are under `runtime/plugin/io_transformations/`.

4.3.2.4.2. FPGA AI Suite PCIe Design Example Build Options

The `runtime` folder in the design example package contains a script to build the runtime called `build_runtime.sh`.

Issue the following command to run the script:

```
./build_runtime.sh <command_line_options>
```

Where <*command_line_options*> are defined in the following table:

Table 13. Command Line Options for the `build_runtime.sh` Script

Command	Description
`-h | --help`	Show usage details
`--cmake_debug`	Call cmake with a debug flag
`--verbosity=<number>`	Large numbers add some extra verbosity
`--build_dir=<path>`	Directory where the runtime build should be placed
`--disable_jit`	If this flag is specified, then the runtime will only support the Ahead of Time mode. The runtime will not link to the precompiled compiler libraries. Use this mode when trying to compile the runtime on an unsupported operating system.
`--build_demo`	Adds several OpenVINO demo applications to the runtime build. The demo applications are in subdirectories of the `runtime/directory`.
`--target_de10_agilex`	Target the Terasic DE10-Agilex Development Board.
`--target_emulation`	Target the software emulator model. Specify this option to build the runtime without a board installed. Inference requests are executed by the software emulator model of the FPGA AI Suite IP.
`--aot_splitter_example`	Builds the AOT splitter example utility for the selected target (Terasic DE10-Agilex Development Board).

continued...

Command	Description
	This option builds an AOT file for a model, splits the AOT file into its constituent components (weights, overlay instructions, etc), and the builds a small utility that loads the model and a single image onto the target FPGA board without using OpenVINO. You must set the `$AOT_SPLITTER_EXAMPLE_MODEL` and `$AOT_SPLITTER_EXAMPLE_INPUT` environment variables correctly. For details, refer to FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility Example Application on page 345.

The FPGA AI Suite runtime plugin is built in release mode by default. To enable debug mode, you must specify the `--cmake_debug` option of the script command.

The `-no_make` option skips the final call to the `make` command. You can make this call manually instead.

FPGA AI Suite hardware is compiled to include one or more IP instances, with the same architecture for all instances. Each instance accesses data from a unique bank of DDR:

- The Terasic DE10-Agilex Development Board has four DDR banks and supports up to four instances.

The runtime automatically adapts to the correct number of instances.

If the FPGA AI Suite runtime uses two or more instances, then the image batches are divided between the instances to execute two or more batches in parallel on the FPGA device.

4.3.3. OFS for PCIe-Attach Design Example Prerequisites

To install the FPGA AI Suite Open FPGA Stack (OFS) for PCIe-attach design example prerequisites:

1. Install the FPGA AI Suite compiler and IP generation tools.
2. Create a working directory.
3. Ensure that your system meets the hardware prerequisites.

   If you are using the Agilex 7 FPGA I-Series Development Kit, ensure your board meets the requirements in Additional Agilex 7 FPGA I-Series Development Kit Configuration on page 94.
4. Ensure that your system meets the Open FPGA Stack (OFS) requirements.
5. Build the OFS for PCIe-attache design example runtime.

OFS for PCIe-Attach Design Example Hardware Prerequisites

The FPGA AI Suite OFS for PCIe-attach design example supports PCIe Attach OFS on the following boards:

- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
- Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)
- FPGA SmartNIC N6001-PL Platform (without Ethernet controller)

Additional Agilex 7 FPGA I-Series Development Kit Configuration

For the Agilex 7 FPGA I-Series Development Kit, the development kit provides a single 16 GB DIMM that you must replace with two 8 GB DIMMs in the board DIMM Sockets.

This design example was developed and tested to work with the following DIMMs:

- Micron MTA8ATF1G64AZ-2G6E1
- Hynix HMA81GU6JJR8N-VK

Open FPGA Stack (OFS) Requirements

For Open FPGA Stack (OFS) support, ensure that you have completed the OFS installation and configuration for your board, including the Open Programmable Acceleration Engine (OPAE) and Device Feature List (DFL) as outlined in the following documents:

- *Getting Started Guide: Open FPGA Stack for Agilex 5 FPGAs Targeting the Agilex 5 FPGA E-Series 065B Modular Development Kit*
- *Getting Started Guide: Open FPGA Stack for Agilex 7 FPGAs Targeting the Agilex 7 FPGA I-Series Development Kit*

Additionally, you might need additional environment configuration such as development permissions such as those provided in the `setup_persmissions.sh` script provided by the oneAPI Accelerator Support Package (ASP).

Related Information

- Agilex PCIe Attach OFS documentation
- Agilex 5 PCIe Attach OFS Workload Development Guide
- Agilex 7 PCIe Attach OFS Workload Development Guide

4.3.3.1. Building the FPGA AI Suite OFS for PCIe Attach Design Example Runtime

The FPGA AI Suite OFS for PCIe Attach design example `runtime` directory contains the source code for the OpenVINO plugins and the `dla_benchmark` program. The CMake tool manages the overall build flow to build the FPGA AI Suite runtime plugin.

4.3.3.1.1. OFS for PCIe Attach Design Example CMake Targets

The top level CMake build target is the FPGA AI Suite runtime plugin shared library, `libcoreDLARuntimePlugin.so`. It will not be built if the target is the software reference. Details on how to target one of the design example boards or the software emulation are specified in OFS for PCIe Attach Design Example Build Options on page 95. The source files used to build the `libcoreDLARuntimePlugin.so` target are located under the following directories:

- `runtime/plugin/src/`
- `runtime/coredla_device/src/`

The flow also builds additional targets as dependencies for the top-level target. The most significant additional targets are as follows:

- The Input and Output Layout Transform library, `libdliaPluginIOTransformations.a`. The sources for this target are under `runtime/plugin/io_transformations/`.

4.3.3.1.2. OFS for PCIe Attach Design Example Build Options

To build the runtime for the OFS for PCIe Attach design example:

1. Ensure that you have created a working directory as described in Creating a Working Directory on page 87.
2. Ensure that the `OPAE_SDK_ROOT` environment variable is set in your build environment. For example, `export OPAE_SDK_ROOT=/usr/`.
3. Run one of the following sets of commands to build the runtime, depending on your PCIe Attach OFS board:
   - Agilex 5 FPGA E-Series 065B Modular Development Kit

     ```
     cd $COREDLA_WORK/runtime

     ./build_runtime.sh  -target_agx5_e_mdk
     ```

   - Agilex 7 FPGA I-Series Development Kit

     ```
     cd $COREDLA_WORK/runtime

     ./build_runtime.sh -target_agx7_i_dk
     ```

   - FPGA SmartNIC N6001-PL Platform

     ```
     cd $COREDLA_WORK/runtime

     ./build_runtime.sh -target_agx7_n6001
     ```

   For other `build_runtime.sh` options, refer to FPGA AI Suite PCIe Design Example Build Options on page 92.

FPGA AI Suite hardware is compiled to include one or more IP instances, with the same architecture for all instances. Each instance accesses data from a unique bank of DDR.

The Agilex 7 FPGA I-Series Development Kit and FPGA SmartNIC N6001-PL Platform both have four DDR banks (two onboard and two DIMM slots) and support up to four FPGA AI Suite IP instances. Install the required DIMMs into each slot that is used. For example, to support four FPGA AI Suite IP instances, you must have both DIMM slots fitted with 8 GB DIMMs. To support two FPGA AI Suite IP instances, you must have the first DIMM slot fitted with an 8GB DIMM.

The four DDR banks are ordered as follows:

1. Onboard DDR 0
2. DIMM slot 0
3. Onboard DDR 1
4. DIMM slot 1

Each DIMM slot can support two FPGA AI Suite IP instances.

The runtime automatically adapts to the correct number of instances.

If the FPGA AI Suite runtime uses two or more instances, then the image batches are divided between the instances to execute two or more batches in parallel on the FPGA device.

4.3.4. Hostless DDR-Free Design Example Prerequisites

To install the FPGA AI Suite Hostless DDR-Free Design Example prerequisites:

1. Install the FPGA AI Suite compiler and IP generation tools.
2. Create a working directory.
3. Ensure that you have a validated version of Quartus Prime Pro Edition software installed.

   The DDR-free design example is validated for use only with the Quartus Prime Pro Edition versions: Version 24.3, Version 25.1 , and Version 25.3.
4. Ensure that you have the required hardware.
5. Ensure that you have the required software.

4.3.4.1. Hostless DDR-Free Design Example Hardware Requirements

This design example requires the following hardware:

- Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)
- Altera FPGA Download Cable

4.3.4.2. Hostless DDR-Free Design Example Software Requirements

This design example requires the following software:

- FPGA AI Suite
- Quartus Prime Programmer (either standalone or as part of Quartus Prime Design Suite).
- Quartus Prime System Console (either standalone or as part of Quartus Prime Design Suite).

Ensure that all the binaries included in the Quartus Prime Design Suite are added to your `$PATH` environment variable so that they can be called from any location.

When you have met these prerequisites, validate that the development kit is connected to the JTAG interface by using the `jtagconfig` utility provided by the Quartus Prime Design Suite. A successful confirmation of the JTAG connection looks like the following example output:

```
$ jtagconfig
1) AGI FPGA Development Kit [1-7.2]
 034BB0DD AGIB027R29A(.|B|C|R0|R1|R3)/..
 020D10DD VTAP10
```

4.3.5. Hostless JTAG Design Example Prerequisites

To install the FPGA AI Suite hostless JTAG design example prerequisites:

1. Install the FPGA AI Suite compiler and IP generation tools.
2. Create a working directory.
3. Ensure that your system meets the hostless JTAG design example hardware prerequisites.
4. Ensure that your system meets the hostless JTAG design example software prerequisites.
5. Build the hostless JTAG design example runtime.

4.3.5.1. Hostless JTAG Design Example Hardware Requirements

The JTAG design example requires one of the following development kits: has the following hardware requirement:

- Agilex 3 FPGA C-Series Development Kit (DK-A3Y135BM16AEA)
- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)

Enable JTAG programming for the development kit as follows:

1. Enable JTAG programming on the development kit board as follows:

Agilex 3 FPGA C-Series Development Kit	Set the board switches SW4[1:4] to [OFF, OFF, OFF, OFF].
Agilex 5 FPGA E-Series 065B Modular Development Kit	Set the board switches SW4[1:2] to [OFF, OFF].

2. On the host computer, ensure that the necessary USB port rules and permissions are enabled by creating a `/etc/udev/rules.d/51-altera-usb-blaster.rules` file with the following content:

```
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6001",
MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6002",
MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6003",
MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6010",
MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6810",
MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="0403", ATTR{idProduct}=="6001",
MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="0403", ATTR{idProduct}=="6002",
MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="0403", ATTR{idProduct}=="6003",
MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="0403", ATTR{idProduct}=="6010",
MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="0403", ATTR{idProduct}=="6810",
MODE="0666"
```

3. Change the port permission of the JTAG-USB connection to `0666`.

   Use the `lsusb` command to determine the bus and device numbers for adjusting permissions.

For example, the command `lsusb` outputs the following bus and device path for devices on the development kit,

```
Bus 002 Device 006: ID 09fb:6010 Altera
Bus 002 Device 007: ID 0403:6010 Future Technology Devices International,
Ltd FT2232C/D/H Dual UART/FIFO IC
```

With this information, adjust the permissions with the following commands:

```
sudo chmod 0666 /dev/bus/usb/002/006

sudo chmod 0666 /dev/bus/usb/002/007
```

4.3.5.2. Hostless JTAG Design Example Software Requirements

You should install the FPGA AI Suiteand OpenVINO according to the steps in the Installing the FPGA AI Suite Compiler and IP Generation Tools on page 67.

To generate the bitstream files for this design example, you require the following components from Quartus Prime Pro Edition Version 25.3 or later:

- Quartus Prime software
- Agilex 3 C-Series or Agilex 5 E-Series device support, depending on your target development kit.

For host-FPGA device communication, and to configure the target development kit, you require the following components from Quartus Prime Pro Edition Version 25.3 or later:

- Quartus Prime Programmer
- Quartus Prime System Console

4.3.5.3. Building the FPGA AI Suite Hostless JTAG Design Example Runtime

Before you can run inference on the design example, build the FPGA AI Suite runtime using the provided `build_runtime.sh` script as follows:

1. Ensure that you have a working directory created as outlined in Creating a Working Directory on page 87.
2. Run the following commands:

```
cd $COREDLA_WORK

source $COREDLA_ROOT/bin/dla_init_local_directory.sh

cd runtime

./build_runtime.sh -target_system_console
```

A successful build of the runtime creates the following objects:

- *<runtime-directory>*/build_Release/libcoreDlaRuntimePlugin.so

  A runtime library that has the APIs for the host to access the FPGA AI Suite IP's registers and memories in the design example.
- *<runtime-directory>*/build_Release/system_console_script.tcl

  A script that the runtime invokes via the Quartus Prime System Console to set up the JTAG services to access the memories and CSR registers in the design example.
- *<runtime-directory>*/build_Release/dla_benchmark/dla_benchmark

  The dla_benchmark application.

If you run into the following error, check if /sbin appears near the start of the list of paths in your PATH environment variable:

```
Imported target "Boost::filesystem" includes non-existent path "/include"
```

Your PATH environment variable does not need to include /sbin for the JTAG design example.

4.3.6. SoC Design Example Prerequisites

To install the FPGA AI Suite SoC design example prerequisites:

1. Install the FPGA AI Suite compiler and IP generation tools.
2. Create a working directory.
3. Ensure that you have one of the supported development kits.
4. Build a bitstream and flash card image for your development kit.

4.3.6.1. SoC Design Example Development Kit Prerequisites

The SoC design example requires one of the following development kits:

- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)

  This development kit features and Agilex 5 E-Series devices (OPN: A5ED065BB32AE6S-R0).

  For more details about this development kit, refer to the following URL: https://www.altera.com/products/devkit/a1jui0000061qabmaa/agilex-5-fpga-and-soc-e-series-modular-development-kit-es.

- Agilex 7 FPGA I-Series Transceiver-SoC Development Kit (DK-SI-AGI027FC)

  This development kit features an Agilex 7 I-Series SoC device with 4 F-Tiles (OPN: AGIB027R31B1E1V).

  Important: The FPGA AI Suite requires DDR4 memory with an x8 or higher component data width. The RAM device provided with the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit provides only a x4 component data width. For more details and recommended RAM modules, refer to Agilex 7 FPGA I-Series Transceiver-SoC Development Kit SoC Design Example Hardware Requirements on page 100.

  For more details about this development kit, refer to the following URL: https://www.altera.com/products/devkit/a1jui0000049utnmam/agilex-7-fpga-i-series-transceiver-soc-development-kit-4x-f-tile

- Arria 10 SX SoC FPGA Development Kit (DK-SOC-10AS066S)

  This development kit features an Arria 10 SX 660 device (OPN: 10AS066N3F40E2SG) with a " -2" speed grade with the included DDR4 HILO memory cards.

  For more details about this development kit, refer to the following URL:

  https://www.altera.com/products/devkit/a1jui0000049utgmam/arria-10-sx-soc-development-kit

In addition, the following hardware components are required:

- SDHC flash card, class 10 speed or faster (minimum 2 GB but 4 GB or more is recommended)
- Mini-USB cable suitable for connecting the development board to a host PC
- Ethernet cable suitable for connecting the development board to a network to provide access from a host PC on the same network

The host PC must use a supported operating system (Red Hat Enterprise Linux 8, Ubuntu 20.04, or Ubuntu 22.04), and must have an internet connection to install the software dependencies.

To build bitstreams, Quartus Prime Pro Edition Version 25.3 must be installed on the host system.

Although the development host system does not need to be the same as the system used to build packages and bitstreams, this guide does not explicitly cover the scenario where they are distinct.

4.3.6.1.1. Agilex 7 FPGA I-Series Transceiver-SoC Development Kit SoC Design Example Hardware Requirements

The FPGA AI Suite SoC design example requires x8 (or wider) DDR4 memory.

The RAM module provided with the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit does not support the FPGA AI Suite SoC design example because the included RAM module provides only an x4 width.

The design example has been verified on a development kit fitted with a Kingston* x8 RDIMM (KSM32RS8/16MFR). Altera recommends using this memory module to help you successfully use the design example.

4.3.6.2. Building the Bitstream and Configuring Your Board for the FPGA AI Suite SoC Design Example

In the topics that follow, some overlay architecture (`.arch`) files referred to in the instructions include the suffix "`LayoutTransform`". This indicates that the FPGA AI Suite internal layout transform (described in the Transforming the Layout of Input Data on page 230) is enabled. On Agilex 7 devices, this internal layout transform must be enabled for S2M operation, and is optional for M2M operation.

To build the bitstream and configure your board for the FPGA AI Suite SoC design example:

1. Complete the initial setup.
2. (Optional) Create an SD card image (`.wic`).
3. Write the SD Card Image (.wic) to an SD Card.
4. Prepare your SoC FPGA development kits for the SoC design example.
5. Add compiled graphs (AOT files) to the SD Card.
6. Verify FPGA device drivers.

4.3.6.2.1. Initial Setup for the SoC Design Example

The SoC design example instructions assume that you have initialized your environment for the FPGA AI Suite with the `init_env.sh` script from a shell that is compatible with the Bourne shell (`sh`).

The FPGA AI Suite `init_env.sh` script might already be part of your shell login script. If not, then use the following command to initialize your shell environment:

```
source /opt/altera/fpga_ai_suite_2025.3.1/dla/bin/setupvars.sh
```

This command assumes that the FPGA AI Suite is installed in the default location. If you are using an FPGA AI Suite version other than 2025.3.1, adjust the path to script accordingly.

4.3.6.2.2. (Optional) Creating an SD Card Image (`.wic`)

An SD card provides the FPGA bitstream and HPS disk image to the SoC FPGA development kit. You can build your own SD card image or use the prebuilt image provided by the FPGA AI Suite SoC design example.

If you want to use the prebuilt image, skip this section and go to Writing the SoC Design Example SD Card Image (.wic) to an SD Card on page 109.

Important: You cannot build the SD card as `root` due to security checks in the BitBake tool used when creating an SD card image.

Installing Prerequisite Software for Building an SD Card Image

Building the SD card image requires the following additional software:

- Quartus Prime Pro Edition Version 25.3
- Ashling RiscFree IDE for Altera
- (Ubuntu 20[12]/22 only) Ubuntu package `libncurses5`
- (Ubuntu 24 only) Ubuntu package `libncurses6`

If you did not install Quartus Prime Pro Edition Version 25.3 when following the instructions in Installing the FPGA AI Suite Compiler and IP Generation Tools on page 67, you must install it now.

Building the SD card image also requires tools provided by Ashling RiscFree IDE for Altera. You can install Ashling RiscFree IDE from a separate installation package or part of your Quartus Prime bundled installation package.

You can download the required software from the following URL: https://www.altera.com/downloads/fpga-development-tools/quartus-prime-pro-current.

To install the prerequisite software for building an SD card image:

1. Install Quartus Prime Pro Edition and Ashling RiscFree IDE for Altera
2. (Ubuntu only) Install Ubuntu package `libcurses` with the following command:
   - Ubuntu 20[12]/22:
     ```
     sudo apt install libncurses5
     ```
   - Ubuntu 24:
     ```
     sudo apt install libncurses6
     ```
3. Ensure that the `QUARTUS_ROOTDIR` environment variable is set properly:
   ```
   echo $QUARTUS_ROOTDIR
   ```
   If the `QUARTUS_ROODIR` is not set, run the following command:
   ```
   export QUARTUS_ROOTDIR=/opt/altera/altera_pro/25.3/quartus
   ```
   If you chose install Quartus Prime in a location other than the default location, adjust the path in `export` command to match your Quartus Prime installation location.
4. Ensure your `$PATH` environment variable includes paths to the installed Quartus Prime and Ashling RiscFree IDE binaries. Adjust the following commands appropriately if you did not install into the default location:
   ```
   export PATH=$PATH:/opt/altera/altera_pro/25.3/niosv/bin
   export PATH=$PATH:/opt/altera/altera_pro/25.3/nios2eds/bin
   export PATH=$PATH:/opt/altera/altera_pro/25.3/riscfree/toolchain/riscv32-unknown-elf/bin
   ```

(12) Standard support for Ubuntu Version 20.04 has ended. As a result, some repositories required for installation might not be available.

5. Confirm that Quartus Prime Pro Edition Version 25.3 is installed by running the following command:

```
quartus_sh -v
```

Related Information

- Ashling RiscFree IDE for Altera User Guide
- Altera FPGA Software Installation and Licensing

Building the FPGA Bitstreams for the SoC Design Example

The FPGA AI Suite SoC design example also includes prebuilt demonstration FPGA bitstreams. If you want to use the prebuilt demonstration bitstreams in your SD card image, skip ahead to Installing HPS Disk Image Build Prerequisites on page 104.

If you build your own bitstreams and do not have an FPGA AI Suite IP license, then your bitstream have a limit of 100,000 inferences. After 100,000 inferences, the unlicensed IP refuses to perform any additional inference. To reset the limit, reprogram the FPGA device.

Building the FPGA Bitstream for the Agilex 5 FPGA E-Series 065B Modular Development Kit

To build the FPGA bitstream for the, Agilex 5 FPGA E-Series 065B Modular Development Kit run the following command:

```
dla_build_example_design.py build \
     --output-dir $COREDLA_WORK/agx5_perf_bitstream \
     -n 1 \
     agx5_soc_s2m \
     $COREDLA_ROOT/example_architectures/AGX5_Performance.arch
```

The bitstream built by this command supports both the M2M execution model and the S2M execution model.

Building the FPGA Bitstream for the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit

To build the FPGA bitstream for the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit, run the following command:

```
dla_build_example_design.py build \
     --output-dir $COREDLA_WORK/agx7_perf_bitstream \
     -n 1 \
     agx7_soc_s2m \
     $COREDLA_ROOT/example_architectures/AGX7_Performance_LayoutTransform.arch
```

The bitstream built by this command supports both the M2M execution model and the S2M execution model.

This design example bitstream is built with a “LayoutTransform” architecture because the design example uses the FPGA AI Suite IP internal layout transform rather than an external layout transform for converting image buffers to the target memory format. The layout transform is required for S2M bitstreams, but is optional for M2M bitstreams. For more information about the layout transform hardware, refer to Input Feature Tensor In-Memory Format on page 231.

Building the FPGA Bitstream for the Arria 10 SX SoC FPGA Development Kit

To build the FPGA bitstream for the Arria 10 SX SoC FPGA Development Kit, run the following command:

```
dla_build_example_design.py build \
     --output-dir $COREDLA_WORK/a10_perf_bitstream \
     -n 1 \
     a10_soc_s2m \
     $COREDLA_ROOT/example_architectures/A10_Performance.arch
```

The bitstream built by this command supports both the M2M execution model and the S2M execution model.

Installing HPS Disk Image Build Prerequisites

The process to build the HPS disk image has additional prerequisites. To install these prerequisites, follow the instructions for your operating system in the following sections:

- Red Hat Enterprise Linux 8 Prerequisites on page 104
- Red Hat Enterprise Linux 9 Prerequisites on page 105
- Ubuntu 20 Prerequisites on page 106[13]
- Ubuntu 22 Prerequisites on page 106
- Ubuntu 24 Prerequisites on page 107

Red Hat Enterprise Linux 8 Prerequisites

To install the prerequisites for Red Hat Enterprise Linux 8:

1. Ensure that you install the prerequisites outlined in Red Hat Enterprise Linux Operating System Requirements on page 70.
2. Enable additional Red Hat Enterprise Linux 8 repository and package manager:

```
sudo subscription-manager repos \
  --enable codeready-builder-for-rhel-8-x86_64-rpms

wget https://dl.fedoraproject.org/pub/epel/epel-release-latest-8.noarch.rpm

sudo dnf install ./epel-release-latest-8.noarch.rpm epel-release

sudo dnf upgrade
```

3. Install the dependency packages:

```
sudo dnf install gawk wget git diffstat unzip texinfo gcc gcc-c++ make \
                 chrpath socat cpio python3 python3-pexpect xz iputils \
                 python3-jinja2 python3-packaging mesa-libEGL SDL xterm \
                 python3-subunit rpcgen zstd lz4 perl-open.noarch \
                 perl-Thread-Queue
```

[13] Standard support for Ubuntu Version 20.04 has ended. As a result, some repositories required for installation might not be available.

4. Install packages required to create the flash card image and FPGA AI Suite runtime and dependencies:

```
cd /tmp

mkdir uboot_tools && cd uboot_tools

wget https://kojipkgs.fedoraproject.org/\
     vol/fedora_koji_archive02/packages/uboot-tools/2018.03/3.fc28/x86_64/\
     uboot-tools-2018.03-3.fc28.x86_64.rpm

sudo dnf install ./uboot-tools-2018.03-3.fc28.x86_64.rpm

sudo dnf install ninja-build fakeroot

sudo -E python3 -m pip install pylint passlib scons patchelf
```

5. Install Make Version 4.3 or later:

```
cd /tmp

mkdir make && cd make

wget https://ftp.gnu.org/gnu/make/make-4.3.tar.gz

tar xvf make-4.3.tar.gz

cd make-4.3

./configure

make

sudo make install
```

6. Add the `/sbin` directory to your `$PATH` environment variable:

```
export PATH="/sbin:$PATH"
```

7. Switch SELinux to permissive mode:

```
sudo setenforce 0
```

Red Hat Enterprise Linux 9 Prerequisites

To install the prerequisites for Red Hat Enterprise Linux 9:

1. Ensure that you install the prerequisites outlined in Red Hat Enterprise Linux Operating System Requirements on page 70.
2. Enable additional Red Hat Enterprise Linux 9 repository and package manager:

```
sudo subscription-manager repos \
  --enable codeready-builder-for-rhel-9-x86_64-rpms

wget https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm

sudo dnf install ./epel-release-latest-9.noarch.rpm epel-release

sudo dnf upgrade
```

3. Install the dependency packages:

```
sudo dnf install gawk wget git diffstat unzip texinfo gcc gcc-c++ make \
                 chrpath socat cpio python3 python3-pexpect xz iputils \
                 python3-jinja2 python3-packaging mesa-libEGL SDL xterm \
                 python3-subunit rpcgen zstd lz4 perl-open.noarch \
                 perl-Thread-Queue scons libomp-devel mtools patch \
                 perl-File-Compare perl-FindBin
```

4. Install packages required to create the flash card image and FPGA AI Suite runtime and dependencies:

```
sudo dnf install ninja-build fakeroot

sudo -E python3 -m pip install pylint passlib scons patchelf
```

5. Confirm that you have Make Version 4.3 or later installed:

```
make --version
```

   The output of this command should include a phrase like `GNU Make 4.3`.

6. Add the `/sbin` directory to your `$PATH` environment variable:

```
export PATH="/sbin:$PATH"
```

Ubuntu 20 Prerequisites

To install the prerequisites for Ubuntu 20[13]:

1. Ensure that you install the prerequisites outlined in Ubuntu Operating System Requirements on page 71.
2. If you have not already installed `libcurses5`, install the library now:

```
sudo apt install libncurses5
```

3. Install the dependency packages:

```
sudo apt install gawk wget git diffstat unzip texinfo gcc build-essential \
                 chrpath socat cpio python3 python3-pip python3-pexpect \
                 xz-utils debianutils iputils-ping python3-git \
                 python3-jinja2 python3-packaging libegl1-mesa \
                 libsdl1.2-dev pylint3 xterm python3-subunit mesa-common-dev
\
                 zstd liblz4-tool device-tree-compiler mtools
```

4. Install packages required to create the flash card image and FPGA AI Suite runtime and dependencies:

```
sudo apt install ninja-build u-boot-tools scons fakeroot
```

5. Add the `/sbin` directory to your `$PATH` environment variable:

```
export PATH="/sbin:$PATH"
```

Ubuntu 22 Prerequisites

To install the prerequisites for Ubuntu 22:

1. Ensure that you install the prerequisites outlined in Ubuntu Operating System Requirements on page 71.
2. If you have not already installed `libcurses5`, install the library now:

```
sudo apt install libncurses5
```

3. Install the dependency packages:

```
sudo apt install gawk wget git diffstat unzip texinfo gcc build-essential \
                 chrpath socat cpio python3 python3-pip python3-pexpect \
                 xz-utils debianutils iputils-ping python3-git \
                 python3-jinja2 python3-packaging libegl1-mesa \
                 libsdl1.2-dev xterm python3-subunit mesa-common-dev zstd \
                 liblz4-tool device-tree-compiler mtools
```

4. Install packages required to create the flash card image and FPGA AI Suite runtime and dependencies:

```
sudo apt install ninja-build u-boot-tools scons fakeroot
```

5. Add the `/sbin` directory to your `$PATH` environment variable:

```
export PATH="/sbin:$PATH"
```

Ubuntu 24 Prerequisites

To install the prerequisites for Ubuntu 24:

1. Ensure that you install the prerequisites outlined in Ubuntu Operating System Requirements on page 71.
2. If you have not already installed `libcurses6`, install the library now:

```
sudo apt install libncurses6
```

3. Install the dependency packages:

```
sudo apt install gawk wget git diffstat unzip texinfo gcc build-essential \
                 chrpath socat cpio python3 python3-pip python3-pexpect \
                 xz-utils debianutils iputils-ping python3-git \
                 python3-jinja2 python3-packaging libegl1-mesa \
                 libsdl1.2-dev xterm python3-subunit mesa-common-dev zstd \
                 liblz4-tool device-tree-compiler mtools
```

4. Install packages required to create the flash card image and FPGA AI Suite runtime and dependencies:

```
sudo apt install ninja-build u-boot-tools scons fakeroot
```

5. Add the `/sbin` directory to your `$PATH` environment variable:

```
export PATH="/sbin:$PATH"
```

(Optional) Downloading the ImageNet Categories

By default, the S2M streaming app prints the category associated with each image after inference.

Optionally, you can use human-readable category names as follows:

1. Download the list of ImageNet categories from a source such as the following URL:

```
https://github.com/xmartlabs/caffeflow/blob/master/examples/imagenet/
imagenet-classes.txt
```

2. Place the contents into the following file:

```
$COREDLA_WORK/runtime/streaming/streaming_inference_app/categories.txt
```

Building the SD Card Image for the SoC Design Example

The SD card image contains a Yocto Project embedded Linux system, HPS packages, and the FPGA AI Suite runtime.

Building the SD card image requires a minimum of 100GB of free disk space.

The SD card image is build with the `create_hps_image.sh` command, which does the following steps for you:

- Build a Yocto Project embedded Linux system.
- Build additional packages required by the SoC design example runtime, including the OpenVINO and OpenCV runtimes.
- Build the design example runtime.
- Combine all of these items and FPGA bitstreams into an SD card image using `wic`.
- Place the SD card image in the specified output directory.

For more details about the `create_hps_image.sh` command, refer to Building the Bootable SD Card Image (.wic) on page 319.

To build the SD card image, run the following commands:

1. **Ubuntu 24.04 Users**: Check if AppArmor is restricting unprivileged user namespaces with the following command:

```
sysctl kernel.apparmor_restrict_unprivileged_userns
```

   If the result is `kernel.apparmor_restrict_unprivileged_userns = 1`, temporarily disable the restriction to allow the Yocto Project BitBake tool to run correctly when building the SD card image with the following command:

```
sudo sysctl kernel.apparmor_restrict_unprivileged_userns=0
```

2. Build the SD card image for your board with one of the following commands:

- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)

```
cd $COREDLA_WORK/runtime

./create_hps_image.sh \
    -f $COREDLA_WORK/agx5_perf_bitstream/hw/output_files \
    -o <output_dir> -u \
  -m agilex5_mk_a5e065bb32aes1
```

- Agilex 7 FPGA I-Series Transceiver-SoC Development Kit

```
cd $COREDLA_WORK/runtime

./create_hps_image.sh \
   -f $COREDLA_WORK/agx7_perf_bitstream/hw/output_files \
   -o <output_dir> -u \
  -m agilex7_dk_si_agi027fa
```

- Arria 10 SX SoC FPGA Development Kit

```
cd $COREDLA_WORK/runtime

./create_hps_image.sh \
   -f $COREDLA_WORK/a10_perf_bitstream/hw/output_files \
   -o <output_dir> -u \
  -m arria10
```

If the command returns errors such as "`bitbake: command not found`", try deleting the `$COREDLA_WORK/runtime/build_Yocto/` directory before rerunning the `create_hps_image.sh` command.

4.3.6.2.3. Writing the SoC Design Example SD Card Image (`.wic`) to an SD Card

Before running the demonstration, you must create a bootable SD card for the FPGA development kit. You can use either the precompiled SD card image or an SD card image that you created.

The precompiled SD card image (`.wic`) is in the following location:

- Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)

```
$COREDLA_ROOT/demo/ed4/agx5_soc_s2m/sd-card/coredla-image-
agilex5_mk_a5e065bb32aes1
```

- Agilex 7 FPGA I-Series Transceiver-SoC Development Kit

```
$COREDLA_ROOT/demo/ed4/agx7_soc_s2m/sd-card/coredla-image-
agilex7_dk_si_agi027fa.wic
```

- Arria 10 SX SoC FPGA Development Kit

```
$COREDLA_ROOT/demo/ed4/a10_soc_s2m/sd-card/coredla-image-arria10.wic
```

If you built your own SD card image following the instructions in (Optional) Creating an SD Card Image (.wic) on page 101, then your SD card image is located in the directory that you specified for the `-o` option of the `create_hps_image.sh` command.

To write the SD card image to an SD card:

1. Determine the device associated with the SD card on the host by running the following command before and after inserting the SD card:

```
cat /proc/partitions
```

   Typical locations for the SD card include `/dev/sdb` or `/dev/sdc`. The rest of these instruction use `/dev/sdx` as the SD card location.

2. Use the `dd` command to write the SD card image as follows:

```
wic_image=<path to SD (.wic) image file>
sudo umount /dev/sdx*
sudo dd if=$wic_image of=/dev/sdx bs=1M
sudo sync
sudo udisksctl power-off -b /dev/sdx
```

After the SD card image is written, insert the SD card into the development kit SD card slot.

If you want to use a Microsoft Windows system to write the SD card image to the SD card, refer to the GSRD manuals available at the following URL: https://www.rocketboards.org/foswiki/Documentation/GSRD.

4.3.6.2.4. Preparing SoC FPGA Development Kits for the FPGA AI Suite SoC Design Example

To prepare an FPGA development kit for the FPGA AI Suite SoC design example:

1. Prepare one of the supported development kits:
   - Prepare the Agilex 5 FPGA E-Series 065B Modular Development Kit.
   - Prepare the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit.
   - Prepare the Arria 10 SX SoC FPGA Development Kit.
2. Configure the SoC FPGA development kit UART connection.
3. Determine the SoC FPGA development kit IP address.

Preparing the Agilex 5 FPGA E-Series 065B Modular Development Kit for the SoC Design Example

Prepare the Agilex 5 FPGA E-Series 065B Modular Development Kit for the FPGA AI Suite SoC design example with the following steps:

1. Confirming the Agilex 5 FPGA E-Series 065B Modular Development Kit Board Setup on page 111.
2. Programming the FPGA device on the board in one of the following ways:

- Programming the Agilex 5 FPGA Device with the JTAG Indirect Configuration (.jic) File on page 112.

  This method programs the QSPI flash memory, which then programs the FPGA device when the board is powered up. With this method, the FPGA programming can be persisted between board power cycles.

  This method is preferred for deployment or testing the other parts of your application after your FPGA bitstream is finalized.

- Programming the Agilex 5 FPGA Device with the SRAM Object File (.sof) on page 114.

  This method programs the FPGA device directly. The FPGA programming is not persisted between board power cycles. This method is typically faster than programming the QSPI flash memory with `.jic` file that then programs the FPGA device.

  This method is preferred when developing or debugging your FPGA bitstream.

3. Connecting the Agilex 5 FPGA E-Series 065B Modular Development Kit to the Host Development System on page 115.

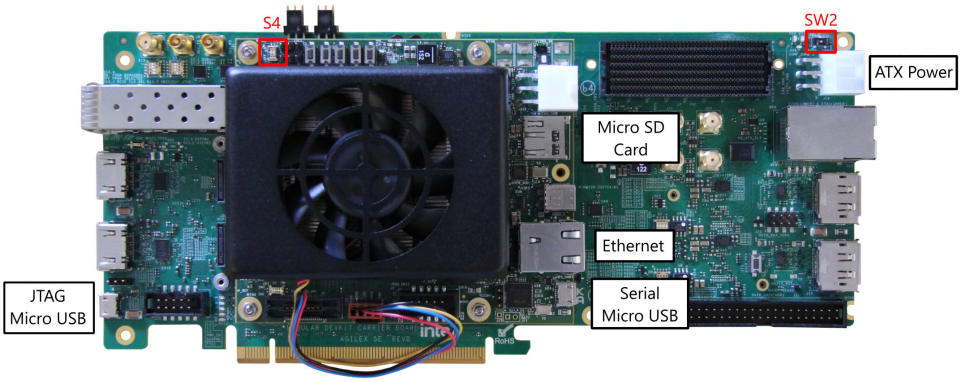
Confirming the Agilex 5 FPGA E-Series 065B Modular Development Kit Board Setup

Confirm the board settings as follows:

1. Ensure that the Agilex 5 FPGA E-Series 065B Modular Development Kit DIP switch and jumpers are set to their default settings. For this design example, you change the settings for some DIP switches depending on what are doing with the board:
   - For programming the FPGA device on the board, you will set the S4 DIP switch for JTAG mode.
   - For booting the FPGA device from flash memory, you will set the S4 DIP switch for QSPI mode.
   - To get power over the ATX connector, ensure that the SW2 switch is set to ATX power mode.

   For more details about default DIP switch and jumper settings, refer to "Default Settings" in the *Agilex 5 FPGA E-Series 065B Modular Development Kit User Guide*.
2. Ensure that the SD card with the programmed Yocto image is installed on the board.

When configured and connected, the Agilex 5 FPGA E-Series 065B Modular Development Kit should resemble the following image:



The board connections serve the following purposes:

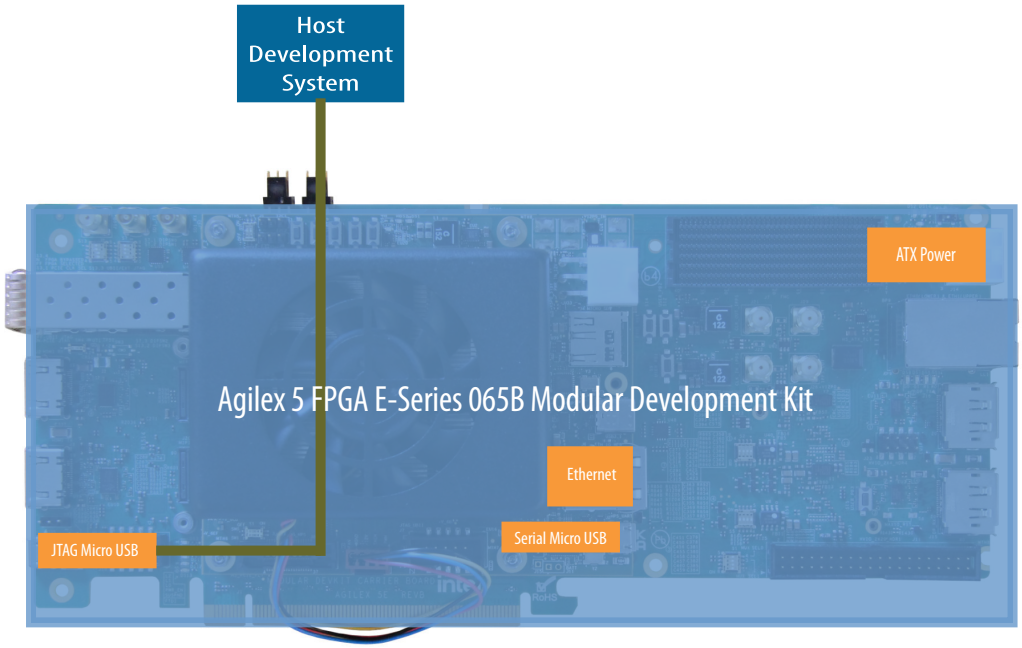
- The JTAG micro USB connector is used to program the FPGA device.
- The Ethernet connector is used for fast data transfer to the HPS.
- The serial micro USB connector is used as follows:
  - ☐ To monitor the serial output from the HPS during operation.
  - ☐ To provide command-line input to the HPS during operation.

Programming the Agilex 5 FPGA Device with the JTAG Indirect Configuration (`.jic`) File

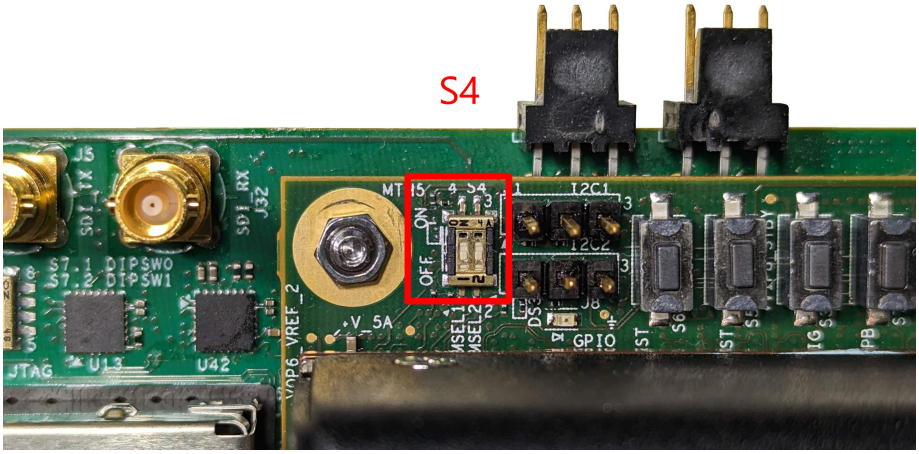
Programming the Agilex 5 device with the JTAG indirect configuration (`.jic`) file programs the QSPI flash memory and allows the FPGA device to be automatically configured when power is applied to the board.

To program the Agilex 5 FPGA device with the JTAG indirect configuration (`.jic`) file:

1. Connect the Agilex 5 FPGA E-Series 065B Modular Development Kit to your host development system via JTAG micro USB connection as shown in the following diagram:



2. Switch the board into JTAG mode by setting the S4[1:2] DIP switch to OFF/OFF:



3. Program the QSPI with the `.jic` file by running the following commands on the host development system:

```
cd $COREDLA_ROOT/demo/ed4/agx5_soc_s2m/sd-card/

quartus_pgm -m jtag -o "pvi;u-boot-spl-dtb.hex.jic@<device_number>"
```

where *<device_number>* is `1` or `2`, depending on whether the HPS is already running (that is, the prior state of the device). Use `1` if the HPS is not running, and `2` if the HPS is already running. If you do not know the state of the device, try `1`. If that fails, try `2`.

4. Switch the board into QSPI mode by setting the S4[1:2] DIP switch to ON/ON and cycle the power to the board.

At boot time, the Agilex 5 FPGA device is configured from the QSPI flash memory.

Programming the Agilex 5 FPGA Device with the SRAM Object File (`.sof`)

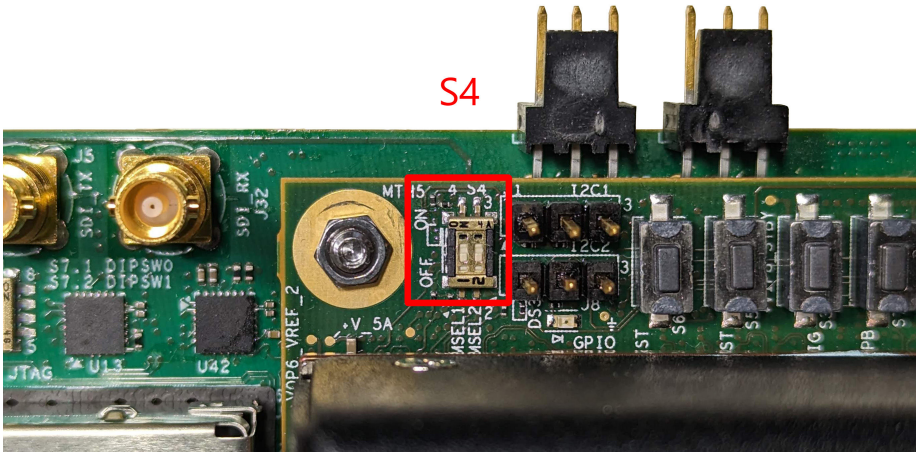
Programming the Agilex 5 device with the SRAM object file (`.sof`) programs FPGA device directly. The FPGA configuration is lost when power is removed from the board.

To program the Agilex 5 FPGA device with the SRAM object file (`.sof`):

1. Connect the Agilex 5 FPGA E-Series 065B Modular Development Kit to your host development system via JTAG micro USB connection as shown in the following diagram:



2. Switch the board into JTAG mode by setting the S4[1:2] DIP switch to OFF/OFF:



3. Program the FPGA device `.sof` file by running the following commands on the host development system:

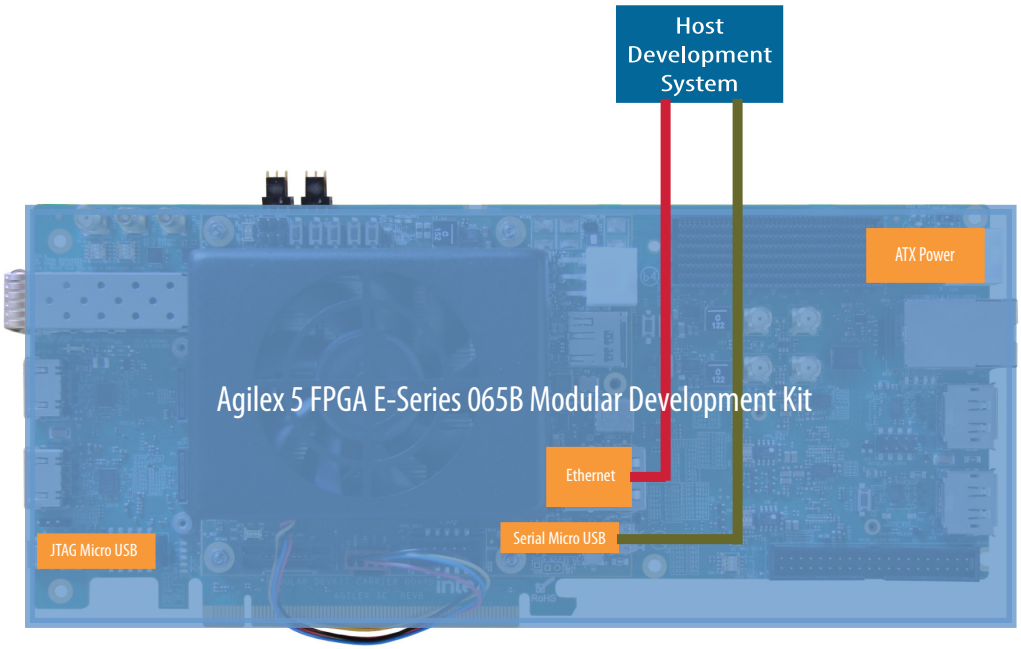
```
cd $COREDLA_ROOT/demo/ed4/agx5_soc_s2m/sd-card/

quartus_pgm -m jtag -o "p;u-boot-spl-dtb.hex.sof"
```

The Agilex 5 FPGA device now boots. The device will lose its configuration when you remove power from the board.

Connecting the Agilex 5 FPGA E-Series 065B Modular Development Kit to the Host Development System

Connect the Agilex 5 FPGA E-Series 065B Modular Development Kit to your host development system via Ethernet and serial micro USB UART connections as shown in the following diagram:



There are 4 COM ports on the one USB connection. The COM port connected to the HPS should be the 3rd available COM port.

Preparing the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit for the SoC Design Example

Prepare the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit for the FPGA AI Suite SoC design example with the following steps:

1. Confirming Agilex 7 FPGA I-Series Transceiver-SoC Development Kit Board Set Up on page 116.
2. Programming the FPGA device on the board in one of the following ways:
   - Programming the Agilex 7 FPGA Device with the JTAG Indirect Configuration (.jic) File on page 117.

     This method programs the QSPI flash memory, which then programs the FPGA device when the board is powered up. With this method, the FPGA programming can be persisted between board power cycles.

     This method is preferred for deployment or testing the other parts of your application after your FPGA bitstream is finalized.
   - Programming the Agilex 7 FPGA Device with the SRAM Object File (.sof) on page 119.

     This method programs the FPGA device directly. The FPGA programming is not persisted between board power cycles. This method is typically faster than programming the QSPI flash memory with `.jic` file that then programs the FPGA device.

     This method is preferred when developing or debugging your FPGA bitstream.
3. Connecting the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit to the Host Development System on page 121.

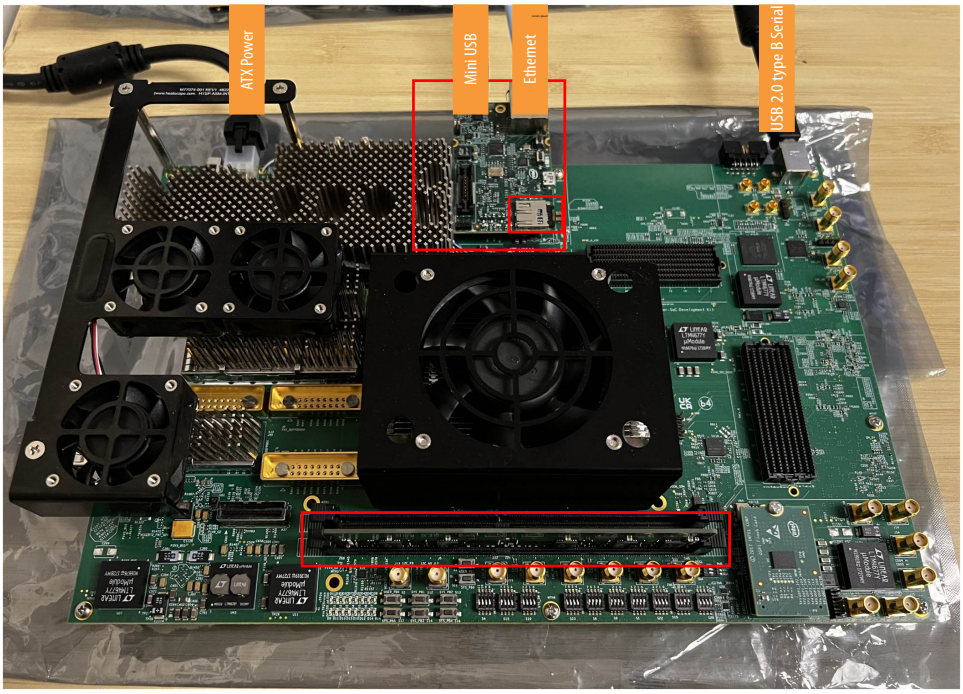
Confirming Agilex 7 FPGA I-Series Transceiver-SoC Development Kit Board Set Up

Confirm the board settings as follows:

1. Ensure that the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit DIP switch and jumpers are set to their default settings. For this design example, you change the settings for the S9 DIP switch depending on what are doing with the board:
   - For programming the FPGA device on the board, you will set the S9 DIP switch for JTAG mode.
   - For booting the FPGA device from flash memory, you will set the S9 DIP switch for QSPI mode.

   For more details about default DIP switch and jumper settings, refer to "Default Settings" in the *Agilex 7 FPGA I-Series Transceiver-SoC Development Kit User Guide*.
2. Ensure that the HPS IO48 OOBE daughter card is installed in connector J4 on the development kit, and the SD card with the programmed Yocto image is installed in the daughter card.
3. Ensure that the DDR4 x8 RDIMM is installed in the PCIe slot furthest from the fan. For RDIMM requirements, refer to Agilex 7 FPGA I-Series Transceiver-SoC Development Kit SoC Design Example Hardware Requirements on page 100.

When configured and connected, the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit should resemble the following image:



The board connections serve the following purposes:

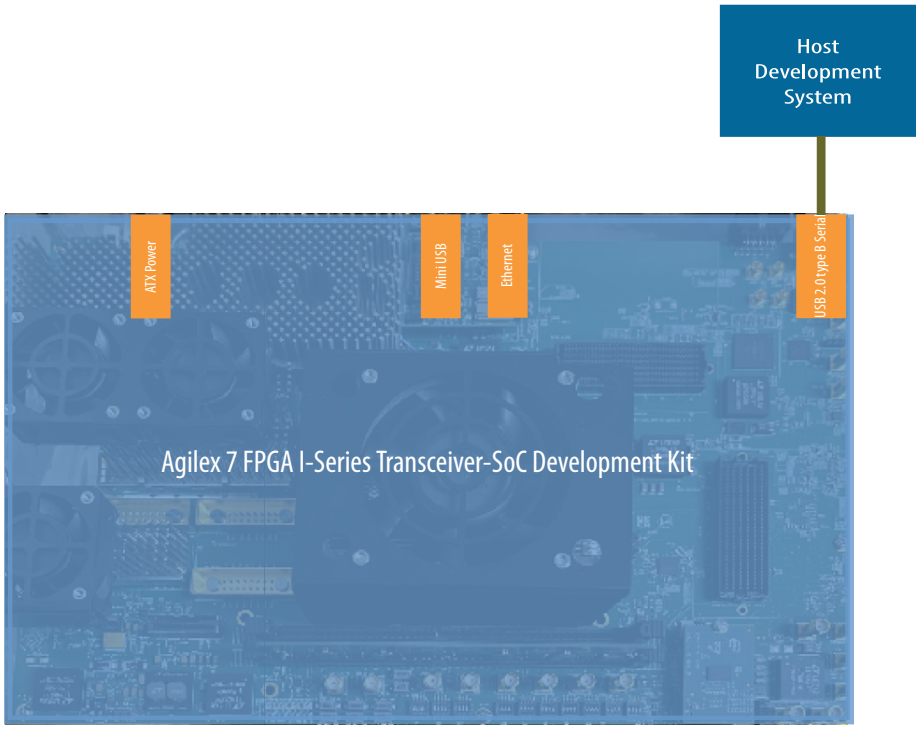
- The USB 2.0 connector is used to program the FPGA device.
- The Ethernet connector is used for fast data transfer to the HPS.
- The micro USB connector is used as follows:
  - ☐ To monitor the serial output from the HPS during operation.
  - ☐ To provide command-line input to the HPS during operation.

Programming the Agilex 7 FPGA Device with the JTAG Indirect Configuration (`.jic`) File

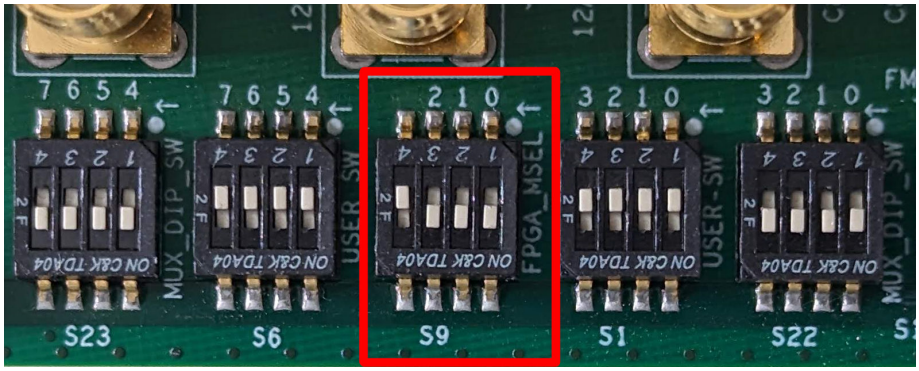
Programming the Agilex 7 device with the JTAG indirect configuration (`.jic`) file programs the QSPI flash memory and allows the FPGA device to be automatically configured when power is applied to the board.

To program the Agilex 7 FPGA device with the JTAG indirect configuration (`.jic`) file:

1. Connect the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit to your host development system via USB 2.0 connection as shown in the following diagram:



2. Switch the board into JTAG mode by setting the S9[1:4] DIP switch to ON/ON/ON/OFF:



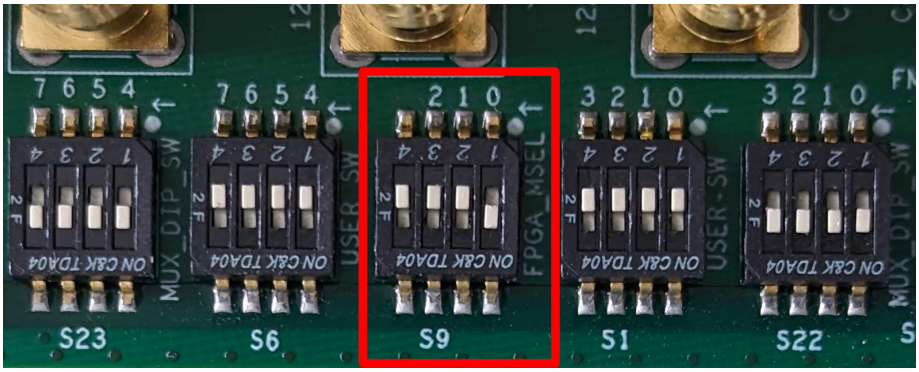
3. Program the QSPI with the `.jic` file by running the following commands on the host development system:

```
cd $COREDLA_ROOT/demo/ed4/agx7_soc_s2m/sd-card/

quartus_pgm -m jtag -o "pvi;u-boot-spl-dtb.hex.jic@<device_number>"
```

where *<device_number>* is `1` or `2`, depending on whether the HPS is already running (that is, the prior state of the device). Use `1` if the HPS is not running, and `2` if the HPS is already running. If you do not know the state of the device, try `1`. If that fails, try `2`.

4. Switch the board into QSPI mode by setting the S9[1:4] DIP switch to ON/OFF/OFF/OFF and cycle power to the board:



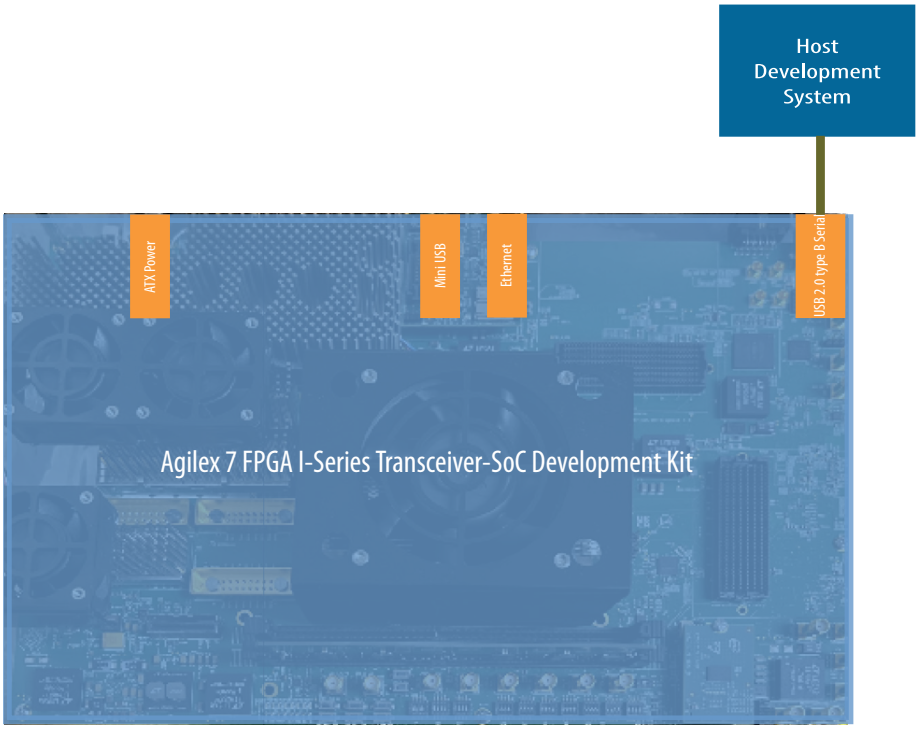
At boot time, the Agilex 7 FPGA device is configured from the QSPI flash memory.

Programming the Agilex 7 FPGA Device with the SRAM Object File (`.sof`)

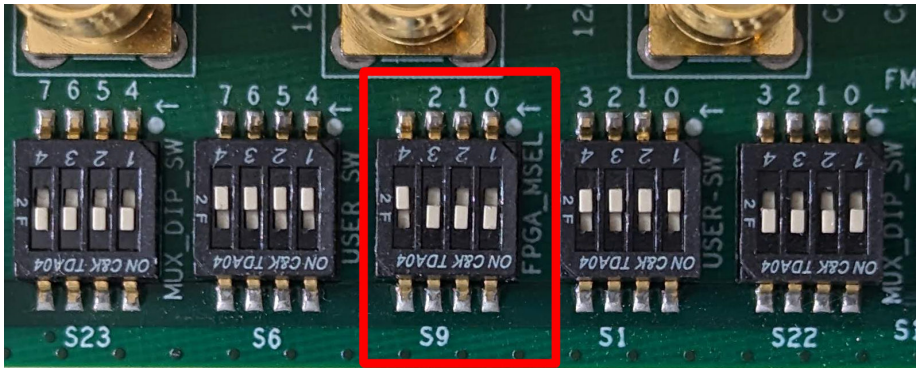
Programming the Agilex 7 device with the SRAM object file (`.sof`) programs FPGA device directly. The FPGA configuration is lost when power is removed from the board.

To program the Agilex 7 FPGA device with the SRAM object file (`.sof`):

1. Connect the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit to your host development system via USB 2.0 connection as shown in the following diagram:



2. Switch the board into JTAG mode by setting the S9[1:4] DIP switch to ON/ON/ON/OFF:



3. Program the FPGA device `.sof` file by running the following commands on the host development system:

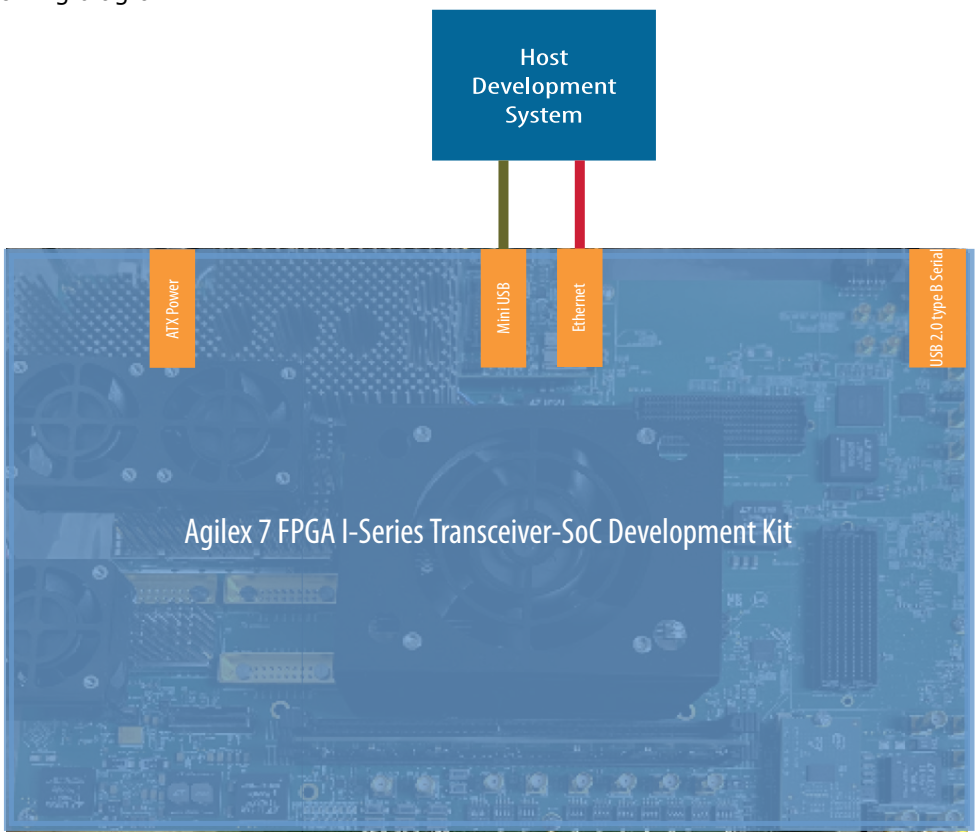
```
cd $COREDLA_ROOT/demo/ed4/agx7_soc_s2m/sd-card/

quartus_pgm -m jtag -o "p;u-boot-spl-dtb.hex.sof"
```

The Agilex 7 FPGA device now boots. The device will lose its configuration when you remove power from the board.

Connecting the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit to the Host Development System

Connect the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit to your host development system via Ethernet and USB UART connections as shown in the following diagram:



Preparing the Arria 10 SX SoC FPGA Development Kit for the SoC Design Example

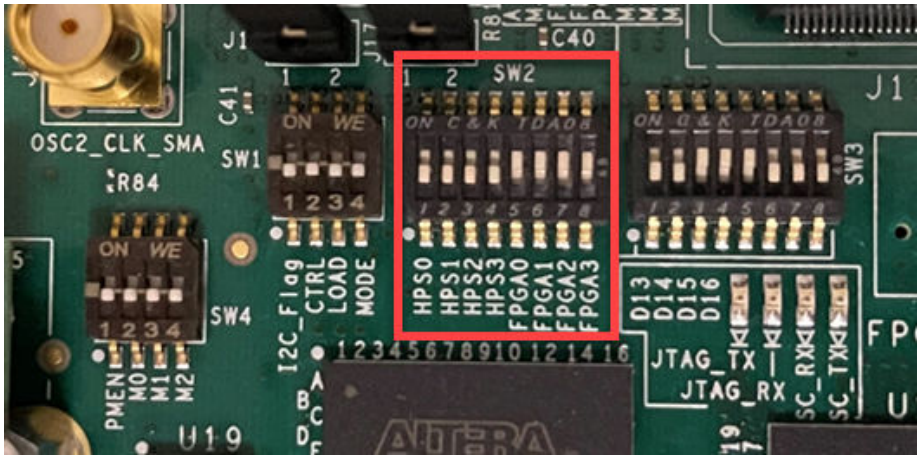
To prepare the Arria 10 SX SoC FPGA Development Kit for the FPGA AI Suite SoC design example:

1. Confirming Arria 10 SX SoC FPGA Development Kit Board Settings on page 121
2. Connecting the Arria 10 SX SoC FPGA Development Kit to the Host Development System on page 123.

Confirming Arria 10 SX SoC FPGA Development Kit Board Settings

Confirm the board settings as follows:

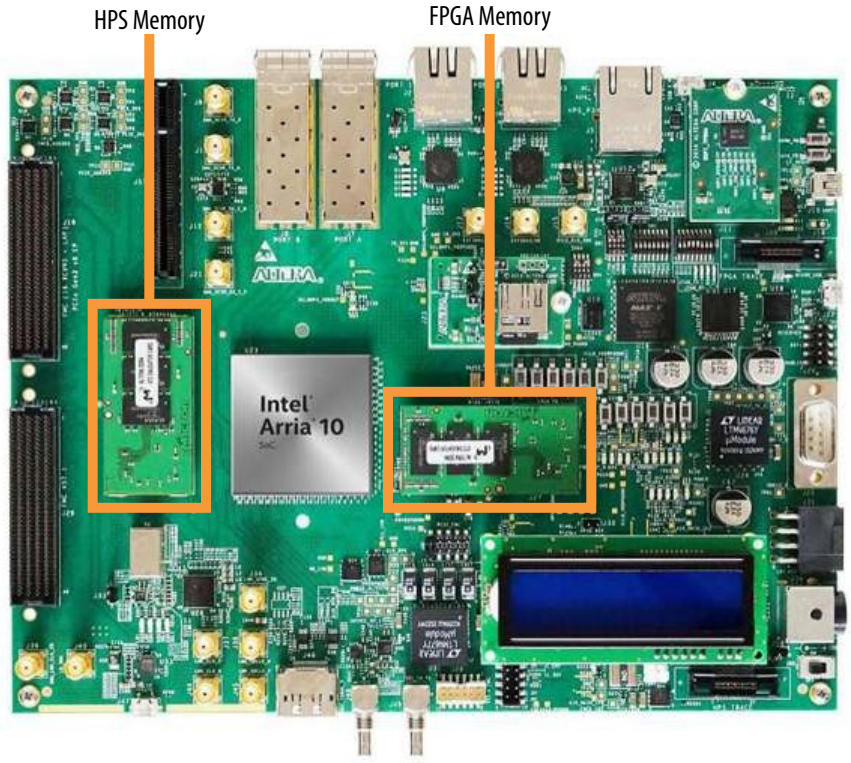
1. Ensure that the Arria 10 SX SoC FPGA Development Kit has the required DIP switch and jumper settings. The SoC design example requires that all DIP switches have their default settings except for SW2 switches 5, 6, 7, and 8, which should be switched ON:



For more details about default DIP switch and jumper settings, refer to *Arria 10 SoC Development Kit User Guide*.

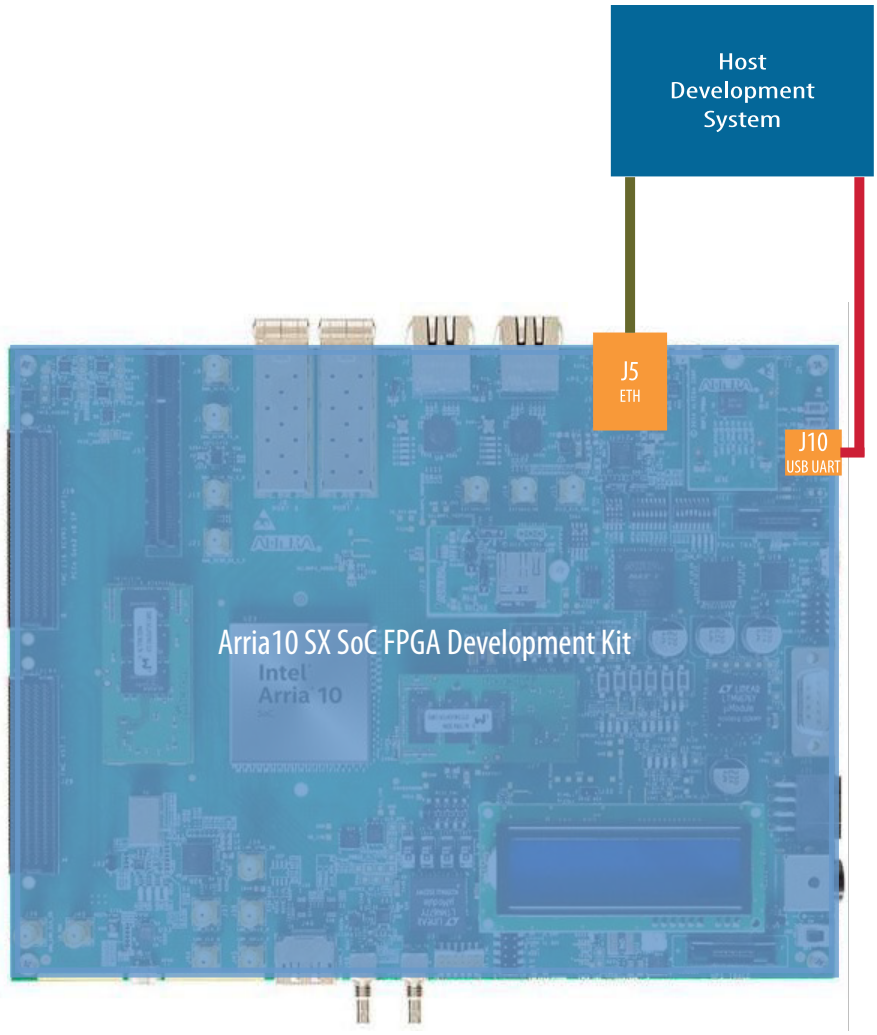
2. Ensure that the HILO cards are fitted correctly.

   The Arria 10 SX SoC FPGA Development Kit includes two DDR4 HILO cards: the HPS memory (1GB) and the FPGA memory (2GB). Both the HPS Memory and FPGA Memory DDR4 HILO modules must be fitted as shown in the following image:



Connecting the Arria 10 SX SoC FPGA Development Kit to the Host Development System

Connect the Arria 10 SX SoC FPGA Development Kit to your host development system via Ethernet and USB UART connections as shown in the following diagram:



Configuring the SoC FPGA Development Kit UART Connection

The SoC FPGA development kit boards have USB-to-serial converters that allows the host computer to see the board as a virtual serial port:

- The Agilex 5 FPGA E-Series 065B Modular Development Kit has a FTDI USB-to-serial converter chip.
- The Agilex 7 FPGA I-Series Transceiver-SoC Development Kit has a USB-to-serial converter on the IO48 daughter card.
- The Arria 10 SX SoC FPGA Development Kit (DK-SOC-10AS066S) has a FTDI USB-to-serial converter chip.

Ubuntu, Red Hat Enterprise Linux, and other modern Linux distributions have built-in drivers for the FTDI USB-to-serial converter chip, so no driver installation is necessary on those platforms.

On Microsoft Windows, the Windows SoC EDS installer automatically installs the necessary drivers. For details, see the SoC GSRD for your SoC FPGA development kit at the following URL: https://www.rocketboards.org/foswiki/Documentation/GSRD

The serial communication parameters are as follows:

- Baud rate: 115200
- Parity: None
- Flow control: None
- Stop bits: 1

On Windows, you can use utilities such as TeraTerm or PuTTY to connect the board. You can configure these utilities from their tool menus.

On Linux, you can use the Minicom utility. Configure the Minicom utility as follows:

1. Determine the device name associated with the virtual serial port on your host development system. The virtual serial port is typically named `/dev/ttyUSB0`.
   a. Before connecting the mini USB cable to the SoC FPGA development kit, determine which USB serial devices are installed with the following command:
      ```
      ls /dev/ttyUSB*
      ```
   b. Connect the mini USB cable from the SoC FPGA development kit to the host development system.
   c. Confirm the new device connection with the `ls` command again:
      ```
      ls /dev/ttyUSB*
      ```
2. If you do not have the Minicom application installed on the host development system, install it now.
   - OnRed Hat Enterprise Linux 8: `sudo yum install minicom`
   - On Ubuntu: `use sudo apt-get install minicom`
3. Configure Minicom as follows:
   a. Start Minicom:
      ```
      sudo minicom -s
      ```
   b. Under **Serial Port Setup** choose the following:
      - Serial Device: **/dev/ttyUSB0** (Change this value to match the system value that you found earlier, if needed)
      - Bps/Par/Bits: **115200 8N1**
      - Hardware Flow Control: **No**
      - Software Flow Control: **No**

      Press **[ESC]** to return to the main configuration menu.
   c. Select **Save Setup as dfl** to save the default setup. Then select **Exit**.

Determining the SoC FPGA Development Kit IP Address

To determine the FPGA development kit IP address:

1. Open a Terminal session to the FPGA development kit via the UART connection and log in using the user name `root` and no password.

```
         Starting Record Runlevel Change in UTMP…
[  OK  ] Finished Record Runlevel Change in UTMP.
     9.553286] random: crng init done
[  OK  ] Finished Load/Save Random Seed.
[   10.084845] socfpga-dwmax ff800000.ethernet eth0: Link is Up - 1Gbps/Full
- flow control off
[   10.103287] IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready

Poky (Yocto Project Reference Distro) 4.0.2 arria10-62747948036a ttyS0

arria10-62747948036a login:
```

2. Issue a `hostname` command to display the network name of the FPGA development kit board:

```
root@arria10-62747948036a:~# hostname
arria10-62747948036a
root@arria10-62747948036a:~#
```

   In this example, the network name of the board is `arria10-62747948036a`.

   Tip: You need this hostname later on to open an SSH connection to the FPGA development kit.

3. Confirm that you have a connection to the development kit from the development host with the `ping` command. Append the `.local` to the host name when you issue the `ping` command:

```
build-host:$ ping arria10-62747948036a.local -c4
PING arria10-62747948036a (192.168.0.23) 56(84) bytes of data.
64 bytes of data from arria10-62747948036a (192.168.0.23): icmp_seq=1 ttl=63
time=1.66ms
64 bytes of data from arria10-62747948036a (192.168.0.23): icmp_seq=1 ttl=63
time=1.66ms
64 bytes of data from arria10-62747948036a (192.168.0.23): icmp_seq=1 ttl=63
time=1.66ms
64 bytes of data from arria10-62747948036a (192.168.0.23): icmp_seq=1 ttl=63
time=1.66ms

--- arria10-62747948036a ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.664/2.037/2.283/0.238 ms
```

You can use the host name when you need to transfer files to the running system by appending the `.local` to the host name. For example, for the host name `arria10-62747948036a`, you can use `arria10-62747948036a.local`.

4.3.6.2.5. Adding Compiled Graphs (AOT files) to the SD Card

An AOT file contains instructions for the FPGA AI Suite IP to "execute" in order to perform inference. For Agilex 5 and Arria 10, the M2M design variant and the S2M design variant require different AOT files. The instructions in this section create both AOT files.

For Agilex 7, the M2M design variant and the S2M design variant use the same AOT file.

To add the compiled graphs to the development kit SD card:

1. Create the $COREDLA_WORK directory, if you have not already done so.
2. Prepare OpenVINO Model Zoo and Model Optimizer.
3. Prepare a model.
4. Confirm that you have the following directory:

   ```
   $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32/
   ```

   If you do not have this directory, confirm that you have completed the first three steps.
5. Compile the graphs.
6. Copy the compiled graphs to the SD card.

Preparing OpenVINO Model Zoo for the SoC Design Example

These instructions assume that you have a copy of OpenVINO Model Zoo 2024.6 in your $COREDLA_WORK/demo/open_model_zoo/ directory.

To download a copy of Model Zoo, run the following commands:

```
cd $COREDLA_WORK/demo
git clone https://github.com/openvinotoolkit/open_model_zoo.git
cd open_model_zoo
git checkout 2024.6.0
```

Preparing a Model for the SoC Design Example

A model must be converted from a framework (such as TensorFlow, Caffe, or Pytorch) into a pair of .bin and .xml files before the FPGA AI Suite compiler (dla_compiler command) can ingest the model.

The following commands download the ResNet-50 TensorFlow model and run theOpenVINO Open Model Zoo tools with the following commands:

```
source ~/build-openvino-dev/openvino_env/bin/activate

omz_downloader --name resnet-50-tf \
    --output_dir $COREDLA_WORK/demo/models/

omz_converter --name resnet-50-tf \
    --download_dir $COREDLA_WORK/demo/models/ \
    --output_dir $COREDLA_WORK/demo/models/
```

The omz_downloader command downloads the trained model to $COREDLA_WORK/demo/models folder. The omz_converter command runs model optimizer that converts the trained model into intermediate representation .bin and .xml files in the $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32/ directory.

The directory $COREDLA_WORK/demo/open_model_zoo/models/public/resnet-50-tf/ contains two useful files that do not appear in the $COREDLA_ROOT/demo/models/ directory tree:

- The README.md file describes background information about the model.
- The model.yml file shows the detailed command-line information given to Model Converter (mo.py) when it converts the model to a pair of .bin and .xml files

For a list OpenVINO Model Zoo models that the FPGA AI Suite supports, refer to Supported Models on page 25.

Troubleshooting OpenVINO Open Model Zoo Converter Errors

You might get the following error while running the `omz_converter` on a TensorFlow model:

```
ValueError: Invalid filepath extension for saving. Please add either a
'.keras' extension for the native Keras format (recommended) or a '.h5'
extension. Use 'model.export(filepath)' if you want to export a SavedModel
for use with TFLite/TFServing/etc.
```

If you get this error, you can follow a process similar to the following example process that convert MobilenetV3 TensorFlow model to an OpenVINO model:

1. Run the following Python code that converts MobileNetV3 to Tensorflow `.savedmodel` format:

```
import os
import tensorflow as tf

COREDLA_WORK = os.environ.get("COREDLA_WORK")
DOWNLOAD_DIR = f"{COREDLA_WORK}/demo/models/"
OUTPUT_DIR = f"{COREDLA_WORK}/demo/models/"

# Set the image data format
tf.keras.backend.set_image_data_format("channels_last")

# Load the MobileNetV3Large model with the specified weights
model = tf.keras.applications.MobileNetV3Large(
    weights=str(
        f"{DOWNLOAD_DIR}/public/mobilenet-v3-large-1.0-224-tf/
weights_mobilenet_v3_large_224_1.0_float.h5"
    )
)
# Save the model to the specified output directory
model.export(filepath=f"{OUTPUT_DIR}/
mobilenet_v3_large_224_1.0_float.savedmodel")
```

2. Run the following command to convert the TensorFlow `.savedmodel` format to OpenVINO model format:

```
mo \
  --input_model=$COREDLA_WORK/demo/models/\
mobilenet_v3_large_224_1.0_float.savedmodel \
  --model_name=mobilenet_v3_large_224_1.0_float \
  --input_shape=[1, 224,224,3]
  --layout nhwc
```

Compiling the Graphs for the SoC Design Example

The precompiled SD card image (`.wic`) provided with the FPGA AI Suite uses one of the following files as the IP architecture configuration file:

- Agilex 5 FPGA E-Series 065B Modular Development Kit

```
AGX5_Performance.arch
```

- Agilex 7 FPGA I-Series Transceiver-SoC Development Kit

```
AGX7_Performance_LayoutTransform.arch
```

- Arria 10 SX SoC FPGA Development Kit

  ```
  A10_Performance.arch
  ```

To create the AOT file for the M2M variant (which uses the `dla_benchmark` utility), run the following command:

```
cd $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32
dla_compiler \
--march $COREDLA_ROOT/example_architectures/<IP arch config file> \
--network-file ./resnet-50-tf.xml \
--foutput-format=open_vino_hetero \
--o $COREDLA_WORK/demo/RN50_Performance_b1.bin \
--batch-size=1 \
--fanalyze-performance
```

where *<IP arch config file>* is one of the IP architecture configuration files listed earlier.

To create the AOT file for the S2M variant (which uses the streaming inference app), run the following command:

```
cd $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32
dla_compiler \
--march $COREDLA_ROOT/example_architectures/<IP arch config file> \
--network-file ./resnet-50-tf.xml \
--foutput-format=open_vino_hetero \
--o $COREDLA_WORK/demo/RN50_Performance_no_folding.bin \
--batch-size=1 \
--fanalyze-performance \
--ffolding-option=0
```

where *<IP arch config file>* is one of the IP architecture configuration files listed earlier.

Agilex 7 devices use the same AOT file created without the `--ffolding-option=0` option for both M2M and S2M operation.

After running either these commands, the compiled models and demonstration files are in the following locations:

Compiled Models	`$COREDLA_WORK/demo/RN50_Performance_b1.bin` `$COREDLA_WORK/demo/RN50_Performance_no_folding.bin`	
Sample Images	`$COREDLA_WORK/demo/sample_images/`	
Architecture File	*Agilex 5*	`$COREDLA_ROOT/example_architectures/AGX5_Performance.arch`
	Agilex 7	`$COREDLA_ROOT/example_architectures/AGX7_Performance_LayoutTransform.arch`
	Arria 10	`$COREDLA_ROOT/example_architectures/A10_Performance.arch`

Copying the Compiled Graphs to the SD card

To copy the required demonstration files to the `/home/root/resnet-50-tf` folder on the SD card:

1. In the serial console, create directories to receive the model data and sample images:

```
mkdir ~/resnet-50-tf
```

2. On the development host, use the secure copy (`scp`) command to copy the data to the board:

```
TARGET_IP=<Development Kit Hostname>.local
TARGET="root@$TARGET_IP:~/resnet-50-tf"
demodir=$COREDLA_WORK/demo
scp $demodir/*.bin $TARGET/.
scp -r $demodir/sample_images/ $TARGET/.
scp $COREDLA_ROOT/example_architectures/<architecture file> $TARGET/.
scp $COREDLA_ROOT/build_os.txt $TARGET/../app/
```

   where *<architecture file>* is one of the following files, depending on your development kit:

   - Agilex 5 FPGA E-Series 065B Modular Development Kit

     ```
     AGX5_Performance.arch
     ```

   - Agilex 7 FPGA I-Series Transceiver-SoC Development Kit

     ```
     AGX7_Performance_LayoutTransform.arch
     ```

   - Arria 10 SX SoC FPGA Development Kit

     ```
     A10_Performance.arch
     ```

3. [Optional] In the serial console run the `sync` command to ensure that the data is flushed to disk.

4.3.6.2.6. Verifying FPGA Device Drivers

The device drivers should be loaded when the HPS boots. Verify that the device drivers are initialized by checking that `uio` files are listed in `/sys/class/uio` by running the following command:

```
ls /sys/class/uio
```

The command should show output similar to the following example:

```
uio0 uio1 uio2
```

If the drivers are not listed, refresh the modules by running the following command before checking again that the drivers are loaded:

```
uio-devices restart
```

5. FPGA AI Suite Tutorial and Design Example Demonstration Applications

5.1. FPGA AI Suite Quick Start Tutorial

The examples in this section assume that you have installed the FPGA AI Suite according to the instructions in Installing the FPGA AI Suite Compiler and IP Generation Tools on page 67.

Ensure that you the required environment variables set. For details, refer to Setting Required Environment Variables on page 85.

A quick command to check that the FPGA AI Suite installation was successful is as follows:

```
dla_compiler --fanalyze-area --march $COREDLA_ROOT/*/AGX7_Generic.arch
```

5.1.1. Preparing OpenVINO Model Zoo for the PCIe Design Example

These instructions assume that you have a copy of OpenVINO Model Zoo 2024.6 in your `$COREDLA_WORK/demo/open_model_zoo/` directory.

To download a copy of Model Zoo, run the following commands:

```
cd $COREDLA_WORK/demo
git clone https://github.com/openvinotoolkit/open_model_zoo.git
cd open_model_zoo
git checkout 2024.6.0
```

5.1.2. Preparing a Model for the PCIe Design Example

A model must be converted from a framework (such as TensorFlow, Caffe, or Pytorch) into a pair of `.bin` and `.xml` files before the FPGA AI Suite compiler (`dla_compiler` command) can ingest the model.

The following commands download the ResNet-50 TensorFlow model and run theOpenVINO Open Model Zoo tools with the following commands:

```
source ~/build-openvino-dev/openvino_env/bin/activate

omz_downloader --name resnet-50-tf \
    --output_dir $COREDLA_WORK/demo/models/

omz_converter --name resnet-50-tf \
    --download_dir $COREDLA_WORK/demo/models/ \
    --output_dir $COREDLA_WORK/demo/models/
```

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

The `omz_downloader` command downloads the trained model to `$COREDLA_WORK/demo/models` folder. The `omz_converter` command runs model optimizer that converts the trained model into intermediate representation `.bin` and `.xml` files in the `$COREDLA_WORK/demo/models/public/resnet-50-tf/FP32/` directory.

The directory `$COREDLA_WORK/demo/open_model_zoo/models/public/resnet-50-tf/` contains two useful files that do not appear in the `$COREDLA_ROOT/demo/models/` directory tree:

- The `README.md` file describes background information about the model.
- The `model.yml` file shows the detailed command-line information given to Model Converter (`mo.py`) when it converts the model to a pair of `.bin` and `.xml` files

For a list OpenVINO Model Zoo models that the FPGA AI Suite supports, refer to Supported Models on page 25.

Troubleshooting OpenVINO Open Model Zoo Converter Errors

You might get the following error while running the `omz_converter` on a TensorFlow model:

```
ValueError: Invalid filepath extension for saving. Please add either a
'.keras' extension for the native Keras format (recommended) or a '.h5'
extension. Use 'model.export(filepath)' if you want to export a SavedModel
for use with TFLite/TFServing/etc.
```

If you get this error, you can follow a process similar to the following example process that convert MobilenetV3 TensorFlow model to an OpenVINO model:

1. Run the following Python code that converts MobileNetV3 to Tensorflow `.savedmodel` format:

```
import os
import tensorflow as tf

COREDLA_WORK = os.environ.get("COREDLA_WORK")
DOWNLOAD_DIR = f"{COREDLA_WORK}/demo/models/"
OUTPUT_DIR = f"{COREDLA_WORK}/demo/models/"

# Set the image data format
tf.keras.backend.set_image_data_format("channels_last")

# Load the MobileNetV3Large model with the specified weights
model = tf.keras.applications.MobileNetV3Large(
    weights=str(
        f"{DOWNLOAD_DIR}/public/mobilenet-v3-large-1.0-224-tf/
weights_mobilenet_v3_large_224_1.0_float.h5"
    )
)
# Save the model to the specified output directory
model.export(filepath=f"{OUTPUT_DIR}/
mobilenet_v3_large_224_1.0_float.savedmodel")
```

2. Run the following command to convert the TensorFlow `.savedmodel` format to OpenVINO model format:

```
mo \
  --input_model=$COREDLA_WORK/demo/models/\
mobilenet_v3_large_224_1.0_float.savedmodel \
  --model_name=mobilenet_v3_large_224_1.0_float \
  --input_shape=[1, 224,224,3]
  --layout nhwc
```

5.1.3. Running the Graph Compiler

The FPGA AI Suite compiler (`dla_compiler` tool) can estimate the performance of a graph and IP architecture combination. The tool can also produce a compiled network that the runtime uses in AOT (ahead-of-time) mode to control inference on the IP.

The following commands generate an AOT file for Agilex 7 devices:

```
cd $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32

dla_compiler \
      --march $COREDLA_ROOT/example_architectures/AGX7_Performance.arch \
      --network-file ./resnet-50-tf.xml \
      --foutput-format=open_vino_hetero \
      --o $COREDLA_WORK/demo/RN50_Performance_b1.bin \
      --batch-size=1 \
      --fanalyze-performance
```

These command specify the following input files:

- The `.arch` file specifies the configuration of the IP architecture (configuration parameters such as large, small, and which activations to use).
- The `.xml` file (along with the `.bin`) describes the network.
- The `.bin` file stores certain parameters along with the model weights.

The FPGA AI Suite compiler uses these files to analyze the performance estimate.

The output file `RN50_Performance_b1.bin` is the compiled network and contains the instructions and graph weights necessary to control the FPGA AI Suite IP on the FPGA device. This output file is specific to the `.arch` file and the graph `.xml` file.

The compiler dot graph shows which layers run on the FPGA and which layers run on the CPU. Using the Graphviz package, the dot file format can be converted into an SVG graphics file named `ResNet-50.svg` that shows each layer. You can view SVG graphics in most web browsers. Use the following command to convert the dot graph file into an SVG file:

```
dot -Tsvg hetero_subgraphs_resnet-50-tf.dot -o ResNet-50.svg
```

5.1.4. Preparing an Image Set

This section describes how to prepare an image set for classification graphs that requires 224x224 input and have been trained on the ImageNet classifications. For the `yolo-v3-tf` and `yolo-v3-tiny-tf` graphs, the instructions in the Running the PCIe-Attach Design Example Demonstration Applications on page 143 describe how to prepare an image set, ground truth data, and how to call the `dla_benchmark` application.

The FPGA AI Suite includes sample validation images in the `$COREDLA_ROOT/demo/sample_images/` folder.

The contents are as follows:

`$COREDLA_WORK/demo/sample_images/`	Sample image directory. A file naming convention is used to provide a sort order for the images.
`$COREDLA_WORK/demo/sample_images/ground_truth.txt`	Ground truth file.
`$COREDLA_WORK/demo/sample_images/TF_ground_truth.txt`	Ground truth file suitable for graphs from the TensorFlow framework.

Image classification graphs trained in the TensorFlow framework require ground truth files that account for a difference in how TensorFlow numbers the output categories (an off-by-one difference). The sample image set includes two ground truth files to account for this.

Due to the small number of images in the sample image set, the `dla_benchmark` commands in this section for the sample image set `-niter=8`. This setting limits the number of inferences executed by the FPGA AI Suite IP.

To help performance benchmarking, the `dla_benchmark` demo generates random image data if it requires more input images than are available in the image set.

Optionally, you can run the demo with the ILSVRC2012 image set. To use this image set, manually download it as follows:

1. The image set is available from https://image-net.org/download-images.php.
2. Download the **2012** "Validation images" and the **2017** development kit.
3. Convert the 2012 images to `.bmp` format, resizing the smallest dimension to 256 while preserving aspect ratio and center crop to 224x224.

The image preprocessing can be done with many different tools. One of the common Linux tools is ImageMagick. For details, refer to https://imagemagick.org/.

The following commands use ImageMagick to prepare the images:

```
mkdir imagenet_jpg

cd imagenet_jpg

tar xf /path/to/ILSVRC2012_img_val.tar

mkdir ../imagenet

echo *.JPEG | xargs mogrify \
    -path ../imagenet \
    -resize '256x256^' \
    -gravity Center \
    -crop 224x224+0+0 \
    -format bmp
```

Create the `TF_ground_truth.txt` file from the 2017 development with these commands:

```
tar zxf ILSVRC2017_devkit.tar.gz

cd ILSVRC/devkit/data

export cnt=1
```

```
cp ILSVRC2015_clsloc_validation_ground_truth.txt TF_ground_truth.txt

cat map_clsloc.txt | sort | (
    while read a; do
        orig=$(echo $a | awk '{print $2}')
        sed -i -e "s/^$orig\$/_$cnt/" TF_ground_truth.txt
        cnt=$(($cnt+1))
    done
)


sed -i -e 's/^_//' TF_ground_truth.txt
```

If you want the demo to use the ImageNet images instead of the sample images bundled with the FPGA AI Suite make the following changes to the `dla_benchmark` example commands in Performing Inference on the PCIe Design Example on page 135:

1. Set `-niter=5000` instead of `-niter=8`.
2. Set `-groundtruth_loc` to specify the `TF_ground_truth.txt` created earlier.
3. Set `-i` to specify the directory containing the converted ILSVRC2012 `.bmp` images.

   The `.bmp` format is lossless and preserves as much details as possible.

5.1.5. Programming the FPGA Device

If you do not have an FPGA board installed, you can still run inference on the software emulation model. For instruction, skip ahead to Performing Inference Without an FPGA Board on page 142.

To complete this section of the tutorial, you must have installed the following supported design example:

- PCIe design example for Agilex 7 devices

  This design example requires the Terasic DE10-Agilex Development Board.

For installation instructions, see Installing FPGA AI Suite on page 61.

Important: The instructions here assume that your machine has only a single board installed.

The PCIe design examples requires a runtime layer that must be compiled before the design is usable. The runtime for the design examples typically needs to be compiled only once. However, a new FPGA bitstream must be generated whenever any of the following parameters change:

- Number of IP instances built in the design example.
  - ☐ PCIe design example for Agilex 7 devices supports up to 4 instances.
- Configuration of the IP architecture (as specified by a `.arch` file)
- Quartus fitter seed (or any other "try-harder" compilation options)

Depending on your design example, follow the instructions in the following section:

- Programming the Agilex 7 Device (Terasic DE10-Agilex Development Board) on page 135

Programming the Agilex 7 Device (Terasic DE10-Agilex Development Board)

To program the DE10-Agilex-B2E2 board required by the PCIe design example for Agilex 7 devices:

1. Build the runtime with the following commands:

```
cd $COREDLA_WORK/runtime

rm -rf build_Release

./build_runtime.sh -target_de10_agilex
```

2. Program the bitstream onto the FPGA device with the following commands:

```
jtagdir=$COREDLA_WORK/runtime/build_Release/fpga_jtag_reprogram

bitsdir=$COREDLA_WORK/demo/bitstreams

$jtagdir/fpga_jtag_reprogram $bitsdir/AGX7_Performance_DE10_Agilex.sof

curarch=$COREDLA_ROOT/example_architectures/AGX7_Performance.arch
```

The bitstream completely reprograms the board, which causes it to disconnect from the PCIe bus. The board is kept accessible with the `$COREDLA_WORK/runtime/fpga_jtag_reprogram` tool.

Precompiled bitstreams are provided in the `$COREDLA_WORK/demo/bitstreams/` directory. Some bitstreams instantiate a large IP, while others instantiate a small IP. There are also differences in which bitstreams support which neural networks. For details, refer to the Supported Models on page 25.

5.1.6. Performing Inference on the PCIe Design Example

Performing Inference Using JIT Mode

The JIT (just-in-time) mode causes the `dla_benchmark` demonstration application to call the `dla_compiler` command in a just-in-time way to compile the neural net graph.

If you do not have images and ground truth files, you can skip the optional `-i` and `-groundtruth_loc` parameter entries in the command that follows. If you skip these parameters, the `dla_benchmark` demonstration application generates randomized image data.

The value for `$curarch` must match the bitstream that you programmed in Programming the FPGA Device on page 134.

```
imagedir=$COREDLA_WORK/demo/sample_images

xmldir=$COREDLA_WORK/demo/models/public/
```

```
$COREDLA_WORK/runtime/build_Release/dla_benchmark/dla_benchmark \
    -b=1 \
    -m $xmldir/resnet-50-tf/FP32/resnet-50-tf.xml \
    -d=HETERO:FPGA,CPU \
    -niter=8 \
    -plugins $COREDLA_WORK/runtime/build_Release/plugins.xml \
    -arch_file $curarch \
    -api=async \
    -perf_est \
    -nireq=4 \
    -bgr \
    -i $imagedir \
    -groundtruth_loc $imagedir/TF_ground_truth.txt
```

Performing Inference Using AOT Mode

In AOT (ahead-of-time) mode, the `dla_benchmark` demonstration application uses a compiled network that was previously produced by the `dla_compiler` compiler command when you followed the steps in Running the Graph Compiler on page 132.

To use AOT mode instead of JIT mode:

1. Add the `-cm` argument to specify the name of the file containing the compiled network
2. Remove the `-perf_est` flag. The `dla_benchmark` demonstration application does not estimate performance in AOT mode.

If you omit `-i` and `-groundtruth_loc` arguments, the `dla_benchmark` demonstration application generates random input data that is useful only for performance benchmarking.

```
gt_file=$COREDLA_WORK/demo/sample_images/TF_ground_truth.txt

$COREDLA_WORK/runtime/build_Release/dla_benchmark/dla_benchmark \
    -b=1 \
    -cm $COREDLA_WORK/demo/RN50_Performance_b1.bin \
    -d=HETERO:FPGA,CPU \
    -niter=8 \
    -plugins $COREDLA_WORK/runtime/build_Release/plugins.xml \
    -arch_file $curarch \
    -api=async \
    -nireq=4 \
    -bgr \
    -i $COREDLA_WORK/demo/sample_images/ \
    -groundtruth_loc $gt_file
```

The `-cm` argument points to the `.bin` file that you created in Running the Graph Compiler on page 132.

Inference APIs

The easiest way to evaluate the ability of the FPGA AI Suite to perform inference is to use the `dla_benchmark` demonstration application that is included in the example runtime and is built as part of the steps described in Programming the FPGA Device on page 134.

The example runtime also includes instructions on how to use the OpenVINO Python API to execute inference using the JIT style described in Performing Inference Using JIT Mode on page 135.

These instructions are located in `$COREDLA_WORK/runtime/python_demos/README.md`.

5.1.7. Building an FPGA Bitstream for the PCIe Design Example

To complete this portion of the tutorial, you must meet the following prerequisites:

- You must have a license for bitstream generation of the FPGA AI Suite IP.
- You must have a specific version of Quartus Prime Pro Edition installed:
  - ☐ The PCIe design example for Agilex 7 devices requires Quartus Prime Pro Edition Version 22.4 or later. This document assumes that Version 25.3 is used.
- You must have the following paths included in your `$PATH` environment variable:
  - ☐ `quartus/bin`
  - ☐ `qsys/bin`

If you do not have a license for FPGA AI Suite, the generated IP has a built-in inference-count limitation. Any inference operations that occur after the limit is reached generate an error message that indicates that a license is required. To reset the inference count limit, you must reprogram the bitstream onto the FPGA device.

Building an FPGA Bitstream for the PCIe Design Example for Agilex 7 Devices

Run the following command for the PCIe design example for Agilex 7 devices:

```
cd $COREDLA_WORK/demo

dla_build_example_design.py build agx7_de10_pcie \
    $COREDLA_ROOT/example_architectures/AGX7_Performance.arch \
    --output-dir $COREDLA_WORK/demo/my_bitstreams/
```

If the `AOCL_BOARD_PACKAGE_ROOT` environment variable is not set, the `dla_build_example_design.py` command returns an error message. To set this environment variable, review the instructions in Additional Software Prerequisites for the FPGA AI Suite PCIe Design Example for Agilex 7 Devices on page 89.

The `dla_build_example_design.py` command has a number of actions it can take. For a full list of actions, run the following command:

```
dla_build_example_design.py --help
```

For a list of options supported by an action, run the following command:

```
dla_build_example_design.py <action> --help
```

For example, the `dla_build_example_design.p build` command provides a `--seed` option that you can use to vary the Quartus Prime random seed.

This commands places the bitstreams into the `my_bitstreams` directory. The bitstream is named `AGX7_Performance.sof`.

After the bitstream is built, you must program it onto the FPGA following the instructions in section Programming the FPGA Device on page 134.

5.1.8. Building the Example FPGA Bitstreams

The supplied bitstreams are generated from the example architectures in `$COREDLA_ROOT/example_architectures`. You can regenerate these bitstreams with the `dla_build_example_design.py build` command.

For example, the example `dla_build_example_design.py build` command in Building an FPGA Bitstream for the PCIe Design Example on page 137 builds the `AGX7_Performance_DE10_Agilex.sof` example bitstream.

5.1.9. Preparing a ResNet50 v1 Model

OpenVINO Model Zoo 2024.6 does not include a ResNet50 v1 model.

The following commands create `graph.xml` and `graph.bin` files for ResNet50 v1, using the `mo_caffe.py` command from OpenVINO Model Converter. These commands assume that you have enabled OpenVINO Model Converter as described Preparing OpenVINO Model Zoo for the PCIe Design Example on page 130.

```
wget https://www.deepdetect.com/models/resnet/ResNet-50-model.caffemodel
```

```
wget https://raw.githubusercontent.com/yihui-he/\
resnet-imagenet-caffe/master/resnet_50/ResNet-50-deploy.prototxt
```

```
source ~/build-openvino-dev/openvino_env/bin/activate
mo \
    --input_shape=[1,3,224,224] \
    --mean_values=[103.53,116.28,123.675] \
    --input=data \
    --output=prob \
    --input_model=ResNet-50-model.caffemodel \
    --input_proto=ResNet-50-deploy.prototxt \
    --model_name=graph
```

5.1.10. Performing Inference on the Inflated 3D (I3D) Graph

Before you try the instructions in this section, ensure that you have completed the following tasks:

- Set up OpenVINO Model Zoo as described in Preparing a Model for the PCIe Design Example on page 130.
- Set up Model Converter as described in Preparing OpenVINO Model Zoo for the PCIe Design Example on page 130.
- Program the FPGA and initialize the `$curarch` environment variable as described in Programming the FPGA Device on page 134

Remember: A model must be converted from a framework (such as Tensorflow, Caffe, or PyTorch) into a `.bin`/`.xml` file pair before the FPGA AI Suite compiler (`dla_compiler` command) can ingest the model.

Preparing the i3d-rgb-tf Model

The following commands download the i3d-rgb-tf TensorFlow model and run Model Converter:

```
source ~/build-openvino-dev/openvino_env/bin/activate

omz_downloader --name i3d-rgb-tf \
   --output_dir $COREDLA_WORK/demo/models/

omz_converter --name i3d-rgb-tf \
   --download_dir $COREDLA_WORK/demo/models/ \
   --output_dir $COREDLA_WORK/demo/models/
```

After running these commands, you can find the model in the following directory:

```
demo/models/public/i3d-rgb-tf/FP32/
```

Inference on the i3d-rgb-tf Model and Accuracy Calculations

This section details how to run inference on i3d-rgb-tf with the sample videos and ground truth files provided. For 3D input, the batch size determines how many "clips" a video should be broken down into.

To run inference on the i3d-rgb-tf model with 3 clips per video, run the following commands:

```
cd $COREDLA_WORK/runtime/build_Release/dla_benchmark

gnd=$COREDLA_WORK/demo/sample_videos/groundtruth_batch_size_3.txt

./dla_benchmark \
   -b=3 \
   -niter=4 \
   -m $COREDLA_WORK/demo/models/public/i3d-rgb-tf/FP32/i3d-rgb-tf.xml \
   -d=HETERO:FPGA,CPU \
   -i=$COREDLA_WORK/demo/sample_videos/ \
   -plugins=../plugins.xml \
   -arch_file=$curarch \
   -api=async \
   -nireq=4 \
   -perf_est \
   -groundtruth_loc=$gnd \
   -folding_option=1
```

To run inference on the i3d-rgb-tf model with 1 clip per video, change the ground truth file (-groundtruth_loc) and the -b flag as follows:

```
cd $COREDLA_WORK/runtime/build_Release/dla_benchmark

gnd=$COREDLA_WORK/demo/sample_videos/groundtruth_batch_size_1.txt
```

```
./dla_benchmark \
   -b=1 \
   -niter=4 \
   -m $COREDLA_WORK/demo/models/public/i3d-rgb-tf/FP32/i3d-rgb-tf.xml \
   -d=HETERO:FPGA,CPU \
   -i=$COREDLA_WORK/demo/sample_videos/ \
   -plugins=../plugins.xml \
   -arch_file=$curarch \
   -api=async \
   -nireq=4 \
   -perf_est \
   -groundtruth_loc=$gnd \
   -folding_option=1
```

Important: Loading the video data with the `dla_benchmark` command places a high memory demand on the host. Using values for the `-niter` option that are less than 100 results in manageable memory requirements for most hosts. If the memory usage is too high, the operating system might abruptly end the running `dla_benchmark` command.

Ground truth files are included for both 3 clips per video, and 1 clip per video. To use a different number of clips per video:

1. Create a new ground truth text file that contains the correct action class for each video being repeated the same amount of time as the amount of clips per video

   See the `groundtruth_batch_size_3.txt` file for reference. The ground truths are repeated 3 times each in that file.
2. Change the `-groundtruth_loc` option to point to the new ground truth text file that you created.

To run the `dla_benchmark` command on the CPU, change the `-d=HETERO:FPGA,CPU` option to `-d=HETERO:CPU`.

5.1.11. Performing Inference on YOLOv3 and Calculating Accuracy Metrics

This tutorial targets Agilex 7 devices and assumes that you have the following prerequisites completed:

- You have a copy of of OpenVINO Model Zoo 2024.6 in your `$COREDLA_ROOT/demo/open_model_zoo/` directory.
- You have completed the steps in Preparing OpenVINO Model Zoo for the PCIe Design Example on page 130.
- You have programmed the Agilex 7 FPGA device with a bitstream that corresponds to `AGX7_Performance.arch`. For example instructions, refer to Building an FPGA Bitstream for the PCIe Design Example on page 137.

The steps in the sections that follow guide you through preparing a YOLOv3 model for inference, preparing a COCO validation dataset and annotations, as well as calculating mAP and COCO AP metrics.

5.1.11.1. Preparing a YOLOv3 Model

As stated in Preparing a Model for the PCIe Design Example on page 130, a model must be converted from a framework (such as TensorFlow, Caffe, or Pytorch) into a pair of `.bin` and `.xml` files before the FPGA AI Suite compiler (`dla_compiler` command) can ingest the model.

The following commands download the YOLOv3 TensorFlow model and run Model Optimizer:

```
source ~/build-openvino-dev/openvino_env/bin/activate

omz_downloader --name yolo-v3-tf \
    --output_dir $COREDLA_WORK/demo/models/


omz_converter --name yolo-v3-tf \
    --download_dir $COREDLA_WORK/demo/models/ \
    --output_dir $COREDLA_WORK/demo/models/
```

These commands create `.bin` and `.xml` files in the `demo/models/public/yolo-v3-tf/FP32/` directory.

5.1.11.2. Preparing a COCO Validation Dataset and Annotations

Use the publicly available COCO 2017 validation images as input to the model and the COCO 2017 annotations as the ground-truth.

You can download the images from the following URL: http://images.cocodataset.org/zips/val2017.zip.

You can download the annotations from the following URL: http://images.cocodataset.org/annotations/annotations_trainval2017.zip.

1. Build the runtime with the following commands:

```
cd $COREDLA_WORK/runtime

rm -rf build_Release

./build_runtime.sh -target_de10_agilex
```

2. Download and extract both `.zip` files into the `coco-images` directory:

```
cd $COREDLA_WORK/runtime

mkdir coco-images

cd coco-images

wget http://images.cocodataset.org/zips/val2017.zip

wget http://images.cocodataset.org/annotations/annotations_trainval2017.zip

unzip annotations_trainval2017.zip

unzip val2017.zip
```

3. The `dla_benchmark` application allows only plain text ground truth files, so use the `convert_annotations.py` script to setup the groundtruth directory as follows:

```
cd $COREDLA_WORK/runtime/coco-images

mkdir groundtruth

python3 \
  ../dla_benchmark/convert_annotations.py \
  annotations/instances_val2017.json \
  groundtruth
```

5.1.11.3. Inference on YOLOv3 and Calculating Accuracy Scores

To run inference on YOLOv3 and calculate the mAP and COCO AP scores, run the following commands:

```
cd $COREDLA_WORK/runtime

./build_Release/dla_benchmark/dla_benchmark \
   -b=1 \
   -niter=5000 \
   -m $COREDLA_WORK/demo/models/public/yolo-v3-tf/FP32/yolo-v3-tf.xml \
   -d=HETERO:FPGA,CPU \
   -i=./coco-images/val2017 \
   -plugins=./build_Release/plugins.xml \
   -arch_file=$COREDLA_ROOT/example_architectures/AGX7_Performance.arch \
   -yolo_version=yolo-v3-tf \
   -api=async \
   -groundtruth_loc=./coco-images/groundtruth \
   -nireq=4 \
   -enable_object_detection_ap \
   -perf_est \
   -bgr
```

The `dla_benchmark` command prints the mAP and COCO AP scores and saves a text file called `ap_report.txt` that contains the scores in the current working directory.

To enable the accuracy checking routine for object detection graphs such as YOLOv3, use the `-enable_object_detection_ap=1` option of the `dla_benchmark` command. This flag directs the `dla_benchmark` command to calculate the mAP and COCO AP for object detection graphs.

Also, specify the version of the YOLO graph that you provide to the `dla_benchmark` command with the `-yolo_version= yolo-v3-tf` option.

The input images folder is specified with `-i=./coco-images` and the ground truth annotations is specified with `-groundtruth_loc=./groundtruth`. If you chose to save the images or the ground truth annotations to a location other than the ones specified in this tutorial, update these parameters to point to the correct location.

5.1.12. Performing Inference Without an FPGA Board

The software emulation model is a C++ software model of the FPGA AI Suite IP that is bit-accurate(*). The emulation models relatively low-level transactions and, in some cases, models processing delays of modules.

To use the software emulation model:

1. Build the runtime with the following commands:

```
cd $COREDLA_WORK/runtime

rm -rf build_Release

./build_runtime.sh -target_emulation
```

2. Run inference with the `-niter=1` and `-nireq=1` options (because the software model is reasonably slow) with the following commands:

```
modeldir=$COREDLA_WORK/demo/models/public

imagedir=$COREDLA_WORK/demo/sample_images

curarch=$COREDLA_ROOT/example_architectures/AGX7_Performance.arch

gnd=$imagedir/TF_ground_truth.txt

cd $COREDLA_WORK/runtime/build_Release/dla_benchmark

./dla_benchmark \
   -b 1  `# Run only a single image` \
   -niter 1  `# Run only a single image` \
   -nireq 1  `# Running emulator: so -nireq=1` \
   -m $modeldir/resnet-50-tf/FP32/resnet-50-tf.xml \
    `# Same as when running on hardware - specify the graph` \
   -d HETERO:FPGA,CPU \
    `# Same as when running on hardware - \
    use FPGA if possible, fallback to CPU` \
   -i $imagedir  \
   `# Same as when running on hardware - specify image directory` \
   -arch_file $curarch \
   `# Same as when running on hardware - specify .arch file` \
   -dump_output \
   `# Dump output result.txt file` \
   -plugins emulation \
   `# Use the software emulator` \
   -groundtruth_loc $gnd
   `# Location of the ground truth file for $imagedir`
```

5.2. FPGA AI Suite Design Example Demonstration Applications

The FPGA AI Suite design examples come with demonstration applications, so that you can get a sense of how the design examples work. Refer to the following sections for instructions on running the design examples:

- Running the PCIe-Attach Design Example Demonstration Applications on page 143
- Running the Hostless DDR-Free Design Example on page 156
- Running the JTAG Design Example Demonstration Application on page 159
- Running the SoC Design Example Demonstration Applications on page 163

5.2.1. Running the PCIe-Attach Design Example Demonstration Applications

This section describes the steps to run the demonstration application and perform accelerated inference using the PCIe-attach design example.

(*) Minor rounding differences between software emulation and hardware will typically result in differences of less than two units of least precision (ulps).

The instructions in this section apply to both the PCIe-attach design example and OFS for PCIe-attach design example.

5.2.1.1. Setup the OFS Environment for the FPGA Device in the OFS for PCIe-Attach Design Example

Restriction: These instructions are required only if you want to run the PCIe-Attach design example on a OFS FPGA board. If you are not using an OFS FPGA board, you can skip this section.

Before you can program the FPGA device with the OFS for PCIe attach design example, you must set up the FPGA device with the OFS framework components and ensure that the OPAE drivers on the host system run correctly.

These steps must be done whenever the system hosting the FPGA board is power-cycled or soft-rebooted.

To set up the FPGA device:

1. Ensure that the PCIe bifurcation BIOS setting on the host machine that hosts the FPGA card is set as follows, depending on the target board:
   - Agilex 7 FPGA I-Series Development Kit: `x8`
   - FPGA SmartNIC N6001-PL Platform: `Auto`
2. Program the FPGA devices with the `.sof` file for the OFS 2025.1-1 slim FIM for your board:
   - Agilex 7 FPGA I-Series Development Kit: https://github.com/OFS/ofs-agx7-pcie-attach/releases/download/ofs-2025.1-1/iseries-dk-slimfim-images_ofs-2025-1-1.tar.gz
   - FPGA SmartNIC N6001-PL Platform: https://github.com/OFS/ofs-agx7-pcie-attach/releases/download/ofs-2025.1-1/n6001-slimfim-images_ofs-2025-1-1.tar.gz

   Program the FPGA with the following command:

   ```
   quartus_pgm -c 1 -m jtag \
               -o "p;<path to the sof file>/ofs_top.sof@1"
   ```
3. Soft-reboot the machine with the following command:

   ```
   sudo reboot
   ```

   A soft reboot is required whenever you program the FPGA with a `.sof` file (SRAM object file) so that the PCIe host can reenumerate the attached devices. A hard reboot or power cycle would require you to reprogram the FPGA device with the earlier command.
4. If you want to use a non-root user to run inference on the FPGA board, complete the following steps:
   a. Set user process resource limits as follows:
      i. Update your `/etc/security/limits.conf` file to remove the limit for locked memory for yourself or for all regular users.

      For details, refer to *Memlock limit* in the following documents:

- OFS Getting Started Guide for Agilex 7 SoC Attach FPGAs
- Getting Started Guide: Open FPGA Stack for Agilex FPGAs Targeting the FPGA SmartNIC N6000/1-PL

ii. Log out of your current session and log back in.

iii. Run the `ulimit -l` command to ensure that limits are set to `unlimited`.

b. Enable huge pages help improve the performance of DMA operations between host and FPGA device. Enable huge pages as follows:

i. Create a rule file at `/etc/sysctl.d/intel-fpga-ofs-sysctl.conf` with the following content:

```
vm.nr_hugepages = 2048
```

ii. Create a rule file at `/sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages` with the following content:

```
2048
```

c. Set the permissions for the OFS device feature list (DFL) framework as follows:

i. Create a rule file at `/etc/udev/rules.d` named `90-intel-fpga-ofs.rules` with the following content:

```
KERNEL=="dfl-fme.[0-9]", ACTION=="add|change", GROUP="root",
MODE="0666", RUN+="/bin/bash -c 'chmod 0666 %S%p/errors/ /dev/%k'"

KERNEL=="dfl-port.[0-9]", ACTION=="add|change", GROUP="root",
MODE="0666", RUN+="/bin/bash -c 'chmod 0666 %S%p/dfl/userclk/
frequency %S%p/errors/* /dev/%k'"
```

Ensure you enter the content as two lines only. The lines are line-wrapped only due to document formatting restrictions.

ii. Run the following commands:

```
sudo udevadm control --reload

sudo udevadm trigger /dev/dfl-fme.0

sudo udevadm trigger /dev/dfl-port.0
```

d. Set the permissions for userspace I/O (UIO) devices as follows:

i. Create a rule file at `/etc/udev/rules.d` named `uio.rules` with the following content:

```
SUBSYSTEM=="uio" KERNEL=="uio*" MODE="0666"
```

ii. Run the following commands:

```
sudo udevadm control --reload

sudo udevadm trigger --subsystem-match=uio --settle
```

e. Initialize the OPAE SDK.

You must initialize the OPAE SDK after every system power cycle or soft reboot. You can make this initialization persistent by using a `systemd` startup service.

To initialize the OPAE SDK:

i. Determine the PCIe B:d.F (system, bus, device, function) of your board by running the `fpgainfo` command:

```
sudo fpgainfo fme
```

In the output, look for a line similar to the following line:

```
PCIe s:b:d.f                    : 0000:03:00.0
```

ii. (For Agilex 7 FPGA I-Series Development Kit only) Assign the first SR-IOF virtual function to the FPGA board with the following command:

```
sudo pci_device s:b:d.f vf 1
```

iii. Initialize `opae.io` with the following command:

```
sudo opae.io init -d s:b:d.1 <your user name>
```

Note: The original function (*f* in *s:b:d.f*) value that the `fpgainfo` command reported is replaced here by `1`.

5. Ensure that the `OPAE_PLATFORM_ROOT` environment variable points to your OFS FPGA interface manager (FIM) `pr_build_template` directory.

5.2.1.2. Exporting Trained Graphs from Source Frameworks in the PCIe Design Example

Before running any demonstration application, you must convert the trained model to the Inference Engine format (`.xml`, `.bin`) with the OpenVINO Model Optimizer.

For details on creating the `.bin/.xml` files, refer to the OpenVINO Model Converter / Conversion API documentation .

5.2.1.3. Compiling Exported Graphs Through the FPGA AI Suite in the PCIe Design Example

The network as described in the `.xml` and `.bin` files (created by the Model Converter) is compiled for a specific FPGA AI Suite architecture file by using the FPGA AI Suite compiler.

The FPGA AI Suite compiler compiles the network and exports it to a `.bin` file that uses the same `.bin` format as required by the OpenVINO Inference Engine.

This `.bin` file created by the compiler contains the compiled network parameters for all the target devices (FPGA, CPU, or both) along with the weights and biases. The inference application imports this file at runtime.

The FPGA AI Suite compiler can also compile the graph and provide estimated area or performance metrics for a given architecture file or produce an optimized architecture file.

For more details about the FPGA AI Suite compiler, refer to Compiling Your Model with the FPGA AI Suite Compiler on page 193.

5.2.1.4. Compiling the PCIe Design Example

Prepackaged bitstreams are available for the PCIe design example. If the prepackaged bitstreams are installed, they are installed in `demo/bitstreams/`.

The prepackaged bitstreams are available from the **Bitstreams and SD Card Images** tab of the FPGA AI Suite download page:

https://www.altera.com/downloads/add-on-development-tools/altera-fpga-ai-suite-current

To build design example bitstreams, you must have a license that permits bitstream generation for the IP, and have the correct version of Quartus Prime software installed. Use the `dla_build_example_design.py` utility to create a bitstream.

For details about this command, the steps it performs, and advanced command options, refer to The dla_build_example_design Command on page 39.

Additional Instructions for the OFS for PCIe Attach Design Example

Before running the `dla_build_example_design.py` utility, ensure that the `OPAE_PLATFORM_ROOT` environment variable points to your OFS FPGA interface manager (FIM) `pr_build_template` directory. If you do not want to compile your own FIM, you can get prebuilt OFS FIM binaries for boards supported by the Agilex 7 OFS for PCIe Attach reference shells on GitHub at the following URL:

```
https://github.com/OFS/ofs-agx7-pcie-attach/releases/
```

The FPGA AI Suite OFS for PCIe Attach design example is based on the OFS 2025.1-1 release of the reference shells.

The `dla_build_example_design.py` utility generates a wrapper that wraps one or more FPGA AI Suite IP instances along with adapters required to connect to the OFS slim FIM.

Important: The OFS FIM uses FPGA resources as well as the FPGA AI Suite IP instances. Keep the FPGA resource limitations in mind when deciding on how many FPGA AI Suite IP instances to use.

Get an estimate of the FPGA resource required for a single FPGA AI Suite IP instance by using the `--fanalyze-area` option of the `dla_compiler`. Use the single instance values to determine the resources required for the number of instances that you want. For more details, see Analyzer Tool Options (dla_compiler Command Options) on page 213.

To generate an FPGA bitstream for the OFS for PCIe Attach design example for the Agilex 7 FPGA I-Series Development Kit with two FPGA AI Suite IP instances, run the following commands:

```
cd $COREDLA_WORK

dla_build_example_design.py build \
  --output-dir build_generic_2inst \
  --seed 1 \
  --num-instances 2
  agx7_iseries_ofs_pcie \
  $COREDLA_ROOT/example_architectures/AGX7_Generic.arch \
```

This command generates a green bitstream (GBS) file called `AGX7_Generic.gbs` that can be found in the `$COREDLA_WORK/build_generics_2inst/` folder.

5.2.1.5. Programming the FPGA Device in the PCIe Design Example

Depending on your target FPGA board, follow the instructions in this section as follows:

- **Agilex 7 FPGA I-Series Development Kit**: Follow the instructions in Programming the FPGA Green Bitstream for an OFS-based FPGA Board on page 148
- **FPGA SmartNIC N6001-PL Platform**: Follow the instructions in Programming the FPGA Green Bitstream for an OFS-based FPGA Board on page 148
- **Terasic DE10-Agilex Development Board**: Follow the instructions in Programming the Terasic DE10-Agilex Development Board on page 148.

Programming the Terasic DE10-Agilex Development Board

You can program the Terasic DE10-Agilex Development Board board using the `fpga_jtag_reprogram` tool.

For details, refer to Programming the FPGA Device on page 134 section of the *Quick Start Tutorial*.

Programming the FPGA Green Bitstream for an OFS-based FPGA Board

Program the FPGA AI Suite design example green bitstream (`.gbs`) to the devices with the following command:

```
fpgaconf -V <path_to_design_example_green_bitstream(.gbs)_file>
```

For example, to program the FPGA device with the green bitstream file generated by the earlier example command, run the following command:

```
fpgaconf -V $COREDLA_WORK/build_generics_2inst/AGX7_Generic.gbs
```

5.2.1.6. Performing Accelerated Inference with the `dla_benchmark` Application in the PCIe Design Example

You can use the `dla_benchmark` demonstration application included with the FPGA AI Suite runtime to benchmark the performance of image classification networks.

5.2.1.6.1. Inference on Image Classification Graphs in the PCIe Design Example

The demonstration application requires the OpenVINO device flag to be either `HETERO:FPGA,CPU` for heterogeneous execution or `HETERO:FPGA` for FPGA-only execution.

The `dla_benchmark` demonstration application runs five inference requests (batches) in parallel on the FPGA, by default, to achieve optimal system performance. To measure steady state performance, you should run multiple batches (using the `niter` flag) because the first iteration is significantly slower with FPGA devices.

The `dla_benchmark` demonstration application also supports multiple graphs in the same execution. You can place more than one graphs or compiled graphs as input, separated by commas.

Each graph can have either a different input dataset or use a commonly shared dataset among all graphs. Each graph requires an individual `ground_truth_file` file, separated by commas. If some `ground_truth_file` files are missing, the `dla_benchmark` continues to run and ignore the missing ones.

When multi-graph is enabled, the `-niter` flag represents the number of iterations for each graph, so the total number of iterations becomes `-niter` × number of graphs.

The `dla_benchmark` demonstration application switches graphs after submitting `-nireq` requests. The request queue holds the number of requests up to `-nireq` × number of graphs. This limit is constrained by the DMA CSR descriptor queue size (64 per hardware instance).

The board you use determines the number of instances that you can compile the FPGA AI Suite hardware for:

- For the Terasic DE10-Agilex Development Board, you can compile up to four instances with the same architecture on all instances.
- For the Agilex 7 FPGA I-Series Development Kit and FPGA SmartNIC N6001-PL Platform, you can compile up to four instances with the same architecture on all instances. Some large architecture might not fit on the board for four instances, such as `AGX7_Performance_Giant`.

Each instance accesses one of the DDR banks on the board and executes the graph independently. This optimization enables multiple batches to run in parallel, limited by the number of DDR banks available. Each inference request created by the demonstration application is assigned to one of the instances in the FPGA plugin.

To enable memory-mapped device (MMD) debug messages when you run the `dla_benchmark` demonstration application. set the `ACL_PCIE_DEBUG` environment variable as follows:

```
ACL_PCIE_DEBUG=1
```

Also, you can test full DDR write and read back functionality when the `dla_benchmark` demonstration application runs by setting the `COREDLA_RUNTIME_MEMORY_TEST` environment variable as follows:

```
COREDLA_RUNTIME_MEMORY_TEST=1
```

To ensure that batches are evenly distributed between the instances, you must choose an inference request batch size that is a multiple of the number of FPGA AI Suite instances. For example, with two instances, specify the batch size as six (instead of the OpenVINO default of five) to ensure that the experiment meets this requirement.

The example usage that follows has the following assumptions:

- A Model Converter IR `.xml` file is in `demo/models/public/resnet-50-tf/FP32/`
- An image set is in `demo/sample_images/`
- The board is programmed with a bitstream that corresponds to `AGX7_Performance.arch`

```
binxml=$COREDLA_WORK/demo/models/public/resnet-50-tf/FP32

imgdir=$COREDLA_WORK/demo/sample_images

cd $COREDLA_WORK/runtime/build_Release

./dla_benchmark/dla_benchmark \
   -b=1 \
   -m $binxml/resnet-50-tf.xml \
   -d=HETERO:FPGA,CPU \
   -i $imgdir \
   -niter=4 \
   -plugins ./plugins.xml \
   -arch_file $COREDLA_ROOT/example_architectures/AGX7_Performance.arch \
   -api=async \
   -groundtruth_loc $imgdir/TF_ground_truth.txt \
   -perf_est \
   -nireq=8 \
   -bgr
```

The following example shows how the FPGA AI Suite IP can dynamically swap between graphs. This example usage assumes that another Model Converter IR `.xml` file has been placed in `demo/models/public/resnet-101-tf/FP32/`. It also assumes that another image set has been placed into `demo/sample_images_rn101/`. In this case, `dla_benchmark` only evaluates the classification accuracy of Resnet50 because we did not provide ground truth for the second graph (ResNet101).

```
binxml1=$COREDLA_ROOT/demo/models/public/resnet-50-tf/FP32
binxml2=$COREDLA_ROOT/demo/models/public/resnet-101-tf/FP32
imgdir1=$COREDLA_ROOT/demo/sample_images
imgdir2=$COREDLA_ROOT/demo/sample_images_rn101
cd $DEVELOPER_PACKAGE_ROOT/runtime/build_Release
./dla_benchmark/dla_benchmark \
   -b=1 \
   -m $binxml1/resnet-50-tf.xml,$binxml2/resnet-101-tf.xml \
   -d=HETERO:FPGA,CPU \
   -i $imgdir1,$imgdir2 \
   -niter=8 \
   -plugins ./plugins.xml \
   -arch_file $COREDLA_ROOT/example_architectures/AGX7_Performance.arch \
   -api=async \
   -groundtruth_loc $imgdir1/TF_ground_truth.txt \
   -perf_est \
   -nireq=8 \
   -bgr
```

5.2.1.6.2. Inference on Object Detection Graphs in the PCIe Design Example

To enable the accuracy checking routine for object detection graphs, you can use the `-enable_object_detection_ap=1` flag.

This flag lets the `dla_benchmark` calculate the mAP and COCO AP for object detection graphs. Besides, you need to specify the version of the YOLO graph that you provide to the `dla_benchmark` through the `-yolo_version` flag. Currently, this routine is known to work with YOLOv3 (graph version is `yolo-v3-tf`) and TinyYOLOv3 (graph version is `yolo-v3-tiny-tf`).

The mAP and COCO AP Metrics in the PCIe Design Example

Average precision and average recall are averaged over multiple Intersection over Union (IoU) values.

Two metrics are used for accuracy evaluation in the `dla_benchmark` application. The mean average precision (mAP) is the challenge metric for PASCAL VOC. The mAP value is averaged over all 80 categories using a single IoU threshold of 0.5. The COCO AP is the primary challenge for object detection in the Common Objects in Context contest. The COCO AP value uses 10 IoU thresholds of .50:.05:.95. Averaging over multiple IoUs rewards detectors with better localization.

Specifying Ground Truth in the PCIe Design Example

The path to the ground truth files is specified by the flag `-groundtruth_loc`.

The validation dataset is available on the COCO official website.

The `dla_benchmark` application currently allows only plain text ground truth files. To convert the downloaded JSON annotation file to plain text, use the `convert_annotations.py` script.

Example of Inference on Object Detection Graphs in the PCIe Design Example

The example that follows makes the following assumptions:

- The Model Converter IR `graph.xml` for either YOLOv3 or TinyYOLOv3 is in the current working directory.

  Model Optimizer generates an FP32 version and an FP16 version. Use the FP32 version.
- The validation images downloaded from the COCO website are placed in the `./mscoco-images` directory.
- The JSON annotation file is downloaded and unzipped in the current directory.

To compute the accuracy scores on many images, you can usually increase the number of iterations using the flag `-niter` instead of a large batch size `-b`. The product of the batch size and the number of iterations should be less than or equal to the number of images that you provide.

```
cd $COREDLA_ROOT/runtime/build_Release

python ./convert_annotations.py ./instances_val2017.json \
   ./groundtruth
```

```
./dla_benchmark/dla_benchmark \
   -b=1 \
   -niter=5000 \
   -m=./graph.xml \
   -d=HETERO:FPGA,CPU \
   -i=./mscoco-images \
   -plugins=./plugins.xml \
   -arch_file=../../example_architectures/AGX7_Performance.arch \
   -yolo_version=yolo-v3-tf \
   -api=async \
   -groundtruth_loc=./groundtruth \
   -nireq=8 \
   -enable_object_detection_ap \
   -perf_est \
   -bgr
```

5.2.1.6.3. Additional `dla_benchmark` Options in the PCIe Design Example

The `dla_benchmark` tool is part of the design example and the distributed runtime includes full source code for the tool.

Table 14. Command Line `dla_benchmark` Options

Command Option	Description
`-nireq=<N>`	This option controls the number of simultaneous inference requests that are sent to the FPGA. Typically, this should be at least twice the number of IP instances; this ensures that each IP can execute one inference request while `dla_benchmark` loads the feature data for a second inference request to the FPGA-attached DDR memory.
`-b=<N>` `--batch-size=<N>`	This option controls the batch size. A batch size greater than 1 is created by repeating configuration data for multiple copies of the graph. A batch size of 1 is typically best for latency System throughput for small graphs, when inference operations are offloaded from a CPU to an FPGA, may improve by using a batch greater than 1. On very small graphs, IP throughput may also improve when using a batch greater than 1. The default value is 1.
`-niter=<N>`	Number of batches to run. Each batch has a size specified by the `--batch-size` option. The total number of images processed is the product of the `--batch-size` option value multiplied by the `-niter` option value.
`-d=<STRING>`	Using `-d=HETERO:FPGA`, CPU causes `dla_benchmark` to use the OpenVINO heterogeneous plugin to execute inference on the FPGA, with fallback to the CPU for any layers that cannot go to the FPGA. Using `-d=HETERO:CPU` or `-d=CPU` executes inference on the CPU, which may be useful for testing the flow when an FPGA is not available. Using `-d=HETERO:FPGA` may be useful for ensuring that all graph layers are accelerated on the FPGA (and an error is issued if this is not possible).
`-arch_file=<FILE>` `--arch=<FILE>`	This specifies the location of the .arch file that was used to configure the IP on the FPGA. The `dla_benchmark` will issue an error if this does not match the `.arch` file used to generate the IP on the FPGA.
`-m=<FILE>` `--network_file=<FILE>`	This points to the XML file from OpenVINO Model Converter that describes the graph. The BIN file from Model Converter must be kept in the same directory and same filename (except for the file extension) as the XML file.

continued...

Command Option	Description
-i=<*DIRECTORY*>	This points to the directory containing the input images. Each input file corresponds to one inference request. The files are read in order sorted by filename; set the environment variable VERBOSE=1 to see details describing the file order.
-api=[sync|async]	The -api=async option allows dla_benchmark to fully take advantage of multithreading to improve performance. The -api=sync option may be used during debug.
-groundtruth_loc=<*FILE*>	Location of the file with ground truth data. If not provided, then dla_benchmark will not evaluate accuracy. This may contain classification data or object detection data, depending on the graph.
-yolo_version=<*STRING*>	This option is used when evaluating the accuracy of a YOLOv3 or TinyYOLOv3 object detection graph. The options are yolo-v3-tf and yolo-v3-tiny-tf.
-enable_object_detection_ap	This option may be used with an object detection graph (YOLOv3 or TinyYOLOv3) to calculate the object detection accuracy.
-bgr	When used, this flag indicates that the graph expects input image channel data to use BGR order.
-plugins_xml_file=<*FILE*>	*Deprecated:* This option is deprecated and will be removed in a future release. Use the -plugins option instead. This option specifies the location of the file specifying the OpenVINO plugins to use. This should be set to $COREDLA_ROOT/runtime/plugins.xml in most cases. If you are porting the design to a new host or doing other development, it may be necessary to use a different value.
-plugins=<*FILE*>	This option specifies the location of the file that specifies the OpenVINO plugins to use. The default behavior is to read the plugins.xml file from the runtime/ directory. This runs inference on the FPGA device. If you want to run inference using the emulation model, specify -plugins=emulation. If you are porting the design to a new host or doing other development, you might need to use a different value.
-mean_values=<*input_name[mean_values]*>	Uses channel-specific mean values in input tensor creation through the following formula: $\frac{\text{input} - \text{mean}}{\text{scale}}$. The Model Converter mean values are the preferred choice and the mean values defined by this option serve as fallback values.
-scale_values=<*input_name[scale_values]*>	Uses channel-specific scale values in input tensor creation through the following formula: $\frac{\text{input} - \text{mean}}{\text{scale}}$. The Model Converter scale values are the preferred choice and the scale values defined by this option serve as fallback values.
-pc	This option reports the performance counters for the CPU subgraphs, if there is any. No sorting is done on the report.
-pcsort=[sort|no_sort|simple_sort]	This option reports the performance counters for the CPU subgraph and sets the sorting option for the performance counter report: • sort: Report is sorted by operation time cost • no_sort: Report is not sorted • simple_sort: Report is sorted by opts time cost but print only executed operations
-save_run_summary	Collect performance metrics during inference. These metrics can help you determine how efficient an architecture is at executing a model. For more information, refer to The dla_benchmark Performance Metrics in the PCIe Design Example on page 154.

5.2.1.6.4. The `dla_benchmark` Performance Metrics in the PCIe Design Example

The `-save_run_summary` option makes the `dla_benchmark` demonstration application collect performance metrics during inference. These metrics can help you determine how efficient an architecture is at executing a model.

Note: The `dla_benchmark` application provides throughput in "frames per second". The time per frame (latency) is 1/throughput.

Statistic	Description
Count	The number of times interference was performed. This is set by the `-niter` option.
System duration	The total time between when the first inference request was made to when the last request was finished, as measured by the host program.
IP duration	The total time the spent-on inference. This is reported by the IP on the FPGA.
Latency	The median time of all inference requests made by the host. This includes any overhead from OpenVINO or the FPGA AI Suite runtime.
System throughput	The total throughput of the system, including any OpenVINO or FPGA AI Suite runtime overhead.
Number of hardware instances	The number of IP instances on the FPGA.
Number of network instances	The number graphs that the IP processes in parallel.
IP throughput per instance	The throughput of a single IP instance. This is reported by the IP on the FPGA.
IP throughput per f_{MAX} per instance	The **IP throughput per instance** value scaled by the **IP clock frequency** value.
IP clock frequency	The clock frequency, as reported by the IP running on the FPGA device. The `dla_benchmark` application treats this value as the IP core f_{MAX} value.
Estimated IP throughput per instance	The estimated per-IP throughput, as estimated by the `dla_compiler` command with the `--fanalyze-performance` option.
Estimated IP throughput per fmax per instance	The **Estimated IP throughput per instance** value scaled by the compiler f_{MAX} estimate.

Interpreting System Throughput and Latency Metrics in the PCIe Design Example

The **System throughput** and **Latency** metrics are measured by the host through the OpenVINO API. These measurements include any overhead that is incurred by both the API and the FPGA AI Suite runtime. They also account for any time spent waiting to make inference requests and the number of available instances.

In general, the system throughput is defined as follows:

$$\text{System Throughput} = \frac{\text{Batch Size} \times \text{Images per Batch}}{\text{Latency}}$$

The **Batch Size** and **Images Per Batch** values are set by the `--batch-size` and `-niter` options, respectively.

For example, consider when `-nireq=1` and there is a single IP instance. The **System throughput** value is approximately the same as the **IP-reported throughput** value because the runtime can perform only one inference at a time. However, if both the -

`nireq` and the number of IP instances is greater than one, the runtime can perform requests in parallel. As such, the total system throughput is greater than the individual IP throughput.

In general, the `-nireq` value should be *twice* the number of IP instances. This setting enables the FPGA AI Suite runtime to pipeline inferences requests, which allows the host to prepare the data for the next request while an IP instance is processing the previous request.

5.2.1.7. Running the Ported OpenVINO Demonstration Applications in the PCIe Design Example

Some of the sample demonstration applications from the OpenVINO toolkit for Linux Version 2024.6 have been ported to work with the FPGA AI Suite. These applications are built at the same time as the runtime when using the `-build_demo` flag to `build_runtime.sh`.

The FPGA AI Suite runtime includes customized versions of the following demo applications for use with the FPGA AI Suite IP and plugins:

- `classification_sample_async`
- `object_detection_demo_yolov3_async`
- `segmentation_demo`

Each demonstration application uses a different graph. The OpenVINO HETERO plugin can fall-back to the CPU for portions of the graph that are not supported with FPGA-based acceleration. However, in a production environment, it may be more efficient to use alternate graphs that execute exclusively on the FPGA.

You can use the example `.arch` files that are supplied with the FPGA AI Suite with the demonstration applications. However, certain example `.arch` files do not enable some of the layer-types used by the graphs associated with the demonstration applications. Using these `.arch` files cause portions of the graph to needlessly execute on the CPU. To minimize the number of layers that are executed on the CPU by the demonstration application, use the following architecture description files located in the `example_architectures/` directory of the FPGA AI Suite installation package to run the demos:

- Agilex 7: `AGX7_Generic.arch`

As specified in Programming the FPGA Device in the PCIe Design Example on page 148, you must program the FPGA device with the bitstream for the architecture being used. Each demonstration application includes a README.md file specifying how to use it.

When the OpenVINO sample applications are modified to support the FPGA AI Suite, the FPGA AI Suite plugin used by OpenVINO needs to know how to find the `.arch` file describing the IP parameterization by using the following configuration key. The following C++ code is used in the demo for this purpose:

```
ie.SetConfig({ { DLIA_CONFIG_KEY(ARCH_PATH), FLAGS_arch_file } }, "FPGA");
```

The OpenVINO demonstration application `hello_query_device` does not work with the FPGA AI Suite due to low-level hardware identification assumptions.

5.2.1.7.1. Example Running the Object Detection Demonstration Application in the PCIe Design Example

You must download the following items:

- `yolo-v3-tf` from the OpenVINO Model Downloader. The command should look similar to the following command:

```
python3 <path_to_installation>/open_model_zoo/omz_downloader \
   --name yolo-v3-tf \
   --output_dir <download_dir>
```

From the downloaded model, generate the `.bin`/`.xml` files:

```
python3 <path_to_installation>/open_model_zoo/omz_converter \
   --name yolo-v3-tf \
   --download_dir <download_dir> \
   --output_dir <output_dir> \
   --mo <path_to_installation>/model_optimizer/mo.py
```

Model Converter generates an FP32 version and an FP16 version. Use the FP32 version.

- Input video from: https://github.com/intel-iot-devkit/sample-videos.
- The recommended video is `person-bicycle-car-detection.mp4`

To run the object detection demonstration application,

1. Ensure that demonstration applications have been built with the following command:

```
build_runtime.sh -target_de10_agilex -build-demo
```

2. Ensure that the FPGA has been configured with the Generic bitstream.
3. Run the following command:

```
./runtime/build_Release/object_detection_demo/object_detection_demo \
   -d HETERO:FPGA,CPU \
   -i <path_to_video>/input_video.mp4 \
   -m <path_to_model>/yolo_v3.xml \
   -arch_file=$COREDLA_ROOT/example_architectures/AGX7_Generic.arch \
   -plugins $COREDLA_ROOT/runtime/plugins.xml \
   -t 0.65 \
   -at yolo
```

Tip: High-resolution video input, such as when using HD camera as input, imposes considerable decoding overhead on the inference engine that can potentially lead to reduced system throughput. Use the the `-input_resolution=<width>x<height>` option that is included in the demonstration application to adjust the input resolution to a level that balances video quality with system performance.

5.2.2. Running the Hostless DDR-Free Design Example

Procedure

To run the hostless DDR-free design example with a ResNet-18 PyTorch Model:

1. Download and prepare the ResNet-18 PyTorch Model with the OpenVINO Open Model Zoo tools with the following commands:

```
source ~/build-openvino-dev/openvino_env/bin/activate

omz_downloader --name resnet-18-pytorch \
    --output_dir $COREDLA_WORK/demo/models/

omz_converter --name resnet-18-pytorch \
    --download_dir $COREDLA_WORK/demo/models/ \
    --output_dir $COREDLA_WORK/demo/models/
```

Important: The OpenVINO Open Model Zoo (OMZ) PyTorch models do not include a softmax operation at the end of the model.

2. Generate the parameter ROMs as `.mif` files by running the FPGA AI Suite compiler with the following command:

```
dla_compiler \
  --batch-size=1 \
  --network-file <path/to/graph> \
  --march $COREDLA_ROOT/example_architectures/
AGX7_Streaming_Ddrfree_Resnet18.arch \
  --foutput-format=open_vino_hetero \
  --o <compiler output .bin file name>  \
  --fplugin HETERO:FPGA \
  --dumpdir $COREDLA_WORK/resnet-18-dlac-out/
```

The `.mif` files are created in a subdirectory of the directory specified by the `--dumpdir` option. This subdirectory is called `parameter_rom`.

For details about creating the `.mif` files required for DDR-free operation, refer to Generating Artifacts for Hostless DDR-Free Operation on page 283.

3. Build the design example with the following command:

```
dla_build_example_design.py build \
  --output-dir <path/to/build/dir> \
  --num-instances 1 \
  --seed 1 \
  --parameter-rom-dir $COREDLA_WORK/resnet-18-dlac-out/parameter_rom/ \
  agx7_iseries_ddrfree \
  $COREDLA_ROOT/example_architectures/AGX7_Streaming_Ddrfree_Resnet18.arch
```

Building the design example creates the bitstream needed to program the FPGA device.

For more information about the `dla_build_example_design` command, refer to The dla_build_example_design Command on page 39.

4. Program the FPGA device with the Quartus Prime Programmer.

The bitstream used to program the device is *<path/to/build/dir>*`/hw/output_files/top.sof`.

Program the FPGA device with the following command:

```
quartus_pgm -c 1 -m jtag -o "p;top.sof@1"
```

For more information about the Quartus Prime Programmer, refer to *Quartus Prime Pro Edition User Guide: Programmer*.

5. To make the JTAG connection stable, lower the JTAG clock speed to 16 MHz or lower with the following command:

```
jtagconfig --setparam 1 JtagClock 16M
```

6. Use the Quartus Prime System Console to run inference on the design example.

   Because this design example is hostless, operations that typically come from the host are performed through Quartus Prime System Console instead. For more information about the Quartus Prime System Console, refer to "Analyzing and Debugging Designs with System Console" in *Quartus Prime Pro Edition User Guide: Debug Tools*.

   Use the System Console to complete the following steps:

   a. (Optional) Update the graph parameters and instructions using the CSR interface.

   b. Store input features in the FPGA on-chip memory.

   c. Prime the FPGA AI Suite IP registers for inference.

   d. Configure an ingress Modular Scatter-Gather DMA (mSGDMA) core to read the input features from on-chip memory and stream data into the FPGA AI Suite IP.

   e. Configure an egress mSGDMA core to stream data from the FPGA AI Suite IP into on-chip memory.

   f. Read the inference results from on-chip memory.

   The design example provides a System Console script to automate these operations for you. You can find the script in the `$CORDLA_ROOT/runtime/streaming/ed0_streaming_example` folder.

   To use the design example System Console script:

   a. (Optional) To update the graph parameters and configurations, generate the update `.mif` files by running the FPGA AI Suite compiler as in step 2, then send the new `.mif` files to the FPGA AI Suite IP with the following command:

   ```
   system-console --script=system_console_script.tcl \
                  --online-reconfiguration <path-to-MIF-directory>
   ```

   b. Run the following command to run inference on the FPGA device:

   ```
   system-console --script=system_console_script.tcl \
                  --input <path-to-img.bin> \
                  --num_inferences <#-of-inferences> \
                  --output_shape <[C H W]> \
                  --functional \
                  --arch=<path-to-architecture-description-file>
   ```

   The design example Quartus Prime System Console script generates a file called `output.bin` that contains the raw inference results.

   c. (Optional) To measure the performance of the design example, run the following command:

   ```
   system-console --script=system_console_script.tcl \
                  --input <path-to-img.bin> \
                  --output_shape <[C H W]> \
                  --core_ip_performance \
                  --arch=<path-to-architecture-description-file>
   ```

   For more information about the System Console script, refer to Hostless DDR-Free Design Example Quartus Prime System Console on page 287.

7. Postprocess the raw inference output for readability with the following command:

```
python3 $COREDLA_ROOT/bin/streaming_post_processing.py <path-to-output.bin>
```

This script cleans the raw output binary file by script some invalid bytes and storing an FP16 formatted `result_hw.txt` file for readability.

5.2.3. Running the JTAG Design Example Demonstration Application

The FPGA AI Suite integrates the JTAG design example with OpenVINO and the FPGA AI Suite `dla_benchmark` example application to perform inference.

This section describes the steps to build the necessary components and commands to launch inference with the `dla_benchmark` application.

5.2.3.1. Building an FPGA Bitstream for the JTAG Design Examples

To generate an FPGA bitstream for the JTAG design example, follow the instructions appropriate for your target development kit:

- Building the Agilex 3 FPGA Bitstream on page 159
- Building the Agilex 5 FPGA Bitstream on page 159

Building the Agilex 3 FPGA Bitstream

To generate an Agilex 3 FPGA bitstream for the JTAG Design Example, run the following commands:

```
cd $COREDLA_WORK

$COREDLA_ROOT/bin/dla_build_example_design.py build \
  --output-dir build_agx3_jtag_ed \
  --num-instances 1 \
  --seed 1 \
  agx3c_jtag \
  $COREDLA_ROOT/example_architectures/AGX3_Small_NoSoftmax.arch
```

This command creates the bitstream file called `AGX3_Small_NoSoftmax.sof` that can be found in the `$COREDLA_WORK/build_agx3_jtag_ed/` folder.

For more information about the `dla_build_example_design.py` command, refer to The dla_build_example_design Command on page 39.

Building the Agilex 5 FPGA Bitstream

To generate an Agilex 5 FPGA bitstream for the JTAG Design Example, run the following commands:

```
cd $COREDLA_WORK

$COREDLA_ROOT/bin/dla_build_example_design.py build \
  --output-dir build_agx5_jtag_ed \
  --num-instances 1 \
  --seed 1 \
  agx5e_modular_jtag \
  $COREDLA_ROOT/example_architectures/AGX5_Generic.arch
```

This command creates the bitstream file called `AGX5_Generic.sof` that can be found in the `$COREDLA_WORK/build_agx5_jtag_ed/` folder.

For more information about the `dla_build_example_design.py` command, refer to The dla_build_example_design Command on page 39.

5.2.3.2. Programming the FPGA Device with JTAG Design Example Bitstream

Before you program the FPGA device, ensure that the USB-JTAG connection and port permissions are set up as described in Hostless JTAG Design Example Hardware Requirements on page 97.

To program the FPGA device with Quartus Prime Programmer and FPGA bitstream that you generated in Building an FPGA Bitstream for the JTAG Design Examples on page 159:

1. Run the following commands:

Agilex 3 FPGA C-Series Development Kit	`cd $COREDLA_WORK/build_agx3_jtag_ed`
	
	`quartus_pgm -c 1 -m jtag -o "p;AGX3_Small_NoSoftmax.sof"`

Agilex 5 FPGA E-Series 065B Modular Development Kit	`cd $COREDLA_WORK/build_agx5_jtag_ed`
	
	`quartus_pgm -c 1 -m jtag -o "p;AGX5_Generic.sof"`

2. Make the JTAG connection stable by lowering the JTAG clock speed to 16 MHz or lower with the following command:

```
jtagconfig --setparam 1 JtagClock 16M
```

5.2.3.3. Preparing JTAG Design Example Graphs for Inference with FPGA AI Suite

Before running any demonstration application, you must convert the trained model to the Inference Engine format (`.xml`, `.bin`) with the OpenVINO Model Optimizer.

For details on creating the `.bin`/`.xml` files, refer to the OpenVINO Model Converter / Conversion API documentation .

The network as described in the `.xml` and `.bin` files (created by the Model Converter) is compiled for a specific FPGA AI Suite architecture file by using the FPGA AI Suite compiler.

The FPGA AI Suite compiler compiles the network and exports it to a `.bin` file that uses the same `.bin` format as required by the OpenVINO Inference Engine.

This `.bin` file created by the compiler contains the compiled network parameters for all the target devices (FPGA, CPU, or both) along with the weights and biases. The inference application imports this file at runtime.

The FPGA AI Suite compiler can also compile the graph and provide estimated area or performance metrics for a given architecture file or produce an optimized architecture file.

For more details about the FPGA AI Suite compiler, refer to Compiling Your Model with the FPGA AI Suite Compiler on page 193.

5.2.3.4. Performing Inference with the JTAG Design Example

After the FPGA device on the development kit has been programmed, you can perform inference.

For this design example, the FPGA AI Suite runtime needs the locations of the following items before performing inference:

- Bitstream file location

  The default bitstream path is `top.sof` located in the `$COREDLA_WORK` directory.

  You can override this default location by specifying the `DLA_SOF_PATH` environment variable.

- Quartus Prime System Console Tcl script (required to properly claim JTAG services)

  The default script is `$COREDLA_WORK/runtime/build_Release/system_console_script.tcl`.

  You can override this value by specifying the `DLA_SYSCON_SOURCE_FILE` environment variable.

Perform inference by running the following command:

Agilex 3 FPGA C-Series Development Kit

```
# Modify MODEL to suit your application

export DLA_SOF_PATH=$COREDLA_WORK/build_agx3_jtag_ed/
AGX3_Small_NoSoftmax.sof
export MODEL=<path-to-model-XML-file>/resnet50.xml

cd $COREDLA_WORK/runtime/build_Release/dla_benchmark

dla_benchmark \
  -b=1 \
  -m=$MODEL \
  -d=HETERO:FPGA,CPU \
  -i <path-to-image-files> \
  -niter=2 \
  -plugins=../../plugins.xml \
  -arch_file= $COREDLA_ROOT/example_architectures/\
AGX3_Small_NoSoftmax.arch \
  -api=async \
  -perf_est \
  -nireq=1 -dump_output -report_lsu_counters
```

Agilex 5 FPGA E-Series 065B Modular Development Kit

```
# Modify MODEL to suit your application

export DLA_SOF_PATH=$COREDLA_WORK/build_agx5_jtag_ed/
AGX5_Generic.sof
export MODEL=<path-to-model-XML-file>/resnet50.xml

cd $COREDLA_WORK/runtime/build_Release/dla_benchmark

dla_benchmark \
  -b=1 \
  -m=$MODEL \
  -d=HETERO:FPGA,CPU \
  -i <path-to-image-files> \
  -niter=2 \
  -plugins=../../plugins.xml \
  -arch_file= $COREDLA_ROOT/example_architectures/
AGX5_Generic.arch \
```

```
  -api=async \
  -perf_est \
  -nireq=1 -dump_output -report_lsu_counters
```

By default, the `dla_benchmark` application records the system-console commands that are executed during inference in a log file named `csr_log.txt` in the current work directory, and preserves the intermediate files created during inference. To prevent the creation of the log file and intermediate files, specify the `–dump_csr=false` of the `dla_benchmark` command.

For more information about the `dla_benchmark` application, refer to Performing Accelerated Inference with the dla_benchmark Application in the PCIe Design Example on page 148.

5.2.3.5. JTAG Design Example Inference Performance Measurement

The `dla_benchmark` application reports inference duration and throughput for the entire design example as well as for the FPGA AI Suite IP.

To perform one inference iteration, the host performs the following steps:

1. Write input data via JTAG to the DDR memory on the FPGA development board.
2. Program CSRs on the FPGA AI Suite IP to start inference.
3. Poll the CSRs until the FPGA AI Suite IP completes the inference.
4. Read the output from the DDR memory to the host via JTAG.

The system duration accounts for all these steps above.

In contrast, the IP duration omits the duration of input and output data transfer.

For this design example, system duration is usually much larger than the IP duration because data transfer over JTAG is relatively slow. Thus, the IP duration and throughput better reflect the performance of the FPGA AI Suite IP.

The following output is an example throughput report generated by the `dla_benchmark` application on the Agilex 5 FPGA E-Series 065B Modular Development Kit after performing 3925 inferences on a quantized ResNet-18 model:

```
[Step 11/12] Dumping statistics report
count:               3925 iterations
system duration:   464549.5363 ms
IP duration:       17945.7971 ms
latency:            118.2524 ms
system throughput: 8.4490 FPS
number of hardware instances: 1
number of network instances: 1
IP throughput per instance: 218.7142 FPS
IP throughput per fmax per instance: 1.0936 FPS/MHz
IP clock frequency: 200.0000 MHz
```

5.2.3.6. JTAG Design Example Known Issues and Limitations

The JTAG design example has the following known issues and limitations:

- The number of inference request (-nireq) must be 1 when running dla_benchmark with the JTAG design example.

- The USB-JTAG connection between the host and the FPGA is relatively slow, so the system throughput is much lower than both the measured and estimated IP throughputs per instance.
- For FPGA AI Suite IP configured with large K_{vec} and C_{vec} parallelism, the peak throughput of the DDR4 interface on the Agilex 5 FPGA E-Series 065B Modular Development Kit can become the bottleneck. When you run the `dla_benchmark` application with the flag `-perf_est`, the application provides a throughput estimation that does not fully account for the limited external memory bandwidth on the development kit, so the estimate might be higher than the measured IP throughput per instance.

  Similarly, peak throughput on the Agilex 3 FPGA C-Series Development Kit is limited by the 32-bit width of the LPDDR4 interface that bottlenecks throughput despite the high data transfer rate.
- On Ubuntu 20 and Ubuntu 22 systems, the runtime might fail when loading model to the FPGA device if the `$DLA_SOF_PATH` environment variable does not point to the correct bitstream file, or if the Quartus Prime System Console `system-console` command is not present in the `$PATH` environment variable.

  The Quartus Prime System Console command is in `$QUARTUS_ROOTDIR/syscon/bin`.

```
[Step 5/12] Resizing network to match image sizes and given batch
[Step 6/12] Configuring input of the model
[[Step 7/12] Loading the model to the device
Generating unsupported layer chains graph (./unsupported_layer_chains.dot)
Using the Tcl setup script at /home/user/coredla-work/runtime/build_Release/
system_console_script.tcl
Saving temporary files to /home/user/Downloads
Segmentation fault (core dumped)
```

- You might occasionally see the following Quartus Prime System Console error when running inference with the runtime on this design example:

```
claim_service: Path cannot be found while executing
"claim_service master $path {}
"\{${::g_const_master_offset_dla} ${::g_const_master_range_dla} EXCLUSIVE\}""
    procedure "claim_dla_csr_service" line 4)
    invoked from within"claim_dla_csr_service"
procedure "initialization" line 4)
 invoked from within
"initialization"
```

  To recover from the issue, reprogram the FPGA with the correct bitstream and rerun the `dla_benchmark` application. To reduce the likelihood of this issue, lower the JTAG clock frequency to 16 MHz or lower before running the run the `dla_benchmark` application:

```
jtagconfig --setparam 1 JtagClock 16M
```

5.2.4. Running the SoC Design Example Demonstration Applications

Depending on the SoC design example mode that you want run, follow the instructions in one of the following sections:

- Running the M2M Mode Demonstration Application in the SoC Design Example on page 164

- Running the S2M Mode Demonstration Application in the SoC Design Example on page 165

Important: Running the demonstration applications requires the terminal session that you opened in Determining the SoC FPGA Development Kit IP Address on page 125.

5.2.4.1. Running the M2M Mode Demonstration Application in the SoC Design Example

The M2M dataflow model uses the `dla_benchmark` demonstration application. The S2M bitstream supports both the M2M dataflow model and the S2M dataflow model.

You must know the host name of the SoC FPGA development kit. If you do not know the development kit host name, go back to Determining the SoC FPGA Development Kit IP Address on page 125 before continuing here.

To run inference on the SoC FPGA development kit:

1. Open an SSH connection to the SoC FPGA development kit:
   a. Start a new terminal session
   b. Run the following command:

   ```
   build-host:$ ssh <devkit_hostname>
   ```

   Where *<devkit_hostname>* is the host name you determined in Determining the SoC FPGA Development Kit IP Address on page 125.

   Continuing the example from Determining the SoC FPGA Development Kit IP Address on page 125, the following command would open an SSH connection:

   ```
   build-host:$ ssh arria10-62747948036a.local
   ```

2. In the SSH terminal, run the following commands:

```
export compiled_model=~/resnet-50-tf/RN50_Performance_b1.bin

export imgdir=~/resnet-50-tf/sample_images

export archfile=~/resnet-50-tf/<architecture file>

cd ~/app

export COREDLA_ROOT=/home/root/app

export LD_LIBRARY_PATH=.

./dla_benchmark \
   -b=1 \
   -cm $compiled_model \
   -d=HETERO:FPGA,CPU \
   -i $imgdir \
   -niter=8 \
   -plugins ./plugins.xml \
   -arch_file $archfile \
   -api=async \
   -groundtruth_loc $imgdir/TF_ground_truth.txt \
   -perf_est \
   -nireq=4 \
   -bgr
```

where *<architecture file>* is one of the following files, depending on your development kit:

- Agilex 5 FPGA E-Series 065B Modular Development Kit

  ```
  AGX5_Performance.arch
  ```

- Agilex 7 FPGA I-Series Transceiver-SoC Development Kit

  ```
  AGX7_Performance_LayoutTransform.arch
  ```

- Arria 10 SX SoC FPGA Development Kit

  ```
  A10_Performance.arch
  ```

The `dla_benchmark` command generates output similar to the following example output for each step. This example output was generated using an Agilex 7 FPGA I-Series Transceiver-SoC Development Kit.

```
[Step 11/12] Dumping statistics report
count:              8 iterations
system duration:    286.0659 ms
IP duration:        64.9427 ms
latency:            138.7106 ms
system throughput: 27.9656 FPS
number of hardware instances: 1
number of network instances: 1
IP throughput per instance: 123.1856 FPS
IP throughput per fmax per instance: 0.3080 FPS/MHz
IP clock frequency measurement: 400.0000 MHz
estimated IP throughput per instance: 137.6405 FPS (500 MHz assumed)
estimated IP throughput per fmax per instance: 0.2753 FPS/MHz
[Step 12/12] Dumping the output values
[ INFO ] Comparing ground truth file /home/root/resnet-50-tf/sample_images/
TF_ground_truth.txt with network Graph_0
top1 accuracy: 100 %
top5 accuracy: 100 %
[ INFO ] Get top results for "Graph_0" graph passed
```

5.2.4.2. Running the S2M Mode Demonstration Application in the SoC Design Example

To run the S2M (streaming) mode demonstration application, you need two terminal connections to the host.

You must know the host name of the SoC FPGA development kit. If you do not know the development kit host name, go back to Determining the SoC FPGA Development Kit IP Address on page 125 before continuing here.

To run the streaming demonstration application:

1. Open an SSH connection to the SoC FPGA development kit:
   a. Start a new terminal session
   b. Run the following command:

      ```
      build-host:$ ssh <devkit_hostname>
      ```

      Where *<devkit_hostname>* is the host name you determined in Determining the SoC FPGA Development Kit IP Address on page 125.

      Continuing the example from Determining the SoC FPGA Development Kit IP Address on page 125, the following command would open an SSH connection:

      ```
      build-host:$ ssh arria10-62747948036a.local
      ```

2. Repeat step 1 to open a second SSH connection to the SoC FPGA development kit.
3. In a terminal session, run the following commands:

```
cd /home/root/app
./run_inference_stream.sh
```

4. In the other terminal session, run the following commands:

```
cd /home/root/app
./run_image_stream.sh
```

The first terminal session (where you ran the `run_inference_stream.sh` command) then shows output similar to the following example:

```
root@arria10-ea80b8d770e7:~/app# ./run_inference_stream.sh
Runtime arch check is enabled. Check started...
Runtime arch check passed.
Runtime build version check is enabled. Check started...
Runtime build version check passed.
Ready to start image input stream.
1 - class ID 776, score = 58.4
2 - class ID 968, score = 90.7
3 - class ID 769, score = 97.8
4 - class ID 769, score = 97.8
5 - class ID 872, score = 99.8
6 - class ID 954, score = 94.4
7 - class ID 954, score = 94.4
```

For more details about the streaming applications and their command line options, refer to Running the Streaming Demonstration on page 329.

5.2.4.3. Troubleshooting the Demonstration Applications in the SoC Design Example

If you receive an error similar to the following error while running either the S2M or M2M applications, check that all the board DIMMs are securely installed:

```
altera-msgdma ff200000.msgdma: dma_sync_wait: timeout! [ 251.812846] DMA Failed
```

6. Creating an Architecture File for the FPGA AI Suite IP

Before you create or modify an FPGA AI Suite IP architecture file, review How Does the FPGA AI Suite Overlay Architecture Work? on page 16 to understand how the FPGA AI Suite IP implements your machine learning (ML) graph.

Related Information

How Does the FPGA AI Suite Overlay Architecture Work? on page 16

6.1. Predefined FPGA AI Suite Architecture Files

Within `$COREDLA_ROOT/example_architectures`, several predefined `.arch` files are available. These files define different configurations of the FPGA AI Suite IP, including the parallelism configuration and modules to enable and disable. Predefined architecture files are carefully generated to demonstrate performance and different features of the FPGA AI Suite Overlay IP described in The FPGA AI Suite IP Overlay Architecture on page 15. These predefined architecture files are a good starting point to evaluate the performance of a given ML graph with the FPGA AI Suite.

Three commonly used architecture presets are:

- Generic Architecture on page 167
- Performance Architecture on page 167
- Giant Architecture on page 168

Compile your model against one or more of these architectures to discover how your model performs under different hardware configurations. To learn how to estimate performance and resource usage, refer to Compiling Your Model with the FPGA AI Suite Compiler on page 193.

Generic Architecture

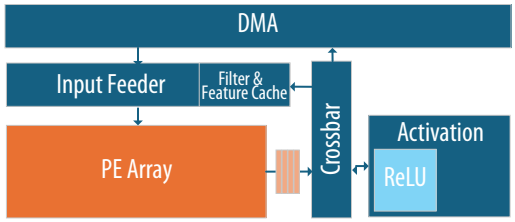
A flexible, adaptable starting point for most models. Usually serves as a baseline architecture for performance and resources utilization benchmarks.

Performance Architecture

>Designed for high parallelism and balanced resource usage. This architecture usually guarantees the best performance for a type of ML graph.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

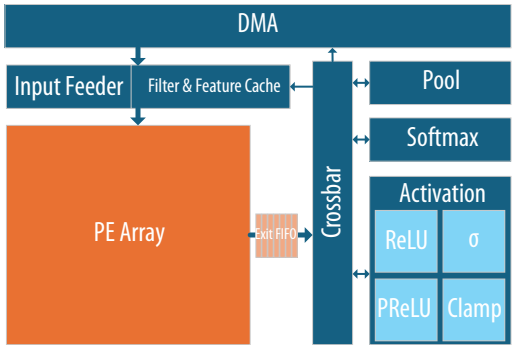
Figure 14. Example Block Diagram of a Performance Architecture



Giant Architecture

Optimized for large input shapes with a focus on throughput, less sensitive to resource constraints. This architecture will take up quite a large portion of the FPGA area.

Figure 15. Example Block Diagram of a Giant Architecture



6.1.1. Architecture Description File Format for Instance Parameterization

The FPGA AI Suite IP has a highly configurable architecture. Configuring the design allows for different trade-offs between inference performance (throughput and latency) and utilization of FPGA resources (area). Configurations are specified through Architecture Description Files. The IP instances corresponding to these configurations can be compiled as part of an FPGA design into an FPGA bitstream.

The architecture determines how much FPGA area is consumed by the FPGA AI Suite IP and strongly affects the achieved inference fps and ease of timing closure.

Achieving the best performance of a given graph for a given FPGA area (or the smallest FPGA area for a given performance target) requires optimizing the architecture. The architecture optimization function of the FPGA AI Suite compiler is designed to produce good architectures for a given graph or set of graphs. For more details about the architecture optimization function of the compiler, refer to Generating an Optimized Architecture on page 253.

The FPGA AI Suite Architecture Description Files use the protobuf format and have a `.arch` file extension. While these files are human readable and editable, manually optimizing an architecture requires a deep knowledge of the FPGA AI Suite IP design and is not recommended.

You must set some of the architecture parameters manually because the Architecture Optimizer does not modify them. For example, the optimizer does not modify the numerical precision (for example, fp16 or fp11) in the architecture file. Similarly, the

optimizer does not modify details related to the AXI interfaces on the IP. In some case, you can improve performance of the resulting optimized architecture by the choice of these values.

When possible, modifying the graph or batch size might also result in performance improvements. For example, a graph that requires FP16 precision might have sufficient accuracy at FP11 or FP12 if a few extra layers are added. Reducing the internal precision enables a large memory and area reduction. Very small and fast graphs might achieve a higher performance on hardware by using a batch size that is greater than one.

The `example_architectures/` directory includes an example that shows how to enable the hardware-accelerated softmax function.

The comment character in the `.arch` format is `#`.

6.2. FPGA AI Suite Architecture File Breakdown

This chapter explores customization parameters within an architecture (`.arch`) file, demonstrates where and how the optimization happens, and hints at the more advanced optimization techniques that are introduced in Optimizing Your FPGA AI Suite IP on page 226. A predefined architecture file (`AGX7_Generic.arch`) is used here as an example.

An architecture file starts with defining the target family and the PE parallelism and precision parameter:

```
family : 'AGX7'
k_vector : 32
c_vector : 16
arch_precision : FP13AGX
stream_buffer_depth : 63488
output_channels_max : 16384
```

Enable element-wise multiplication and the constraints on filter size.

```
enable_eltwise_mult : true
filter_size_width_max : 28
filter_size_height_max : 28
```

PE array interleave, described below. Choosing `exit_fifo_depth` is a system design consideration that costs more area.

```
pe_array {
  num_interleaved_features : 5
  num_interleaved_filters : 1
  exit_fifo_depth : 1024
}
```

Filter scratchpad depth controls the amount of filter buffered.

```
filter_scratchpad {
  filter_depth : 512
  bias_scale_depth : 512
}
```

Work with FPGA team to set DMA parameters.

```
dma  {
  csr_addr_width : 11
  csr_data_bytes : 4
  ddr_addr_width : 32
  ddr_burst_width : 4
  ddr_data_bytes : 64
  ddr_read_id_width : 2
}
```

Enable activation functions.

```
activation {
  generic_aux_parameters {
    k_vector : 16
  }
  enable_clamp : true
  enable_leaky_relu : false
  enable_sigmoid : true
  enable_prelu : true
  }
```

Enable hardened pooling module. If not included, the pooling will be executed on the host.

```
pool {
  generic_aux_parameters {
    k_vector : 4
  }
  max_window_height : 13
  max_window_width : 13
  max_stride_vertical : 4
  max_stride_horizontal : 4
}
```

The crossbar acts as a central hub for gathering output from the PE, performs auxiliary operations, and writes back the results for the next convolution. The crossbar has the following main parameters:

xbar_in_ports
: Defines a connection that receives output feature from the PE array.

xbar_ports
: Defines the connection of several auxiliary modules: activation, hardened pooling, and hardened softmax.

  The output feature from the PE array, if needed, are sent to these auxiliary modules for further processing. The activation and softmax modules connect to the xbar_in_port input connection but the pool module connects to the activation connection. These connections means that the output feature can go to activation or softmax directly, but it must go through activation for pooling.

xbar_out_ports
: Allows the output feature to be sent to input_feeder then to the PE for the next convolution; to output_writer for writing out the result; since the input_connection has xbar_in_port,

the input can bypass all modules connected to the crossbar and directly goes out; similarly, the result from activation, pool, and softmax can be sent out.

```
xbar {
  xbar_k_vector : 16
  max_input_interfaces : 5
  max_output_interfaces : 5
  xbar_ports {
    xbar_aux_port {
      name : 'activation'
      input_connection : 'xbar_in_port'
    }
    xbar_aux_port {
      name : 'pool'
      input_connection : 'xbar_in_port'
      input_connection : 'activation'
    }
    xbar_aux_port {
      name : 'softmax'
      input_connection : 'xbar_in_port'
    }
  }
  xbar_in_port {
    external_connection : 'pe_array'
  }
  xbar_out_port {
    external_connection : 'input_feeder'
    external_connection : 'output_writer'
    input_connection : 'xbar_in_port'
    input_connection : 'pool'
    input_connection : 'activation'
    input_connection : 'softmax'
  }
}
```

The configuration network is connected to all other modules since it decodes the instructions from the compiled model to the FPGA device and orchestrates inference by controlling all other modules. As mentioned in FPGA AI Suite IP Datapath Component Organization on page 22, the configuration network provides little configurability in the architecture file.

Related Information

FPGA AI Suite IP Datapath Component Organization on page 22

6.2.1. FPGA AI Suite IP Supported Layers and Hyperparameter Ranges

Each layer in the intermediate representation (IR) must be validated against the layer and primitive constraints. This step ensures that the design flow only progresses with operator dimensions and hyperparameters supported by the FPGA AI Suite hardware.

Validation at this stage prevents errors during architecture generation and guarantees that all layers can be synthesized into FPGA-ready IP without unsupported fallbacks.

The following table lists the hyperparameter ranges supported by key primitive layers in the FPGA AI Suite IP:

Table 15. Supported Layers and Hyperparameter Ranges

Layer / Primitive	Hyperparameter	Supported Range
Fully connected	None	n/a
2D Convolution	Filter Size	Width = [1..28] Height = [1..28] Height does not have to equal width. Default value for each is 14.
	Stride	Maximum stride is 15
	Pad	Maximum pad is (2^{16}) - 1
3D Convolution	Filter Size	Width = [1..28] Height = [1..28] Depth = [1..14] Filter volume should fit into the filter cache size.
	Stride	Maximum stride is 15.
	Pad	Maximum pad is (2^{16}) - 1
Depthwise	Filter Size	Same as 2D Conv filter size Depth = 1
	Stride	Same as 2D Conv stride Depth = 1
	Pad	Same as 2D conv padding Depth = 1
Scale-Shift	Scale factor	FP16 float range
	Bias term	FP16 float range
Deconvolution / Transpose Convolution	Filter Size	Any – Same as convolution, and height/width can be different Depth = 1
	Stride	1, 2, 4, 8 (stride width == stride height) Depth = 1
	Pad	Restricted to filter_[height, width] - 1 Depth = 1
ReLU	n/a	n/a
pReLU	Scaling parameter (a) (1 per filter / conv output channel)	float range Depth = 1
Leaky ReLU	Scaling parameter (a) (1 per tensor)	float range
Clamp	Limit parameters (a, b) (1 per tensor)	float range
Round_Clamp	Limit parameters (a, b) (1 per tensor)	float range
H-sigmoid	n/a	n/a
H-swish	n/a	n/a

continued...

Layer / Primitive	Hyperparameter	Supported Range
Sigmoid	n/a	FP16 float range
Swish	n/a	FP16 float range
Tanh	n/a	FP16 float range
Max Pool	Window Size	up to 13x13x13
	Pad	1, 2
	Stride	1, 2, 3, 4
Average Pool	Window Size	Up to 27x27 (one less than the maximum 2D convolution size) Width == Height Depth = 1 or 2
	Pad	1, 2
	Stride	1, 2, 3, 4
Softmax	Maximum Number of Channels	4096
Elementwise Multiplication of feature * filter and feature * feature tensors.(14)	n/a	Tensor sizes are expanded if necessary to support the multiplication. Depth = 1
ChannelToSpace DepthToSpace PixelShuffle	block_mode	`blocks_first` or `blocks_last`
	block_size	2, 4, 8
Upsampling Downsampling	Interpolation Type	`Bilinear` (upsampling only) or `Nearest Neighbor` (upsampling and downsampling
	Filter Size	Similar to 2D convolution except that width and height must be symmetrical for bilinear interpolation. Downsampling supports a sampling factor (ratio between an output dimension and input dimension) of only 2x or 4x.

6.2.2. Architecture Description File Parameters

6.2.2.1. Parameter Group: Global Parameters

Parameter: `family`

This parameter specifies the target FPGA device family for the architecture.

Legal Values

Table 16. Valid Values for `family` Global Parameter

Value	Description
`A10`	Target Arria 10 devices.
`AGX3`	Target Agilex 3 devices.
`AGX5`	Target Agilex 5 devices.

continued...

(14) This is an element-wise multiplication, not a matrix multiply operation.

Value	Description
AGX7	Target Agilex 7 devices.
C10	Target Cyclone 10 devices.
S10	Target Stratix 10 devices.

Parameter: k_vector

This parameter, also called KVEC, describes the number of filters that the PE Array is able to process simultaneously.

Typically the architecture optimizer is used to set this parameter.

Legal values: [4-128]

- The k_vector value must be a multiple of the c_vector value.
- The k_vector value must be divisible by the xbar_k_vector and auxiliary k_vector values.
- When you use the depthwise module, the k_vector value must equal the c_vector value.

Parameter: c_vector

This parameter, also called CVEC, describes the size of the dot product within each PE in the PE Array.

Typically the architecture optimizer is used to set this parameter.

Legal values: [4,8,16,32,64]

- When you use the depthwise module, the c_vector value must equal the k_vector value.

Parameter: num_lanes

This parameter describes how many output-height slices the architecture can compute in parallel.

Using the num_lanes architecture parameter has the following effects:

- Setting the num_lanes parameter scales the PE array in the FPGA AI Suite IP by the given number and provides additional parallelism at the cost of more DSPs and area.
- The total stream buffer size scales with the num_lanes parameter. Because the feature surface of a graph is divided across multiple lanes, adjust the stream_buffer_depth parameter listed in the .arch file by the inverse of the num_lanes parameter value. For example, a 4-lane architecture with 10k stream buffer depth indicates a 40k total stream buffer size.

Legal values: [1,2,4]

Parameter: `arch_precision`

This parameter sets the precision (in bits) of the internal numeric representation used by FPGA AI Suite IP. Lower values increase fps and reduce area, but at the cost of inference accuracy.

Each internal precision option corresponds to a different number of sign and mantissa bits, and uses either two's complement or sign+magnitude. For details, refer to the table in FPGA AI Suite IP Parameterization on page 189.

The FP16 precision significantly increases the size of the resulting IP, but can improve accuracy (particularly in models that have not been retrained for low precision).

All numeric options use block floating point format. In block floating point format, each block of size CVEC shares a common exponent. Both CVEC (`c_vector`) and `arch_precision` affect the accuracy of the inference. However, the impact of `c_vector` is generally small, while the impact of the `arch_precision` setting is relatively large.

The block floating point format used by the FPGA AI Suite IP is directly compatible with graphs that use INT8 symmetric quantization. INT8 symmetric quantization requires that all operations going from floating point to integer, and vice versa, require only scaling (multiplication or division). When given a graph with INT8 weights, the FPGA AI Suite compiler sets the exponent of the block floating point weights so the original INT8 weights can be used directly as the mantissa. This setting limits the use of INT8 weights to architectures where the mantissa is 8-bits or larger.

The use of INT8 graphs does not significantly affect either the inference speed or the FPGA resource consumption. All inference, regardless of whether block floating point is used, is performed with the same hardware.

In addition to selecting a compatible numeric precision, set the `pe_array/enable_scale` parameter to true in order to support graphs with INT8 quantization.

The example architectures that are included with the FPGA AI Suite are already set to the recommended `arch_precision` parameter values for their supported FPGA family. In some cases, it is useful to select a different `arch_precision` value. FP11 is the lowest precision option, but requires the least number of RAM blocks, and slightly reduces the amount of external memory traffic. The FP12AGX significantly reduces the number of DSPs required to implement the PE array, but logic utilization may increase.

Agilex 3 and Agilex 5 FPGA devices implement enhanced DSPs with AI tensor blocks. To take advantage of AI tensor blocks, set the `arch_precision` value to FP12AGX or FP11 and use interleave values of `12x1`.

For more details about the block floating point format, refer to the *Low-Precision Networks for Efficient Inference on FPGAs* white paper.

Legal values:

FPGA Device Family	Supported `arch_precision` Values
Agilex 3	• FP11 • FP12AGX • FP13AGX • FP16 (less common)
Agilex 5	• FP11 • FP12AGX • FP13AGX • FP16 (less common)
Agilex 7	• FP11 • FP13AGX • FP16 (less common)
Arria 10	• FP11 • FP16 (less common)
Cyclone 10 GX	• FP11 • FP16 (less common)
Stratix 10	• FP11 • FP16 (less common)

Table 17. Multiplication Operations per DSP

Precision	FPGA Device Family		
	Arria 10 Cyclone 10 GX Stratix 10	Agilex 7	Agilex 3 Agilex 5
FP11	4	4	20(*)
FP12AGX	–	–	20(*)
FP13AGX	–	4	4(**)
FP16	2	2	2

INT8 symmetric quantization is enabled by FP12AGX and higher precision options.

The total number of multipliers required by the PE Array will be equal to CVEC * KVEC * num_lanes. Due to quantization, this calculation underpredicts the number of DSPs required when using Agilex 3 or Agilex 5 DSP tensor mode. In addition, the PE Array requires KVEC * num_lanes DSPs to build the FP32 accumulators.

Parameter: `enable_mixed_precision`

This parameter enables the mixed precision features. For more details, refer to Using the Mixed Precision Feature on page 243.

(*) Assumes a 12x1 interleave.

(**) The Agilex 3 and Agilex 5 DSP blocks support six multipliers per block, but currently FPGA AI Suite uses only four of them.

Parameter: `stream_buffer_depth`

This parameter controls the depth of the stream buffer. The stream buffer is used as the on-chip cache for feature (image) data. Larger values increase area (logic and block RAM) but also increase performance.

Typically the architecture optimizer is used to set this parameter.

Legal values: [2048-262144]

Parameter: `enable_eltwise_mult`

This parameter enables the Elementwise multiplication layer. This layer is required for MobileNetV3.

Parameters: `filter_size_width_max`, `filter_size_height_max`

These parameters determine the maximum size of a convolution filter, which also relates the maximum window size for Average Pool.

The maximum window size for Average Pool is no larger than the value determined by the following formula: $\min(\text{filter_size_width_max}, \text{file_size_height_max}) - 1$. In addition, the Average Pool window size may be limited by the filter_scratchpad and filter_depth parameters.

Legal values: [14,28]

Parameters: `output_image_height_max`, `output_image_width_max`, `output_channels_max`

These parameters control the maximum size of the output tensor.

The default maximum size is 128x128, with up to 8192 channels

Parameter: `enable_debug`

This parameter toggles the FPGA AI Suite debug network to allow forwarding of read requests from the CSR to one of many externally-attached debug-capable modules.

Generally not required for production architectures.

Legal values: [`true`,`false`]

Parameter: `enable_layout_transform`

The parameter enables the dedicated input tensor transform module in the FPGA AI Suite IP. When enabled, the dedicated layout transform hardware transforms the input tensor format and folds the inputs into channels.

You can use the layout transform in streaming and non-streaming configurations of the FPGA AI Suite IP. The transform is particularly useful for doing fast and deterministic tensor preprocessing in hostless applications, or applications where the hard-processor is slow or highly loaded.

However, the layout transform comes with an FPGA area cost that scales mainly with the input data bus width, maximum tensor/stride dimensions, and CVEC. In particular, instances where the value of `max_stride_width` × `max_stride_height` × `max_channels` is greater than the CVEC value consume significant memory resources due to the buffer space required for the overflowing CVEC.

In graphs where the first convolution stride dimensions are unity, no folding can be done, and the layout transform cannot optimize the layout of the input tensor. In such a case, try doing a lighter-weight transformation operation outside of the FPGA AI Suite IP.

When this parameter is enabled, configure the transform with the parameters as described in Parameter Group: layout_transform_params on page 187. For information about the layout transformation operation and hardware, refer to Transforming the Layout of Input Data on page 230.

The hardware layout transform is supported in the SoC streaming to memory (S2M) mode only for Agilex 7 devices. Other devices use a lightweight, external transform module.

Support for the layout transform in the streaming-to-memory (S2M) SoC design example is inconsistent. Only the Agilex 7 variant of the S2M SoC design example supports the hardware layout transform. The S2M SoC example designs for other families assume the layout transform is not used.

6.2.2.2. Parameter Group: `activation`

This parameter group configures the activation module. These activation functions are common in deep learning, and it is beyond the scope of this document to describe them.

Different activation functions can be enabled or disabled to suit the graph to be run. Disabling unnecessary activations functions can reduce area.

Parameter: `activation/enable_relu`

This parameter enables or disables the Rectified Linear Unit (ReLU) activation function.

Legal values: [true, false]

Parameter: `activation/enable_leaky_relu`

This parameter enables or disables the Leaky ReLU activation function. This activation function is a superset of the ReLU activation function.

Legal values: [true, false]

Parameter: `activation/enable_prelu`

This parameter enables or disables the Parametric ReLU activation function. This activation function is a superset of the Leaky ReLU activation function.

Legal values: [true, false]

Parameter: activation/enable_clamp

This parameter enables or disables the clamp function. Enabling the clamp function also enables a ReLU6 activation function.

Legal values: [true, false]

Parameter: activation/enable_round_clamp

This parameter enables or disables the round_clamp function. Enabling the round_clamp function also enables ReLU.

If both enable_clamp and enable_round_clamp are set, enable_round_clamp takes priority over enable_clamp when implementing ReLU.

Legal values: [true, false]

Parameter: activation/enable_sigmoid

This parameter enables or disables the Sigmoid and Swish activation functions.

As a side-effect, enabling these activation functions also enables the Tanh and Reciprocal activation function. This side-effect might change in a future release, so the best practice is to enable activation function explicitly instead of depending on the side-effect.

Legal values: [true, false]

Parameter activation/enable_tanh

This parameter enables or disables the Tanh activation function.

As a side-effect, enabling this activation functions also enables the Sigmoid, Swish, and Reciprocal activation functions. This side-effect might change in a future release, so the best practice is to enable activation functions explicitly instead of depending on the side-effect.

Legal value: [true, false]

6.2.2.3. Parameter Group: pe_array

This parameter group configures the PE Array. The PE Array is used to calculate dot products.

Parameter: pe_array/dsp_limit

Use this parameter to force the PE array to implement multipliers in ALM logic on the FPGA.

The number of multipliers that the PE requires is determined by the `k_vector` and `c_vector` global parameters. Given the value of the `arch_precision` global parameter and the target architecture (for example, Arria 10 or Agilex 7), the number

of multipliers determines the number of DSPs that the PE Array tries to use. If this number exceeds the value set in the `dsp_limit` parameter, then some multipliers are implemented in ALM logic to ensure that the PE Array DSP usage does not exceed the limit set by the `dsp_limit` parameter.

If this option is omitted, then all multipliers are implemented in the FPGA AI Suite IP as DSPs.

Typically, this parameter is set by the architecture optimizer.

Parameters: `pe_array/num_interleaved_features`, `pe_array/num_interleaved_filters`

To support layers with bias values, the PE array uses a threaded accumulator that is time-multiplexed to handle multiple accumulations. Each accumulation corresponds to an output filter and feature.

Common Values:	*Agilex 3 devices*	12x1
	Agilex 5 devices	12x1
	Agilex 7 devices	5x1, 3x2
	Arria 10 devices	4x1, 2x2
	Stratix 10 devices	5x1, 3x2

All architectures support a 1x1 interleave. Selecting a 1x1 interleave typically reduces ALM consumption, but the IP associated with this architecture does not support layers with bias. Because most deep learning graphs include bias, the 1x1 interleave is typically not used.

The architecture optimizer does not modify the `num_interleaved_features` and `num_interleaved_filters` values. You must set them manually.

The filter interleave multiplies the effective KVEC, which means that graphs with a depthwise convolution (such as certain versions of MobileNet) might perform best when using `num_interleaved_filters=1`. Multilayer perceptron graphs might perform best when using `num_interleaved_features=1`.

Except in the 1x1 case, the value of `num_interleaved_features` multiplied by `num_interleaved_filters` must meet the following requirements:

Agilex 3 devices	The value of `num_interleaved_features` multiplied by `num_interleaved_filters` must be greater than or equal to five. To use the Tensor DSP mode, the value of `num_interleaved_features` must be greater than or equal to 12 and the value of `num_interleaved_filters` must be one.

Agilex 5 devices	The value of `num_interleaved_features` multiplied by `num_interleaved_filters` must be greater than or equal to five. To use the Tensor DSP mode, the value of `num_interleaved_features` must be greater than or equal to 12 and the value of `num_interleaved_filters` must be one.
Agilex 7 devices	The value of `num_interleaved_features` multiplied by `num_interleaved_filters` must be greater than or equal to five.
Arria 10 devices	The value of `num_interleaved_features` multiplied by `num_interleaved_filters` must be greater than or equal to four.
Stratix 10 devices	The value of `num_interleaved_features` multiplied by `num_interleaved_filters` must be greater than or equal to five.

There is no advantage in choosing interleave factors larger than the minimum required.

Parameter: `pe_array/exit_fifo_depth`

This parameter controls the depth of the PE Array exit FIFO. Larger values might reduce the incidence of stalling, but at the cost of area.

Typically, this parameter is not modified.

Parameter: `pe_array/enable_scale`

This parameter controls whether the IP supports scaling feature values by a per-channel weight. This is used to support batch normalization and INT8 scaling (in graphs that are INT-8 quantized and do not use block floating point).

In most graphs, the graph compiler (`dla_compiler` command) adjusts the convolution weights to account for scale, so this option is usually not required. (Similarly, if a shift is required, then the convolution bias values are adjusted).

Legal values: true, false

6.2.2.4. Parameter Group: `pool`

This parameter group configures the pool module. The pool module is used for max pooling only; average pooling is performed using the convolution engine.

Parameter: `pool/k_vector`

This parameter controls the width of the pool interface.

Typically, the architecture optimizer is used to set this parameter.

Legal values: [1,2,4,8,16,32, 64]

Parameters: `pool/max_window_height`, `pool/max_window_width`

These parameters set the maximum window height and width that the architecture can support.

Typically, you set this value to the size of the largest max pooling window in your graph. Larger values cost more FPGA area.

Legal values: [3-7]

Parameters: `pool/max_stride_vertical`, `pool/max_stride_horizontal`

These parameters set the maximum stride values that the architecture can support.

Typically, you set these to the largest value your graph requires after a max pool. Larger values cost FPGA area.

Legal values: [1-4]

6.2.2.5. Parameter Group: `depthwise`

This parameter group configures the depthwise module. The depthwise module accelerates the depthwise convolutions.

If a depthwise layer has more parameters than the depthwise module can support, the layer is executed on the general PE array.

Parameter: `depthwise/k_vector`

This parameter controls the width of the depthwise interface. Typically, the architecture optimizer is used to set this parameter.

Legal Values: [16, 32, 64]

Parameters: `depthwise/max_window_height`, `depthwise/max_window_width`

These parameters set the maximum window height and width that the architecture can support. Typically, you set this value to the size of the largest max depthwise window in your graph.

Larger values cost more FPGA area.

Legal Values: [3, 5, 7]

Parameters: `depthwise/max_stride_vertical`, `depthwise/max_stride_horizontal`

These parameters set the maximum stride values that the architecture can support. Typically, you set these to the largest value your graph requires after a depthwise convolution.

Larger values cost FPGA area.

Legal Values: [1-4]

Parameters: `depthwise/max_dilation_vertical`, `depthwise/max_dilation_horizontal`

These parameters set the maximum dilation values that the architecture can support. Typically, you set these to the largest value your depthwise convolutions in your graph require

Larger values cost FPGA area.

Legal Values: [1-6]

6.2.2.6. Module: `softmax`

The `softmax` module is enabled or disabled by including a custom auxiliary primitive (`custom_aux_primitive`) with a `layer_type` and `name` set to `softmax`. The primitive must connect to the crossbar (`xbar`).

The file `example_architectures/Generic_softmax.arch` has an example of this connectivity.

Parameter: `softmax/max_num_channels`

This parameter sets the maximum number of channels for SoftMax that the architecture can support.

Legal Values: [1-4096]

6.2.2.7. Parameter Group: `dma`

This parameter group defines key attributes related to the external AXI interface for the IP.

Parameters: `dma/csr_addr_width`, `dma/csr_data_bytes`

These parameters define the interface to the CSR.

Parameters: `dma/ddr_addr_width`, `dma/ddr_burst_width`, `dma/ddr_data_bytes`, `dma/ddr_read_id_width`

These parameters define the AXI interface to off-chip memory.

6.2.2.8. Parameter Group: xbar

For each layer of the graph, data passes through the convolution engine (referred to as the processing element [PE] array), followed by zero or more auxiliary modules. The auxiliary modules perform operations such as activation or pooling.

After the output data for a layer has been computed, it can be sent to one of the following places:

- An internal buffer while waiting for the start of the next convolution layer.
- The external memory for reading by the host program.

Internally, a crossbar (xbar) connects the modules together. The `xbar` parameters specify the connections between the PE array, the auxiliary modules, the input feeder (which holds data waiting for the next convolution layer), and the output writer (which writes the data to the external memory).

Consider the following example:

```
xbar {
      xbar_k_vector : 16
      max_input_interfaces : 4
      max_output_interfaces : 4
      xbar_ports {
        xbar_aux_port {
          name : 'activation'
          input_connection : 'xbar_in_port'
        }
        xbar_aux_port {
          name : 'pool'
          input_connection : 'xbar_in_port'
          input_connection : 'activation'
        }
      }
      xbar_in_port {
        external_connection  : 'pe_array'
      }
      xbar_out_port {
        external_connection : 'input_feeder'
        external_connection : 'output_writer'
        input_connection    : 'xbar_in_port'
        input_connection    : 'pool'
      }
    }
```

The crossbar always has the following elements:

- An `xbar_in_port` element that accepts the incoming connection from the PE array.
- An `xbar_out_port` element that connects externally to the input feeder and output writer.

This example architecture also has two auxiliary modules defined with `xbar_aux_port` elements: a pool module and an activation module. The `xbar_aux_port` elements are specified in the `xbar_ports` section.

In this configuration, the activation module can accept data from the `xbar_in_port` (the PE array), while the pool module can accept data from the `xbar_in_port` or from the activation module.

Finally, the output port can accept data either directly from the `xbar_in_port` or from the pool module. These connections limit how data flows through the system.

In this example, the activation module cannot write out from the layer. Activations must be followed by pooling layers. Also, an activation layer cannot follow a pooling layer. However, the convolution can be followed by a pooling layer without an activation layer in between.

Connections cost area and can reduce f_{MAX}. You can reduce area by including only the connections that are required for a given graph (sometimes called "depopulating" the crossbar). If you use this architecture as the starting point for the Architecture Optimizer, the result can improve throughput.

To see examples of other crossbar configurations, review the `example_architectures/` directory.

Parameter: `xbar/xbar_k_vector`

This parameter defines the width of the interface into the crossbar. Typically, this parameter is set to be equal to the width of the widest interface into any of the auxiliary modules.

Typically, the architecture optimizer is used to set this parameter.

Legal values: [2,4,8,16,32,64]

6.2.2.9. Parameter Group: `filter_scratchpad`

These parameters define the on-chip cache used for the filter weights, bias values, and scale values (if scale and bias are enabled).

Parameter: `filter_scratchpad/filter_depth`

This parameter defines the size of the on-chip cache used to store convolution filter weights. Larger values use more FPGA resources, but might increase fps.

Typically, the architecture optimizer is used to set this parameter.

Legal values: Depends on the setting of `enable_parameter_rom`:

- `enable_parameter_rom=false`

  2^n where n is `9`, `10`, or `11`
- `enable_parameter_rom=true`

  2^n where n is 4 or greater.

Parameter: `filter_scratchpad/bias_scale_depth`

This parameter defines the size of the on-chip cache used to store feature bias and scale weights, if the corresponding support is enabled in the PE Array. Larger values use more FPGA resources, but might increase inference throughput.

Typically this is set equal to the `filter_scratchpad/filter_depth` parameter.

Legal values: Depends on the setting of `enable_parameter_rom`:

- `enable_parameter_rom=false`
  2^n where *n* is `9`, `10`, or `11`
- `enable_parameter_rom=true`
  2^n where *n* is 4 or greater.

6.2.2.10. Parameter Group: `input_stream_interface`

Enable and configure the width of the input AXI4-Stream interface. If enabled, input streaming requires that the layout transform is also enabled. Neural network graphs compiled with an input streaming-enabled architecture cannot be sliced in the input layer, therefore the stream buffer depth must be large enough to fit the input layer.

Parameter: input_stream_interface/enable

Enables the input streaming module. Enable_layout_transform must also be true, and the transform must be configured for the target neural network graph.

Legal values: true, false

Parameter: input_stream_interface/data_width

Sets the width of the AXI4-streaming input bus in bits.

Legal values: 2^n where *n* is 4 or greater.

Parameter: input_stream_interface/fifo_depth

Sets the depth of the input FIFO

Legal values: 2^n where *n* is 4 or greater.

For best performance, set the `fifo_depth` value to the size of one transformed input (refer to Input Transform Mapping on page 232) For example, if the transformed input is 32×28×28×x1 and the `c_vector` size is 16, set `fifo_depth` to 2048.

For example, an architecture with a 128-bit input AXI streaming interface would include the following options:

```
input_stream_interface {
 enable: true
 data_width: 128
 fifo_depth: 2048
}
```

6.2.2.11. Parameter Group: `output_stream_interface`

Enable and configure the AXI4-Stream output interface.

Parameter: `output_stream_interface/enable`

Enables the output streaming module.

Legal values: true, false

Parameter: `output_stream_interface/data_width`

Sets the width of the AXI4-steaming output interface in bits.

Legal values: 2^n where n is 4 or greater.

Parameter: `output_stream_interface/fifo_depth`

Sets the depth of the output FIFO.

Legal values: 2^n where n is 4 or greater.

For best performance, the FIFO depth should hold the entire output $\text{ceil}\left(\frac{\text{channel}}{\text{xbar k_vector}}\right) \times \text{width} \times \text{height} \times \text{depth}$. For example, if the output is 64×10×10×1 and the xbar k_vector is 32, the FIFO depth should be set to 256.

For example, and architecture file with an 128-bit wide output AXI streaming interface would include the following options:

```
output_stream_interface {
 enable: true
 data_width: 128
 fifo_depth: 1024
}
```

6.2.2.12. Parameter Group: `config_network`

These parameters define the configuration network that connects internal components within the IP.

Typically, this section is changed only minimally when modifying the examples in the `example_architectures/` directory.

6.2.2.13. Parameter Group: `layout_transform_params`

These parameters configure the input tensor layout transformation module of the FPGA AI Suite IP.

Parameter: `layout_transform_params/enable_bias_scale`

When true, the layout transform can apply bias and scale values to the input as part of the transform operation. The bias and scale values are set by the compiler. Otherwise, this operation is done in the runtime software if applicable, or in the PE array.

Parameter: `layout_transform_params/input_type`

This parameter indicates the data type of the input tensor elements.

Legal values: [F16, U8, U16]

Parameters: `layout_transform_params/max_channels`, `layout_transform_params/max_feature_height`, `layout_transform_params/max_feature_width`, `layout_transform_params/max_feature_depth`, `layout_transform_params/max_stride_height`, `layout_transform_params/ max_stride_width`, `layout_transform_params/max_stride_depth`, `layout_transform_params/max_pad_top`, `layout_transform_params/max_pad_left`, `layout_transform_params/max_pad_depth`, `layout_transform_params/max_filter_width`, `layout_transform_params/max_filter_height`, `layout_transform_params/max_filter_depth`

This group configures the range of feature shapes, padding, and convolution strides that the layout transform hardware module supports. The values in this configuration represent the maximum allowed values for each parameter. However, the resource usage of the layout transform is sensitive to the parameterization, so set these values as close to the actual values as possible.

The parameters of the first convolution in the graph must fit within the maximum range configured here. The exact parameters required by the layout transform module are reported by the FPGA AI Suite compiler in a file named `input_transform_dump_<graphname>.csv`.

For more information about the input tensor layout transform, refer to Input Feature Tensor In-Memory Format on page 231.

6.2.2.14. Parameter Group: `lightweight_layout_transform_params`

These parameters configure the lightweight version of the input tensor layout transformation module in the FPGA AI Suite IP.

Parameter: `lightweight_layout_transform_params/channels`

An integer representing the number of channels in the input tensor.

Parameter: `lightweight_layout_transform_params/input_type`

This parameter indicates the data type of the input tensor elements.

Legal values: [F16, U8, U16]

Parameter: `lightweight_layout_transform_params/input_bus_width`

Sets the width of the data bus, in bits, for the input layout transform. This parameter has the following constraints:

- In streaming designs, this value must exactly match the width of the input AXI-streaming interface. (In DDR-based designs, width conversion is done if required).
- The value of input bus width must be a multiple of 8 (byte multiple).
- The input bus width must also be a multiple of the width of a pixel. That is, it must be an integer multiple of `channels x sizeof(input_type)`.

Legal values: `input_bus_width = N x (channels x |input_type|)`, for a positive integer `N`*

* Given `N`, the input bus is said to contain `N` pixels in parallel

Parameter: `lightweight_layout_transform_params/pixel_fifo_depth`

This parameter sets the depth of the exit FIFO to tune the elasticity of the input interface. This parameter applies to the exit FIFO for each pixel in parallel.

Legal values: Integers greater than 1

Parameter: `lightweight_layout_transform/enable_bias_scale`

When `true`, the lightweight layout transform can apply bias and scale values to the input as part of the transform operation. The bias and scale values are set by the compiler. Otherwise, this operation is done in the runtime software if applicable, or in the PE array.

Legal values: `[true,false]`

6.3. FPGA AI Suite IP Parameterization

The FPGA AI Suite IP block has many important parameters that describe arithmetic precision, feature set, size of various modules (such as the PE Array), and details regarding the internal buses and the external AXI interface.

The following table summarizes the effects of these parameters and parameter groups:

Table 18.

Effect	Input Feeder	Filter Scratchpad	PE Array	Crossbar	Aux Modules
Data movement efficiency Data storage efficiency	input_stream_interface output_stream_interface stream_buffer_depth	filter_depth arch_precision	exit_fifo_depth	xbar_k_vector	generic_aux_parameters
Computation speed			k_vector c_vector num_lanes num_interleaved_features		

continued...

Effect	Input Feeder	Filter Scratchpad	PE Array	Crossbar	Aux Modules
			num_interleaved_filters		
Extra functionality enablement	enable_parameter_rom enable_layout_transform layout_transform_params		enable_scale		pool depthwise softmax activation
Inference accuracy			arch_precision		

Configurable parameters are specified in the Architecture Description (`.arch`) file, as described in Architecture Description File Format for Instance Parameterization on page 168 and Architecture Description File Parameters on page 173.

The table below shows the major parameters, some of which are not configurable, that describe the IP block.

Common Parameter Name	Description	Valid Range
c_vector (CVEC)	Size of the dot product performed by each PE in the PE Array. Typically optimized when generating an optimized architecture with the FPGA AI Suite compiler.	[4,8,16,32,64]
k_vector (PE KVEC)	Number of PEs in the PE Array	[4-128] Must be a multiple of c_vector
num_lanes	Number of execution lane. Each lane contains a set of auxiliary modules (aux_modules) and a number (equal to k_vector) of PEs	[1,2,4]
N/A	Number of auxiliary modules connecting to the crossbar (XBAR)	[1-4]
pool k_vector (Pool KVEC)	Width of the pool interface. Typically optimized when generating an optimized architecture with the FPGA AI Suite compiler.	[1,2,4,8,16,32,64]
pool max_window_height pool max_window_width	Size of the pooling window	[3x3, 7x7,13x13]
depthwise k_vector (Depthwise KVEC)	Number of output channels processed in parallel. Typically optimized when generating an optimized architecture with the FPGA AI Suite compiler.	[16, 32, 64] Must be equal to k_vector
depthwise max_window_height depthwise max_window_width	Size of the depthwise filter	[3x3, 5x5, 7x7]
depthwise max_dilation_vertical depthwise max_dilation_horizontal	Maximum supported value for the depthwise dilation	[1-6]
activation k_vector (Activation KVEC)	Width of the activation interface. Typically optimized when generating an optimized architecture with the FPGA AI Suite compiler.	[2,4,8,16,32,64]
enable_clamp	Enables clamp activation function	[true, false]
enable_relu	Enables ReLU activation function	[true, false]
enable_leaky_relu	Enables Leaky ReLU activation function	[true, false]
enable_prelu	Enables PReLU activation function	[true, false]

continued...

Common Parameter Name	Description	Valid Range
enable_round_clamp	Enables round clamp activation function	[true, false]
enable_sigmoid	Enables Sigmoid and Swish activation functions	[true, false]
enable_tanh	Enables Tanh activation function	[true, false]
enable_parameter_rom	Enables storing graph parameters in on-chip memory, which requires input and output streaming to be enabled and configured. For details about DDR-free operation, refer to Generating Artifacts for Hostless DDR-Free Operation on page 283.	[true, false]
arch_precision (PE precision)	Precision of features and weights in the PE Array. For details about how this parameter affects DSP utilization, refer to the "Parameter: arch_precision" section of Parameter Group: Global Parameters on page 173.	"FP11" (INT7-BFP / 1s.6m.5e) "FP12AGX" (INT8-BFP / 8m.5e, two's complement) "FP13AGX" (INT9-BFP / 9m.5e, two's complement) "FP16" (INT12-BFP / 1s.11m.5e)
PE bias add precision	Precision of accumulator bias value in the PE Array.	fp16
PE accumulator precision	Precision of the accumulators in the PE Array.	fp32
PE drain precision	Precision of values drained from the PE Accumulators to the XBAR and AUX Modules.	fp16
PE interleave factor	Multi-threading factor for the features x filters in the PE array accumulators.	**Agilex 3 devices:** 2x3, 3x2, 5x1, 1x5, 12x1 **Agilex 5 devices:** 2x3, 3x2, 5x1, 1x5, 12x1 **Agilex 7 devices:** 2x3, 3x2, 5x1, 1x5 **Arria 10 devices:** 2x2, 4x1, 1x4 **Stratix 10 devices:** 2x3, 3x2, 5x1, 1x5 1×1 supported for graphs with no bias
PE scale precision	Precision of scale multiplier in the PE array	fp16
Aux module precision	Precision of the Aux Modules	fp16
Memory port width	Width of memory port	[64, 128, 256, 512]
enable_debug	Toggle the FPGA AI Suite debug network that includes interface profiling counters that can be queried with the CSR. Enabled by default.	[true, false]
enable_layout_transform	Enables the dedicated input tensor layout transform module. The standard layout transform module cannot be enabled when the lightweight layout transform is enabled.	[true, false]
enable_lightweight_layout_transform	Enables the area-optimized (lightweight) input tensor layout transform module.	[true, false]

continued...

Common Parameter Name	Description	Valid Range
	The lightweight layout transform cannot be enabled when the standard layout transform is enabled.	
enable_layout_transform_writeback	Enables the layout transform to write its results to external memory rather than the stream buffer. Enabling this mode requires additional CSR commissioning steps.	[true, false]

The major constraints include:

- PE KVEC must be a multiple of CVEC
- PE KVEC must be divisible by XBAR and AUX KVECs
- PE drain width must be equal to XBAR KVEC

Graph limitations include:

- Convolution filter size: 1×1 -> 28×28, including asymmetric
- Convolution filter stride: 1 .. 15
- No limitation on convolution padding
- The limits of the depthwise layers are the same as normal convolution. Depthwise convolution is handled with software emulation using regular convolution passes.

The maximum supported DDR size is 4GB.

7. Compiling Your Model with the FPGA AI Suite Compiler

You can use the FPGA AI Suite compiler in a number of modes when you are developing your graph and architecture.

These use modes require your graphs to be processed into an OpenVINO intermediate representation (IR) by the OpenVINO Model Converter tool. The Model Converter generates IR files that describe your model network topology (`.xml`) and the weights and biases for your model (`.bin`).

These OpenVINO IR files also serve as inputs to the OpenVINO Inference Engine.

Before you use the compiler, it is helpful to understand the FPGA AI Suite IP overlay architecture. You can learn more by reviewing How Does the FPGA AI Suite Overlay Architecture Work? on page 16 and Creating an Architecture File for the FPGA AI Suite IP on page 167.

Use the FPGA AI Suite compiler (`dla_compiler` command) to do the following tasks:

- Compile the IR of your graph to an FPGA bitstream.
- Generate artifacts for DDR-free operation
- Estimate the performance of a graph.
- Estimate the FPGA area required by an architecture.
- Generate an optimized architecture.
- Generate an optimized architecture for a frame rate target value.

Related Information

OpenVINO documentation

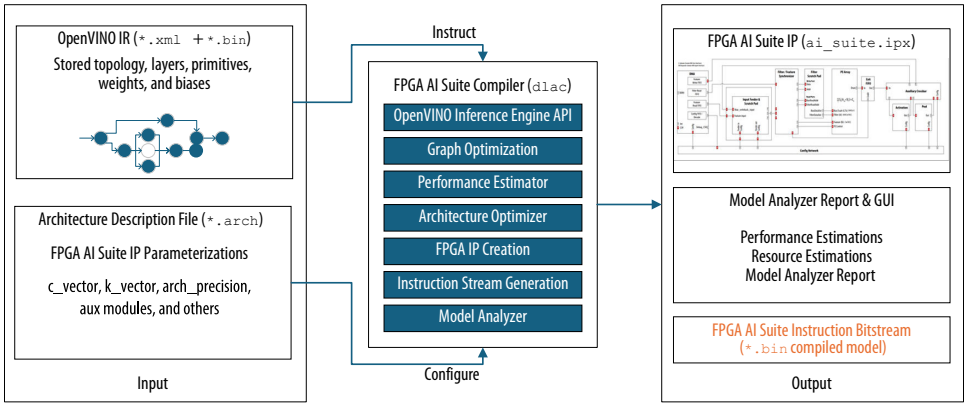
7.1. Compiling a Graph

Before you compile your first graph with the FPGA AI Suite compiler, you should have a good understanding of the OpenVINO intermediate representation (IR) that represents the machine learning graph and the FPGA AI Suite architecture description file that parametrizes of the FPGA AI Suite IP.

With those concepts in mind, the input, output, and functionality of the FPGA AI Suite compiler can be summarized in the following diagram:

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

Figure 16. FPGA AI Suite Compiler Operation



For compiling a model and analyzing model performance, use the following `dla_compiler` command options:

Option	Description
`--network-file`	[**Required**] The path to the OpenVINO Inference Engine network topology (`.xml`) file or files. If you have multiple files, list them as a sequence of space-delimited arguments. The `.bin` weights file must be in the same directory as the `.xml` file, and the file must have the same file name (except for the `.bin` extension).
`--march`	[**Required**] The architecture parameterization to use when compiling the graph.
`--plugins-file`	[**Optional**] The location of the plugins.xml file.
`--batch-size`	[**Optional**] Override the network batch size.
`--bin-data`	[**Optional**] Forces graph to be compiled for binary data input, regardless of input tensor shape. Binary data is read as the input layer type. Otherwise, graphs with 1- or 3-channel inputs are converted to U8 inputs and binary input files are discarded.
`--ffolding-option`	[**Optional**] Controls how (or if) folding happens on the first convolution layer.
`--o`	[**Optional**] File name of the exported compiled graph when using `open_vino_hetero` format.
`--dumpdir`	[**Optional**] Directory for dumping additional debug output. The file path location specified by the `--dump_dir` option command is also used as the destination location for most of the `.csv`, `.txt`, and `.dot` files created by the compiler. If you do not specify the `--dump_dir` option, these files are placed in the current working directory.
`--foutput-format`	[**Optional**] Specifies the export format: *`open_vino_hetero`* — The `open_vino_hetero` format dumps the graph from the OpenVINO level. Choose this format for use with the supplied example designs. *`dla_compiled_result`* — The `dla_compiled_result` format dumps the configuration and weights buffer. Do not use this format unless you are writing your own network import routines.
`--fplugin`	[**Optional**] Specifies the OpenVINO plugins. Typically set to `HETERO:FPGA,CPU`, which first tries to use the FPGA IP to implement each portion of the graph and falls back onto the host CPU only for portions that the FPGA IP does not support.

The simplest `dla_compiler` command format for compiling a graph is as follows:

```
dla_compiler \
   --network-file <list of  OpenVINO IR XML files> \
   --march <path to .arch file>
```

Example Command

```
dla_compiler \
  --network-file ResNet50.xml \
  --march $COREDLA_ROOT/example_architectures/A10_Generic.arch
```

7.2. Estimating Graph Performance

To estimate the performance of a graph on an architecture, use the `--fanalyze-performance` `dla_compiler` command option.

The `dla_compiler` command compiles the graph for the specified architecture to estimate its performance. The performance estimator assumes that any portions of the graph that are assigned to the CPU run inference with zero latency. That is, the performance estimate accounts only for the FPGA portions of the graph.

The performance estimator also estimates the average memory bandwidth and the memory requirements of the graph. The estimated memory requirement is typically an underestimate because the memory estimates assume one input buffer and one output buffers while the FPGA AI Suite runtime uses a default of five of each.

For DDR-free architectures, the performance estimate excludes performance of the input/output streamer. The estimate applies only to the FPGA AI Suite IP itself.

The list of required and optional arguments includes all the required and optional arguments from Compiling a Graph *on page 193*. In addition, the following options are specific to estimating graph performance:

Option	Description
`--fanalyze-performance`	[**Required**] Enables the performance estimator.
`--fassumed-fmax-core=<`*assumed* f_{MAX}`>`	[**Optional**] Specifies the assumed f_{MAX} of the compiled FPGA AI Suite IP. The performance estimator does not have the ability to estimate f_{MAX} of a given IP parameterization, nor does it know which speed grade the IP targets. Typically, the IP achieves 300 MHz or higher on a C2 Arria 10 device. The default f_{MAX} depends on the FPGA device family: • Arria 10 - 265 MHz • Agilex 3 - 350 MHz • Agilex 5 - 350 MHz • Agilex 7 - 500 MHz • Cyclone 10 GX - 265 MHz • Stratix 10 - 265 MHz
`--fassumed-memory-bandwidth`	[**Optional**] Specifies the available average DDR bandwidth in MB/s for each instance of the FPGA AI Suite IP. Do no set this value if the IP does not use DDR. The default DDR bandwidth depends on the FPGA device family: • Arria 10 - 19200 MB/s • Agilex 3 - 6400 MB/s • Agilex 5 - 6400 MB/s

continued...

Option	Description
	• Agilex 7 - 21328 MB/s • Cyclone 10 GX - 6400 MB/s • Stratix 10 - 19200 MB/s
`--fdump-performance-report`	[**Optional**] An optional output file for the performance estimate, otherwise the performance summary is displayed on the terminal.

The simplest `dla_compiler` command format for estimating the performance of a graph is as follows:

```
dla_compiler \
   --network-file <path to graph.xml> \
   --march <path to .arch file> \
   --fanalyze-performance
```

Performance Summary

The performance summary consists of the following metrics:

Table 19. Performance Summary Metrics Provided by the FPGA AI Suite Compiler

Metric	Description
PE-only Conv Throughput No DDR	Throughput of the PE array (that is, only layers that are mapped into convolutions) assuming that there is no limit to DDR bandwidth.
PE-only Conv Throughput	Estimate that accounts for the performance impact of fetching filter data from external memory.
Overall throughput Inf PE Buf Depth (zero MPBW)	Models the latency impact of most writes to external memory and latency of the activation modules. The latency estimate is pessimistic.
Overall throughput Zero PE Buf Depth	Same as the previous row, but using a optimistic methodology to estimate the latency impact.
Overall Throughput Inv PE Buf Depth	Estimate that accounts for external memory bottlenecks that affect the feature prefetch. The latency estimate is pessimistic.
Overall Throughput Zero PE Buf Depth	Same as previous row, but using an optimistic methodology to estimate the latency impact.

The final throughput number is further adjusted to more closely match on-board performance measurements. The majority of the adjustment is to account for the difference in practical versus theoretical DDR bandwidth.

Example Command

```
dla_compiler \
   --network-file ResNet50.xml ResNet101.xml \
   --march $COREDLA_ARCH/example_architectures/A10_Generic.arch \
   --fanalyze-performance \
   --fassumed-fmax-core=300 \
   --fassumed-memory-bandwidth=19200
```

7.3. Estimating the Area and Power of an Architecture

To estimate the FPGA resource usage (area) and power consumption of an IP parameterization described by an architecture (`.arch`) file, use the **`--fanalyze-area`** option.

Running the FPGA AI Suite compiler with this option displays a report that shows area usage and generates a `.pta` file (`power-parameters.pta`) that you can open with the FPGA Power and Thermal Analyzer. The FPGA Power and Thermal Analyzer can then provide you with estimates for the static power, dynamic power, and total power consumption of the architecture. For more information about the FPGA Power and Thermal Analyzer, refer to the *FPGA Power and Thermal Analyzer User Guide*

Option	Description
`--fanalyze-area`	[**Required**] Enable the area estimator and generate a `.pta` file that you can open with the FPGA Power and Thermal Analyzer to view power consumption estimates for the architecture.
`--march`	[**Required**] The architecture parameterization to assume.
`--fdump-area-report`	[**Optional**] Path specifying an output file for the area estimate summary.
`--fdump-pta-report`	[**Optional**] Path specifying a `.pta` output file for use with the FPGA Power and Thermal Analyzer.

The simplest command format for estimating the area of an FPGA AI Suite IP instance is as follows:

```
dla_compiler \
   --fanalyze-area \
   --march <path to .arch file>
```

Example Command

```
dla_compiler \
   --fanalyze-area \
   --march $COREDLA_ARCH/example_architectures/A10_Generic.arch
```

7.4. The FPGA AI Suite Compiler Report

After running the FPGA AI Suite compiler with the performance or resources estimation flags, you get a detailed report for the architecture and the network. Subject to the f_{MAX} assumptions for this compilation, you receive estimates for the following metrics:

Table 20. Compiler Report Metrics

Metric	Description
Total DDR Space Required	The estimated total DDR space including static buffer for config data, static buffer for filter, and buffer for features. Changing the *arch_precision* parameter in your .arch file changes the storage size of the filter and affects this estimation.
Total DDR Transfers Required	The estimated total DDR transfers for one instance. Similarly, changing the *arch_precision* parameter in your .arch file affects this estimation.
Minimum Average DDR Bandwidth Required	An average estimation on the DDR bandwidth. Please note that the peak DDR transfer will exceed this estimation. Use as a baseline for system budgeting.
PE-only Conv Throughput No DDR	The estimated throughput of the PEs assuming no DDR transfers, i.e. the design DDR-Free and weights are stored on-chip. This estimation may appear to be the highest since it eliminates all the system and memory overhead.
PE-only Conv Throughput	The estimated throughput of the PEs for a single IP instance. This estimation isolates the performance of the PE, excluding input and output memory transfer. It highlights the computational efficiency of the PE array of your architecture.
Final throughput	The total estimated throughput of the IP. This estimation includes any overhead of the memory transfers. This is for a single IP instance.

For ALMs, DSPs, and M20Ks, leave some unused on the FPGA device to ensure that your FPGA design can achieve timing closure and to ensure that the design is routable and can fit.

Work with your FPGA team to set a resources utilization target. After such target being set, budget according to the target. As a general rule, aim for a maximum ALM utilization of 70%, a maximum DPS usage of 95%, and maximum M20K usages of 95%.

7.4.1. Partitioning Table Report

When the `dla_compiler` tool invokes the OpenVINO HETERO plugin, the plugin creates a file named `hetero_subgraphs_<graph_name>.dot` that is a visual representation of the graph where the graph layers are annotated to indicate if the layer runs on the CPU or the FPGA device.

You can use the `dot` program from the Graphviz open source graph visualization software package to convert `.dot` files into formats that are more easily viewable, such as SVG.

You can open an SVG file directly in most web browsers, including Mozilla* Firefox*.

For example, to create an SVG file from a `hetero_subgraphs_ResNet50.dot` file generated by the OpenVINO HETERO plugin, run the following command:

```
dot -Tsvg hetero_subgraphs_ResNet50.dot -o subgraphs.svg
```

Open `subgraphs.svg` in a web browser to view the visualization of the graph layers.

Related Information

https://graphviz.org

7.4.2. Model Analyzer Reports

During compilation, the FPGA AI Suite compiler analyzes your model and generates the following reports:

- Model analysis report
- Unsupported layer chains report
- Per-layer latency report
- Per-layer parallel efficiency report

Model analysis report

The model analysis report (`model_analyzer_report.txt`) contains information about the model, the layer placement information, the reason why layers were unsupported, and how internal layers relate to the layers from the original OpenVINO graph passed to the FPGA AI Suite compiler.

Unsupported layer chains report

The unsupported layer chains report (`unsupported_layer_chains.dot`) contains a visual representation of the unsupported layers of the model.

The report is structured to highlight nodes as follows:

- Nodes that are directly unsupported by the FPGA AI Suite compiler are highlighted in red.
- Nodes that are unsupported due to other nodes being unsupported are highlighted in orange.
- Nodes from the original model that have been unsupported are highlighted in blue.

Trace from any unsupported node in the original graph to find the root causes of lack of support.

You can convert this report to SVG format which can be displayed in a web browser by running the following command:

```
dot -Tsvg unsupported_layer_chains.dot -o unsupported_layer_chains.svg
```

Per-layer latency report

This report is generated only if you specified one or both of the following FPGA AI Suite compiler command options:

- `--fanalyze-performance`
- `--festimate-per-layer-latencies`

The per-layer latency report (`per_layer_latency_report.txt`) reports information about the estimated throughput on three levels of granularity:

- The clock cycle count for the graph as a whole
- The clock cycle count for each of the subgraphs
- The clock cycle count for each layer in the graph.

The per-layer clock cycle information is accompanied by the internal layer name, and the original layer names from the OpenVINO graph that map to each of the internal layer.

Per-layer parallel efficiency report

This report is generated only if you specified the `--fanalyze-performance` FPGA AI Suite compiler command options.

The per-layer parallel efficiency report (`per_layer_parallel_efficiency_report.txt`) reports information about the parallel efficiency based on the number of the input/output channels of each layer with regard to the cvec and kvec of the architecture.

For example, if the cvec is 16 and the number of input channels is 60. The parallel efficiency is $\text{Total compute slots need} = \text{ceil}\left(\frac{60}{4}\right) \times 16 = 64$. The efficiency is $\frac{60}{64} = 93.75\%$.

The layer information is accompanied by the internal layer name and the original layer names from the OpenVINO graph that maps to each of the internal layer.

7.4.3. Visualizing How FPGA AI Suite Implements Your Graph

In addition to the text-based reports that the Model Analyzer provides, the Model Analyzer also provides a standalone graphical interface that can open visualizations of the Model Analyzer reports in a web browser. The *Model Analyzer GUI* aggregates information about the machine learning graph from the Model Analyzer reports generated by the FPGA AI Suite compiler.

The Model Analyzer GUI is an HTML page (`model_analyzer_gui.html`) that is output along with the other compiler outputs such as the `.txt`, `.dot`, and `.csv` reports. The compiler outputs these files to the `$COREDLA_WORK` directory unless you use the `--dumpdir` compiler option to specify an output destination directory.

To use the Model Analyzer GUI, open the `model_analyzer_gui.html` file in any of the following supported web browsers:

- Google* Chrome* Version 86 or newer
- Mozilla Firefox Version 75 or newer
- Microsoft Edge* Version 86 or newer

If the FPGA AI Suite compiler processes multiple graphs, each input graph has their own copy of `model_analyzer_gui.html`. there will be one copy of the GUI for each input graph.

Restriction: You cannot open the `.dot` files in the `visualizations` directory with the Model Analyzer GUI. To view the `.dot` files in the `visualizations` directory, convert the `.dot` files to `.svg` file with the Graphviz graph visualization software.

Related Information

https://graphviz.org

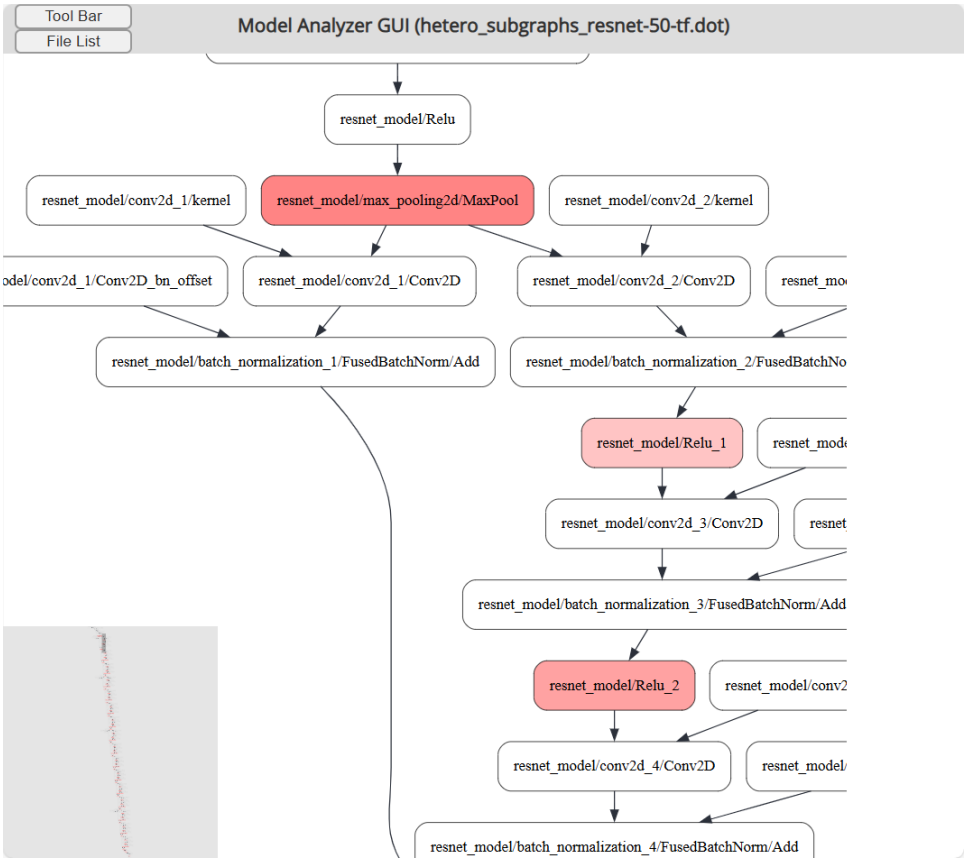
7.4.3.1. Model Analyzer GUI Description

When initially open the Model Analyzer GUI, you are prompted to open a `.dot` file or a compiler reports folder. You can also drag-and-drop files or folders to the Model Analyzer GUI.

If you have multiple model visualization available in a folder, you can switch between them by clicking **File List** and selecting a `.dot` file to view.

The Model Analyzer GUI header indicates either the folder name and open `.dot` file name (if you opened a folder) or only the open `.dot` file name (if you opened only a `.dot` file).

Figure 17. Example Model Analyzer GUI



Clicking on node in the graph shows additional information about the node in a **Details** pane

The Model Analyzer GUI has the following main components:

- **Main Window**

  Displays the DOT graph. You can zoom and pan the graph to view closeups of parts of your graph with zooming and panning.

  You can also drag the graph around in the main window to view different parts of your graph.

  You can zoom into the graph by scrolling your mouse wheel. Holding down the CTRL key while scrolling the mouse while zooms in and out quickly.
- **Tool Bar button**: Click **Tool Bar** and choose actions to open files or folders, hide constant nodes in the graph, highlight nodes with latency impacts, view help.
- **File List button**: Click **File List** to view the list of available `.dot` graph files to display. Click a file in the list to display the graph in the main window.
- **Details Pane**: Opens on the right when you click a node in a graph. The details pane shows detailed information about the selected nodes or layer, including attributes and performance metrics.
- **Minimap:** The lower-left corner shows a minimap view of the graph which indicates which part of the graph is visible in the main window. You can drag the box in the view to quickly navigate the entire graph.

Viewing Layer Details

Click on a node inside the Main Window to open Details Pane. The information for that layer will be displayed as follows:

- For `hetero_subgraph .dot` files the Details Pane shows information about the OpenVINO IR name for the layer, the OpenVINO nGraph Operation name, assigned device for non-constant layers; precision, shape, and value for constant layers.
- For `unsupported_layer .dot` files the Details Pane shows layer name, FPGA support, origin of the layer in the original OpenVINO IR, the type of the layer, and the reasons that the layer is assigned to CPU instead of FPGA.

Information displayed in the Details Pane is parsed from the DOT graph and matches with the manually converted image.

Viewing Layer Latencies

When you have a `hetero_subgraph .dot` file open, you can color-code the layers based on their execution latency by selecting **Tool Bar ➤ Layer Latency Heatmap**. This color-coding provides you with a visual cue of performance-critical sections of your graph.

Remember: Layer latency information is available only if you specified one or both of the following FPGA AI Suite compiler command options when compiling your graph:

- `--fanalyze-performance`
- `--festimate-per-layer-latencies`

Constant nodes and nodes without reported layer latency appear in white.

Layers with reported latency values appear in shades of red with darker shades indicating a higher latency and lighter shades indicating a lower latency.

To see the latency report details for a node, click the node of interest to display the latency report in the Details Pane.

Per-layer latency information is not available when displaying `unsupported_layer .dot` files.

Hiding Constant Nodes

When you have a `hetero_subgraph .dot` file with many constant nodes (node in white color), you can reduce the visual clutter of seeing them in your graph by selecting **Tool Bar ➤ Hide Constant Node**.

To display the constant node after hiding them, select **Tool Bar ➤ Hide Constant Node** again.

7.4.3.2. Fixing Unsupported Layers Assigned To CPU

If you encountered the warning message from the FPGA AI Suite compiler saying that your ML graph is split into too many subgraphs, or you suspect that many graph layers are assigned to the CPU and are causing low performance, check if the graph contains unsupported layers.

Graph layers that are unsupported by the FPGA cause the compiler to split the computation graph to heterogeneous subgraphs. Each heterogeneous subgraph has its corresponding DOT file. CPU subgraphs, when introduced unnecessarily, are bad for performance because moving data from FPGA to CPU, and then the reverse, costs wasted cycles. In general, unless the CPU execution of the subgraphs is intentional or necessary (for example, the output layers of YOLO are handled by CPU), eliminate as many of them as possible.

Unsupported layers that are assigned to CPU are usually caused by the following reasons:

- Your ML graph uses an unsupported layer, or the hyperparameter of a layer is not within the supported range by the FPGA AI Suite. In this case, consider searching for a supported alternative, or workaround by replacing the unsupported layer with one or a combination of other layers that achieves the same outcome as the unsupported layer. To learn about the supported layers and their supported ranges, refer to The FPGA AI Suite IP Overlay Architecture on page 15.
- If the layer is a supported layer, and the hyperparameters are within the supported range, but the compiler still creates CPU subgraph, then you might be using an architecture that does not enable the essential computation modules.

Use the Model Analyzer GUI to view the unsupported layers by opening an unsupported layer DOT file in the compiler report files. Click on the nodes to investigate the assignment reasons in the Details Pane and fix the unsupported layers accordingly.

7.5. FPGA AI Suite Compiler Command Line Options

The `dla_compiler` command options that you use change depending on your use mode. To see what command options are required and optional for the various use modes, review Compiling Your Model with the FPGA AI Suite Compiler *on page 193*.

7.5.1. Inputs (`dla_compiler` Command Options)

`--network-file` Option

Syntax `--network-file=<network_file> <network_file_2> ... <network_file_n>`

Description This option specifies the path to your OpenVINO network topology (`.xml`) file or files. You can specify multiple files with this option to allow for a multigraph flow.

The network weight file (`.bin`) associated with each network topology file must be in the same folder as the network topology file and must use the same graph name.

Example `--network-file_name=/user_path1/graph1.xml /user_path2/graph2.xml`

Parsed As `(vector<string>)`

`--plugins-file` Option

Syntax `--plugins-file=path_to_plugins.xml`

Description This option is intended for expert use only and is rarely needed. Incorrect use of this option can result in obscure error messages. This option specifies the path to a custom `plugins.xml` file that initializes the `InferenceEngine::Core` with a custom `plugins.xml` file.

The `plugins.xml` file indicates the locations for your custom FPGA AI Suite `.so` files: `libcoreDLAHeteroPlugin.so` and `libcoreDLAAotPlugin.so`.

Example `--plugins-file=./lib/plugins.xml`

Parsed As `(string)`

`--batch-size` Option

Syntax `--batch-size=<batch_size>`

Description This option overrides of the network specified batch size.

For multigraphs, you can specify a single batch size to use for all graphs.

Example `--batch=16`

Parsed As `(int)`

`--bin-data` Option

Syntax `--bin-data`

Description Forces graph to be compiled for binary data input, regardless of input tensor shape. Binary data is read as the input layer type.

Otherwise, graphs with 1- or 3-channel inputs are converted to U8 inputs and binary input files are discarded.

Parsed As `(bool)`

7.5.2. Outputs (`dla_compiler` Command Options)

`--o` Option

Syntax `--o <file_name>`

Description This option specifies the output file name for the exported compiled graph.

Example `--o ResNet50_GenericArch.bin`

Parsed As `(string)`

`--dumpdir` Option

Syntax `--dumpdir path`

Description This option specifies the path to the folder for export file and debug file outputs.

The file path location specified by the `--dump_dir` option command is also used as the destination location for most of the `.csv`, `.txt`, and `.dot` files created by the compiler. If you do not specify the `--dump_dir` option, these files are placed in the current working directory.

Example `--dumpdir /tmp/my_output/`

Parsed As `(string)`

`--foutput-format` Option

Syntax `--foutput-format=<exported_graph_format>`

Description This option specifies the format of the exported compiled graph file.

The <*exported_graph_format*> parameter can take one of the following values:

- `open_vino_hetero`

  The export function is called from the OpenVINO HETERO plugin. This option serializes and dumps the following outputs to a binary file:

  - ☐ OpenVINO input information
  - ☐ OpenVINO output information
  - ☐ FPGA AI Suite compiled results

- `dla_compiled_result`

  Only the FPGA AI Suite compiled results are dumped to the binary file.

Default Value `open_vino_hetero`

Parsed As `(string)`

7.5.3. Reporting (`dla_compiler` Command Options)

When a graph is compiled, the following CSV files are created by default to provide additional information about the input and output tensors and how they map to FPGA AI Suite IP tensors:

- `input_transform_dump`
- `input_transform_mapping`
- `output_transform_dump`
- `output_transform_mapping`

You can disable the generation of these reports with the `--fno-transform-tables` command option.

`input_transform_dump` Report

The `input_transform_dump` file is a CSV file with input tensor dimension, striding, padding, and precision. The `input_transform_dump` report is provided in a file named `input_transform_dump_`*<graph_name>*`.csv`

An example of a simple output CSV file created from an AlexNet graph is as follows:

Table 21. (`input_transform_dump_AlexNet.csv`)

	depth	height	width	height stride	width stride	top pad	left pad	Bot pad	right pad	precision
Input num 0	48	57	57	4	4	0	0	1	1	fp16

Additional information about the initial scale and shift performed on the tensor is included separately.

`input_transform_mapping` Report

The `input_transform_mapping` file is a CSV file that shows the element-wise mapping of the logical input tensor elements to the FPGA AI Suite input tensor format. The `input_transform_mapping` report is provided in a file named `input_transform_mapping_<graph_name>.csv`.

The transform mapping report has columns corresponding to the offset and subscript indices for the logical input tensor elements, and the corresponding elements in the transformed FPGA AI Suite input tensor.

For more information about contents of the input transform mapping report, refer to Input Transform Mapping on page 232.

`output_transform_dump` Report

The `output_transform_dump` file is a CSV file with output tensor offsets and dimension (batch, depth, height, width). Output transforms tables are named `output_transform_dump_<Graph_name>.csv`

An example of a simple output CSV file created from an AlexNet graph is as follows:

Table 22. (`output_transform_dump_AlexNet.csv`)

	batch dim	depth dim	height dim	width dim	batch offset	depth offset	height offset	width offset
output 0 tensor 0	1	1000	1	1	0	0	0	0

`output_transform_mapping` Report

The report is a CSV file that show the element-wise mapping of the FPGA AI Suite output tensor format to the logical output tensor. The `output_transform_mapping` report is provided in a file named `output_transform_mapping_<graph_name>.csv`.

This mapping in the `output_transform_mapping` report is the inverse of the input mapping described in the `input_transform_mapping` report.

For more information about the output tensor format, refer to Output Tensor In-Memory Format on page 233.

`--fno-transform-tables` Option

Syntax `--fno-transform-tables`

Description This option disables the creation of the CSV report files.

Parsed As `(bool)`

7.5.4. Compilation Options (`dla_compiler` Command Options)

`--fplugin` Option

Syntax `--fplugin=<OpenVINO_plugin>`

Description This option specifies the plugin for the OpenVINO inference engine.

The <*OpenVINO_plugin*> parameter can take one of the following values:

- `HETERO:FPGA`

  The OpenVINO HETERO plugin is used, and it tries to use the FPGA for all inference operations.

  If any layers are not supported by the FPGA device, then it issues an error.

  Use this plugin to ensure that the CPU support is not used for any layers.

- `HETERO:CPU`

  The OpenVINO HETERO plugin is used, and all inference operations are allocated to the CPU.

  Use this plugin to test the data flow through `dla_benchmark` application when an FPGA device is not available.

- `HETERO:FPGA,CPU`

  The OpenVINO HETERO plugin is used, and inference operations are allocated to the FPGA device where possible. The CPU is used for layers that are not supported on the FPGA device.

  The graph is partitioned into parts that run on the FPGA device and parts that must run on the CPU.

Default Value `HETERO`

Parsed As `(string)`

`--fassumed-fmax-core` Option

Syntax `--fassumed-fmax-core=<assumed` f_{MAX}`>`

Description This option sets the f_{MAX} used for calculating performance. Specify the <*assumed* f_{MAX}> value in MHz.

Default Value The default value depends on the device family:

- Arria 10 - `265`
- Agilex 3 - `350`
- Agilex 5 - `350`
- Agilex 7 - `500`
- Cyclone 10 GX - `265`
- Stratix 10 - `265`

Parsed As `(int)`

`--ffolding-option` Option

Syntax `--ffolding-option=<folding_option>`

Description This parameter controls channel-folding optimization during inference. Folding changes the shape of the input feature tensor to increase computational efficiency on the FPGA.

For more details on folding, refer to Folding Input on page 226.

Channel folding can be applied by the following system components:

- Host runtime software
- FPGA AI Suite IP:
  - ☐ Dedicated layout transform module
  - ☐ Convolution engine (in a limited capacity)

The <*folding_option*> parameter can take one of the following values: `0`, `1`, `2`, or `3`.

- **0**

  Folding is applied neither externally nor by any FPGA AI Suite IP module.

- **1**

  Folding is performed by the runtime software on the host or by the FPGA AI Suite IP dedicated layout transform module if it is available.

  This is the default option.

- **2**

  Folding performed by the following components:

  - ☐ Runtime software on the host or by the FPGA AI Suite IP dedicated layout transform module if it is available.
  - ☐ FPGA AI Suite convolution engine

- 3

  Folding is preformed by the FPGA AI Suite IP convolution engine only. The FPGA AI Suite IP dedicated layout transform module is not used.

Choosing A Value for the <*folding_option*> Parameter

Consider the following criteria to choose a value for the <*folding_option*> parameter:

- If your overlay architecture includes the full input layout transform module, use option `1` or `2` to maximize folding efficiency.
- If the full input layout transform module is not available, use option `0` or `3` to reduce preprocessing time in software.
- If you are using the lightweight input transform, you must use option `0` or `3`.
- If you are unsure, use option `1`. This option is recommended for most applications.

Default Value 1

Parsed As `(int)`

`--fmin-subgraph-layers` Option

Syntax `--fmin-subgraph-layers=<min_subgraph_layer>`

Description This option sets the minimum number of layers that are allowed in a subgraph that runs on FPGA. A subgraph with fewer layers than this value runs on CPU in the OpenVINO HETERO plugin.

Default Value `2`

Parsed As `(int)`

7.5.5. Architecture Options (`dla_compiler` Command Options)

`--march` Option

Syntax `--march=<architecture_filename>`

Description This option sets the path to the architecture file to use when compiling a graph.

Default Value `./example_architectures/Generic.arch`

Parsed As `(string)`

7.5.6. Architecture Optimizer Options (`dla_compiler` Command Options)

`--interleave-search` Option

Syntax `--interleave-search`

Description This option causes the architecture optimizer to evaluate different legal feature and filter interleave options. This significantly increases the run time of the architecture optimizer, by as much as 50%.

Due to the run time penalty, after an optimal interleave has been found for a given graph, the recommendation is to place the optimized interleave into the initial input `.arch` file and avoid using `--interleave-search` during any future fine-tuning.

For more information, refer to Parameter Group: pe_array on page 179.

Default Value N/A

Example `--interleave-search`

Parsed As `(bool)`

`--mmax-resources` Option

Syntax `--mmax-resources=<max_ALMs>,<max_M20Ks>,<max_DSPs>`

Description This option specifies the maximum number of ALMs, M20Ks, and DSPs to use when optimizing the architecture.

If you do not specify this option, the architecture uses as many resources as needed. In rare cases, the optimizer might use more resources than are available on the FPGA device.

Default Value N/A

Example `--mmax-resources=50000,500,400`

Parsed As `(vector<int>)`

The `vector` must be of length 3. The `int` values can be separated by one of the following characters:

- `,`
- `;`
- `|`
- `\`

`--mmax-resources-alm-util` Option

Syntax `--mmax-resources-alm-util=<percent_alm_utilization>`

Description Set this option only if you also set the `--mmax-resources` option.

This option sets a percentage target of the <*max_ALMs*> value from the `--mmax-resources` option that architecture optimizer aims to use for the architecture. The remaining ALMs are used by the Quartus Prime software to improve timing closure.

Valid values for this option are integers in the 1-100 range.

Default Value `100`

Example `--mmax-resources-alm-util=75`

Parsed As `(int)`

`--mtarget-fps` Option

Syntax `--mtarget-fps=<target_framerate>`

Description This option specifies the target frame rate (fps) for an architecture optimized for minimum resource usage.

The architecture optimizer can use in excess of 128GB of RAM when using this option, and thus requires a server-class machine. The required memory varies significantly depending on the graph. For example, a ResNet 50-type graph can often require up to 256 GB of memory.

To reduce the number of threads and memory used by the architecture optimizer when you use the `--mtarget-fps`, set the `COREDLA_TARGET_FPS_THREAD_LIMIT` environment variable to specify the maximum number of threads to use. For example, issuing a `export COREDLA_TARGET_FPS_THREAD_LIMIT=1` command, limits the architecture optimizer to running on one thread. Limiting the number of threads reduces the computational parallelism and can increase the runtime of the architecture optimizer.

Example `--mtarget-fps=25.0`

Parsed As `(double)`

`--network-weighting` Option

Syntax `--network-weighting=<network_weight_1> <network_weight_2> <network_weight_3>...<network_weight_n>`

Description This option defines the relative optimization weighting of input graphs.

For example, if two graphs are specified and have throughput values of fps_1 and fps_2, then the overall throughput is maximized using the user-specified weights w_1 and w_2 as follows:

$$\text{fps}_{\text{total}} = 1/\left(\frac{w_1}{\text{fps}_1} + \frac{w_2}{\text{fps}_2}\right)$$

When you specify multiple networks, the list of weights must be space-delimited.

If this option is not specified, all networks are equally weighted.

Example `--network-weightings=1 2`

Parsed As `(vector<int>)`

7.5.7. Analyzer Tool Options (`dla_compiler` Command Options)

`--fanalyze-area` Option

Syntax `--fanalyze-area`

Description Set this option to have the compiler estimate the area of the specified architecture and generate a `.pta` file that you can open with the FPGA Power and Thermal Analyzer to see power usage estimates for the architecture.

Parsed As `(bool)`

`--fanalyze-performance` Option

Syntax `--fanalyze-performance`

Description When this option is set, the compiler estimates the performance of the given graph on the specified architecture. The performance report provides the estimated kernel throughput for each network graph provided.

The compiler also estimates the average memory bandwidth and the memory requirements. The estimated memory requirement is typically an underestimate because it excludes the effects of memory fragmentation.

An example performance report is as follows:

```
Graph reported throughput: 12.345678
```

Parsed As `(bool)`

`--fassumed-memory-bandwidth` Option

Syntax `--fassumed-memory-bandwidth=<assumed_DDR_bandwidth_in_MB/s>`

Description The FPGA AI Suite IP throughput is directly correlated to the DDR bandwidth in your system. Use this option with the `--fanalyze-performance` option to have the compiler estimate the performance based on the assumed DDR bandwidth in MB/s. If the architecture file is a DDR-free architecture, do not set this option.

As an example, a performance report that includes this option (set as `--fassumed-memory-bandwith=19200`) includes the following line:

```
ASSUMED DDR BANDWIDTH PER IP INSTANCE = 19200 MB/s
```

Default Value The default value depends on the device family:

- Arria 10 - `19200`
- Agilex 3 - `6400`
- Agilex 5 - `6400`
- Agilex 7 - `21328`
- Cyclone 10 GX - `6400`
- Stratix 10 - `19200`

Parsed As `(double)`

`--fdump-area-report` Option

Syntax `--fdump-area-report=<filepath>`

Description Optional output file for the area estimate.

This option also enables the `--fanalyze-area` option.

Parsed As `(string)`

`--fdump-performance-report` Option

Syntax `--fdump-performance-report=<dump_performance>`

Description Optional output file for the performance estimate.

This option also enables the `--fanalyze-performance` option.

Parsed As `(string)`

`--fdump-pta-report` Option

Syntax `--fdump-pta-report=<filepath>`

Description Optional output file for the FPGA Power and Thermal Analyzer parameters files generated as part of the area estimation.

This option also enables the `--fanalyze-area` option.

Parsed As `(string)`

`--fdisplay-device` Option

Syntax `--fdisplay-device`

Description The option enables displaying the device (for example, CPU or FPGA) for each subgraph for all networks.

Parsed As `(bool)`

7.5.8. Miscellaneous Options (`dla_compiler` Command Options)

`--h` or `--help` Option

Syntax
- `--h`
- `--help`

Description This option displays the help message for the `dla_compiler` command.

Parsed As `(bool)`

7.6. Compiler Inputs and Outputs

The FPGA AI Suite compiler requires input files in specific formats and generates a variety of output files including reports about the compiled graph and its use of FPGA resources.

Review the compiler command options for descriptions of these input and outputs.

Additional information about compiler inputs and outputs are provided in the following sections:

- FPGA AI Suite Input File Formats on page 216
- FPGA AI Suite Compiler Graph Export Formats on page 216
- Partitioning Table Report on page 198
- Model Analyzer Reports on page 198
- Model Visualizations

Related Information

- Inputs (dla_compiler Command Options) on page 204
- Outputs (dla_compiler Command Options) on page 205
- Reporting (dla_compiler Command Options) on page 206

7.6.1. FPGA AI Suite Input File Formats

OpenVINO Graphs (`.xml`/`.bin`)

These files are specified with the `--network-file` compiler command option.

Your must convert your models to the OpenVINO Inference Engine format (`*.xml` and `*.bin` files) by using the OpenVINO Model Optimizer tool.

For details, refer to the OpenVINO documentation at the following URL:

https://docs.openvino.ai/2024/documentation/legacy-features/transition-legacy-conversion-api/legacy-conversion-api.html

FPGA AI Suite IP Architecture Description (`.arch`)

This file is specified with the `--march` compiler command option.

For details about this file format, review the Creating an Architecture File for the FPGA AI Suite IP on page 167.

Related Information

Inputs (dla_compiler Command Options) on page 204

7.6.2. FPGA AI Suite Compiler Graph Export Formats

Use the `--foutput-format` option of the FPGA AI Suite compiler command to set the graph export format. You can export the graph in the following formats:

OpenVINO `HETERO` Graph Export File Format

For the `open_vino_hetero` output file format (as set by the `--foutput-format` option), the export function is called from the OpenVINO HETERO plugin. This plugin serializes and dumps OpenVINO input and output information and the FPGA AI Suite (`dla_compiler`) compiled results to a binary file.

Compiled Results Graph ExportFile Format

For `dla_compiled_result` output file format (as set by the `--foutput-format` option), the export function is called from the OpenVINO HETERO plugin. Only the FPGA AI Suite (`dla_compiler`) compiled results are dumped to a binary file.

Related Information

Outputs (dla_compiler Command Options) on page 205

8. Generating the FPGA AI Suite IP for Integration into an FPGA Design

The FPGA AI Suite IP generation utility reads an input Architecture Description File (`.arch`) and places generated IP into an IP library that can be imported into Visual Designer Studio or Platform Designer, or used directly in a pure RTL design. The generated IP is configured based on the Architecture Description (`.arch`) File, which defines the following items:

- Top RTL parameters are set to reflect values in the Architecture Description File
- The Discovery ROM MIF is created to specify an architecture hash and FPGA AI Suite compiler version string

The generation utility also copies all the required RTL files into a single directory. As convenience options, the utility can also help to create `.qsf` files and wrappers for the PCIe design example or to compile an instance of the IP with a dummy BSP interface.

The IP generation utility generates either an unlicensed or a licensed copy of the IP:

Unlicensed IP The unlicensed IP has a limit of 100,000 inferences. After 100,000 inferences, the unlicensed IP refuses to perform any additional inference and a bit in the CSR is set. For details about the CSR bit, refer to "DMA Descriptor Queue" in CSR Map and Descriptor Queue on page 264.

Licensed IP The licensed IP has no inference limitation

The IP generation utility checks for an FPGA AI Suite IP license before generating the IP. The utility prints messages to `stdout` that show the license status.

You can use either licensed and unlicensed IP for bitstream generation so that you can fully test your design during the evaluation process.

8.1. IP Generation Utility Execution Flows

The IP generation utility (`dla_create_ip` command) has the following flows:

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

Table 23. IP Generation Flows

Flow	Command Option	Description
Create an IP Directory	`--flow create_ip`	Creates a new FPGA AI Suite IP library directory, generate an IP, and place it in the library.
Add an Architecture to an IP Directory	`--flow add_arch`	Generates an IP and place it in an existing FPGA AI Suite IP library directory.
List Architectures in an IP Directory	`--flow list`	Lists the IPs in an existing IP library directory.
Removing an Architecture from an IP Directory	`--flow remove_arch`	Removes an IP from an existing IP library directory.

Create an IP Directory

This flow is typically first flow that you use. It creates an FPGA AI Suite IP library directory and adds an architecture as follows:

1. Creates the IP library directory and copies all contents from *<ai_suite_root>*/`fpga/ip_template/` into the new directory. The copied folder (*<ip_template>*) contains some basic Tcl scripts and Platform Designer IP configurations.
2. Creates a *<ip_directory>/Verilog* directory and copies over RTL files that are common to any generated FPGA AI Suite IP architecture.

   These files are listed in `static_files.tcl`, which is used to ensure that other flows (the Platform Designer flow and design assembly flows outside Platform Designer) have access to the list of RTL source files.
3. Creates the following directory for each architecture-family pair to add to the IP library:

   <ip_directory>`/Verilog/`*<architecture>_<family>*

   This location stores architecture-specific files.
4. Invokes an internal utility to read the architecture description file (`.arch`).

   Output files are architecture-specific and are copied to the *<ip_directory>*`/Verilog/`*<architecture>_<family>* directory.
5. For the specific architecture, creates a `generated_files.tcl` file and a `dla_ip.qsf` file in the *<ip_directory>*`/Verilog/`*<architecture>_<family>* directory.

Add an Architecture to an Existing IP Directory

This flow adds a new architecture to an existing IP library directory as follows:

1. Creates a *<ip_directory >/Verilog* directory and copies over RTL files that are common to any generated FPGA AI Suite IP architecture.

   These files are listed in `static_files.tcl`, which is used to ensure that other flows (the Platform Designer flow and design assembly flows outside Platform Designer) have access to the list of RTL source files.
2. Creates the following directory for each architecture-family pair to add to the IP library:

   <ip_directory>`/Verilog/`*<architecture>_<family>*

This location stores architecture-specific files.

3. Invokes an internal utility to read the architecture description file (`.arch`).

   Output files are architecture-specific and are copied to the *<ip_directory>*/`Verilog/`*<architecture>_<family>* directory.

4. For the specific architecture, creates a `generated_files.tcl` file and a `dla_ip.qsf` file in the *<ip_directory>*`/Verilog/`*<architecture>_<family>* directory.

List Architectures in an IP Directory

This flow lists all available architectures in the IP library directory. The utility looks for all *<architecture_family>* folders and displays them.

Remove an Architecture from an IP Directory

This flow removes an architecture from an IP library directory. The utility looks for a *<architecture_family>* folder and removes it.

8.2. IP Generation Utility Inputs

The only external inputs to the IP creation flow are FPGA AI Suite architecture description files. The file format for architecture descriptions files is described in Architecture Description File Format for Instance Parameterization on page 168.

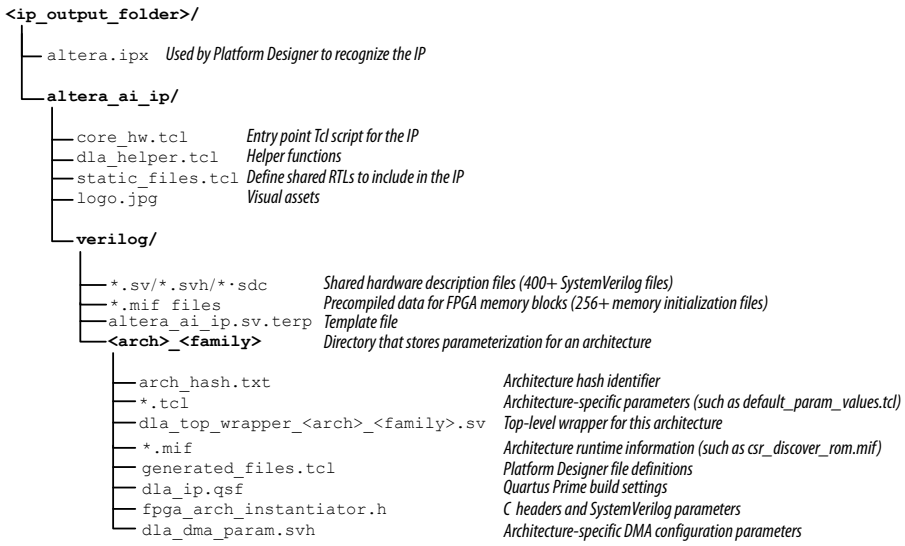
The internal inputs to the scripts are:

- *<ai_suite_root>*/fpga/*<modules>*. Module folders that contain RTL source files.
- *<ai_suite_root>*/fpga/ip_template. Contains basic files used to create the IP in the Platform Designer flow.

8.3. IP Generation Utility Outputs

The FPGA AI Suite IP generation utility (`dla_create_ip` command) generates an FPGA AI Suite IP directory as its output.

The default directory structure is as follows:



8.4. IP Generation Utility Command Line Options

The actions of the IP generation utility (`dla_create_ip` command) are controlled primarily by the `--flow` option. Depending on your choice for the `--flow` option, you might need to specify other command options.

The `--flow` Option

This option specifies the IP generation utility execution flow to run.

Syntax

`--flow <flow>`

Where <*flow*> specifies the execution flow to run.

Valid values

- `create_ip`

  For details, refer to The --flow create_ip Flow on page 223.
- `add_arch`

  For details, refer to The --flow add_arch Flow on page 224.
- `list`

  For details, refer to The --flow list Flow on page 224.
- `remove_arch`

  For details, refer to The --flow remove_arch Flow on page 225.

Default Value

`create_ip`

The `--arch` Option

This option specifies one or more FPGA AI Suite architecture description files (`.arch`) to use in the flow.

If you specify this option, you cannot specify the `--arch_dir` option.

Syntax `--arch <architecture> [<architecture2>] [<architecture3>] ...`

For the `create_ip` and `add_arch` flows, specify a path to the architecture description file or files to add to an IP library.

For the `remove_arch` flow, specify the name of the architecture description file or files to remove from an IP library.

The `--arch_dir` Option

Use this option with the `create_ip` and `add_arch` flows to specify the path to a directory that contains architecture description files. If you specify this option, the IP generation utility adds each architecture in the specified directory to the FPGA AI Suite IP library.

If you specify this option, you cannot specify the `--arch` option.

Syntax `--arch_dir <path_to_arch_folder>`

The `--ip_dir` Option

This option specifies the path of the FPGA AI Suite IP library directory to use in the flow.

In the `create_ip` flow, this directory is created if it does not exist. For all other flows, the specified IP library directory must exist.

Syntax `--ip_dir <ip_directory>`

Default Value `<ai_suite_root>/ip`

The `--unlicenced/--licenced` Options

Generate an unlicensed or licensed copy of the FPGA AI Suite IP:

- **Unlicensed IP:** Unlicensed IP has a limit of 10000 inferences. After 10000 inferences, the unlicensed IP refuses to perform any additional inference and a bit in the CSR is set. For details about the CSR bit, refer to "DMA Descriptor Queue" in CSR Map and Descriptor Queue on page 264.
- **Licensed IP:** Licensed IP has no inference limitation.

If you generate a licensed copy of the IP but do not have a license, then Quartus cannot generate a programming bitstream for your FPGA.

If neither option is specified, then the tool queries the `lmutil` license manager to determine the correct option.

Syntax `--unlicenced`

`--licenced`

The `--overwrite` Option

For the `create_ip` flow, use this option to overwrite the IP library directory when you specify an existing IP library directory.

If you specify an existing directory with the `create_ip` flow and do not specify the `--overwrite` option, you are asked if you want to overwrite the directory.

Syntax `--overwrite`

8.4.1. The `--flow create_ip` Flow

The default flow for the IP generation utility (`dla_create_ip` command) is the `--flow create_ip` flow. This flow creates a new IP library directory (`<ip_directory>`).

To generate a new IP, provide one of the following options as the location of architecture description file or files:

- Use the `--arch` option to specify one or more architecture description files (`.arch`).
- Use the `--arch_dir` option to specify a directory that contains one or more architecture description files. All architecture description files in the directory are added to the IP library.

Use the `--ip_dir` option to specify the output FPGA AI Suite IP library directory. If unspecified, the default value of *`<ai_suite_root>`*`/ip` is used as the IP library directory.

To overwrite the IP directory and create a new IP if the IP directory already exists, specify the `--overwrite` option. . If you do not set the `--overwrite` option, the script prompts you to confirm that you want to overwrite the IP library directory.

Usage Synopsis

```
dla_create_ip [--flow create_ip] \
   --arch <path to .arch File> [<path to .arch file> ...] \
   [--ip_dir <ip_directory>] \
   [--overwrite]
```

```
dla_create_ip [--flow create_ip] \
   --arch_dir=<path to  directory with .arch files> \
   [--ip_dir <ip_directory>] \
   [--overwrite]
```

Sample Call

```
dla_create_ip --flow create_ip \
   --arch=$COREDLA_ROOT/example_architectures/A10_Generic.arch \
   --overwrite \
   --ip_dir ./ip
```

Sample Output

```
============================================================
            Start IP Creation Flow
============================================================
Generate file path ip/altera_ai_ip/Verilog/Generic_A10
============================================================
            IP Creation finished
============================================================
```

8.4.2. The `--flow add_arch` Flow

The `--flow add_arch` flow adds additional generated IP to an existing IP library directory (previously created with the `--flow create_ip` flow).

If you add an architecture that already exists in the IP library directory, the architecture is overwritten with newly generated RTL files.

Usage Synopsis

```
dla_create_ip --flow add_arch \
   --arch <path to .arch file> \
   [--ip_dir <ip_directory>]
```

Sample Call

```
dla_create_ip --flow add_arch \
   --arch $COREDLA_ROOT/example_architectures/A10_Generic.arch \
   --ip_dir ./ip
```

Sample Output

```
============================================================
       Adding Generic_A10 to the existing IP
============================================================

Generate file path ./ip/altera_ai_ip/Verilog/Generic_A10

============================================================
       Finished adding Generic_A10
============================================================
```

8.4.3. The `--flow list` Flow

Use `--flow list` flow to list all available architecture in an IP library folder. An architecture description file (`.arch`) specifies the target FPGA family device.

If an architecture has been added to the IP library with different FPGA families, those architecture–FPGA family combinations are displayed as different architectures in the IP folder.

Usage Synopsis

```
dla_create_ip --flow list [--ip_dir <ip_directory>]
```

Sample Call

```
dla_create_ip --flow list --ip_dir $COREDLA_ROOT/example_ip_cores
```

Sample Output

```
================================================================
     Listing available architectures from <ai_suite_rootdir>/ip
================================================================
1x1x16x16_fp11_sb30240_reluk16_poolk16_A10
1x1x16x16_fp11_sb30240_reluk16_A10
```

8.4.4. The `--flow remove_arch` Flow

Remove an architecture from the IP library with the `--flow remove_arch` flow.

If you specify an architecture description file that does not exist in the IP library, the IP generation utility returns a warning.

Usage Synopsis

```
dla_create_ip --flow remove_arch \
   --arch <path to .arch file> \
  [--ip_dir <ip_directory>]
```

Sample Call

```
dla_create_ip --flow remove_arch \
   --arch A10_Generic.arch \
   --ip_dir ./ip
```

Sample Output

```
================================================================
removing Generic_A10 from existing IP
================================================================
```

9. Optimizing Your FPGA AI Suite IP

Optimization of the FPGA AI Suite IP generated by the FPGA AI Suite is essential for achieving application-specific performance, latency, and resource utilization targets.

The FPGA AI Suite provides a comprehensive toolchain that integrates model compilation, architecture tuning, and IP generation for deployment on advanced FPGA platforms. A central part of this toolchain is the architecture configuration file (`.arch`) that defines key parameters such as processing element (PE) array dimensions, data precision (for example, INT8, FP16), memory interface widths, and activation function implementations.

Fine-tuning these parameters in the architecture file enables efficient mapping of neural network workloads to FPGA resources. Additional optimization tactics include layer fusion, operator reordering, and partitioning of heterogeneous graphs to isolate unsupported layers. These techniques, when used in conjunction with the FPGA AI Suite compiler and performance estimation tools, allow for iterative refinement of the architecture to meet throughput, latency, and area constraints across a wide range of AI workloads.

9.1. Folding Input

Many graphs, particularly those processing image data, have very shallow input channels. In high CVEC instantiations, very shallow input channels can lead to low computational efficiency.

To improve the computational efficiency on these graphs, you can transform these shallow input channels to a deeper channel vector through a process called *input folding*.

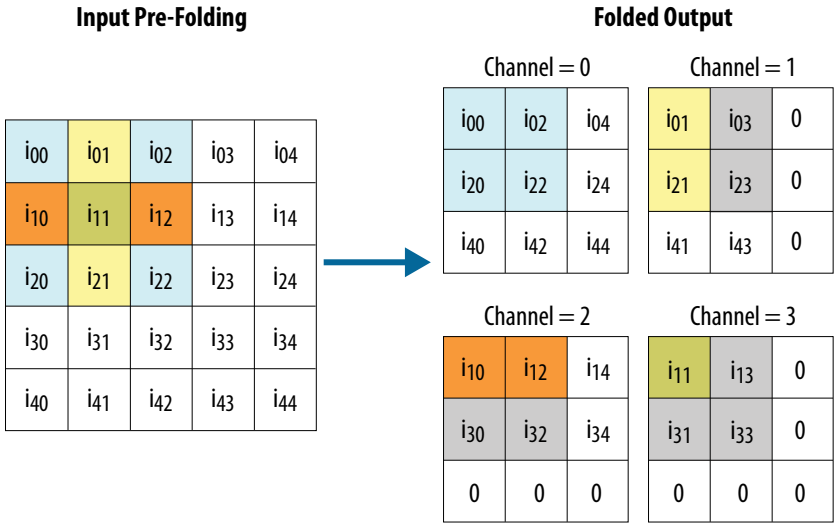
Folding the input can be done by the host runtime or a soft IP block that you write working in conjunction with the FPGA AI Suite compiler, or it can be performed in hardware using the hardware layout transforms described in Transforming the Layout of Input Data on page 230.

Input folding is typically most beneficial to the first layer of a graph.

The following figure illustrates an example of the functionality of the folding transform performed by the FPGA AI Suite compiler.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

Figure 18. Illustration of the folding transform for a 1x1×5×5 input, 1×1×3×3 filter and stride_height = stride_width = 2



In this transformation, the input depth, height, and width are folded into the channel dimension by a factor corresponding to the stride of the first convolution of a network. In the earlier figure, this factor corresponds to transforming the input channels from 1 to 4 ($1 \times \text{stride_depth} \times \text{stride_height} \times \text{stride_width}$), input height from 5 to 3 ($\text{ceil}\left(\frac{5}{2}\right)$) and input width from 5 to 3. Each color corresponds to the new filter window, which in this case would be 4×1×2×2, with the gray boxes corresponding to 0 padding for the filters. Folding is done in a similar way for inputs with depths greater than one, but the folding transform illustration excludes it for simplicity.

The FPGA AI Suite IP has various enhancements that reduce, but not eliminate the efficiency hit of shallow first layers. In many cases, you can disable first layer folding in the compiler and pass shallow-channel tensors directly to the IP hardware.

To enable input folding and control how it is applied during inference, set the `--ffolding-option` compiler option. For more information, refer to `--ffolding-option` Option in Compilation Options (dla_compiler Command Options) on page 208.

9.2. Parallelizing Inference Using FPGA AI Suite with Multiple Lanes and Multiple Instances

After folding strategies for resource balancing, the next stage in design refinement is optimization of performance through parallelization. At this stage, the objective shifts from fitting the model to maximizing throughput and minimizing latency.

Architectural Vector Scaling

The primary mechanism for compute parallelism within the FPGA AI Suite IP is adjustment of c_vector and k_vector.

- **k_vector** specifies the number of filters the PE Array can process in parallel. Legal values: 4–128, with the requirement that the value be a multiple of c_vector.
- **c_vector** defines the dot-product width per PE, with supported values of 4, 8, 16, 32, and 64.

Increasing these parameters reduces folding in the compiler output and increases the amount of work executed per cycle. This technique improves MAC throughput until constrained by available DSPs, BRAM, or achievable f_{MAX}.

Streaming Optimization and Multilane Scaling

As architectural width increases, external DDR bandwidth becomes the limiting factor. Transitioning to DDR-free streaming removes this bottleneck by routing feature maps directly into and out of the accelerator pipeline.

- The **num_lanes** parameter scales the PE array by duplicating data lanes. Legal values are 1, 2, and 4.
- The total stream buffer size scales proportionally with the number of lanes, requiring adjustment of stream_buffer_depth by the inverse of num_lanes.
- To achieve best performance with DDR-free inference mode, enable multilane operation.

This optimization reduces per-frame latency and increases effective throughput by aligning data movement with compute capacity.

Graph-Level Optimization

Graph-level optimization enhances runtime efficiency by restructuring execution flow and maximizing hardware utilization without modifying the accelerator IP itself. These optimizations target scheduling, memory access, and computational overlap.

- **Batching** allows multiple inputs to be processed as a grouped workload, amortizing setup and initialization overhead across multiple samples. This approach increases throughput for inference scenarios where latency per frame is less critical.
- **Job Queuing** ensures the accelerator remains active by overlapping host-to-device transfers, compute execution, and device-to-host transfers. This minimizes idle cycles within the pipeline.
- **Operator Fusion** combines adjacent layers (e.g., convolution, bias, activation) into a single execution kernel, reducing intermediate memory writes and reads. This decreases bandwidth demand and improves latency.
- **Graph Partitioning** splits the model across multiple compute partitions, enabling sections of the graph to execute concurrently on independent accelerator instances or host-device pipelines. Partitioning aligns with multi-instance deployment strategies.

- **Weight Preloading** ensures that frequently reused weights are cached in on-chip memory, reducing DDR bandwidth usage when the same filters are applied repeatedly across multiple inputs.
- **Asynchronous Execution** leverages runtime APIs to queue multiple graphs or subgraphs, overlapping workloads across FPGA compute engines, thereby achieving concurrency at the application layer.
- **Dynamic Scheduling** enables runtime-level decisions on task ordering, prioritizing low-latency jobs or throughput-optimized batches based on workload characteristics.

These tactics extend beyond folding and vector scaling by addressing bottlenecks at the graph compilation and scheduling level. When combined with architectural parallelism and streaming, graph-level optimizations create a balanced pipeline where compute, data movement, and scheduling are jointly optimized.

Multi-Instance Deployment

Scaling beyond a single FPGA AI Suite IP core is achieved by deploying multiple accelerator instances within the same FPGA fabric. This approach allows for concurrent processing of multiple inference tasks, improving overall throughput and enabling parallel execution of workloads.

- **Instance Configuration**: Each FPGA AI Suite IP instance operates independently, with its own control and status registers, DMA interfaces, and memory resources. This isolation ensures that each instance can process data independently without interference.
- **Resource Allocation**: Proper allocation of FPGA resources is crucial. Each instance requires a portion of the FPGA DSPs, BRAMs, and logic elements. The total resource usage scales with the number of instances deployed.
- **Interconnect Design**: The FPGA interconnect fabric must be designed to support multiple instances, ensuring that data can be routed efficiently between the host and each FPGA AI Suite IP instance.
- **Clock Management**: Each instance may operate on a separate clock domain, requiring careful management of clocking resources and timing constraints to ensure stable operation.
- **Power Considerations**: Deploying multiple instances increases the overall power consumption of the FPGA. Power budgeting must account for the cumulative power requirements of all active instances.

Composite Optimization Strategy

An optimized FPGA AI Suite design applies parallelization across multiple layers:

1. **Architectural vector scaling (`c_vector`, `k_vector`)** for compute parallelism.
2. **Streaming with multilane (`num_lanes`)** for bandwidth and latency optimization.
3. **Batching and job queues** for graph-level utilization.
4. **Multi-instance replication** for workload concurrency.

Parallelization in FPGA AI Suite IP is achieved through progressive application of these methods, each addressing specific bottlenecks in the compute, memory, or workload domain.

Next Optimization Step

After you parallelize operations as much as possible, the next optimization step is With parallelization optimizations complete, the next stage in optimizing your IP is to transforming the input data layout to further align model execution with hardware efficiency.

9.3. Transforming the Layout of Input Data

The FPGA AI Suite provides two hardware transforms for the layout of input data:

- A standard transform that supports input folding for any feature, stride, and padding values.
- A lightweight transform that exchanges some folding operation support for FPGA area savings.

The results of either layout transform can be written directly to the stream buffer or written to external memory.

The lightweight layout transform also has additional restrictions on the input bus width (must be a multiple of the pixel width) to reduce resource usages.

In both the standard and lightweight input layout transforms, only input tensors with HWC ordering are supported.The data elements can be in either FP16, U8, or U16 format.

When active, the input layout transforms operate as follows:

Standard input layout transform	The standard layout transform hardware folds the input tensor and converts it to the CHWCvec format as described in Input Feature Tensor In-Memory Format on page 231. If configured for U8 or U16 inputs, the data elements are also converted to FP16 format before tensors are sent downstream for inference. Bias and scale values are applied to the input within the layout transform hardware module if required by the graph.
Lightweight input layout transform	The lightweight layout transform converts tensors to CHWCvec order without applying the folding operation. The data is converted to FP16. Bias and scale values are applied to the input within the lightweight layout transform hardware module if required by the graph.

To avoid input slicing when either hardware layout transform is enabled, size the stream buffer to accommodate the entire input feature.

Use the hardware layout transform with the `--ffolding_option 1` compiler option described in Compilation Options (dla_compiler Command Options) on page 208. The layout transform hardware does not currently support 5-dimensional input tensors.

9.3.1. Full Input Layout Transform

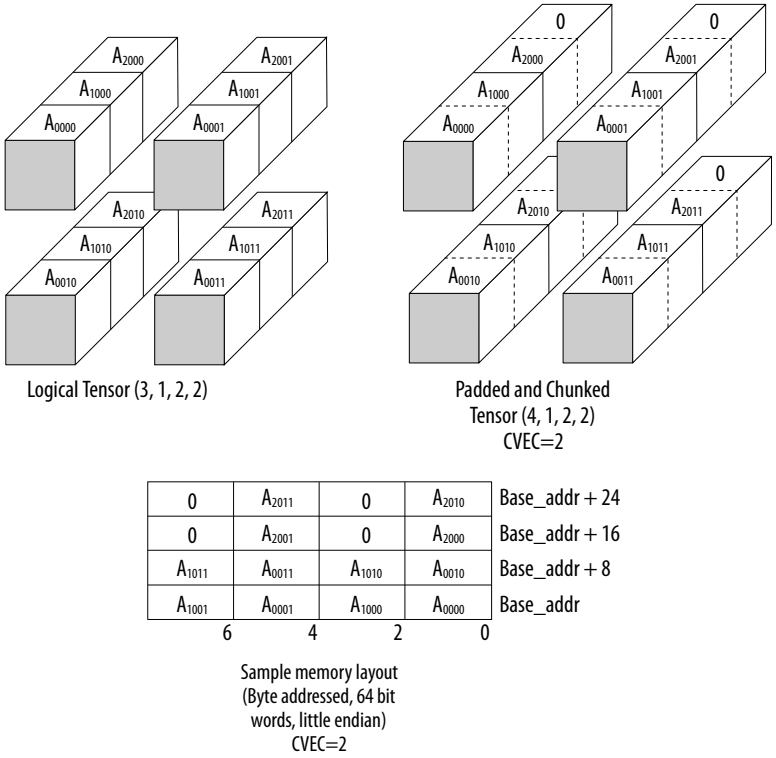
9.3.1.1. Input Feature Tensor In-Memory Format

Input features are stored in FP16 format. FP16 format has 1 sign bit, 10 mantissa bits, and 5 exponent bits. The input features are converted by the IP hardware to its native format using a *round to nearest, ties to even* (RNE) rounding rule.

Feature elements are packed into CVEC-sized chunks in the channel dimension from low to high. The final CVEC chunk, at a given (d,h,w), is padded with zeros. The CVEC chunks are stored in NCDHW format. The order is as follows: batch, channel, depth, height, width, and CVEC, where CVEC is the fastest changing index and batch the slowest.

The following figure shows a sample memory layout for a 1×3×1×2×2 input tensor to a CVEC=2 architecture:

Figure 19. Input Tensor In-Memory Layout



9.3.1.1.1. Multiple Input Graphs

For graphs with more than one input, each tensor is structured as described in the previous section. The multiple input tensors must be packed together at address offsets as reported by the FPGA AI Suite compiler.

The compiler generates CSV files that describe the input and output tensor, unless you specify the `--fno-transform-tables` option. Each row of the CSV file gives information about one input. For more details, refer to the Reporting (dla_compiler Command Options) on page 206.

For multiple inputs, the inputs are stored by batch and then by input number. For example, for 3 inputs and 2 batches, the input tensors would be stored as follows:

- input 1, batch 1
- input 1, batch 2
- input 2, batch 1
- input 2, batch 2
- input 3, batch 1
- input 3, batch 2

9.3.1.1.2. Input Scale and Shift

Many graphs require that input data be pre-scaled and pre-shifted. These scale and shift operations are supported in the FPGA AI Suite IP if they are sent to the device. Depending on the folding options specified, the method of support differs for the FPGA AI Suite IP. Input preprocessing is not supported for 5D inputs.

When you set the `--ffolding-option` FPGA AI Suite compiler option to `--ffolding-option=1` or `--ffolding-option=2`, external modules of the FPGA AI Suite are responsible for replacing the zero padding of the input data with non-zero shift values received as input from the FPGA AI Suite compiler.

When you set the `--ffolding-option` FPGA AI Suite compiler option to `--ffolding-option=0` or `--ffolding-option=3`, the scaling and shifting are performed entirely by the FPGA AI Suite IP without any additional support from the host or external modules. If the scale and shift operation is not mapped to the device, then it is typically performed as an operation fused with the conversion to FP16. This fused operation is performed either by a host CPU or by an external hardware block.

9.3.1.1.3. Input Transform Mapping

To help make sense of how input data is transformed for a compiled graph, the FPGA AI Suite compiler creates the following CSV files:

- `input_transform_dump_<graph-name>.csv`

  This file describes the tensor shape, padding, and stride for each input tensor.

- `input_transform_mapping_<graph-name>.csv`

  This file shows the element-wise mapping of the logical input tensor elements (domain) to the FPGA AI Suite IP input tensor format (co-domain) described earlier.

The transform mapping file has columns that correspond to the offset and subscript indices for the logical input tensor elements, and the corresponding elements in the transformed FPGA AI Suite input tensor.

For a graph with the input example given in Input Tensor In-Memory Layout, the transform mapping CSV output from the DLA compiler would be as follows:

Table 24. Example Transform Mapping CSV Output

Logical Tensor Offset	Input Channel (C)	Input Depth (D)	Input Height (H)	Input Width (W)	→	Input Tensor Offset	Transformed Channel (C)	Transformed Depth (D)	Transformed Height (H)	Transformed Width (W)	Transformed C-vector (Cvec)
0	0	0	0	0	→	0	0	0	0	0	0
1	0	0	0	1	→	2	0	0	0	1	0
2	0	0	1	0	→	4	0	0	1	0	0
3	0	0	1	1	→	6	0	0	1	1	0
4	1	0	0	0	→	1	1	0	0	0	0
5	1	0	0	1	→	3	1	0	0	1	0
6	1	0	1	0	→	5	1	0	1	0	0
7	1	0	1	1	→	7	1	0	1	1	0
8	2	0	0	0	→	8	0	0	0	0	1
9	2	0	0	1	→	10	0	0	0	1	1
10	2	0	1	0	→	12	0	0	1	0	1
11	2	0	1	1	→	14	0	0	1	1	1

The tensor offsets in this table are logical, not address offsets and are thus independent of data type. The transformation typically implicitly adds zero padding to the data. As such, not all transformed output logical offsets are mapped from a given input logical offset.

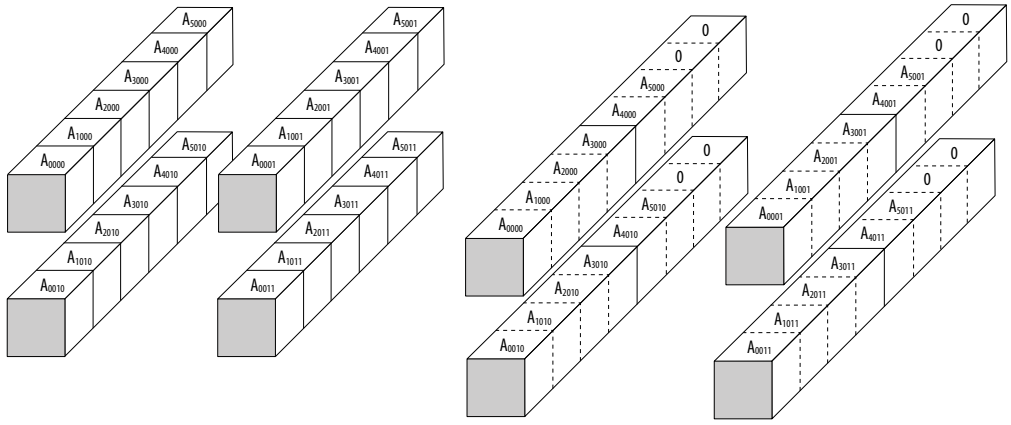
9.3.1.2. Output Tensor In-Memory Format

The output tensor in-memory format is similar to the input tensor in-memory format described in Input Feature Tensor In-Memory Format on page 231. However, the output tensor is padded to the nearest multiple of KVEC rather than CVEC, with the padding being done at the boundaries between FPGA AI Suite IP outputs rather than strictly at the edge of the logical tensor output.

While the logical tensor output might be a single tensor, the FPGA AI Suite IP might compute this output as one single output or by slicing the output into smaller pieces.

Each scenario can result in a different output layout, with the first resulting in padding only on the boundary of the tensor as shown in the following figure:

Figure 20. Output Tensor In-Memory Layout Without Slicing

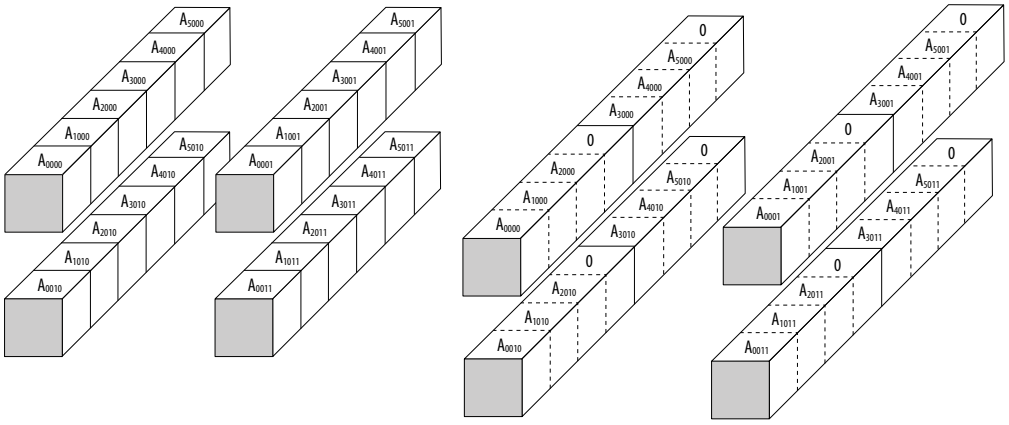


0	0	0	0	Base_addr + 56
0	0	0	0	Base_addr + 48
A_{5011}	A_{4011}	A_{5010}	A_{4010}	Base_addr + 40
A_{5001}	A_{4001}	A_{5000}	A_{4000}	Base_addr + 32
A_{3011}	A_{2011}	A_{3010}	A_{2010}	Base_addr + 24
A_{3001}	A_{2001}	A_{3000}	A_{2000}	Base_addr + 16
A_{1011}	A_{0011}	A_{1010}	A_{0010}	Base_addr +8
A_{1001}	A_{0001}	A_{1000}	A_{0000}	Base_addr
6	4	2	0	

Sample memory layout
(Byte addressed, 64 bit words, little endian)
CVEC=2, KVEC=4

The second scenario results in padding within the logical tensor as well as on the boundary of the logical tensor as shown in the following figure:

Figure 21. Output In-Memory Layout With Slicing



0	A_{5011}	A_{4011}	A_{3011}	Base_addr + 56
0	A_{5010}	A_{4010}	A_{3010}	Base_addr + 48
0	A_{5001}	A_{4001}	A_{3001}	Base_addr + 40
0	A_{5000}	A_{4000}	A_{3000}	Base_addr + 32
0	A_{2011}	A_{1011}	A_{0011}	Base_addr + 24
0	A_{2010}	A_{1010}	A_{0010}	Base_addr + 16
0	A_{2001}	A_{1001}	A_{0001}	Base_addr +8
0	A_{2000}	A_{1000}	A_{0000}	Base_addr
6	4	2	0	

Sample memory layout
(Byte addressed, 64 bit words, little endian)
KVEC=4

To enable the plugin to determine the appropriate output layout and offsets, the compiler provides a `DlaRuntimeOutputConfiguration` object. One of the fields of this object is the `output_tensor_mapping` field, which provides a mapping from FPGA AI Suite IP tensor output to logical tensor output.

To show how the IP output tensor data is mapped to the logical output tensor for a compiled graph, the FPGA AI Suite compiler creates the following CSV files:

- `output_transform_dump_<graph-name>.csv`

  This file describes the tensor shape and offsets.

- `output_transform_mapping_<graph-name>.csv`

  This file shows the element-wise mapping of the FPGA AI Suite output tensor to the logical output tensor. This mapping is in the inverse of the input mapping described in Input Transform Mapping on page 232. Refer to the example in that topic for a description of the transform mapping CSV file.

9.3.2. Lightweight Layout Transform

The term "lightweight" differentiates this design from the "full" layout transform to" indicate that this layout transform trades-off performance (FPGA AI Suite IP throughput) in favor of reduced resource utilization.

When to Use Lightweight Layout Transform

Consider enabling the lightweight layout transform in the FPGA AI Suite IP if any of the following criteria apply to your graph:

- The full layout transform consumes more ALMs or DSPs than budgeted and the device family is more sensitive to area usage.
- The ML model has a small input feature, and the performance enhancement from folding the input feature may be marginal.
- The first convolution in the ML model has a stride of 1x1, where folding does not provide extra performance gain.

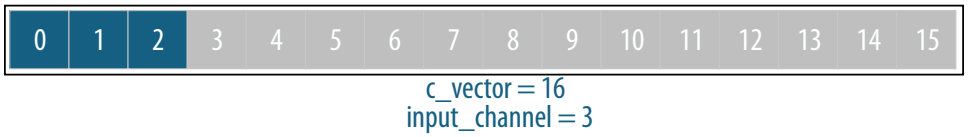
9.3.2.1. How the Lightweight Layout Transform Works

Since data usually arrives at the input feeder from a device as a raster scan channels-first format, the shape of the raw input tensors may have spatial dimension first and then channel last. For instance, for an 8-bit RGB image, the input shape is height by width by channel (HxWxC), where C is 3.

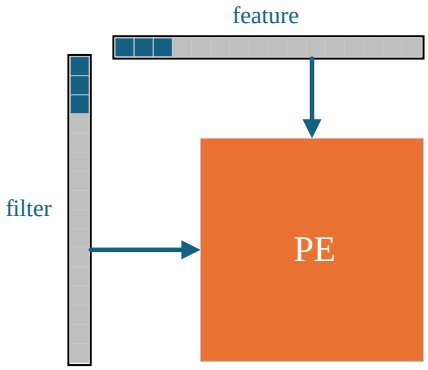
The PE array is capable of doing dot-product operations for a pair of *c_vector*-sized input features and filter at FP16 precision in each cycle. Therefore, the input tensors to the PE array engine are required to have a channel dimension that matches the value of c_vector.

When the lightweight layout transform is enabled, if an input pixel arrives at the input feeder with a channel dimension is less than c_vector, the channel dimension must be padded with zeros. For instance, an 8-bit RGB pixel in a 16 c_vector input is padded with 12 zeros.

Figure 22. Input Pixel with Zero Padding



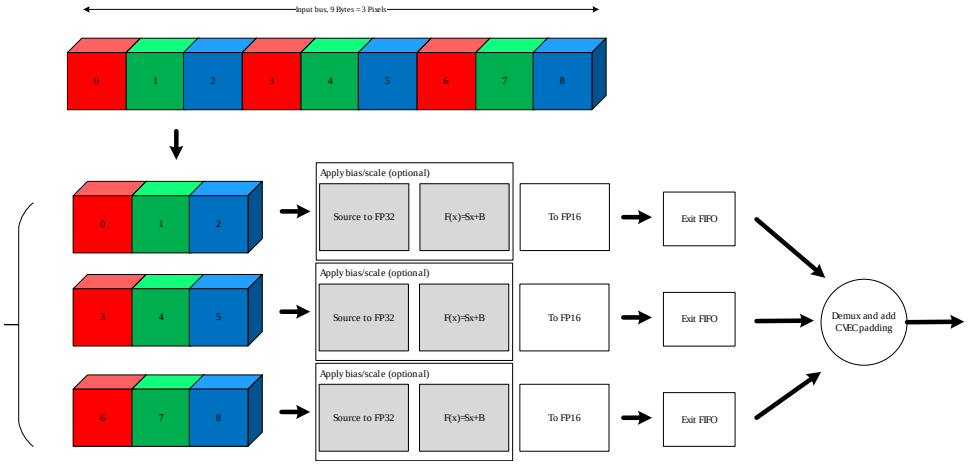
Then, elements in this pixel are converted to FP16, to match with the PE precision. After the conversion, the lightweight layout transform for this input feature is complete and it is queued in an exit pixel buffer ready to be consumed by the PE.



While the preceding description considered the lightweight transform for an input of one pixel at a time, in most scenarios multiple pixels can arrive at the input feeder on the data bus.

In the illustration that follows, three pixels arrive at the input feeder in parallel. Each pixel has three channels of 8-bit RGB value as an unsigned integer (U8). Therefore, the width of the input bus is 72 = 3 (parallel pixels) * 3 (channels per pixel) * 8 (bit per feature element).

Figure 23. Multiple Pixels In the Lightweight Layout Transform



Given N pixels arriving at the input feeder in parallel, each pixel has width of W_{pixel}, the input data bus width is $W_{\text{bus}} = N \times W_{\text{pixel}}$.

When these pixels arrive at the input feeder, some graphs require a bias and scale to be applied to the input tensor. The lightweight layout transform can optionally apply a bias and scale to input values as they arrive.

Because the lightweight layout transform neither spends extensive logic resources to fold the spatial dimensions into channel dimension, nor does it track partial pixel transactions or buffering partial results, it splits the input into N individual pixels, creates one of the transform processing pipeline described earlier for each of the pixel, and then selectively sends the transformed feature vector downstream to be consumed by PE.

Functional Restrictions

The lightweight layout transform has the following functional restrictions:

- Tensors with channel dimensions greater than c_vector cannot be handled by the lightweight layout transform.
- Input bus width must be a multiple of the pixel width.

Comparison to the Full Layout Transform

Resource savings in the lightweight transform compared to the full layout transform come mainly from the restriction of the input bus width to being a multiple of the pixel width and not supporting folding.

Conversely, the full layout transform has no restriction on the input bus width aside from being a multiple of the element width – i.e., multiple of 8 bits for U8 inputs, or multiple of 16 bits for FP16/U16 inputs. This means that the full layout transform must keep track of state information between transfers, and store partial results.

The following table compares the estimated performance using `dla_compiler` at 500MHz and aera for resnet-50-tf between AGX7_Performance_LayoutTransform.arch with hardware full layout transform and AGX7_LightweightLayoutTransform.arch. Arch files are slightly modified so that unused auxiliary modules are turned off and c_vector, k_vector, and stream buffer depth are aligned.

Table 25. Input Layout Transform Resource Utilization and Throughput Comparison

Input Layout Transform	ALMs	DSPs	ALUTs	M20K	Min Avg DDR	IP Throughput
Lightweight	60314	586	77506	2124	7547MB/s	149fps
Full	96730	594	139543	2853	8480MB/s	171fps

Related Information

Parallelism in the FPGA AI Suite IP on page 18

9.3.2.2. Enabling the Lightweight Layout Transform

The lightweight layout transform is parameterized by the FPGA AI Suite architecture File. The transform configuration options are as follows

Table 26. Lightweight Transform Parameters

Parameter	Type	Range	Default
channels	int	[1..CVEC]	3
input_bus_width	int	> 8	48
pixel_fifo_depth	int	> 1	16
enable_bias_scale	bool	{true, false}	TRUE
input_type	enum	{F16, U8, U16}	F16

Setting the Lightweight Layout Transform Parameters

Set the lightweight layout transform parameters as follows:

- channels

  The number of channels in the input feature. This value can be found by examining the input source.

- input_bus_width

  The width of the input bus depends on the upstream data source. Consider it as a system-level consideration when building the application. Decide on the number of parallel pixels to the FPGA AI Suite IP. The resource utilization of the layout transform scales primarily with the number of parallel pixels, N. This is because the data conversion to FP16 and the application of bias and scale is done in parallel for each pixel, and thus the hardware is duplicated for each pixel. Each pixel has its own configurable-size exit FIFO for added elasticity.

- pixel_fifo_depth

  The maximum number of transformed input feature queued in the FIFO downstream. The FIFO depth affects the resources usage since the number of exit FIFO scales with the number of parallel pixels.

  Considering the scenario of single consumer and multiple producer case, each cycle the consumer (downstream) consumes from one exit FIFO, whereas the producers (lightweight layout transform pipelines) queue data to every exit FIFOs concurrently. If the exit FIFO depth is too shallow, then the producers saturate the FIFOs too quickly before the consumer can consume. Therefore, the FIFOs backpressure the upstream, which is undesirable. To avoid this constant backpressure, set a value at least larger than the number of parallel pixels, N.

- enable_bias_scale

  Enables the IP to compute normalization for input features by adding bias and scaling.

- input_type

  The data type for each element in the raw input pixel. For example, the input_type for 8-bit RGB pixel is U8.

Example Architecture File

An example of a typical instantiation of the lightweight layout transform in the architecture file for a 3-channel FP16 input and a 240-bit-wide input bus and bias scale enabled is shown below. To enable the lightweight layout transform, enable_lightweight_layout_transform should also be set to true.

```
enable_lightweight_layout_transform : true

# 5 pixels in parallel
lightweight_layout_transform_params {
  channels : 3
  input_bus_width : 240
  input_type : F16
  enable_bias_scale : true
}
```

Compile and Run IP That Uses the Lightweight Layout Transform

If you enable the lightweight layout transform, you must turn off folding in the FPGA AI Suite compiler and runtime:

dla_compiler	To turn off folding, specify the `--ffolding-option 0` option.
dla_benchmark	To turn off folding, specify the `--folding_option 0` option.

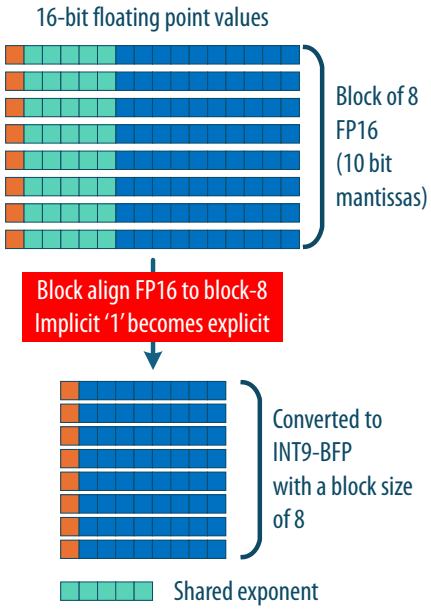
When bias-and-scale is enabled in the layout transform, the bias and scale values are automatically extracted from the graph and set to the transform at runtime. The lightweight layout transform is enabled and tested in both streaming and external memory-based designs.

9.4. Make Precision vs. Performance Trade-offs for Your FPGA AI Suite IP

9.4.1. Block Floating Point (BFP)

The following diagram illustrates the conversion from floating point (fp16) to block floating point (in this example, INT9-BFP). The mantissas of the inputs are aligned to the largest exponent value in the group, which becomes the shared exponent.

Figure 24. Conversion to block floating point



The Architecture Description File defines the block size parameter (`c_vector`) and block floating point precision parameter (`arch_precision`).

The following table below summarizes the block floating point notation used by the FPGA AI Suite. The mantissa widths given here include the implicit leading 1.

Table 27. Block Floating Point Notation Convention

arch_precision	Block floating point	Meaning
FP11	INT7-BFP	1s.6m.5e (unsigned integer mantissa)
FP12AGX	INT8-BFP	8m.5e (two's complement mantissa)
FP13AGX	INT9-BFP	9m.5e (two's complement mantissa)
FP16	INT12-BFP	1s.11m.5e (unsigned integer mantissa)

Due to the architecture of its DSP blocks, the Agilex 7 FPGA fabric is optimally configured at INT9-BFP. The Agilex 5 FPGA fabrics are optimally configured in DSP tensor mode at INT8-BFP.

When converting from floating point to block floating point, numerical precision is lost in the following ways:

- If input values in the block have different exponents, those values with smaller exponents lose lowermost precision bits through the shift-align-round operation.
- If the BFP mantissa format has *fewer* bits of precision than the input floating point format, lowermost bits of precision are lost regardless of the exponent value.

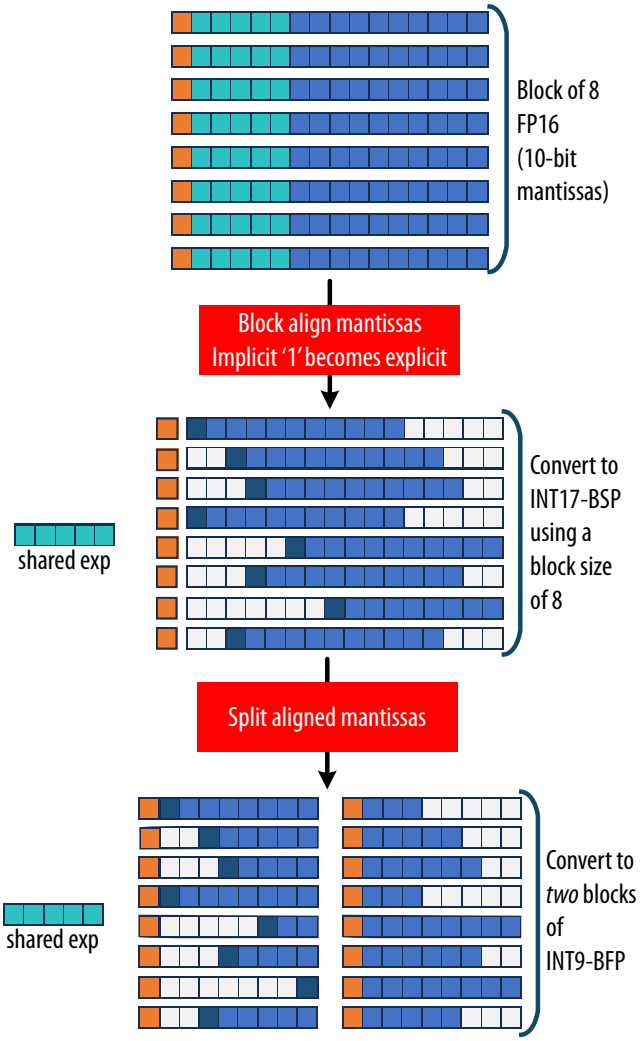
Numerical experiments have demonstrated that the quantization to lower-precision block floating point formats results in relatively small accuracy loss for many popular networks. For details, refer to the Intel white paper "*Low-Precision Networks for Efficient Inference on FPGAs*".

9.4.2. Improving Layer Accuracy by using Mixed Precision

For ML tasks that are sensitive to precision, although using a lower precision saves area, the inference accuracy may be impacted. Mixed precision feature enables running designated layers in the ML graph to run at a higher precision to achieve a better accuracy.

Figure 25. Conversion to high-precision block floating point

The following diagram illustrates the conversion from floating point (fp16) to high-precision block floating point (in this example, 2 x INT9-BFP).



Since the block floating point block alignment step uses a mantissa width *larger* than the fp16 mantissa, there is little to no loss of precision.

The only situation in which the high-precision blocked values lose mantissa precision relative to the fp16 inputs occurs when values of very different magnitude (i.e. having very different exponents) are blocked together. In this situation, a large bit shift is required to block align the mantissas, which can cause some low-precision bits of smaller values in the block to be lost. The 7th blocked value in the diagram illustrates this case.

Using the high precision block floating point numerical decomposition, a PE array parameterized to handle INT9-BFP can perform convolutions at INT17-BFP precision on select layers. The table below summarizes what "high precision BFP" entails for different FPGA AI Suite IP arch_precision parameter values.

Table 28. High-Precision BFP vs. Default Precision BFP

arch_precision	Block floating point	High precision BFP
FP11	INT7-BFP	INT13-BFP(not supported)
FP12AGX	INT8-BFP	INT15-BFP
FP13AGX	INT9-BFP	INT17-BFP
FP16	INT12-BFP	INT23-BFP(not supported)

A high-precision convolution layer has 4x the computational cost of a default precision convolution layer. With high precision BFP decomposition, both features and filters are represented as the sum of two terms. The resulting feature-filter product of sums has four terms.

The computational cost can be reduced by using high precision BFP for only the features, leaving the filters at default precision. Such a layer has 2x the computational cost of a default precision layer.

9.4.3. Using the Mixed Precision Feature

To enable mixed precision support in the FPGA AI Suite IP, manually edit the Architecture Description File to add the line:

```
enable_mixed_precision:true
```

Specifying Graph Layers to Run at High Precision

To enable mixed precision, specify the desired precision for a graph layer by manually annotating it onto the OpenVINO Model Converter output, the OpenVINO graph (`graph.xml`).

If you rerun the OpenVINO Model Converter during development, the `graph.xml` file is regenerated and you must re-annotate this new file.

Annotating the OpenVINO Graph

Refer to the FPGA AI Suite documentation for instructions on running the OpenVINO Model Converter tool on your trained network.

The primary OpenVINO Model Converter outputs are the OpenVINO graph (`graph.xml`) that describes your network topology, and the weights and biases for your model (`graph.bin`). These OpenVINO outputs are the inputs of the FPGA AI Suite `dla_compiler` tool.

To annotate the precision of a layer precision onto the OpenVINO graph, you must modify that layer name string directly in `graph.xml`. The precision annotation is appended at the end of the layer name, following a double underscore, as shown in the example below.

```
<layer id="2" name="model/conv1/Conv2D__fpga_precision=high" type="Convolution"
version="opset1">
   <data strides="1, 1" dilations="1, 1" pads_begin="0, 0" pads_end="0, 0"
auto_pad="same_upper"/>
   <input>
     <port_id="0" precision="FP32">
...
```

Do not modify the `graph.bin` file.

You can specify the following `fpga_precision` annotation values:

Table 29. FPGA Precision Annotation Values

Annotation	Description
`__fpga_precision=default`	Run this layer at the default BFP precision. The default BFP arch precision is indicated by the `arch_precision` field in the Architecture Description File.
`__fpga_precision=high`	Run this layer with both **feature** and **filter** at high BFP precision.
`__fpga_precision=high-feature`	Run this layer with the **feature** at high BFP precision. The filter is represented at the default `arch_precision`.
(no annotation used)	Equivalent to `__fpga_precision=default`.

Selecting Which Graph Layers to Annotate

The FPGA AI Suite IP runs the most computationally intensive graph layers at block floating point precision. Other graph layers operate at fp16 precision. For details, refer to Block Floating Point (BFP) on page 240.

The following table summarizes the information that you need to select which graph layers to annotate with precision directives. For each graph layer/primitive, the table tells describes the following information:

- The corresponding layer type in the `graph.xml`
- The FPGA AI Suite IP block to which the `dla_compiler` tool maps layers of this type.
- The default precision at which this IP block operates (that is, if `__fpga_precision=default`).
- Whether this layer type is a legal target for the `fpga_precision` annotation.

Table 30. Graph Layer Mapping and Precision

Layer / Primitive	OpenVINO IR (.xml) Layer Type	IP Block Mapping	Default Precision	Supports Mixed Precision?
Convolution(2D or 3D)	Convolution	PE array	arch_precision(bfp)	Yes
Convolutionincluding bias(2D or 3D)	Convolution + Add	PE array	arch_precision(bfp)	Yes
Depthwise	GroupConvolution + Add (if bias is present)	Depthwise auxiliary block (if possible) Otherwise, maps to the PE array	fp16 arch_precision(bfp)	No (if aux block) Yes (if PE array)
Scale-Shift	Multiply (single input)	PE array	arch_precision(bfp)	Yes
Deconv / Transpose Convolution	ConvolutionBackpropData	PE array	arch_precision(bfp)	No
Elementwise addition of feature + feature tensors	Add	PE array	arch_precision(bfp)	Yes

continued...

Layer / Primitive	OpenVINO IR (.xml) Layer Type	IP Block Mapping	Default Precision	Supports Mixed Precision?
Elementwise multiplication of feature * filter tensors	Multiply (single input)	PE array	arch_precision(bfp)	Yes
Elementwise multiplication of feature * feature tensors	Multiple(two inputs)	PE array	arch_precision(bfp)	No
Fully connected	Matmul + Add (if bias is present)	PE array	arch_precision(bfp)	Yes
ChannelToSpace DepthToSpace PixelShuffle	DepthToSpace	PE array	arch_precision(bfp)	No
ReLU pReLU Leaky ReLU	ReLU, PReLU	Activation auxiliary block	fp16	N/A
Clamp Round Clamp	Clamp	Activation auxiliary block	fp16	N/A
H-sigmoid	Hsigmoid	Activation auxiliary block	fp16	N/A
H-swish	HSwish	Activation auxiliary block	fp16	N/A
Sigmoid	Sigmoid	Activation auxiliary block	fp16	N/A
Swish	Swish	Activation auxiliary block	fp16	N/A
Max Pool	MaxPool	Pooling auxiliary block	fp16	N/A
Max Pool (for 3D inputs, i.e. DHWC)	MaxPool	Pooling auxiliary block + PE array	fp16 &arch_precision(bfp)	No
Average Pool	AvgPool or ReduceMean	PE array	arch_precision(bfp)	Yes
Softmax	Softmax	Activation auxiliary block	fp16	N/A

Most graph layers that map to the FPGA AI Suite PE array are legal targets for the `fpga_precision` annotation. These are marked **Yes** in the **Supports Mixed Precision?** column of the preceding table.

Layers that are mapped to auxiliary blocks of the FPGA AI Suite IP are computed at fp16 precision. This is "high precision" by default. These layers are marked "N/A" in the above table.

The `dla_compiler` tool processes your annotations as follows:

- When a precision annotation is successfully applied, `dla_compiler` adds this precision to the layer visualization produced by the compiler as **"...after_all_passes.dot"** generated under `$coredla_word/visualizations`. This will show up in the layer as Precision_Mode: "Selected Precision" (High, High_Feature).

You can convert this report to SVG format which can be displayed in a web browser by running the following command:

```
dot -Tsvg ...after_all_passes.dot -o ...after_all_passes.svg
```

- For an annotation applied to an unsupported layer type (that is, "No*" or "No" in Graph Layer Mapping and Precision.), `dla_compiler` issues an error and terminates the compilation.
- If `dla_compiler` maps a layer to an auxiliary block (that is, "N/A" in Graph Layer Mapping and Precision), it will ignore any associated precision annotation and will print Precision Mode: Arch, which means the default arch precision.

Achieving Your Numerical Objectives

The mixed precision feature is useful for preserving fine-grained information in the input feature data. Information encoded in the least significant mantissa bits of the feature fp16 input is lost when data is quantized to the default block floating point arch_precision.

Use the `__fpga_precision=high` or `__fpga_precision=high-feature` annotations as follows:

- Apply them to the *initial layers* of a graph. Applying the annotations at this level preserves fine-grained input data, which would be lost when quantizing to default-precision block floating point.
- Apply them to *consecutive* layers of a graph. It is unclear whether there is significant benefit to running high-BFP precision layers after the first default block floating point precision layer.
- Apply them to the *fewest possible graph layers* needed to achieve your numerical objectives. Running a layer at high BFP precision is more computationally intensive, which reduces the overall inference throughput.

If your graph contains depthwise convolution layers, leverage the dedicated depthwise auxiliary block:

- Enable the depthwise auxiliary block by modifying the Architecture Description File. The depthwise auxiliary block provides superior throughput *and* higher numerical precision for depthwise convolutions.
- Ensure that the parameterization of the depthwise auxiliary block (for maximum window size, stride, and dilation) are set large enough to support all of the depthwise convolutions in your graph.
- Use the FPGA AI Suite Area Estimator tool to estimate the FPGA resource utilization required by the depthwise auxiliary block.

For information on using the depthwise auxiliary block, refer to Parameter Group: depthwise on page 182.

9.4.3.1. Performance Impact of Mixed Precision

When you enable the mixed precision feature, expect the following effects on area and throughput:

- **Area impact**.

  Adding `enable_mixed_precision:true` to the Architecture Description File results in a small increase in the ALM utilization of the FPGA AI Suite IP.

- **Throughput impact**

  The throughput impact of mixed precision depends on your specific graph and the layers you configure to run at `__fpga_precision=high` and `__fpga_precision=high-feature`. The per-layer throughput impact is summarized in the following table:

Table 31. Throughput Impact of FPGA Precision Annotations

Annotation	Relative Computation Required
`__fpga_precision=default`	N/A
`__fpga_precision=high`	Requires **4x** the multiply-add operations (compared to a default precision layer).
`__fpga_precision=high-feature`	Requires **2 x** the multiply-add operations (compared to a default precision layer).
(no annotation used)	N/A

For FPGA AI Suite Version 2025.3, the following usage constraints apply to the mixed precision feature:

- Some graph layers cannot be run at high precision OR not yet supported for mixed precision. For a complete list of layer types and constraints, refer to Graph Layer Mapping and Precision for a complete list of layer types and constraints.
- Only select `arch_precision` values that are supported for use with mixed precision. For supported values, refer to Block Floating Point Notation Convention.

The following known issues apply to the mixed precision:

- A rare corner case may arise if the feature input contains many severely subnormal fp16 inputs (for example, with only 1 or 2 significant mantissa digits). If this issue occurs, the `dla_benchmark` program triggers an assertion. Ensure that your input data is scaled appropriately to prevent this issue.

A critical aspect of optimizing your FPGA AI Suite IP is the ability to make precision versus performance trade-offs. This trade-off involves balancing the computational accuracy of your models with the desired inference speed and resource utilization on the FPGA. The FPGA AI Suite uses Block Floating Point (BFP) as its core numeric representation, enabling both high-throughput and efficient use of FPGA DSP resources. For information about Block Floating Point, refer to Block Floating Point (BFP) on page 240.

- FP16 (INT12-BFP) – Highest default precision supported
- FP13 (INT9-BFP) – Optimal for Agilex 7
- FP12 (INT8-BFP) – Optimal for Agilex 5
- FP11 (INT7-BFP) – Highest performance, lowest precision; may require quantization-aware training (QAT)

Trade-off considerations:

- FP11 offers significant performance boosts and lower memory usage but may reduce model accuracy.
- FP13 is often the best default for modern vision models and is supported by many pre-compiled design examples.

Recommended defaults by device

- Agilex 5: Optimal at FP12 in DSP tensor mode
- Agilex 7: Optimal at FP13

Use the Model Converter and Calibration Tools provided with the suite to evaluate precision impact and adjust the model accordingly.

9.5. FPGA AI Suite IP Supported Layers and Hyperparameter Ranges

Each layer in the intermediate representation (IR) must be validated against the layer and primitive constraints. This step ensures that the design flow only progresses with operator dimensions and hyperparameters supported by the FPGA AI Suite hardware.

Validation at this stage prevents errors during architecture generation and guarantees that all layers can be synthesized into FPGA-ready IP without unsupported fallbacks.

The following table lists the hyperparameter ranges supported by key primitive layers in the FPGA AI Suite IP:

Table 32. Supported Layers and Hyperparameter Ranges

Layer / Primitive	Hyperparameter	Supported Range
Fully connected	None	n/a
2D Convolution	Filter Size	Width = [1..28] Height = [1..28] Height does not have to equal width. Default value for each is 14.
	Stride	Maximum stride is 15
	Pad	Maximum pad is (2^{16}) - 1
3D Convolution	Filter Size	Width = [1..28] Height = [1..28] Depth = [1..14] Filter volume should fit into the filter cache size.
	Stride	Maximum stride is 15.
	Pad	Maximum pad is (2^{16}) - 1
Depthwise	Filter Size	Same as 2D Conv filter size Depth = 1
	Stride	Same as 2D Conv stride Depth = 1
	Pad	Same as 2D conv padding Depth = 1

continued...

Layer / Primitive	Hyperparameter	Supported Range
Scale-Shift	Scale factor	FP16 float range
	Bias term	FP16 float range
Deconvolution / Transpose Convolution	Filter Size	Any – Same as convolution, and height/width can be different Depth = 1
	Stride	1, 2, 4, 8 (stride width == stride height) Depth = 1
	Pad	Restricted to filter_[height, width] - 1 Depth = 1
ReLU	n/a	n/a
pReLU	Scaling parameter (a) (1 per filter / conv output channel)	float range Depth = 1
Leaky ReLU	Scaling parameter (a) (1 per tensor)	float range
Clamp	Limit parameters (a, b) (1 per tensor)	float range
Round_Clamp	Limit parameters (a, b) (1 per tensor)	float range
H-sigmoid	n/a	n/a
H-swish	n/a	n/a
Sigmoid	n/a	FP16 float range
Swish	n/a	FP16 float range
Tanh	n/a	FP16 float range
Max Pool	Window Size	up to 13x13x13
	Pad	1, 2
	Stride	1, 2, 3, 4
Average Pool	Window Size	Up to 27x27 (one less than the maximum 2D convolution size) Width == Height Depth = 1 or 2
	Pad	1, 2
	Stride	1, 2, 3, 4
Softmax	Maximum Number of Channels	4096
Elementwise Multiplication of feature * filter and feature * feature tensors.(15)	n/a	Tensor sizes are expanded if necessary to support the multiplication. Depth = 1

continued...

(15) This is an element-wise multiplication, not a matrix multiply operation.

Layer / Primitive	Hyperparameter	Supported Range
ChannelToSpace DepthToSpace PixelShuffle	block_mode	`blocks_first` or `blocks_last`
	block_size	2, 4, 8
Upsampling Downsampling	Interpolation Type	`Bilinear` (upsampling only) or `Nearest Neighbor` (upsampling and downsampling
	Filter Size	Similar to 2D convolution except that width and height must be symmetrical for bilinear interpolation. Downsampling supports a sampling factor (ratio between an output dimension and input dimension) of only 2x or 4x.

9.6. FPGA AI Suite IP Parameterization

The FPGA AI Suite IP block has many important parameters that describe arithmetic precision, feature set, size of various modules (such as the PE Array), and details regarding the internal buses and the external AXI interface.

The following table summarizes the effects of these parameters and parameter groups:

Table 33.

Effect	Input Feeder	Filter Scratchpad	PE Array	Crossbar	Aux Modules
Data movement efficiency Data storage efficiency	input_stream_interface output_stream_interface stream_buffer_depth	filter_depth arch_precision	exit_fifo_depth	xbar_k_vector	generic_aux_parameters
Computation speed			k_vector c_vector num_lanes num_interleaved_features num_interleaved_filters		
Extra functionality enablement	enable_parameter_rom enable_layout_transform layout_transform_params		enable_scale		pool depthwise softmax activation
Inference accuracy			arch_precision		

Configurable parameters are specified in the Architecture Description (`.arch`) file, as described in Architecture Description File Format for Instance Parameterization on page 168 and Architecture Description File Parameters on page 173.

The table below shows the major parameters, some of which are not configurable, that describe the IP block.

Common Parameter Name	Description	Valid Range
c_vector (CVEC)	Size of the dot product performed by each PE in the PE Array.	[4,8,16,32,64]

continued...

Common Parameter Name	Description	Valid Range
	Typically optimized when generating an optimized architecture with the FPGA AI Suite compiler.	
k_vector (PE KVEC)	Number of PEs in the PE Array	[4-128] Must be a multiple of c_vector
num_lanes	Number of execution lane. Each lane contains a set of auxiliary modules (aux_modules) and a number (equal to k_vector) of PEs	[1,2,4]
N/A	Number of auxiliary modules connecting to the crossbar (XBAR)	[1-4]
pool k_vector (Pool KVEC)	Width of the pool interface. Typically optimized when generating an optimized architecture with the FPGA AI Suite compiler.	[1,2,4,8,16,32,64]
pool max_window_height pool max_window_width	Size of the pooling window	[3x3, 7x7,13x13]
depthwise k_vector (Depthwise KVEC)	Number of output channels processed in parallel. Typically optimized when generating an optimized architecture with the FPGA AI Suite compiler.	[16, 32, 64] Must be equal to k_vector
depthwise max_window_height depthwise max_window_width	Size of the depthwise filter	[3x3, 5x5, 7x7]
depthwise max_dilation_vertical depthwise max_dilation_horizontal	Maximum supported value for the depthwise dilation	[1-6]
activation k_vector (Activation KVEC)	Width of the activation interface. Typically optimized when generating an optimized architecture with the FPGA AI Suite compiler.	[2,4,8,16,32,64]
enable_clamp	Enables clamp activation function	[true, false]
enable_relu	Enables ReLU activation function	[true, false]
enable_leaky_relu	Enables Leaky ReLU activation function	[true, false]
enable_prelu	Enables PReLU activation function	[true, false]
enable_round_clamp	Enables round clamp activation function	[true, false]
enable_sigmoid	Enables Sigmoid and Swish activation functions	[true, false]
enable_tanh	Enables Tanh activation function	[true, false]
enable_parameter_rom	Enables storing graph parameters in on-chip memory, which requires input and output streaming to be enabled and configured. For details about DDR-free operation, refer to Generating Artifacts for Hostless DDR-Free Operation on page 283.	[true, false]
arch_precision (PE precision)	Precision of features and weights in the PE Array. For details about how this parameter affects DSP utilization, refer to the "Parameter: `arch_precision`" section of Parameter Group: Global Parameters on page 173.	"FP11" (INT7-BFP / 1s.6m.5e) "FP12AGX" (INT8-BFP / 8m.5e, two's complement) "FP13AGX" (INT9-BFP / 9m.5e, two's complement) "FP16" (INT12-BFP / 1s.11m.5e)
PE bias add precision	Precision of accumulator bias value in the PE Array.	fp16

continued...

Common Parameter Name	Description	Valid Range
PE accumulator precision	Precision of the accumulators in the PE Array.	fp32
PE drain precision	Precision of values drained from the PE Accumulators to the XBAR and AUX Modules.	fp16
PE interleave factor	Multi-threading factor for the features x filters in the PE array accumulators.	**Agilex 3 devices:** 2x3, 3x2, 5x1, 1x5, 12x1 **Agilex 5 devices:** 2x3, 3x2, 5x1, 1x5, 12x1 **Agilex 7 devices:** 2x3, 3x2, 5x1, 1x5 **Arria 10 devices:** 2x2, 4x1, 1x4 **Stratix 10 devices:** 2x3, 3x2, 5x1, 1x5 1×1 supported for graphs with no bias
PE scale precision	Precision of scale multiplier in the PE array	fp16
Aux module precision	Precision of the Aux Modules	fp16
Memory port width	Width of memory port	[64, 128, 256, 512]
enable_debug	Toggle the FPGA AI Suite debug network that includes interface profiling counters that can be queried with the CSR. Enabled by default.	[true, false]
enable_layout_transform	Enables the dedicated input tensor layout transform module. The standard layout transform module cannot be enabled when the lightweight layout transform is enabled.	[true, false]
enable_lightweight_layout_transform	Enables the area-optimized (lightweight) input tensor layout transform module. The lightweight layout transform cannot be enabled when the standard layout transform is enabled.	[true, false]
enable_layout_transform_writeback	Enables the layout transform to write its results to external memory rather than the stream buffer. Enabling this mode requires additional CSR commissioning steps.	[true, false]

The major constraints include:

- PE KVEC must be a multiple of CVEC
- PE KVEC must be divisible by XBAR and AUX KVECs
- PE drain width must be equal to XBAR KVEC

Graph limitations include:

- Convolution filter size: 1×1 -> 28×28, including asymmetric
- Convolution filter stride: 1 .. 15
- No limitation on convolution padding
- The limits of the depthwise layers are the same as normal convolution. Depthwise convolution is handled with software emulation using regular convolution passes.

The maximum supported DDR size is 4GB.

9.7. Generating an Optimized Architecture

You can generate an optimized architecture in two ways: optimize the architecture for the highest performance subject to a constraint on the maximum allowable FPGA area or optimize the architecture for the smallest possible FPGA area subject to a constraint on the minimum target frame rate (fps).

9.7.1. Generating an Architecture for Highest Performance

To generate an architecture that is optimized for a graph or set of graphs, the FPGA AI Suite architecture optimizer uses a base architecture and modifies parameters to achieve the highest throughput in frames per second (fps). The best architecture is saved as an architecture description file with a file name based on the architecture parameters.

Some parameters (such as the precision specified in the `.arch` file) are not modified during optimization. Parameters that are not optimized take their value from the input architecture file specified by the required `--march` option. Important IP parameters to set manually include the `arch_precision`, `num_interleaved_features`, and `num_interleaved_filters` parameters. IP parameters are described in the FPGA AI Suite IP Parameterization on page 189.

Architecture optimization is a slow process. The optimizer stores the intermediate architecture iterations in the `arch_gen_reports/` directory.

When you use the `--mtarget-fps` option, the architecture optimizer might use in excess of 128GB of memory. The required memory varies significantly depending on the graph. For example, a ResNet 50-type graph can often require up to 256 GB of memory.

If this memory is not available, the operating system might stop the `dla_compiler` process, and the user shell prints the message "`Killed`". For information about how to use the `COREDLA_TARGET_FPS_THREAD_LIMIT` environment variable to control the resource consumption of the architecture optimizer, refer to the description of `--mtarget-fps` in Architecture Optimizer Options (dla_compiler Command Options) on page 211.

If multiple input graphs are specified by using multiple `--network-file` options, then the optimizer calculates a weighted objective. For example, if two graphs are specified and have throughput values of fps_1 and fps_2 then the overall throughput is maximized using the user-specified weights w_1 and $w_2$as follows:

$$\text{fps}_{\text{total}} = 1/\left(\frac{w_1}{\text{fps}_1} + \frac{w_2}{\text{fps}_2}\right)$$

The architecture optimization process uses the compiler, the performance estimator, and the area estimator. Accordingly, its command line options are as follows:

- All required and optional arguments from *Compiling a Graph*, including the required `--march` option.
- All optional arguments from *Estimating the Performance of a Graph*.
- All optional arguments from *Estimating the Area of an Architecture*.
- The following arguments for architecture optimization:

Option	Description
`--gen-arch`	[**Required**] Enable the architecture optimizer.
`--mmax-resources=<max_ALMs>,<max_M20K_blocks>,<max_DSP_blocks>`	[**Optional**] Sets the maximum number of resources that the output architecture can use (as estimated by the area estimator). Specify the resources as a comma-delimited sequence of max ALMs, M20k blocks, and DSP blocks. If you do not specify this option, the architecture uses as many resources as needed. The optimizer might use more resources than are available on the FPGA device.
`--mmax-resources-alm-util=<%_max_ALM_utilization>`	[**Optional**] Set this option only if you also set the `--mmax-resources` option. This option sets a percentage target of the *<max_ALMs>* value from the `--mmax-resources` option that architecture optimizer aims to use for the architecture. The remaining ALMs are used by the Quartus Prime software to improve timing closure. The default value is `100` (100% utilization). Designers typically target full-chip logic utilization values lower than `100` (100% utilization) to improve routing and timing closure of the design. As an example, a target of 80% ALM utilization (`--mmax-resources-alm-util=80`) tells the architecture optimizer that it should use only 80% of the ALMs that were specified by the `--mmax-resources` option. The other 20% are used by Quartus Prime software to improve timing closure.
`--mtarget-fps`	[**Optional**] Sets the minimum frames-per-second (fps) that the output architecture must achieve (as estimated by the performance estimator). This option significantly increases the runtime and memory requirements of the architecture optimizer. The architecture optimizer can use in excess of 128GB of RAM when using this option, and thus requires a server-class machine. The required memory varies significantly depending on the graph. For example, a ResNet 50-type graph can often require up to 256 GB of memory
`--interleave-search`	[**Optional**] Causes the architecture optimizer to evaluate different legal feature and filter interleave options. This significantly increases the run time of the architecture optimizer, by as much as 50%. Due to the run time penalty, after an optimal interleave has been found for a given graph, the recommendation is to place the optimized interleave into the initial input `.arch` file and avoid using `--interleave-search` during any future fine-tuning. For more information, refer to Parameter Group: pe_array on page 179.
`--gen-min-sb`	[**Optional**] Minimum size stream buffer. Can be specified explicitly to reduce the search space for the optimizer. For more information about stream buffer size, refer to the "Parameter: `stream_buffer_depth`" section in the Parameter Group: Global Parameters on page 173.
`--network-weightings="<network_weight_1> <network_weight_2> <network_weight_3>...<network_weight_n>"`	[**Optional**] Space-delimited specification of network weights when multiple networks are specified. If not specified, then all networks are equally weighted.
`--gen-arch-file`	[**Optional**] Name of the output architecture (`.arch`) file.

`--max-archsetM-percentage`
`--arch-limit`
`--time-limit`
	[**Optional**] Used when performing a larger search of the optimization space, as described in `dla_compiler --help`. Not recommended for general use.

The architecture optimizer supports only 1x*N* and *N*x1 interleave, as described in the release notes.

For more information about modifying interleaving, refer to the "Parameters: `pe_array/num_interleaved_features`, `pe_array/num_interleaved_filters`" section in the Parameter Group: pe_array on page 179.

The simplest command format to optimize an Architecture Description (`.arch`) file for a graph is as follows:

```
dla_compiler \
   --gen-arch \
   --mmax-resources=<max_ALMs>,<max_M20K_blocks>,<max_DSP_blocks> \
   --network-file <path or paths to graph.xml> \
   --march=<path to input .arch file>
```

Example Command

```
dla_compiler \
   --gen-arch \
   --mmax-resources=427200,2713,1518 \
   --gen-min-sb=2048 \
   --network-file ResNet50.xml Mobilenet_v1.xml \
   --march=./example_architecture/A10_Performance.arch \
   --mmax-resources-alm-util=75 \
   --fassumed-fmax-core=300 \
   --network-weightings=1 2
```

9.7.2. Generating an Architecture Optimized for a Frame Rate Target Value

To generate the smallest possible architecture with a minimum target frame rate (fps) performance constraint, use the `--mtarget-fps` option in addition to the options used to generate an optimized architecture. If you specify both the `--mtarget-fps` and `--mmax-resources` options, the optimizer tries to satisfy both constraints.

The output files are the same as when generating a performance-optimized architecture.

The function uses as many processor cores available as possible to search for a potential minimum architecture achieving the target frame rate.

Example Command

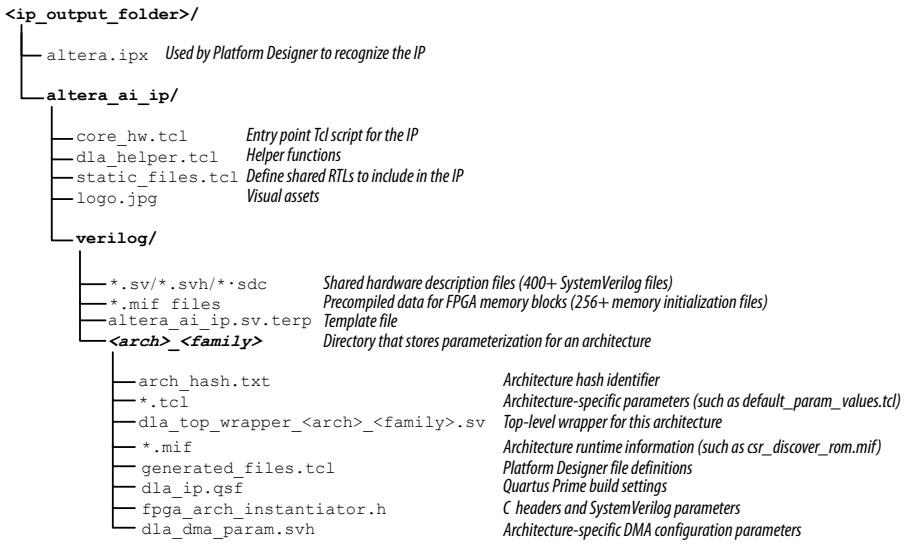
```
dla_compiler \
   --gen-arch \
   --gen-min-sb=2048 \
   --network-file Mobilenet_v1.xml \
   --march=./example_architecture/AGX7_Performance.arch \
   --mmax-resources-alm-util=75 \
   --mmax-resources=427200,2713,1518 \
   --fassumed-fmax-core=300 \
   --mtarget-fps=100.0
```

10. Integrating FPGA AI Suite IP into an FPGA Design

At the end of FPGA AI Suite IP generation, the compiler generates a verilog description of the IP as well as an .ipx file for integration into Platform Designer. The compiler creates a directory containing the files required to add the FPGA AI Suite IP to your FPGA design (refer to diagram below).

10.1. FPGA AI Suite IP Directory Structure

The compiler generates the files required to integrate the FPGA AI Suite IP into your FPGA design. The default directory structure of the generated IP is as follows:



The FPGA AI Suite compiler generates a .ipx file for integration into Platform Designer.

10.2. Interfacing FPGA AI Suite IP an FPGA Design for a Typical System

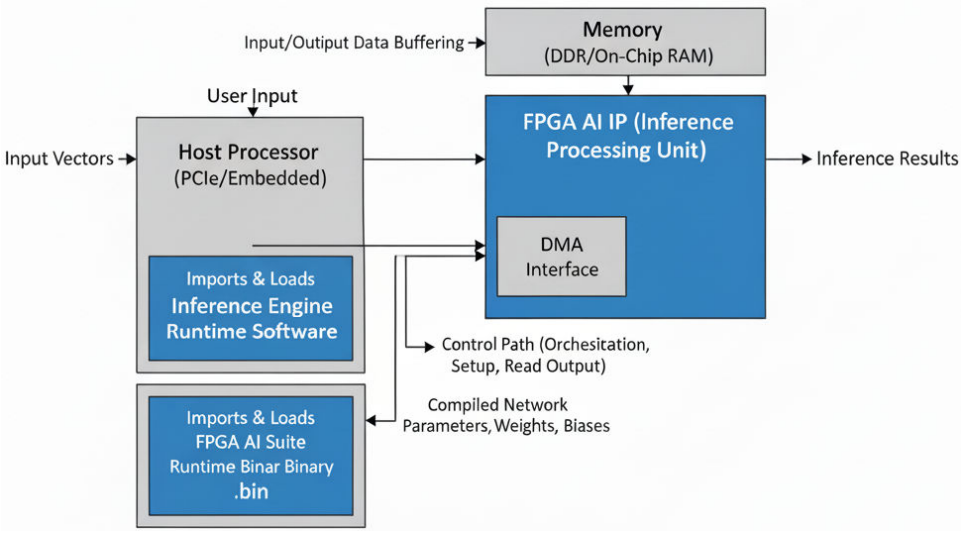
In a typical FPGA AI Suite IP system, a host processor (either a PCIe host or an embedded host) serves as the control path to the IP and does the following actions:

- Orchestrating the input
- Sets up the FPGA AI Suite IP
- Reads the output of the FPGA AI Suite IP
- Executes the Inference Engine runtime software (which imports and loads the compiled network parameters, weights, and biases contained in FPGA AI Suite runtime binary (`.bin` file)

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

Typically, a memory (either external interface to DDR or on-chip memory) is used to store or buffer the input vectors to the FPGA AI Suite IP. The FPGA AI Suite IP presents a DMA interface that can be programmed by the control path processor to manage and orchestrate the inference process at run time.

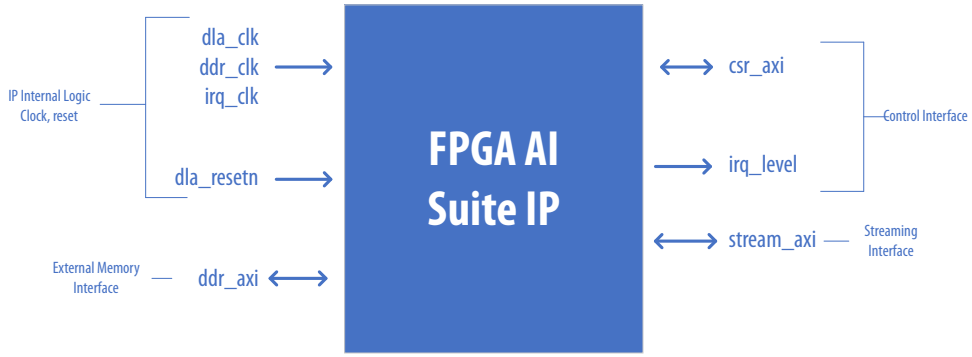
Figure 26. FPGA AI Suite System Architecture



10.3. FPGA AI Suite IP Interface

The following block diagram shows the high-level interface of the FPGA AI Suite IP.

Figure 27. FPGA AI Suite IP Interface Block Diagram



The interface to the IP contains the following signals:

- Clock for FPGA AI Suite IP logic
- Reset for FPGA AI Suite IP logic
- Memory interface from IP (each instance of the IP contains a DMA engine that handles memory transfer to and from memory. For details, refer to "DMA Descriptor Queue" in CSR Map and Descriptor Queue on page 264.
- Control Interface to the host process
- AXI Streaming Interface on page 259

To integrate generated IP in your design, connect these signals in your design and ensure that the software layers are designed to handle the integrated logic and your IP instance. This section covers the interface details for the IP and explains how to connect these in your design. For software development considerations, refer to Developing Software Applications with the FPGA AI Suite on page 339.

To instantiate the IP in your design, use Platform Designer and make the connections yourself or use the FPGA AI Suite design examples as a starting point to quickly prototype and evaluate your IP in hardware using one of the supported development kits.

In some cases, this document uses the terms *initiator* and *responder* where the terms *master* and *slave* might have been used in the past.

10.3.1. Clock and Reset

Table 34. Clocks

Name	Description
`dla_clk`	Clock used by internal processing logic
`ddr_clk`	Clock used by DDR memory and CSR interfaces. The layout transform also operates on this clock when the transform enabled,
`irq_clk`	Clock used for interrupt request (IRQ) interface

Table 35. Resets

Name	Description
`dla_resetn`	Global asynchronous reset This reset must be held for at least three cycles of the slowest of the clocks listed in the Clocks table. The IP becomes responsive sometime after the reset is released, but not immediately due to an internal reset cycle in the FPGA AI Suite IP.

10.3.2. AXI Interfaces

Table 36. AXI Control Interface

Name	Type	Clock	Reset	Description
CSR Responder	AXI4-Lite	`ddr_clk`	`dla_resetn`	Exposes IP MMIO region The CSR initiator operates on the `ddr_clk` clock.

Table 37. AXI External Memory Interface

Name	Type	Clock	Reset	Description
DDR0 Initiator	AXI4	ddr_clk	dla_resetn	Initiator port for connecting to DDR memory

Table 38. AXI Interrupt Interface

Name	Type	Clock	Reset	Description
Interrupt Initiator	Interrupt Sender	irq_clk	dla_resetn	Level sensitive interrupt

The following parameters are used by the AXI interfaces. The parameter values can be modified in the Architecture Description files as described in Generating the FPGA AI Suite IP for Integration into an FPGA Design on page 218.

Name	Supported Value	Entry in Architecture Description
C_CSR_AXI_ADDR_WIDTH	11	= dma.csr_addr_width
C_CSR_AXI_DATA_WIDTH	32	= dma.csr_data_bytes * 8
C_DDR_AXI_ADDR_WIDTH	1~32	= dma.ddr_addr_width
C_DDR_AXI_BURST_WIDTH	1~8	= dma.ddr_burst_width
C_DDR_AXI_DATA_WIDTH	64, 128, 256, 512 (bits)	= dma.ddr_data_bytes * 8
C_DDR_AXI_THREAD_ID_WIDTH	2	= ddr_read_id_width

AXI Streaming Interfaces

For information about the FPGA AI Suite IP AXI streaming interfaces, refer to AXI Streaming Interface on page 259.

10.3.3. AXI Streaming Interface

The FPGA AI Suite can be configured to accept AXI-streaming (AXI4-Stream) input features, produce AXI-streaming (AXI4-Stream) output features, or both. When input streaming is enabled, graph inputs must fit entirely in the stream buffer. This section describes the interfaces and operation of the input and output streaming modes. The available modes of operation for DDR-free (discussed in Using FPGA AI Suite in Hostless DDR-Free Mode on page 283) and DDR-enabled designs are summarized in the following table:

Table 39. Summary of Streaming Operation Modes

Input Mode	Output Mode	Description
DDR-free supported configurations		
Streaming	Streaming	The only supported mode of operation when external memory is not available. Useful for data processing pipelines where pre- and post- processing stages are on the FPGA device and have AXI-streaming interfaces. Does not support CPU subgraphs or sliced inputs (that is, the stream buffer must be large enough to stage the entire input).
DDR-enabled supported configurations		
Streaming	Streaming	For data processing pipelines where pre- and post- processing stages are on the FPGA device and have AXI-streaming interfaces. Does not support sliced inputs.

continued...

Input Mode	Output Mode	Description
DDR-memory	Streaming	For pipelines where the host CPU is typically the data source, and post-processing stages are on the FPGA device with an AXI-streaming interface. Supports CPU subgraphs and sliced inputs.
Streaming	DDR-memory	For pipelines where the data source or preprocessing stage is an AXI-streaming IP, and the output should go typically to the CPU host for post-processing. Does not support sliced inputs.
DDR-memory	DDR-memory	For applications where the host CPU is the typically the data source and the sink. Supports CPU subgraphs and sliced inputs.
Stream-to-External memory	Streaming	For scenarios where the input feature does not fit in the stream buffer, or where it is useful to have greater elasticity at the input. Inputs are transformed and written to a buffer in external memory with a circular buffer policy. Supports sliced input layers.

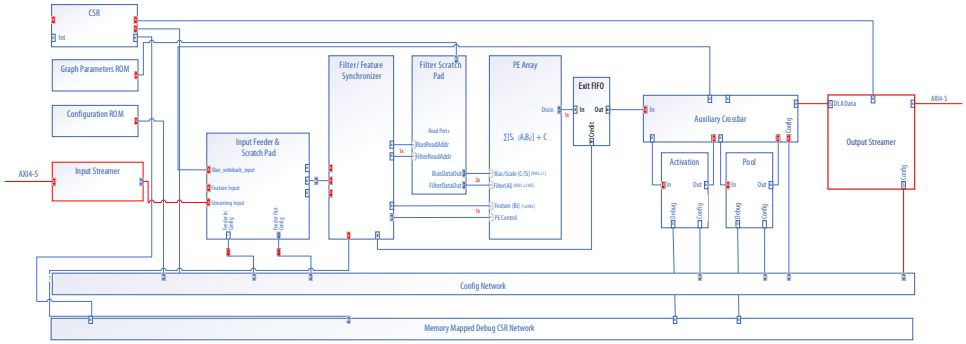
Either the streaming or the non-streaming modes can be used to connect other system components. The DDR-memory input (output) are typically used when the CPU host is the input (output), although a small soft-logic state machine or controller can instead be used to co-ordinate data transfer to/from the DDR memory.

If one of the layers in the graph is not supported by the FPGA, then a CPU host is normally required to execute the unsupported layer. However, you can build custom soft logic on the FPGA to implement the unsupported layer. The output from the IP (either streaming-out or DDR-based) can be sent to the custom soft logic.

After the custom soft logic has finished computation, the output data from the custom soft logic can be sent (via streaming or via DDR) to another instance of the FPGA AI Suite IP to execute the remainder of the graph. Alternately, you can send the output of the custom soft logic back to the same, original, instance of the FPGA AI Suite IP but with the FPGA AI Suite IP configured to use a different instruction stream. For details, refer to "DMA Descriptor Queue" in CSR Map and Descriptor Queue on page 264.

Figure 28. FPGA AI Suite IP with Input and Output Streaming

When input and output streaming is enabled, the system architecture can be described as shown in the following diagram:

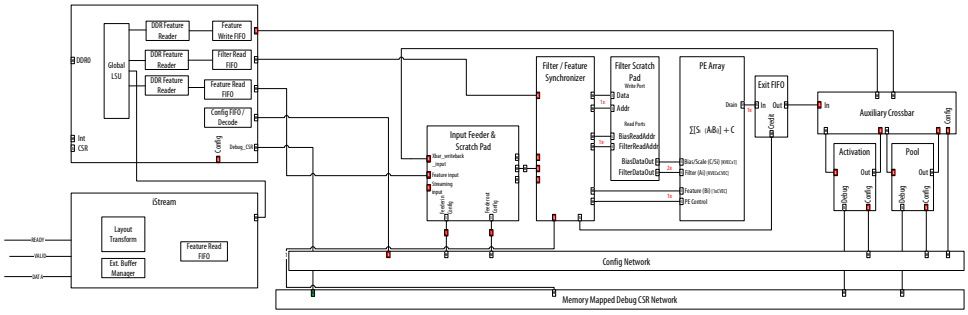


For more information about the configuration and operation of these streaming interfaces, refer to the following sections:

- Input Streaming on page 261
- Output Streaming on page 263

Figure 29. FPGA AI Suite with Input Streaming and External Memory Input Write-Back

When streaming is enabled with external memory input staging the system architecture can be summarized by the following diagram:



The address and range of the write-back buffer space, and size of the feature input must be written to the FPGA AI Suite CSR. Refer to "Layout Transform External Memory Registers" in section 10.3.4. for more information on commissioning the IP in this mode.

10.3.3.1. Input Streaming

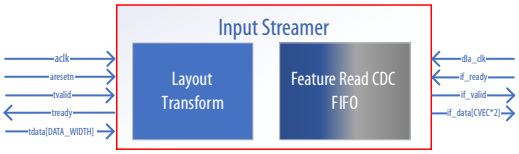
When input streaming is configured in the IP system architecture as described in Parameter Group: input_stream_interface on page 186, and the `Activate_streaming` DMA control register is set to `1`, the FPGA AI Suite IP accepts inputs through the AXI4-Stream signals that are exposed at the top-level instead of being memory mapped. The exposed signals are as follows:

Table 40. FPGA AI Suite IP AXI4-Stream Input Interface Signals

Signal	Source	Width	Description
`ACLK`	Clock	1	Data source clock
`ARESETn`	Reset	1	Data source active-low reset
`TVALID`	Data source	1	Input signal that indicates whether the values in `TDATA` are valid.
`TREADY`	FPGA AI Suite IP	1	Output signal that indicates whether the FPGA AI Suite IP is ready to accept data.
`TDATA`	Data source	`DATA_WIDTH`	Input data bus.

Schematically, the input streaming component is constructed as follows:

Figure 30. Input Streamer Schematic View



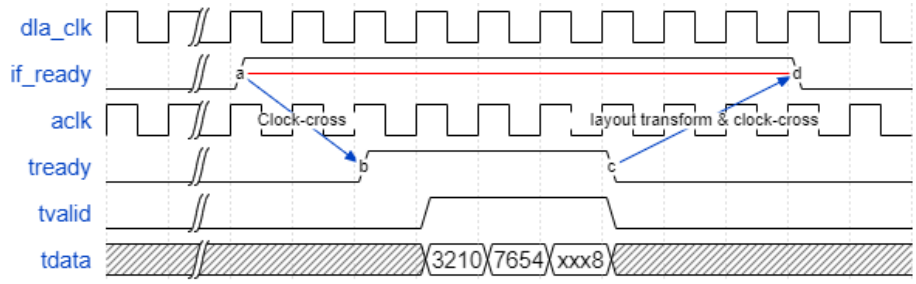
The streamed input tensor format must be in HWC, where `channels` is the fastest-changing dimension. The HWC tensors are internally folded and vectorized to CHWCvec tensors that the PE array can ingest (as described in Input Feature Tensor In-Memory Format on page 231). Data also internally crosses from the source clock domain to the FPGA AI Suite IP internal clock domain.

This subset of the AXI4 streaming protocol signals implements a streaming interface where transfers take place whenever the `TREADY` and `TVALID` signals are asserted. The input streaming interface does not implement the `TSTRB` or `TKEEP` signals, which means that all data in a valid `TDATA` signal transfer must be valid unless it is the last transfer. The `TLAST` signal is also unused at the input, because the FPGA AI Suite IP tracks the state of the transfer internally.

Any data beyond the boundary of the input tensor in the final transfer of an input feature that is not a multiple of `DATA_WIDTH` is ignored. The data stream producer is responsible for padding features, if needed, so that adjacent features to not share the same data transfer at the boundary.

The following timing example shows a 3x3x1 input tensor with monotonically increasing pixel values and DATA_WIDTH of 4 bytes. Note the padding in the third transfer.

Figure 31. Streaming Input Waveform



The `TREADY` signal is asserted by the FPGA AI Suite IP whenever the IP is ready for a new input feature and streaming has been activated in the CSR. The `TREADY` signal comes from the input streamer module and is first asserted once the FPGA AI Suite IP is configured and the input streamer FIFO is not full. The FPGA AI Suite IP accepts the input feature as long as there is space in the FIFO and no backpressure from the downstream system, which means the `TREADY` signal can be deasserted mid input feature.

When the input streaming interface is enabled, it requires the layout transform (standard/input or lightweight) to be enabled and configured as described in Parameter Group: input_stream_interface on page 186.

When streaming data is received, it is converted from HWC format, where channels is the fastest-changing dimension in the memory representation, to CHWCvec format, where the input has been vectorized into Cvec-lines that can be input to the PE array.

The layout transform can be configured to either accept FP16 input data or accept uint16 or uint8 input data that is converted internally to FP16.

The input streaming module handles clock-domain crossing from the input stream clock domain to that of the FPGA AI Suite IP and also handles width conversion from DATA_WIDTH to CVEC.

10.3.3.2. Output Streaming

Use the output streaming component to stream output data from FPGA AI Suite IP to a downstream module using an AXI4-Stream interface.

When the output streaming interface enabled, it produces data in HWC format (where channels is the fastest changing dimension), on the output bus. The bus width is configured in the architecture parameters as described in Parameter Group: output_stream_interface on page 186. The data is converted from the FPGA AI Suite IP internal clock domain to the clock domain of the AXI4-Stream receiver.

The output streaming interface supports only graphs with a single tensor output. Graphs that produce multiple tensors must either concatenate the tensors or use the non-streaming output to external memory.

The following signals implement the output AXI4-Stream interface:

Table 41. FPGA AI Suite IP AXI4-Stream Output Interface Signals

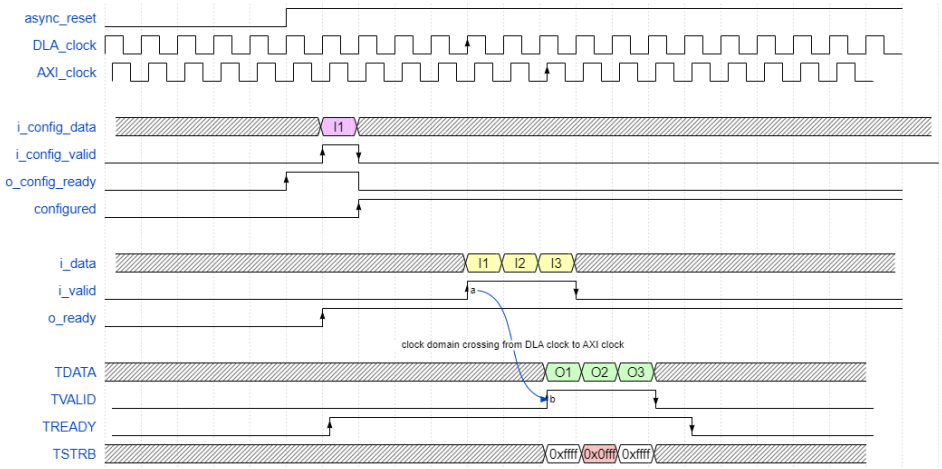
Signal	Source	Width	Description
clk_axi	Clock	1	Downstream AXI clock
i_resetn_axi	Reset	1	Downstream AXI active-low reset
o_ostream_axi_t_valid	FPGA AI Suite IP	1	Output signal that indicates whether the values in TDATA are valid
i_ostream_axi_t_ready	Downstream AXI User	1	Output signal that indicates whether the AXI receiver is ready to accept data
o_ostream_axi_t_data	FPGA AI Suite IP	DATA_WIDTH	Output data bus
o_ostream_axi_t_strb	FPGA AI Suite IP	DATA_WIDTH/8	Output signal that indicates which bytes of TDATA are valid
o_ostream_axi_t_last	FPGA AI Suite IP	1	Indicates the last transmission for the current frame

Data from the FPGA AI Suite IP is produced in CVEC multiples. When the number of output channels is not a multiple of CVEC, the last AXI4-Stream transaction for a single pixel in height/width dimensions might have only some valid elements that are indicated by the o_ostream_axi_t_strb signal.

Consider the following example: Assume the output data tensor has a shape of 3x3x10 (HWC), with CVEC = 8 elements (each 16 bits), and AXI TDATA_WIDTH = 128 bits. For each pixel in the 3x3 surface, we need to produce 10 channels, which fit in two AXI transactions. The first transaction has all valid elements (o_ostream_axi_t_strb = 0xffff). The second transaction has only two valid elements and the rest are zeros (o_ostream_axi_t_strb = 0x000f). The downstream receiver of AXI transactions is responsible for intercepting the o_ostream_axi_t_strb signal and processing only the valid elements.

The figure that follows demonstrates an example transaction flow with 3 input transactions (I1, I2 and I3, each 128 bits) and three output AXI transactions (O1, O2, and O3, each 128 bits). In this example the first and third 128-bit transactions are all

valid data (`o_ostream_axi_t_strb` =0xffff), but the second transaction has only 6 valid elements (16 valid bytes). For the second transaction, the `o_ostream_axi_t_strb` =0x0fff (marked in red color). Notice that data transactions happen only when the block is configured and ready to produce output.



10.3.4. CSR Map and Descriptor Queue

The CSR interface uses a 32-bit data path in which all accesses are aligned to 32 bits; however the address is a byte address. The size of the CSR address space is 2048 bytes (11 bit addressable). The regions within the CSR address space are listed in the table that follows.

Table 42. CSR Address Space Regions

Feature	Base Address
Discovery ROM (512 word)	0x000
Interrupt Control	0x200
DMA Descriptor Queue	0x210
DMA Control Registers	0x220
Layout Transform External Memory Registers	0x230
Performance Registers	0x240
Debug Network Registers	0x250
DMA License Register	0x260
DMA Transaction Counters	0x264
Model Update Registers	0x300

Register and Bit Attribute Definitions

The following notation describes the CSR registers.

Table 43. Register and Bit Attribute Definitions

Attribute	Expansion	Description
RW	Read/Write	This bit can be read or written by software.
RO	Read Only	The bit is set by hardware only. Software can only read this bit. Writes have no effect.
RW1C	Read/Write 1to Clear	Software can read or clear this bit. Software must write 1 to clear this bit. Writing zero to an RW1C bit has no effect. A multibit RW1C field can exist. In that case, all bits in the field are cleared if a 1 is written to any of the bits.
RsvdZ	Reserved and zero	Reserved for future RW1C implementations. When you write to a register with RsvdZ bits, only write zeros to these bits.

Discovery ROM

The discovery ROM stores metadata. The metadata includes a hash for the architecture that the IP corresponds to and the FPGA AI Suite version that was used to create the IP.

The host runtime can use this information to determine whether the incoming inference job can be run on the IP instances. For example, if the architectures do not match each other, then inference is not possible.

The layout of the discovery ROM is as follows:

Table 44. Discovery ROM Layout

Base Byte Address	Length (in bytes)	Feature
0x000	16	Hash of the Architecture Description File (`.arch`)
0x010	32	Human-readable FPGA AI Suite version string

Interrupt Control

The interrupt control feature registers are as follows:

Table 45. Interrupt Control Feature Registers

Register	Offset	Attribute	Description
ICR	0x000	RW1C	DMA Interrupt control register
IMR	0x004	RW	DMA Interrupt mask register

The DMA optionally generates level sensitive interrupt signals in response to various events.

The hardware sets the corresponding bit within the ICR register whenever such an event occurs.

An interrupt is generated upon a 0-to-1 transition of a bit within ICR only if the corresponding bit in the IMR is set to one. A 0-to-1 transition of a bit within the IMR also generates an interrupt if the corresponding bit within the ICR is set to 1.

Table 46. Interrupt Control Register (ICR) Fields

Field	Bit	Description
Reserved	31:2	RsvdZ (Reserved; software must write 0)
Inference_complete	1	Indicates that an inference request has completed
Error	0	Indicates that an error condition has been triggered

Table 47. Interrupt Mask Register (IMR) Fields

Field	Bit	Description
Reserved	31:2	RsvdZ (Reserved; software must write 0)
Inference_complete_mask	1	Set to one to enable interrupt generation on inference completion
Error_mask	0	Set to one to enable interrupt generation on error condition

DMA Descriptor Queue

The DMA contains a single descriptor FIFO for enqueuing inference requests. Descriptors potentially require multiple register writes and are added to the queue upon writing to the `desc_input_output_base_addr` register.

The `desc_cfg_filter_base_addr` and `desc_cfg_num_words` are registers that hold their value.

If you already enqueued a DMA descriptor and want to enqueue another descriptor with the same values for the `desc_cfg_filter_base_addr` and `desc_cfg_num_words` registers, then write to the `desc_input_output_base_addr` register.

If you want to change the `desc_cfg_filter_base_addr` and `desc_cfg_num_words` registers for the next descriptor, then you must set new values before writing to the `desc_input_output_base_addr` register.

Table 48. DMA Descriptor Queue Registers

Register	Offset	Attribute	Description
`desc_cfg_filter_base_addr`	0x000	RW	Base address pointer for the configuration buffer and for the filter buffer. The filters are located at `desc_cfg_filter_base_addr` + `desc_cfg_num_words`, which is encoded in the address provided to the filter reader as configuration data. Must be aligned to a multiple of the DDR word size.
`desc_cfg_num_words - 2`	0x004	RW	Length of the configuration buffer - 2, in config words (64 bits – 32 for instruction, 32 for data)
`desc_input_output_base_addr`	0x008	RW	Base address pointer for the input feature data and output inference results (written to an offset from the base address). Must be aligned to a multiple of the DDR word size. Writing to this register enqueues a descriptor into the internal DMA descriptor queue.
`desc_diagnostics`	0x00C	RO	This register is useful for debugging. Production software should not need to read from this. Bit 0: Asserts if the descriptor queue overflows; this is a sticky bit which only clears after reset.

continued...

Register	Offset	Attribute	Description
			Bit 1: Descriptor queue is full or almost full. Bit 2: Asserts if the inference limit for an unlicensed IP is reached. When asserted, inference requests are rejected. All other bits are reserved.

DMA Control Registers

Table 49. DMA Control Registers

Register	Offset	Attribute	Description
`Intermediate_ddr_base_address`	0x000	RW	Base address for the DDR intermediate data. This is a shared address across all graphs. Only required to be set once upon startup. Must be aligned to a multiple of the DDR word size.
`Inference_completion_count`	0x004	RO	Number of inference request completions by the FPGA AI Suite IP.
`IP_reset`	0x008	RW	Write any non-zero value to this address to trigger a reset of the FPGA AI Suite IP. The value is automatically cleared upon reset. Reading from this register always returns `0`.
`Activate_streaming`	0x00C	RW	When streaming is enabled in the architecture, writing "1" to this register makes the FPGA AI Suite IP begin queuing descriptors and start listening for streaming inputs. Writing "0" stops queuing descriptors and turns off the input streaming interface.

Layout Transform External Memory Registers

To properly commission the layout transform external memory, the registers in this section must be written in order (EXT_MEM_LT_BASE_ADDR, EXT_MEM_LT_FRAME_SIZE, then EXT_MEM_LT_ADDR_RANGE). The FPGA AI Suite compiler reports the input frame size in its output with the string "`Graph input size:`", which is helpful in setting the frame size register correctly for your graph.

Table 50. Layout Transform External Memory Registers

Register	Offset	Attribute	Description
EXT_MEM_LT_BASE_ADDR	0x00	RW	Base address of the input layout transform external memory space used for staging input frames.
EXT_MEM_LT_FRAME_SIZE	0x04	WO	Size of the input frame after input transformations. For help, the model compiler outputs the post-input transform feature size.
EXT_MEM_LT_ADDR_RANGE	0x08	WO	The range allocated to the input feature staging area. This value must be set to at least three times the frame size set in EXT_MEM_LT_FRAME_SIZE.

Performance Registers

Hardware counters are provided to measure how many clock cycles that the IP is active. A job is considered active after the first word of its descriptor is read from the descriptor queue. A job is considered finished just before the `done` interrupt is raised and the completion count is updated.

The IP and supporting host form an elastic pipeline in which multiple jobs can be in flight. The IP tracks both the overall latency (for example, the length of time required to process 100 jobs) as well as the average latency for each of those jobs. The hardware tracks the total latency of every job but knowing the total number of jobs software can compute the average.

64-bit counters mitigate against overflow. There is no synchronization between reading the lower or upper 32 bits of a counter, therefore the software should not read the counters while the IP is active.

Table 51. Performance Registers

Register	Offset	Attribute	Description
Total clocks active (lower 32 bits)	0x000	RO	On each clock cycle, if any IP job is active, increment the counter by 1.
Total clocks active (upper 32 bits)	0x004	RO	Same as above.
Total clocks for all jobs (lower 32 bits)	0x008	RO	On each clock cycle, if there are *N* IP jobs active, increment the counter by *N*.
Total clocks for all jobs (upper 32 bits)	0x00C	RO	Same as above.

Debug Network Registers

The debug network has the following registers available from the CSR:

Table 52. Debug Network Registers

Register	Offset	Attribute	Description
DLA_DMA_CSR_OFFSET_DEBUG_NETWORK_ADDR	0x000	RO	Address that the debug network uses to issue a read request.
DLA_DMA_CSR_OFFSET_DEBUG_NETWORK_VALID	0x004	RO	Indicates that a read response has been received from the debug network.
DLA_DMA_CSR_OFFSET_DEBUG_NETWORK_DATA	0x008	RO	Data from debug network.

DMA License Register

Table 53. DMA License Register

Register	Offset	Attribute	Description
`license_flag`	0x000	RO	Indicates whether the IP is licensed: • `0`: unlicensed • `1`: licensed

DMA Transaction Counters

Hardware counters are provided to measure the number of data words accessed by the DMA from the external DDR memory.

The counter values are separated into input feature reads, input weights and biases reads, and output feature writes. The width of each memory word in bytes matches the `dma/ddr_data_bytes` value in the architecture description file.

Table 54. DMA Transaction Counter Registers

Register	Offset	Attribute	Description
Total number of input feature words read by the FPGA AI Suite IP (lower 32 bits)	0x000	RO	This counter is incremented by 1 for every input feature word transferred from the external memory to the IP DMA on the AXI4 read bus.
Total number of input feature words read by the FPGA AI Suite IP (upper 32 bits)	0x004	RO	Same as above.
Total number of input filter and biases words read by the FPGA AI Suite IP (lower 32 bits)	0x008	RO	This counter is incremented by 1 for every filter-bias word transferred from the external memory to the IP DMA on the AXI4 read bus.
Total number of input filter and biases words read by the FPGA AI Suite IP (upper 32 bits)	0x00C	RO	Same as above.
Total number of output feature words written by the FPGA AI Suite IP (lower 32 bits)	0x010	RO	This counter is incremented by 1 for every feature word written to the external memory by the IP DMA on the AXI4 write bus.
Total number of output feature words written by the FPGA AI Suite IP (upper 32 bits)	0x00C	RO	Same as above.

Model Update Registers

When the FPGA AI Suite IP is configured as DDR-Free, its model can be updated via writes to the following registers:

Register	Offset	Attribute	Description
MODEL_UPDATE_WORD_0	0x000	W	32-bit chunk of a scratchpad or configuration word, index 0 (least significant)
MODEL_UPDATE_WORD_1	0x004	W	Same as above, index 1
MODEL_UPDATE_WORD_2	0x008	W	Same as above, index 2
MODEL_UPDATE_WORD_3	0x00C	W	Same as above, index 3
MODEL_UPDATE_WORD_4	0x010	W	Same as above, index 4
MODEL_UPDATE_WORD_5	0x014	W	Same as above, index 5
MODEL_UPDATE_WORD_6	0x018	W	Same as above, index 6
MODEL_UPDATE_WORD_7	0x01C	W	Same as above, index 7
MODEL_UPDATE_WORD_8	0x020	W	Same as above, index 8
MODEL_UPDATE_WORD_9	0x024	W	Same as above, index 9
MODEL_UPDATE_WORD_10	0x028	W	Same as above, index 10

continued...

Register	Offset	Attribute	Description
MODEL_UPDATE_WORD_11	0x02C	W	Same as above, index 11
MODEL_UPDATE_WORD_12	0x030	W	Same as above, index 12
MODEL_UPDATE_WORD_13	0x034	W	Same as above, index 13
MODEL_UPDATE_WORD_14	0x038	W	Same as above, index 14
MODEL_UPDATE_WORD_15	0x03C	W	Same as above, index 15
MODEL_UPDATE_WORD_16	0x040	W	Same as above, index 16
MODEL_UPDATE_WORD_17	0x044	W	Same as above, index 17
MODEL_UPDATE_WORD_18	0x048	W	Same as above, index 18
MODEL_UPDATE_WORD_19	0x04C	W	Same as above, index 19
MODEL_UPDATE_WORD_20	0x050	W	Same as above, index 20
MODEL_UPDATE_WORD_21	0x054	W	Same as above, index 21
MODEL_UPDATE_WORD_22	0x058	W	Same as above, index 22
MODEL_UPDATE_WORD_23	0x05C	W	Same as above, index 23
MODEL_UPDATE_WORD_24	0x060	W	Same as above, index 24
MODEL_UPDATE_WORD_25	0x064	W	Same as above, index 25
MODEL_UPDATE_WORD_26	0x068	W	Same as above, index 26
MODEL_UPDATE_WORD_27	0x06C	W	Same as above, index 27
MODEL_UPDATE_WORD_28	0x070	W	Same as above, index 28
MODEL_UPDATE_WORD_29	0x074	W	Same as above, index 29
MODEL_UPDATE_WORD_30	0x078	W	Same as above, index 30
MODEL_UPDATE_WORD_31	0x07C	W	Same as above, index 31
MODEL_UPDATE_CONTROL	0x080	W	Type of word and target address of the update

To see how to use these registers to update the DDR-free model on the FPGA device, refer to Updating Hostless DDR-Free MIF Files Through the CSR on page 292

10.3.5. Interfacing the FPGA AI Suite IP to Avalon Memory Map (AVMM)

The default output of the generated FPGA AI Suite IP presents AXI interfaces. If your FPGA design has components that require Avalon Memory Mapped (AV MM) interface then you can covert between AXI and AVMM in the following ways:

- In Quartus Prime Platform Designer, you can automatically connect the AXI and AV MM ports, Platform Designer interconnect handles the translation for you.

  For more information about the interconnect, refer to "Platform Designer Interconnect" in *Quartus Prime Pro Edition User Guide: Platform Designer*.
- For explicit control over the conversion, can use an AXI to AVMM bridge IP.

  or more information about these IPs, refer to the following topics in *Quartus Prime Pro Edition User Guide: Platform Designer*:

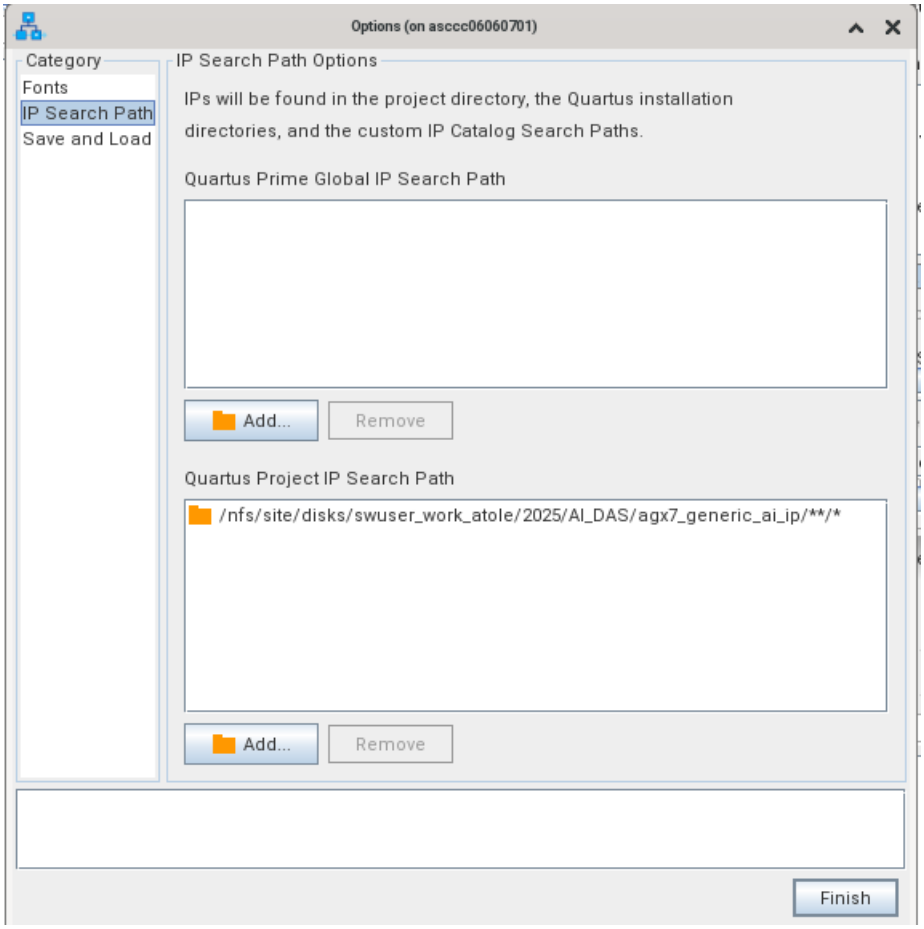
  ☐ Avalon Memory Mapped Pipeline Bridge IP

  ☐ AXI Bridge IP

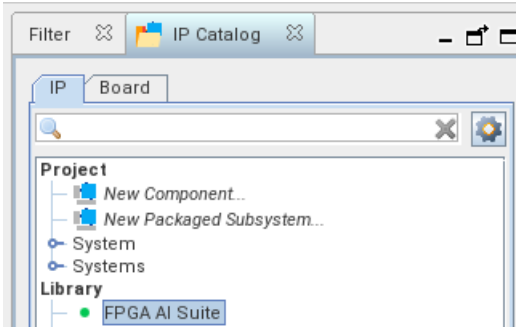
10.4. Instantiating the FPGA AI Suite IP in Platform Designer

Use Platform Designer to instantiate or integrate the FPGA AI Suite IP in your FPGA design. The FPGA AI Suite compiler produces an IP index file (`.ipx`) file that you can use to add the design in Platform Designer. For more information about IP component description files like an IP index file, refer to "IP Component Description Files" in *Quartus Prime Pro Edition User Guide: Platform Designer*.

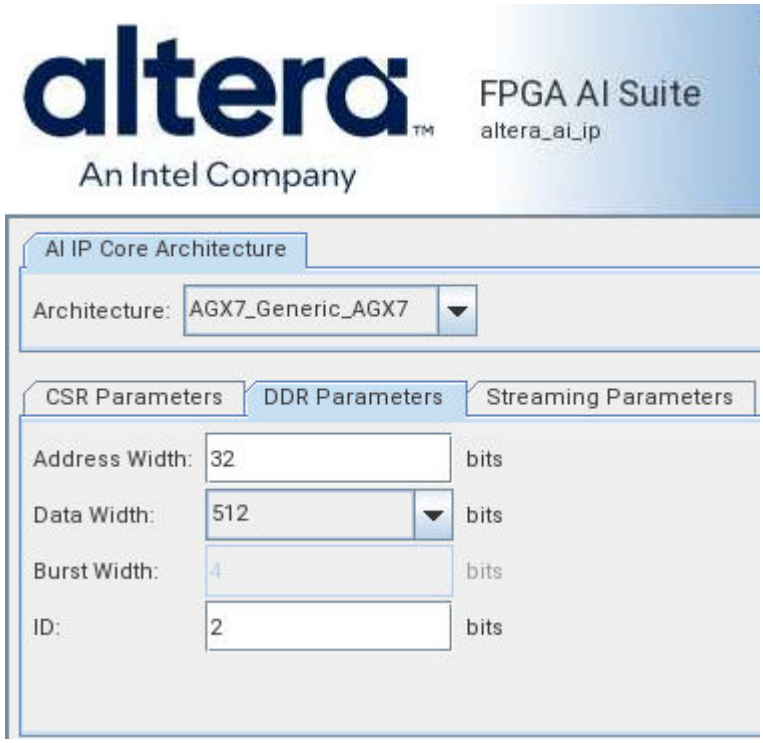
For Platform Designer IP catalog to show the FPGA AI Suite IP, add the location of the `.ipx` file for the IP to search path for your project as shown in the following image.



After you update the IP search path, the Platform Designer IP catalog refreshes to show the FPGA AI Suite IP as shown in the image that follows. . For more information about the IP search path in Platform designer, refer to to "IP Search Path Recursive Search" in *Quartus Prime Pro Edition User Guide: Platform Designer*.



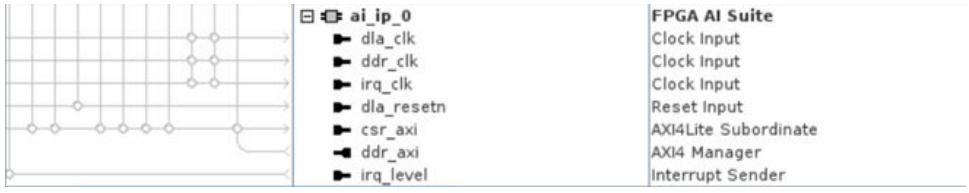
When you view the FPGA AI Suite IP in Platform Designer, most of the parameters cannot be changed in the IP properties dialog box. The IP properties are set when you generate the FPGA AI Suite IP. You can modify some of the DDR interface parameters in the IP properties dialog box:



Note: For this example, the FPGA AI Suite IP was generated with the following command:

```
dla_create_ip --flow create_ip \
  --arch=$COREDLA_ROOT/example_architectures/AGX7_Generic.arch \
  --overwrite \
  --ip_dir ./ip
```

After the IP is instantiated as shown in the following image, connect your IP to the exposed signals.



Related Information

- Quartus Prime Pro Edition User Guide: Platform Designer
- Generating the FPGA AI Suite IP for Integration into an FPGA Design on page 218

10.4.1. Resource Utilization of FPGA AI Suite IP

Quartus Prime synthesis provides the pre-fit resource utilization. You can view the FPGA AI Suite IP resource utilization after you have an FPGA AI Suite IP instance added in your FPGA design and run synthesis successfully. Running synthesis on your design helps get an estimate on how full the device is which can impact routing and power consumption on your device. The synthesis results also provide insight into

resource availability for additional hardware or additional instances of the FPGA AI Suite IP in your design. The post-fit usage can be seen in the compilation reports after the Quartus Prime fitter has run.

For more information about Quartus flows and resource utilization reports, refer to the Quartus Prime Pro Edition Software User Guides.

Related Information

Quartus Prime Software User Guides

11. Using FPGA AI Suite as a PCIe-Attach Platform

The FPGA AI Suite PCIe-attach design examples demonstrate how the Intel Distribution of OpenVINO toolkit and the FPGA AI Suite support the look-aside deep learning acceleration model.

The PCIe-attach design examples are implemented with the following components:

- FPGA AI Suite IP
- Intel Distribution of OpenVINO toolkit
- A supported FPGA board:

Table 55. FPGA Boards Supported by the PCIe-Attach Design Examples

PCIe-Attach Interface	Supported Board
BSP	Terasic DE10-Agilex Development Board (DE10-Agilex-B2E2)
OFS	Agilex 5 FPGA E-Series 065B Modular Development Kit (MK-A5E065BB32AES1)
	Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES)
	FPGA SmartNIC N6001-PL Platform (without Ethernet controller)

- Sample hardware and software systems that illustrate the use of these components

Prebuilt FPGA bitstreams are available as a separate package. For details, refer to Downloading Precompiled Bitstreams and SD Card Images on page 86. These bitstreams use a variable of the example architecture files that are shipped with FPGA AI Suite. However, the design example build scripts let you choose from a variety of architecture files and build (or rebuild) your own bitstreams, provided that you have a license permitting bitstream generation.

These designs are provided with the FPGA AI Suite as examples that show how to incorporate the IP into a design. The designs are not intended for unaltered use in production scenarios. Any potential production application that uses portions of these design examples must review them for both robustness and security.

To learn more about the software stack of the runtime host application, refer to Understanding the FPGA AI Suite Runtime Software Stack on page 339.

To learn more about the PCIe design examples and to build and execute the designs, refer to the Altera FPGA Developer Site Example Designs.

Related Information

- Altera FPGA Developer Site Example Design
- PCIe Design Example Prerequisites on page 87
- OFS for PCIe-Attach Design Example Prerequisites on page 93

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

11.1. PCIe-Attach Design Example System Architecture

The PCIe-Attach design example is derived from the BSP provided by the Terasic DE10-Agilex Development Board.

11.1.1. PCIe-Attach System Overview

The system consists of the following components connected to a host system via a PCIe interface as shown in the following figure.

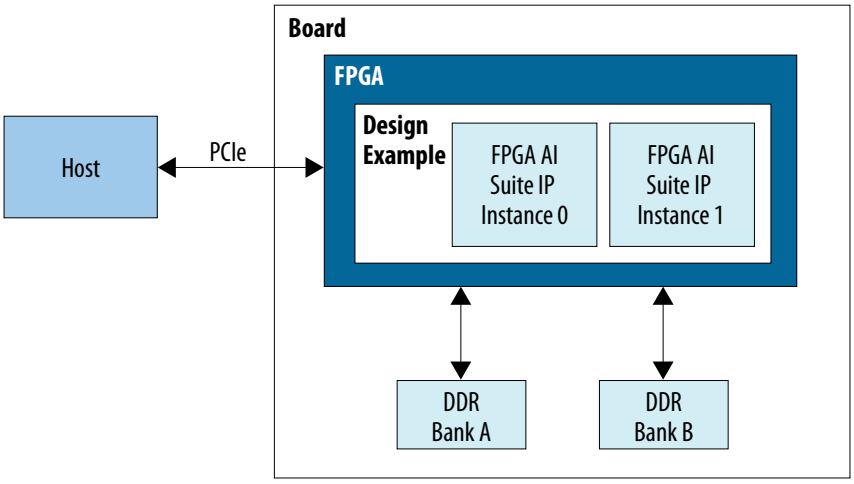
- A board with the FPGA device
- On-board DDR memory

The FPGA image consists of the FPGA AI Suite IP and an additional logic that connects it to a PCIe interface and DDR. The host can read and write to the DDR memory through the PCIe port. In addition, the host can communicate and control the FPGA AI Suite instances through the PCIe connection which is also connected the direct memory access (DMA) CSR port of FPGA AI Suite instances.

The FPGA AI Suite IP accelerates neural network inference on batches of images. The process of executing a batch follows these steps:

1. The host writes a batch of images, weights, and configuration data to DDR where weights can be reused between batches.
2. The host writes to the FPGA AI Suite CSR to start execution.
3. FPGA AI Suite computes the results of the batch and stores them in DDR.
4. Once the computation is complete, FPGA AI Suite raises an interrupt to the host.
5. The host reads back the results from DDR.

Figure 32. FPGA AI Suite Design Example System Overview



11.1.2. PCIe-Attach Design Example Hardware

A top-level view of the design example is shown in FPGA AI Suite Design Example Top Level.

The instances of FPGA AI Suite IP are on the right (`dla_top.sv`). All communication between the FPGA AI Suite IP systems and the outside occurs via the FPGA AI Suite IP DMA. The FPGA AI Suite IP DMA provides a CSR (which also has interrupt functionality) and reader/writer modules which read/write from DDR.

The host communicates with the board through the PCIe protocol. The host can do the following things:

1. Read and write the on-board DDR memory (these reads/writes do not go through FPGA AI Suite IP).
2. Read/write to the FPGA AI Suite IP DMA CSR of the instances.
3. Receive interrupt signals from the FPGA AI Suite IP DMA CSR of both instances.

Each FPGA AI Suite IP instance can do the following things:

1. Read/write to its DDR bank.
2. Send interrupts to the host through the interrupt interface.
3. Receive reads/writes to its DMA CSR.

From the perspective of the FPGA AI Suite, external connections are to the PCIe interface and to the on-board DDR4 memory. The DDR memory is connected directly to `mem.qsys` block, while the PCIe interface is converted into Avalon memory mapped (MM) interfaces in `pcie_ed.qsys` block for communication with the `mem.qsys` block.

The `mem.qsys` blocks arbitrate the connections to DDR memory between the reader/writer modules in FPGA AI Suite IP and reads/writes from the host. Each FPGA AI Suite IP instance in this design has access to only one of the DDR banks. This design decision implies that the number simultaneous FPGA AI Suite IP instances that can exist in the design is limited to the number of DDR blocks available on the board. Adding an additional arbiter would relax this restriction and allow additional FPGA AI Suite IP instances.

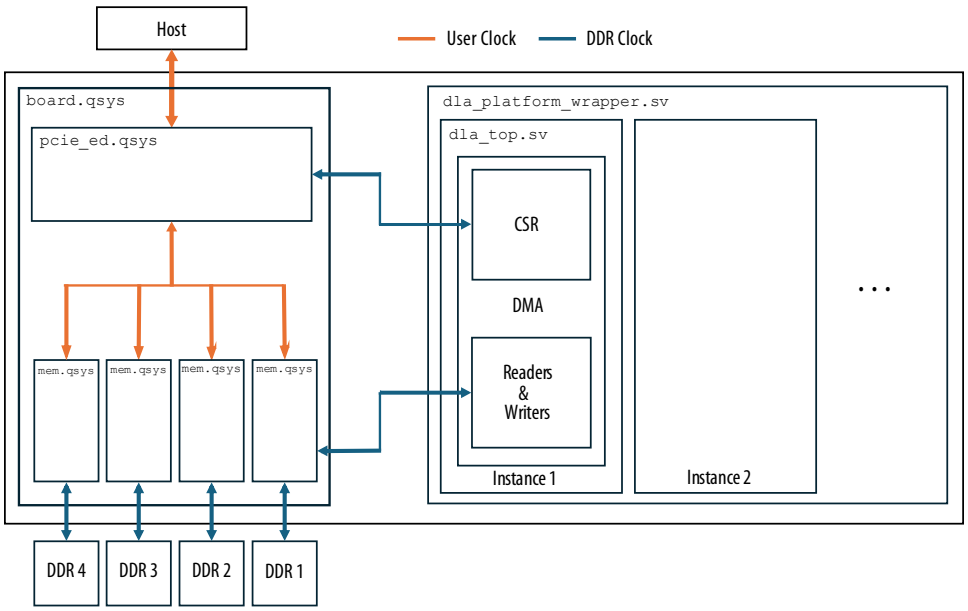
Much of `board.qsys` operates using the Avalon Memory-mapped (MM) interface protocol. The FPGA AI Suite DMA uses AXI protocol, and `board.qsys` has Avalon MM interface to AXI adapters just before each interface is exported to the FPGA AI Suite IP (so that outside of the Platform Designer system it can be connected to FPGA AI Suite IP). Clock crossing is also handled inside of `board.qsys`. For example, the host interface must be brought to the DDR clock to talk with the FPGA AI Suite IP CSR.

There are three clock domains: host clock, DDR clock, and the FPGA AI Suite IP clock. The PCIe logic runs on the host clock. FPGA AI Suite DMA and the platform adapters run on the DDR clock. The rest of FPGA AI Suite IP runs on the FPGA AI Suite IP clock.

FPGA AI Suite IP protocols:

- Readers and Writers: 512-bit data (width configurable), 32-bit address AXI4 interface, 16-word max burst (width fixed).
- CSR: 32-bit data, 11-bit address

Figure 33. FPGA AI Suite Design Example Top Level



The `board.qsys` block contains two major elements; the `pcie_ed.qsys` block and the `mem.qsys` blocks. The `pcie_ed.qsys` block interfaces between the host PCIe data and the `mem.qsys` blocks. The `mem.qsys` blocks interface between DDR memory, the readers/writers, and the host read/write channels.

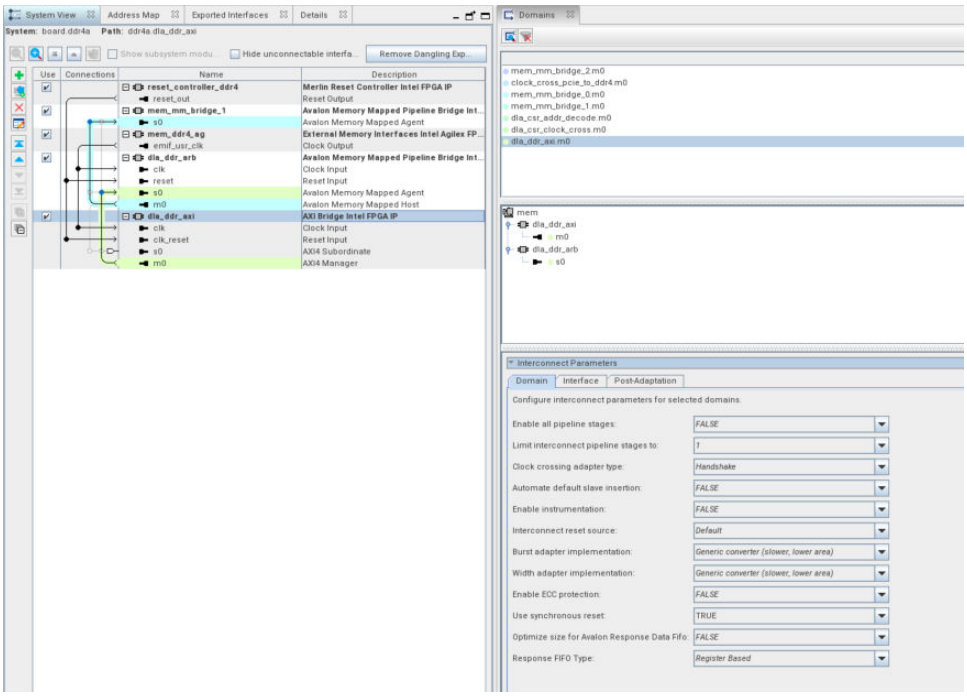
- Host read is used to read data from DDR memory and send it to the host.
- Host write is used to read data from the host into DDR memory.
- The MMIO interface performs several functions:
  - ☐ DDR read and write transactions are initiated by the host via the MMIO interface
  - ☐ Reading from the AFU ID block. The AFU ID block identifies the AFU with a unique identifier and is required for the OPAE driver.
  - ☐ Reading/writing to the DLA DMA CSRs where each instance has its own CSR base address.

Note: Avalon MM/AXI4 adapters in Platform Designer might not close timing.

Platform Designer optimizes for area instead of f_{MAX} by default, so you might need to change the interconnect settings for the inferred Avalon MM/AXI4 adapter. For example, we made some changes as shown in the following figure.

Figure 34. Adjusting the Interconnect Settings for the Inferred Avalon MM/AXI4 Adapter to Optimize for f_{MAX} Instead of Area.

Note: This enables timing closure on the DDR clock.



To access the view in the above figure:

- Within the Platform Designer GUI choose **View -> Domains**. This brings up the **Domains** tab in the top-right window.
- From there, choose an interface (for example, `dla_ddr_axi`).
- For the selected interface, you can adjust the interconnect parameters, as shown on the bottom-right pane.
- In particular, we needed to change **Burst adapter implementation** from **Generic converter (slower, lower area)** to **Per-burst-type converter (faster, higher area)** to close timing on the DDR clock.

This was the only change needed to close timing, however it took several rounds of experimentation to determine this was the setting of importance. Depending on your system, other settings might need to be tweaked.

11.1.2.1. PCIe-Attach Design Example PLL Adjustment

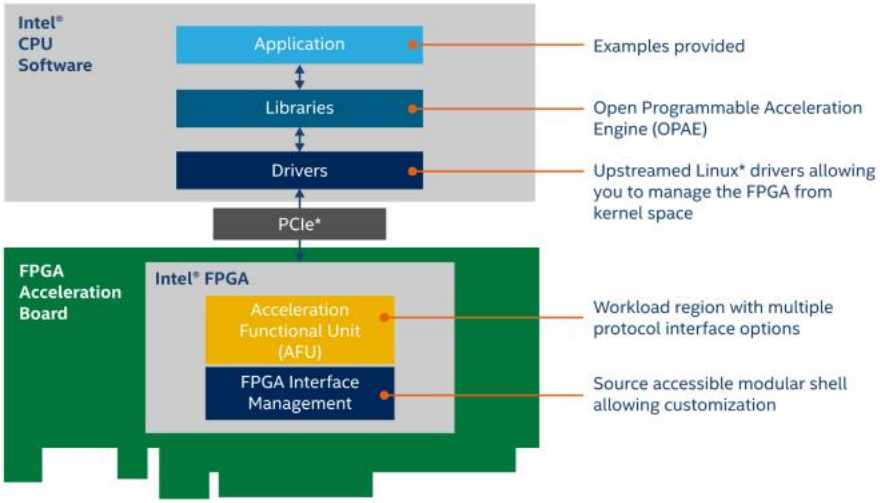
The design example build script adjusts the PLL driving the FPGA AI Suite IP clock based on the f_{MAX} that the Quartus Prime compiler achieves.

A fully rigorous production-quality flow would re-run timing analysis after the PLL adjustment to account for the small possibility that change in PLL frequency might cause a change in clock characteristics (for example, jitter) that cause a timing failure. A production design that shares the FPGA AI Suite IP clock with other system components might target a fixed frequency and skip PLL adjustment.

11.2. OFS PCIe-Attach Design Example Components

11.2.1. OFS PCIe-Attach Hardware Components

The FPGA AI Suite OFS for PCIe attach design example is based on OFS (Open FPGA Stack). The following diagram shows a high-level view of a typical OFS system/A software stack runs on the host CPU (applications, OFS libraries, FPGA drivers) that connects via a PCIe connection to an FPGA board.



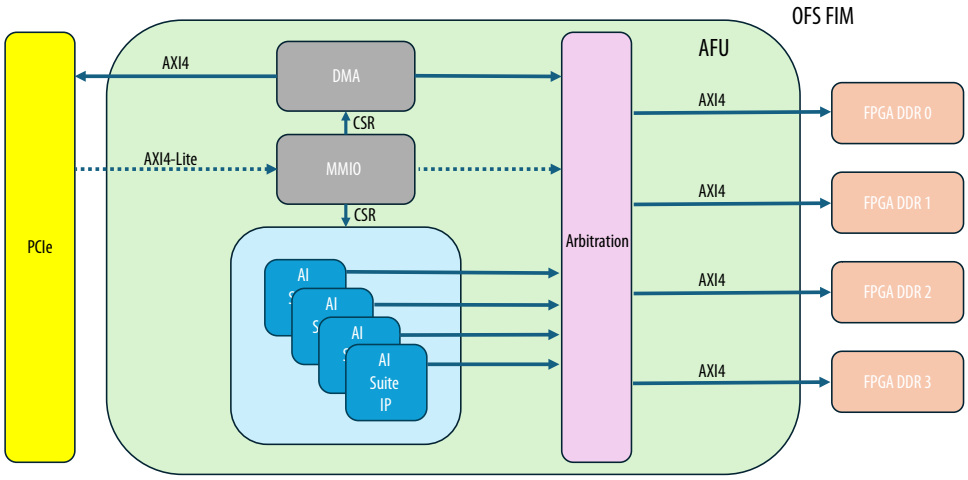
The design example hardware implementation, sits in the AFU (acceleration functional unit) region and uses the OFS FIM (FPGA interface manager) to connect to both the host CPU via a PCIe connection and also to the on-board DDR4 memory.

For more information about OFS, refer to the Open FPGA Stack (OFS) documentation.

The Agilex 7 FPGA I-Series Development Kit has four banks of DDR4 memory on board: two banks are soldered-on 8 GB of DDR4 memory each, two banks are DIMM slots for DDR4 DIMMs. For the design example, the DIMMs must also be 8GB in size each to match the soldered DDR4 memories. Larger size memories are currently not supported.

The FPGA SmartNIC N6001-PL Platform has four banks of DD4 memory on board. All banks are soldered-on 4 GB DDR4 DIMMs.

The following diagram shows how the OFS for PCIe attach design example is implemented within the OFS AFU:



The OFS FIM provides the following external interfaces:

- Towards the host CPU via the PCIe connection two interfaces are exposed:
  - ☐ A high-throughput AXI4 agent that initiates reads and writes from the FPGA fabric over the PCIe connection to the host CPU memory that is used by the DMA controller.
  - ☐ An AXI4-Lite host so that initiates reads and writes from the host CPU to the FPGA fabric. This interface is used, mainly for configuring the FPGA AI Suite IP CSRs, the DMA CSR, and unaligned MMIO accesses of the FPGA DDRs.
- Towards the on-board FPGA DDR banks.

  Four AXI4 agents, each of which connects to one DDR memory bank, connect to arbitration logic to enable the following paths into the DDR memory banks:
  - ☐ DMA controller to DDR
  - ☐ MIMO to DDR
  - ☐ FPGA AI Suite IP to DDR.

That design example has three different clock domains:

- PCIe core clock (500 MHz)
- DDR core clock (333 MHz)
- User clock (configurable, typically around 600 MHz)

At the entry from the PCIe interface to the AFU, there are clock crossers from the PCIe core clock into the DDR core clock. The DDR core clock is used in all the DMA, CSR and arbitration logic to help benefit timing closure, while always still maintaining the full bandwidth to all four FPGA DDR banks.

The FPGA AI Suite IP runs on a configurable user clock and is set accordingly after the Quartus compile to match the maximum supported frequency of the IP in its chosen configuration. Typically, this frequency is 600 MHz and above for designs with only one FPGA AI Suite IP instance and just below 600 MHz for four FPGA AI Suite IP instances.

Caution: The FPGA SmartNIC N6001-PL Platform card is designed for its specified power budget. If a majority of the FPGA DSPs on the device are operating at 600 MHz, you can exceed this power budget. Exceeding the power budget causes the power regulators on the card to shut the card down, which renders the card invisible on the PCIe bus.

If the card shuts down in this way, the host machine issues a kernel panic (in Linux) and either freezes or reboots automatically.

If this occurs, you must reduce the target frequency of the FPGA AI Suite IP or reduce the number of FPGA AI Suite IP instances in your instantiation of the design example.

11.2.2. OFS PCIe-Attach Software Components

The OFS for PCIe attach design example contains a sample software stack for the runtime flow. For details about the sample software stack, refer to Design Example Software Components on page 43.

12. Using FPGA AI Suite in Hostless DDR-Free Mode

To avoid use of external memory for storing graph weights and FPGA AI Suite IP configurations during inference, you can store these parameters within the IP using on-chip memory.

In this case, three types of memory initialization files (`.mif`) are required:

- `ddrfree_filter_hw*.mif`: Contains the graph filters.
- `ddrfree_bias_scale_hw*.mif`: Contains the graph biases and scaling factors.
- `ddrfree_config.mif`: Stores the FPGA AI Suite instructions for the compiled graph.

These files are generated using the `dla_compiler` tool that takes the architecture definition (`.arch` file) and the target neural network graph as inputs.

You must also enable input and output streaming when using DDR-free operation. The graph must have a large enough stream buffer depth to accommodate all of the intermediate results during inference.

The `dla_build_example_design.py` command can build design examples with DDR-free operation. The only streaming design example option currently supported is the `agx7_iseries_ddrfree` option. The design example created by this option targets the Agilex 7 FPGA I-Series Development Kit ES2 (DK-DEV-AGI027RBES).

To compile a bitstream with a DDR-free architecture, specify the directory that contains the DDR-free `.mif` files with the `--parameter_rom_dir` option of the `dla_build_example_design.py build` command.

For details about creating the `.mif` file required for DDR-free operation, refer to Generating Artifacts for Hostless DDR-Free Operation on page 283

12.1. Generating Artifacts for Hostless DDR-Free Operation

When a graph is compiled with an architecture file that has the `enable_parameter_rom` FPGA AI Suite IP block configuration option enabled, the FPGA AI Suite compiler (`dla_compiler`) produces a set of memory initialization (`.mif`) files in a directory called `parameter_rom` under the export directory specified by the `--dumpdir` compiler command option. These files are required to build the bitstream for DDR-free operation and to run software inference. You do not need to specify any additional compiler options when compiling an architecture with DDR-free operation enabled.

For an architecture with DDR-free operation enabled, the compiler produces the following files:

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

- `./ddrfree_filter_hw*.mif`

  Contain the graph filters that are used to build the bitstream for DDR-free operation.
- `./ddrfree_bias_scale_hw*.mif`

  Contain the graph biases and scaling factors used to build the bitstream for DDR-free operation
- `./ddrfree_filter_emu*.mif`

  Contain the graph filters in software emulator form.
- `./ddrfree_bias_scale_emu*.mif`

  Contains the graph biases and scaling factors in software emulator format
- `./ddrfree_config.mif`

  Stores the FPGA AI Suite instructions for the compiled graph.

The filter and bias/scale `.mif` files are postfixed with an integer that represents the PE that the parameters are loaded into.

These files are generated using the `dla_compiler` tool that takes the architecture definition (`.arch` file) and the target neural network graph as inputs.

A total of `k_vector` DDR-free filter `.mif` files are generated. Each file is postfixed with an integer that represents which PE the filter is to be loaded to. Similarly, a total of `k_vector` DDR-free bias and scale files are generated, using the same postfix scheme to indicate the PE that should load this file.

The FPGA AI Suite IP requires a stream of instructions that describe the order in which convolutions, activations, and other operations are performed. This instruction stream is stored in a single `.mif` file (`ddrfree_config.mif`).

You must also enable input and output streaming when using DDR-free operation. The graph must have a large enough stream buffer depth to accommodate all of the intermediate results during inference.

Related Information

- Changing the ML Graph in a Hostless DDR-Free Architecture on page 291
- Updating Hostless DDR-Free MIF Files Through the CSR on page 292
- Model Update Registers on page 269

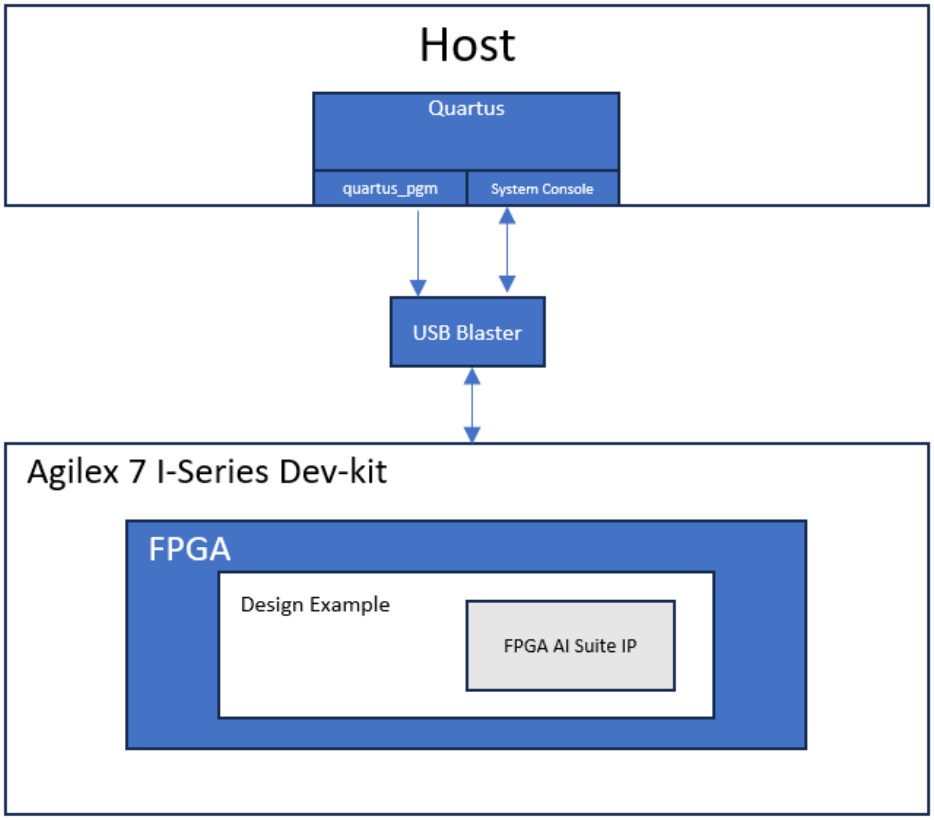
12.2. Hostless DDR-Free Design Example System Architecture

12.2.1. Hostless DDR-Free Design Example System Overview

The hostless DDR-free design example FPGA image consists of the FPGA AI Suite IP and additional logic that connects the IP to a JTAG interface. The DDR-Free design example does not use the `dla_benchmark` runtime. Instead, it allows for communication and control of the FPGA AI Suite IP through a JTAG-Quartus Prime System Console connection. In addition, the DDR-Free design example showcases the FPGA AI Suite IP streaming functionality. For more information about feature input and output streaming, refer to AXI Streaming Interface on page 259.

The system configuration of this design example is shown in the following block diagram:

Figure 35. DDR-Free Design Example System Configuration



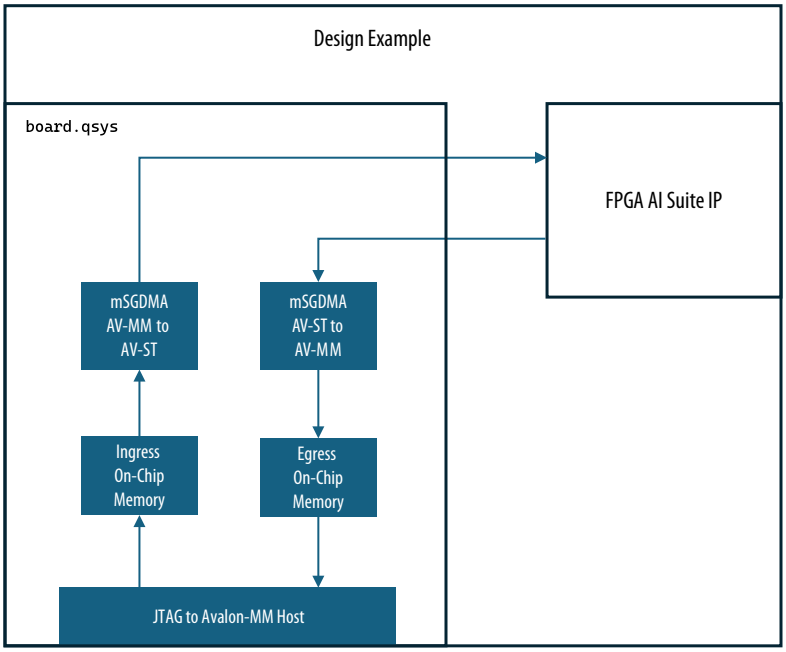
12.2.2. Hostless DDR-Free Design Example Hardware

This section provides an in-depth description of the hostless DDR-free design example, focusing on the integration and functionality of the JTAG to Avalon-MM host, the ingress and egress mSGDMA engines, the FPGA AI Suite IP for inference, and the on-chip memory modules. It covers the configuration and control mechanisms, as well as the interaction between different components to achieve efficient AI inference on an FPGA device.

A top-level view of the design example that illustrates the data flow is shown in Figure 36 on page 286. The DDR-Free design example is currently limited to one FPGA AI Suite IP instance.

All components are connected to the JTAG to Avalon-MM host and are memory-mapped on the JTAG bus, allowing for efficient communication and control from the Quartus Prime System Console. Address offsets for each component is provided in Hostless DDR-Free Design Example JTAG to Avalon MM Host Register Map on page 291.

Figure 36. DDR-Free Design Example System Architecture



12.2.2.1. The Modular Scatter-Gather DMA (mSGDMA) Engines in the Hostless DDR-Free Design Example

The data flow within the system is orchestrated by the modular scatter-gather DMA (mSGDMA) engines and the FPGA AI Suite IP (which performs the inference computation). The following mSGDMA engines are used in the design example:

- **Ingress mSGDMA**

  The ingress mSGDMA engine performs memory-mapped reads from the on-chip memory and streams the data into the FPGA AI Suite IP. It converts Avalon-MM transactions to Avalon Streaming format.

- **Egress mSGDMA**

  The egress mSGDMA engine receives the streamed inference results from the FPGA AI Suite IP and stores them into the egress on-chip memory using MM operations. It converts Avalon-ST transactions back to Avalon-MM format.

The mSGDMA engines are configured to use 128-bit streaming transfer sizes.

For more information about how to use the modular scatter-gather DMA core, refer to "Modular Scatter-Gather DMA Core" in *Embedded Peripherals IP User Guide*.

12.2.2.2. On-Chip Memory Modules in the Hostless DDR-Free Design Example

The on-chip memory modules store input data and final inference results. These memories are accessible via Avalon-MM interfaces. There are two modules: one for staging input memory and one for staging output inference results. These are referred to as ingress and egress on-chip memory, respectively. The sizes of the on-chip memory modules are defined in Table 56 on page 287.

- **Ingress On-Chip Memory**

  This module is dedicated to storing the input data before it is processed by the FPGA AI Suite IP. It serves as the staging area for data that will be read by the ingress mSGDMA engine and streamed into the inference IP.

- **Egress On-Chip Memory**

  This module is used to store the final inference results after they have been processed by the FPGA AI Suite IP. The egress mSGDMA engine writes the inference results from the FPGA AI Suite IP to this memory, making it available for retrieval and further use.

The following table provides the specific sizes allocated for each on-chip memory module, ensuring that the system has adequate storage for both input data and inference result:

Table 56. On-Chip Memory Module Sizes

On-Chip Memory Module	Size (in bytes)
Ingress	524288
Egress	131072

12.2.2.3. Platform Designer System in the Hostless DDR-Free Design Example

The on-chip memory modules, mSGDMA engines, and other components are instantiated and interconnected within the `board.qsys` Platform Designer system. This comprehensive system design is then instantiated as an IP block within the `top.sv` file, ensuring seamless integration and efficient operation of the entire design.

12.2.2.4. PLL Adjustment in the Hostless DDR-Free Design Example

The design example build script adjusts the PLL driving the FPGA AI Suite IP clock based on the f_{MAX} that the Quartus Prime compiler achieves.

12.3. Hostless DDR-Free Design Example Quartus Prime System Console

The hostless DDR-free design example design example requires user interaction on the host system through Quartus Prime System Console. For more information about the Quartus Prime System Console, refer to "Analyzing and Debugging Designs with System Console" in *Quartus Prime Pro Edition User Guide: Debug Tools*.

The system console user interface communicates over JTAG to a JTAG to Avalon-MM host IP that enables the following functions:

- Read/write to the FPGA AI Suite IP DMA CSR

  For more information about the FPGA AI Suite IP CSR map, refer to CSR Map and Descriptor Queue on page 264

- Read/write to ingress and egress on-chip memory
- Read/write to ingress and egress modular scatter-gather DMA (mSGDMA) CSR

  For more information about the mSGDMA CSR, refer to "Register Map of mSGDMA"in *Embedded Peripherals IP User Guide*.

You can find the Quartus Prime System Console Tcl script in the following location:

```
$COREDLA_ROOT/runtime/streaming/ed0_streaming_example/system_console_script.tcl
```

12.3.1. Hostless DDR-Free Design Example Quartus Prime System Console Script Options

The Quartus Prime System Console script facilitates various operations related to performance testing and functional evaluation of the design example. The following table provides an explanation of the options that you can use with the script:

Table 57. System Console Script Options

Option	Usage
`--arch`	Path to the architectural description (`.arch`) file. This argument is required for running inference.
`--input`	Path to input binary (`*.bin`) file Must be in either fp16 or uint8 depending on layout transform used (full vs lightweight). Refer to Parameter: `layout_transform_params/input_type`
`--num_inferences`	Number of inferences. For some operation modes, the number of inferences is hardcoded for optimal performance measurements: • Core IP Performance: 1 • IP Performance: 32
`--online_reconfiguration`	Path to the directory containing `.mif` files that contain the parameters and instructions of a new graph. If you specify this option, a graph update via JTAG occurs and all inference arguments are ignored.
`--output_shape`	Output shape of graph in CHW format. For example: `[1000 1 1]`.
`--functional`	Mode of operation. You can use only one of these options at a time. • Functional: Used for functional testing for large number of inferences. • Core IP Performance: Evaluates core IP Performance. • IP Performance: End to end IP performance including input and output streamer.
`--core_ip_performance`	
`--ip_performance`	

12.3.2. Hostless DDR-Free Design Example Inference Functionality

The process of executing an inference on the hostless DDR-free design example involves the following steps in the Quartus Prime System Console. Each step translates to a specific Tcl process in the `system_console_script.tcl` script:

1. Prime the FPGA AI Suite IP CSR and resets the input and egress SGDMAs for streaming inference.

   ```
   initialize_coredla{}
   ```

2. Load raw input features into ingress on-chip memory

   ```
   stage_input{}
   ```

3. Queue a descriptor into the ingress SGDMA for MM to streaming operation

   ```
   queue_ingress_descriptor{}
   ```

4. Queue a descriptor into the egress SGDMA for streaming to MM operation

```
queue_egress_descriptor{}
```

5. Reads inference results from the egress on-chip memory

```
read_output{}
```

12.3.3. Hostless DDR-Free Design Example System Reset

In most FPGA AI Suite Design Examples, system resets are typically managed through software running on a host. However, because the hostless DDR-free design example operates without a host, this design example uses In-System Sources and Probes to perform a reset via JTAG.

This approach enables remote control of the reset process, ensuring both flexibility and accessibility. For the DDR-Free design example, the reset operation is initiated by writing a reset bit through the system console via JTAG. The following Tcl code snippet demonstrates the reset process.

Figure 37. System-Console TCL Reset Code for DDR-Free Design Example

```
# Initiate reset via source/probe IP
proc assert_reset {} {
 set issp_index 0
 set issp [lindex [get_service_paths issp] 0]
 set claimed_issp [claim_service issp $issp mylib]
 set source_data 0x0
 issp_write_source_data $claimed_issp $source_data
 set source_data 0x1
 issp_write_source_data $claimed_issp $source_data
 }
```

Related Information

"Design Debugging Using In-System Sources and Probes" in *Quartus Prime Pro Edition User Guide: Debug Tools*

12.3.4. Hostless DDR-Free Design Example Input Data Conversion

The hostless DDR-free design example streams data into the FPGA AI Suite IP from the ingress on-chip memory. To achieve this, the system console script must stage input data in a `.bin` file format instead of `.bmp` format. The `.bin` file must be in organized in HWC (height-width-channel) format and in the following precision modes, depending on your input type:

- Layout transform (full or lightweight) *input_type* is FP16, the precision mode of the `.bin` file must be FP16 (half-precision floating-point).
- Otherwise, the precision mode of the `.bin` file must be U8 (unsigned 8-bit).

To facilitate this conversion from values in a `.bmp` file to FP16 values, refer to the following example Python code. This script reads a `.bmp` file, converts the image data to FP16 format, and saves it in the required `.bin` format.

Figure 38. Python Example Code for `.bmp` to `.bin` Conversion

```
import sys
from PIL import Image
import numpy as np

def convert_image_to_bin(input_image_name):
    # Read the BMP file
    img = Image.open(input_image_name)
    output_file_name = 'array_hwc_fp16.bin'

    # Convert the image to a numpy array
    arr = np.array(img)

    # Convert the image to FP16 format
    arr_fp16 = arr.astype(np.float16)

    # Save the FP16 HWC formatted data to a .bin file
    with open(output_file_name, 'wb') as f:
        arr_fp16.tofile(f)

    print(f"Converted {input_image_name} to {output_file_name}")

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Usage: python bmp_to_bin_converter.py <input_image_name>")
        sys.exit(1)

    input_image_name = sys.argv[1]
    convert_image_to_bin(input_image_name)
```

12.3.5. Measuring Performance in the Hostless DDR-Free Design Example

The system console script allows you to measure the following types of performance:

- Core IP Performance

  Specifying the `--core_ip_performance` operation mode option sets the `--num_inferences` option to 1 and stages the input into the `input_streamer` before starting the inference.

  This operation mode measures the latency of 1 inference through the IP without the time to fill the input streamer FIFO.

- System Throughput

  Specifying the `--ip_performance` operation mode sets the `--num_inferences` option to 32 and measures the throughput of the graph. This mode only saves the last output since offloading the output after every inference could limit the performances.

  This operation mode measures the throughput of the whole system including input streamer and output streamer.

12.4. Hostless DDR-Free Design Example JTAG to Avalon MM Host Register Map

Table 58. JTAG to Avalon MM Host Register Map

IP	Offset	Description
FPGA AI Suite IP 0	0x0003_8000 – 0x0003_87ff	Refer to CSR Map and Descriptor Queue on page 264
Ingress On-Chip Memory	0x0020_0000 – 0x0027_ffff	Refer to "On-Chip Memory II (RAM or ROM) IP" in *Embedded Peripherals IP User Guide*.
Egress On-Chip Memory	0x0028_0000 – 0x0029_ffff	
Ingress mSGDMA (MM to Streaming)	CSR: 0x0003_0000 – 0x0003_001f	Refer to "Modular Scatter-Gather DMA Core" in *Embedded Peripherals IP User Guide*.
	Descriptor: 0x0003_0020 – 0x0003_002f	
Egress mSGDMA (Streaming to MM)	CSR: 0x0003_0040 – 0x0003_005f	
	Descriptor: 0x0003_0060 – 0x0003_006f	

12.5. Changing the ML Graph in a Hostless DDR-Free Architecture

Each DDR-free architecture stores filter, bias, scale, and configuration data in M20K embedded memories rather than in external memory. There are two logical memories used to store this data, one called the *filter-bias-scratchpad* and the other called the *configuration memory*. The memory contents are encoded by the memory initialization (`.mif`) files.

For a hostless DDR-free architecture, you can update the ML graph stored in the FPGA AI Suite IP in the following ways:

- Programming new memory contents in the FPGA AI Suite IP CSR interface from the `.mif` files. This method keeps the programmed FPGA device online.
- Modify the FPGA bitstream and reprogram the FPGA with the modified bitstream.

For both approaches, the first step is to generate a new set of `.mif` files as described in Generating Artifacts for Hostless DDR-Free Operation on page 283. You must guarantee the filter-bias-scale memory depth and configuration memory depth of the existing FPGA AI Suite IP are sufficient for holding the parameters and configurations of the new model. The compiler returns an error message if the memory size configured into the FPGA AI Suite IP is too small.

Updating the Model via FPGA AI Suite IP CSR

To update the ML graph by programming new memory contents in the FPGA AI Suite IP CSR interface from the `.mif` files, refer to Updating Hostless DDR-Free MIF Files Through the CSR on page 292.

Updating the Model by Reprogramming the FPGA Device

You can update the contents of the M20Ks through the Quartus Prime tools. The commands regenerate the `top.sof` bitstream file that needs to be reprogrammed on the device.

The Quartus Prime tools do not change the architecture of the FPGA AI Suite IP. The update only changes the contents of the on-chip M20Ks that store the graph information and the FPGA AI Suite IP configuration.

To update the contents of the M20K on-chip memory:

1. Recompile the `.mif` files for the new graph as described in Generating Artifacts for Hostless DDR-Free Operation on page 283.
2. Replace the `.mif` files under "*<path/to/build/dir>*`/coredla_ip/intel_ai_ip/verilog/.`" with the files that were created in the previous step.
3. Run the following commands from "*<path/to/build/dir>*`/hw/`":

```
quartus_cdb top -c top --update_mif

quartus_asm --read_settings_files=on --write_settings_files=off top -c top
```

12.5.1. Updating Hostless DDR-Free MIF Files Through the CSR

The FPGA AI Suite IP compiler generates the following MIF files in the `parameter_rom` subdirectory of the compiler output directory:

- `ddrfree_filter_hw_`*<kvec-index>*`.mif`

  Store the graph weights targeting the M20Ks in each K-vector.

- `ddrfree_bias_scale_hw_`*<kvec-index>*`.mif`

  Stores the K-vector bias-scale values.

- `ddrfree_config.mif`

  The configuration MIF file.

To update the M20K content with a set of new MIF files via the CSR interface, follow this pseudocode:

```
1  # Update the Filter-Bias-Scale scratchpad
2  # Load all filter-bias-scale MIFs into memory.
3  # All filter MIFs are named as ddrfree_filter_hw_<kvec_id>.mif
4  # All bias-cale MIFs are named as ddrfree_bias_scale_hw_<kvec_id>.mif
5  fbs_mifs[] = load_mifs(fbs_mif_path);
6  for mif in fbs_mifs:
7    for addr, word in mif:
8      for i in [0, 32):
9        # Perform a 32-bit write to the CSR register. The base address is
numerated as “byte address”
10       write_to_csr(MODEL_UPDATE_WORD_0 + i*4, word[32*i+31:32*i] if i <
number_of_bytes(word)/4 else 0x0);
11     is_bias_scale = 1 if mif.type == bias_scale else 0;
12     write_to_csr(MODEL_UPDATE_CONTROL, (1 << 31) | (is_bias_scale << 30) |
(mif.kvec_id << 16) | addr));
13 # Update configuration memory.
14 # Load the configuration MIF file, ddrfree_config.mif
15 conf_mif = load_mif(conf_mif_path);
16 for addr, word in conf_mif:
17   for i in [0, 32):
18     # Perform a 32-bit write to the CSR register. The base address is
numerated as “byte address”
19     write_to_csr(MODEL_UPDATE_WORD + i*4, word[32*i+31:32*i] if i <
number_of_bytes(word)/4 else 0x0));
20   write_to_csr(MODEL_UPDATE_CONTROL, addr & 0xFFFF);
21 # Wait for data to propagate to the M20Ks.
```

```
22 sleep
23 # Invalidate FIFOs holding outdated configuration words
24 assert_reset()
```

Each word in an MIF file needs to be broken down into 32-bit chunks and written to the Model Update Word registers, with the least significant chunk targeting Model Update Word 0. Unused Model Update Word registers should be set to 0x0. Once all the chunks of the MIF word are sent, perform another write to register Model Update Control to indicate the type of word and k-vector index (only necessary if the MIF file contains filter weights or bias-scales) information to commit the update, according to the table that follows. The write to the Model Update Control update would cause the CSR interface to assemble the chunks into a memory word, and send it to the embedded memory.

Table 59. Bit layout of the Model Update Control register

Bit Indices	31	30	29:22	21:16	15:0
Configuration	1'b0	Reserved			Word Address in RAM
Filter	1'b1	1'b0	Reserved	K-vector Index	
Bias-Scale	1'b1	1'b1	Reserved		

Finally, wait for 1024 cycles on the `ddr_clk` for the data to propagate to the M20Ks, and issue a reset to the FPGA AI Suite IP to invalidate FIFOs holding outdated configuration words.

Related Information

- Model Update Registers on page 269
- Generating Artifacts for Hostless DDR-Free Operation on page 283

13. Using the FPGA AI Suite in Hostless JTAG Mode

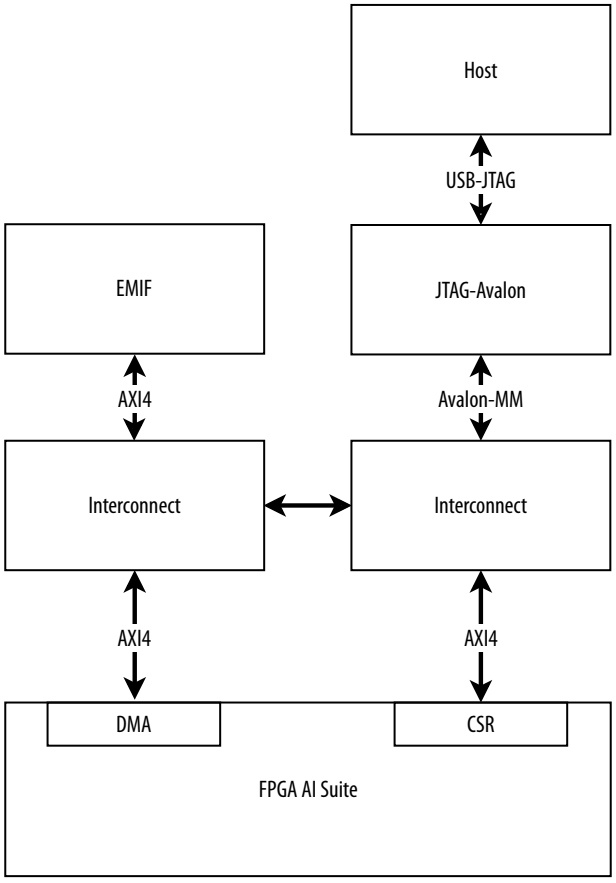
Use the FPGA AI Suite in hostless JTAG mode when you want direct control of FPGA-based inference through a JTAG interface, typically in lab environments or tightly controlled edge systems.

13.1. Hostless JTAG Design Example Components

13.1.1. Hostless JTAG Hardware Components

The FPGA AI Suite hostless JTAG design example uses a JTAG-USB connection on the development kit to facilitate host-FPGA transactions. The following diagram shows a simplified block diagram of the design.

Figure 39. Simplified System Design of the Hostless JTAG Design Example



© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

The design example stores filter weights, biases, configurations for the FPGA AI Suite IP, graph inputs, outputs, and intermediate data on the lower 2GB of a memory interface, similar to the PCIe design examples. An external memory interface (EMIF) IP manages the memory and toggles the 32-bit interface as follows:

Agilex 3 FPGA C-Series Development Kit	LPDD4 memory with the 32-bit interface toggled at 2133 Mbps
Agilex 5 FPGA E-Series 065B Modular Development Kit	DDR4 memory with the 32-bit interface toggled at 1600 Mbps

For the Agilex 3 design, the FPGA AI Suite IP can typically meet timing at around 400 MHz and accesses the EMIF IP via a 128-bit AXI4 data interface clocked at 200 MHz. The IP can only access the first 2GB of the memory.

For the Agilex 5 design, the FPGA AI Suite IP can typically meet timing at around 300 MHz and accesses the EMIF IP via a 128-bit AXI4 data interface clocked at 200 MHz. The IP can only access the first 2GB of the memory.

The host uses a JTAG-Avalon Bridge IP to access the external memory, as well as the CSR registers of the FPGA AI Suite IP via a 32-bit Avalon-MM interface clocked at 100 MHz. As seen from the JTAG-Avalon Bridge IP, the subordinates reside at the addresses outlined in the following table:

Table 60. Subordinate Addresses Accessible from the JTAG-Avalon Interface

Memory Address Range	Description
External Memory	0x0000_0000 – 0x7FFF_FFFF
FPGA AI Suite IP CSRs	0x8000_0000 – 0x8000_0FFF

To investigate and modify the implementation of the hardware components, review the source files of the system design part of the design example in the `$COREDLA_ROOT/platform/agx3c_jtag` or `$COREDLA_ROOT/platform/a5e_modular_devkit_jtag` directories.

13.1.2. Hostless JTAG Software Components

The JTAG design example relies on the following software components to executed FPGA inference via the dla_benchmark application:

- OpenVINO Toolkit
- FPGA AI Suite Runtime Plugin
- Quartus Prime System Console

When the build target is this design example, the FPGA AI Suite Runtime Plugin instantiates an MMD Wrapper object that converts external memory and FPGA AI Suite CSR access commands into Quartus Prime System Console calls. The MMD wrapper depends on the Boost C++ Libraries. The definitions of the MMD wrapper object and its methods are implemented in `$COREDLA_WORK/runtime/coredla_device/mmd/system_console/mmd_wrapper.cpp`.

Upon instantiation, the MMD wrapper initializes the JTAG services via the routine in a Quartus Prime System Console script: `$COREDLA_WORK/runtime/coredla_device/mmd/system_console/system_console.tcl`.

14. Using FPGA AI Suite as an Embedded Platform

The FPGA AI Suite SoC design examples provide two variants for demonstrating the FPGA AI Suite operation.

All designs are Platform Designer based systems.

There is a single top-level Verilog RTL file for instantiating the Platform Designer system.

These two variants demonstrate FPGA AI Suite operations in the two most common usage scenarios. These scenarios are as follows:

- **Memory to Memory (M2M)**: In this variant, the following steps occur:
  1. The Arm processor host presents input data buffers to the FPGA AI Suite that are stored in a system memory.
  2. The FPGA AI Suite IP performs an inference on these buffers.
  3. The host system collects the inference results.

  This variant demonstrates the simplest use-case of the FPGA AI Suite.
- **Streaming to Memory (S2M)**: This variant offers a superset of the M2M functionality. The S2M variant demonstrates sending streaming input source data into the FPGA AI Suite IP and then collecting the results. An Avalon streaming input captures live input data, stores the data into system memory, and then automatically triggers FPGA AI Suite IP inference operations.

  You can use this variant as a starting point for larger designs that stream input data to the FPGA AI Suite IP with minimal host intervention.

  On the Agilex 7, the S2M mode uses the FPGA AI Suite IP internal layout transform capability instead of the external demonstration layout transform described in The Layout Transform IP as an Application-Specific Block in the SoC Design Example on page 303. The internal transform capability allows for a wider range of input bus widths and supports folding. For more information about the internal transform capability, refer to Transforming the Layout of Input Data on page 230.

14.1. FPGA AI Suite SoC Design Example Inference Sequence Overview

The FPGA AI Suite IP works with system memory. To communicate with the system memory, the FPGA AI Suite has its own multichannel DMA engine.

This DMA engine pulls input commands and data from the system memory. It then writes the output data back to this memory for a Host CPU to collect.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

When running inferences, the FPGA AI Suite continually reads and writes intermediate buffers to and from the system memory. The allocation of all the buffer addresses is done by the FPGA AI Suite runtime software library.

Running an inference requires minimal interaction between the host CPU and the IP Registers.

The system memory must be primed with all necessary buffers before starting a machine learning inference operation. These buffers are setup by the FPGA AI Suite runtime library and application that runs on the host CPU.

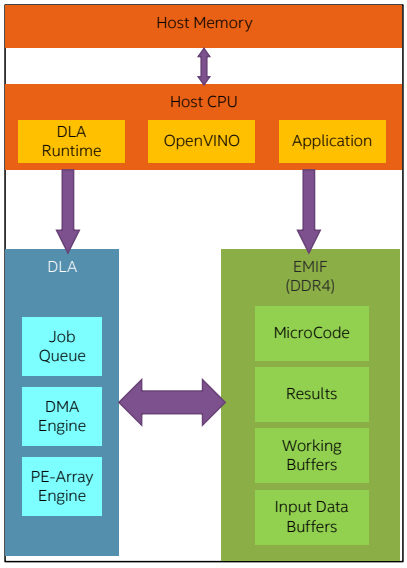
After the setup is complete, the host CPU pushes a job into the IP registers.

The FPGA AI Suite IP now performs a single inference. The job-queue registers in the IP are FIFO based, and the host application can store multiple jobs in the system memory and then prime multiple jobs inside the IP. Each job stores results in system memory and results in a CPU interrupt request.

For each inference operation in the M2M model, the host CPU (HPS) must perform an extensive data transfer from host (HPS) DDR memory to the external DDR memory that is allocated to the FPGA AI Suite IP. As this task has not been FPGA-accelerated in the design, the host operating system and FPGA AI Suite runtime library must manually transfer the data. This step consumes significant CPU resources. The M2M design uses a DMA engine to help with the data transfer from HPS DDR to the allocated DDR memory.

The FPGA AI Suite inference application and library software are responsible for keeping the FPGA AI Suite IP primed with new input data and responsible for consuming the results.

Figure 40. FPGA AI Suite SoC Design Example Inference Sequence Overview



For a detailed overview of the FPGA AI Suite IP inference sequence, refer to the The FPGA AI Suite IP Overlay Architecture on page 15.

14.2. Memory-to-Memory (M2M) Variant Design

The memory-to-memory (M2M) variant of the SoC design example illustrates a technique for embedded (SoC) FPGA AI Suite operations where the input data sets are primarily drawn from a memory or file sources. In this scenario, the data is typically not real time and is processed as fast as possible.

This design combines the HPS SoC FPGA device with an additional DMA engine to allow for efficient transfer of data to and from the CPU and system memory. The HPS is an Arm Cortex*-A76/A55 on Agilex 5, an Arm Cortex-A53 on Agilex 7, and Arm Cortex-A9 on Arria 10.

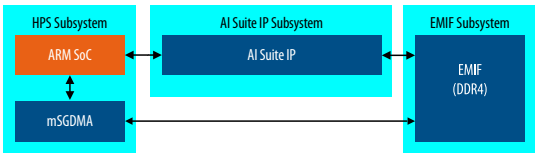
In the M2M design, the source data originally resides within the host CPU domain on an SD card. The application uses the DMA controller to move the host-side data to the device side domain. This movement mimics the process that an application would typically do.

The test program then initiates FPGA AI Suite IP inference operations and wait for the IP to complete its process. Command-line options to the user application define how many inferences are executed.

After the inference operation is completed, the application uses the DMA to transfer the results back from external memory to the host domain. The results are then displayed on the Linux console.

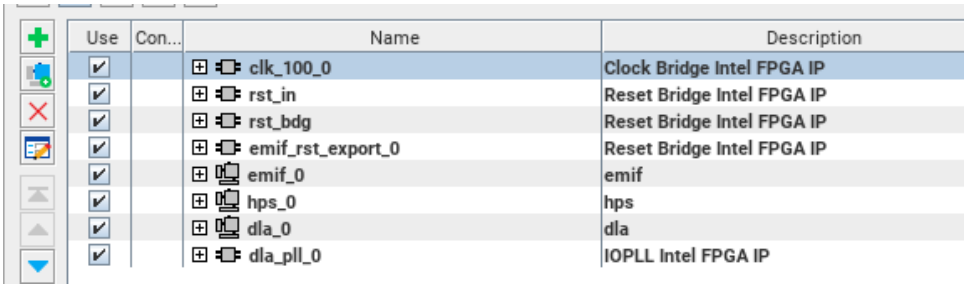
The modular scatter-gather direct memory access (mSGDMA) controller IP provides this DMA facility.

Figure 41. Block Diagram of M2M Variant



The M2M variant appears in Platform Designer as follows:

Figure 42. M2M Variant in Platform Designer



14.2.1. The mSGDMA FPGA IP

The modular scatter-gather direct memory access (mSGDMA) FPGA IP used in this design example serves as an example of how you can integrate a DMA into your own system. You can replace this DMA engine by another 3rd party controller.

The FPGA AI Suite runtime software must be modified if you want to use another DMA engine.

14.2.2. RAM considerations

An FPGA-based external memory interface is used to store all machine learning input, output, and intermediate data.

The FPGA AI Suite IP uses the DDR memory extensively in its operations.

Typically, you dedicate a memory to the FPGA AI Suite IP and avoid sharing it with the host CPU DDR memory. Although a design can use the host memory, other services that use the DDR memory impact the FPGA AI Suite IP performance and increase non-determinism in inference durations. Consider this impact when you choose to use a shared DDR resource.

The FPGA AI Suite IP requires an extensive depth of memory that prohibits the use of onboard RAM such as M20Ks. Consider the RAM/DDR memory footprint when you design with the FPGA AI Suite IP.

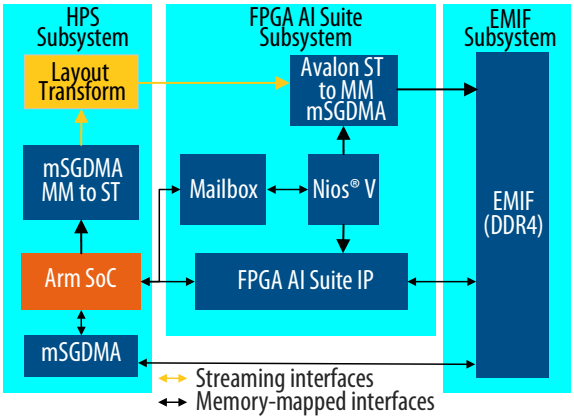
14.3. Streaming-to-Memory (S2M) Variant Design

The streaming-to-memory (S2M) variant of the SoC design example builds on top of the M2M design to demonstrate a method of using the FPGA AI Suite IP with continuously streaming input data.

The application example is a typical video stream being processed with ResNet50 to detect physical objects in the images, such as a person, cat, or dog.

In the example, test images are stored on the SD card file system. These images are loaded into host memory and a DMA (memory-to-streaming) IP is used to create a simulated video stream.

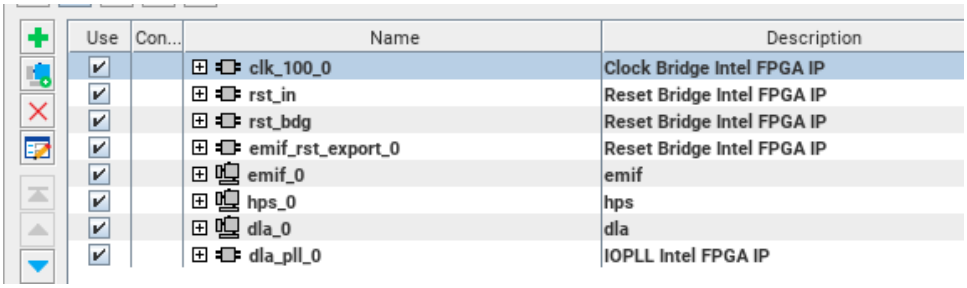
Figure 43. Block Diagram of S2M Variant forAgilex 5 and Arria 10



In the Agilex 7 version of the S2M variant, the “Layout Transform” block is not present. The layout transform occurs within the FPGA AI Suite IP.

The S2M variant appears in Platform Designer as follows:

Figure 44. S2M Variant in Platform Designer



14.3.1. Streaming Enablement for FPGA AI Suite

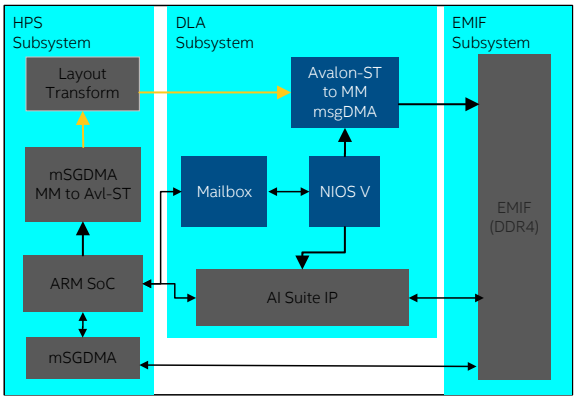
In an M2M system, input buffers are provided by the host CPU. However, in a streaming system (S2M), input buffers are created by an external hardware stream. For the FPGA AI Suite IP to process this external stream, several operations must happen in a coordinated way:

- The raw stream data must pass through a layout transform (either the FPGA AI Suite IP internal layout transform or layout transform in an external IP) to reformat the raw data into an FPGA AI Suite compliant data format
- The formatted data must be written into system memory at specific locations, known only to the host application and the FPGA AI Suite software library at run time.
- The FPGA AI Suite IP job queue must be primed at the correct time, in synchronization with the input stream buffers, such that the FPGA AI Suite IP starts an inference immediately upon a new input buffer becoming ready.

Within Platform Designer, a Nios® V based subsystem is added alongside the FPGA AI Suite IP to provide the streaming capabilities. This subsystem highlighted in blue in the block diagram that follows.

In the diagram, the yellow interconnect lines indicate Avalon streaming interfaces, and the black interconnect lines indicate memory-mapped interfaces.

Figure 45. Nios V Streaming Subsystem



14.3.2. Nios V Subsystem

Three IP modules make up the Nios V subsystem:

- **mSGDMA** (Avalon streaming to memory-mapped mode). This module is used to take the formatted input data stream and place it into system memory to create the FPGA AI Suite IP input buffers.
- **Mailbox** (On-Chip Memory II IP). This module is used to provide a communication API between the host-application and the Nios Subsystem. FPGA AI Suite IP command and status message are conveyed through this interface.
- **Nios V processor**. This module manages the FPGA AI Suite IP job-queue, mailbox and mSGDMA buffer allocation. Using the Nios V processor offloads the latency-sensitive ingest and buffer management from the HPS.

All C source-code to the Nios V application is provided. You can modify the Nios software to enable third-party DMA controllers, if required.

14.3.3. Streaming System Operation

Two operations must occur in parallel for streaming systems to work:

- Buffers must be managed appropriately. That is, buffers of streaming data must be written into system memory at a specific location ready for the FPGA AI Suite IP to process.
- Inference jobs must be managed appropriately. That is, inference jobs must be primed at the correct time to process the new buffer.

14.3.3.1. Streaming System Buffer Management

Before machine learning inference operations can occur, the system requires some initial configuration.

As in the M2M variant, the S2M application allocates sections of system memory to handle the various FPGA AI Suite IP buffers at startup. These include the graph buffer, which contains the weights, biases and configuration, and the input and output buffers for individual inference requests.

Instead of fully managing these buffers, the input-data buffer management is offloaded to the Nios processor. The Nios processor owns the Avalon streaming to memory-mapped mSGDMA, and the processor programs this DMA to push the formatted data into system memory.

As the buffers are allocated at startup, the input buffer locations are written into the mailbox. The Nios V processor then holds onto these buffers until a new set is received. All stream data is now constantly pushed into these buffers in a circular ring-buffer concept.

14.3.3.2. Streaming System Inference Job Management

In a M2M system, the host CPU handles pushing jobs into the FPGA AI Suite IP job queue by writing to the IP registers. In the streaming configuration, this task is offloaded to the Nios system and must be done in a coordinated way with the input buffer writing.

In streaming mode, the job-queue management is pushed to the mailbox instead of being managed by the host application. The job queue entry is then received by the Nios processor. After an input buffer is written, the mSGDMA interrupts the Nios processor, and the Nios processor now pushes one job into the FPGA AI Suite IP.

For every buffer stored by the mSGDMA, the Nios processor attempts to start another job.

For more details about the Nios V Stream Controller and the mailbox communication protocol, refer to Using FPGA AI Suite in Video Applications on page 323.

14.3.4. Resolving Input Rate Mismatches Between the FPGA AI Suite IP and the Streaming Input

When designing a system, the stream buffer rate should be matched to the FPGA AI Suite IP inferencing rate, so that the input data does not arrive faster than the IP can process it.

The SoC design example has safeguards in the Nios subsystem for when the input data rate exceeds the FPGA AI Suite processing rate.

To prevent input buffer overflow (potentially writing to memory still being processed by the FPGA AI Suite IP), the Nios subsystem has a buffer dropping technique built into it. If the subsystem detects that the FPGA AI Suite IP is falling behind, it starts dropping input buffers to allow the IP to catch up.

Using mailbox commands, the host application can check the queue depth level of the Nios subsystem and see if the subsystem needs to drop input data.

Depending on the buffer processing requirements of a design, dropping input data might not be considered a failure. It is up to you to ensure that the IP inference rate meets the needs of the input data.

If buffer dropping is not desired, you can try to alleviate buffer dropping and increase FPGA AI Suite IP performance with the following options:

- Configure a higher performance `.arch` file (IP configuration), which requires more FPGA resource. The `.arch` can be customized for the target machine learning graphs.
- Increase the system clock-speed.
- Reduce the size of the machine learning network, if possible.
- Implement multiple instances of the FPGA AI Suite IP and multiplex input data between them.

14.3.5. The Layout Transform IP as an Application-Specific Block in the SoC Design Example

For Agilex 5 and Arria 10, the layout transformation IP in the S2M design is provided as RTL source as an example layout transformation within a video inferencing application.

For Agilex 7, the layout transform is part of the FPGA AI Suite IP. For details about layout transform within the FPGA AI Suite IP, refer to Transforming the Layout of Input Data on page 230.

The flexibility of the FPGA AI Suite and the scope of projects it can support means that a layout transformation IP cannot serve all inference applications.

Each target application typically requires its own layout transformation module to be designed. System architectures need to budget for this design effort within their project.

Input data to the FPGA AI Suite IP must be formatted in memory so that the data matches the structure of the IP PE array and uses FP16 values.

The structure of the PE array is defined by the architecture file and the `c_vector` parameter setting describes the number of layers required for the input buffer. Typical `c_vector` values are 8, 16, and 32.

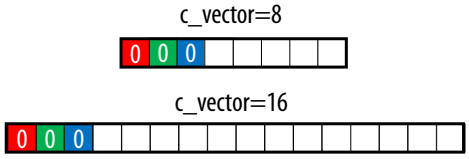
When considering streaming data, the `c_vector` can be viewed in comparison to the number of input channels of data that are present. For example, video has red, green, and blue channels that make up each pixel color. The following diagram shows how the video channels map to the input data stream required by the FPGA AI Suite IP.

Figure 46. Input Data Steam Mapping



The S2M design demonstrates an example of video streaming. Pixel data is sent through the layout-transform as RGB pixels, where each color is considered an input channel of data.

As the input data comprise only three channels of input data, the input data must be padded with zeros for any unused channels. The following diagram shows an example of two architectures, one with `c_vector` value of 8 and another with `c_vector` value of 16.



In the first example where `c_vector` is set to 8, the first pixel of RGB is placed on the input stream filling the first 3 channels, but there are 5 more channels remaining that must be initialized. These are filled with zero (represented by the white squares). This padded stream is then fed into the Nios subsystem.

This example layout transform does not support input folding. Input folding is an input preprocessing step that reduces the amount of zero padding in the `c_vector`. This folding then enables more efficient use of the dot product engine in the FPGA AI Suite IP. The efficiency gains can be significant depending on the graph and C_VEC. For more details, refer to Folding Input on page 226.

Related Information

Parameter Group: Global Parameters on page 173

14.3.5.1. Layout Transform Considerations

Pixels are typically 8-bit integer values, and the FPGA AI Suite requires FP16 values. As well as the `c_vector` padding, the layout transformation module converts the integer values to floating-point values.

The S2M example is a video-oriented demonstration. For networks such as ResNet50, the input pixel data must further be manipulated with a "mean" and "variance" value. The layout transformation module performs basic operation of Y=A*B+C operation on each pixel to meet the needs of a ResNet50 graph trained for ImageNet.

14.3.5.2. Layout Transform IP Register Map

The layout transform IP has the following register address space:

Table 61. Register Address Space of Layout Transformation IP

Register	Address Range	Description
Control	0x00	Main Control Register
C-Vector	0x04	Global C-Vector Control
Reserved	0x08 .. 0x03f	
Variance	0x40 .. 0x7f	Variance values per plane
Mean	0x80 .. 0xbf	Mean Values per plane
Reserved	0xc0 .. 0xff	

Control Register (0x00)

This is the global control register. When altering other CSR registers, software should issue a reset to the LT module via this register to commission the new settings. The stream then generates outputs based on the new settings and flush out all stale data.

Attempting to alter the configuration registers of the LT during active streaming generates undefined output data.

Table 62. Control Register Layout

Bit Location	Register Description	Attributes
0	**Reset** '1' = **In reset**: All streaming input data is discarded and no output data generated. '0' = **Running**: Streaming input data produces LT output data.	RW
31:1	Reserved	RO

C-Vector Register (0x04)

This register should be configured to match the C-Vector value of the architecture built into the FPGA AI Suite. This value need only be written once at startup as the architecture of the FPGA AI Suite is fixed at build time.

Table 63. C-Vector Register Layout

Bit Location	Register Description	Attributes
5:0	**C-vector** Value must match architecture of the DLA as defined in the .arch file	RW
31:6	Reserved	RO

Variance Registers (0x40 .. 0x7F)

Each plane has a unique variance value. Software must configure a value for each plane. The values are stored in FP32 format.

There are 16 registers in this section, where each register relates to a given plane. This register is write-only and returns 0xFFFFFFFF when reading.

Table 64. Variance Registers Layout

Bit Location	Register Description	Attributes
31:0	**FP32 formatted Variance value per plane**	WO

Mean Registers (0x80 .. 0xBF)

Each plane has a unique a mean value. Software must configure a value for each plane. The values are stored in FP32 format.

There are 16 registers in this section, where each register relates to a given plane. This register is write-only and returns 0xFFFFFFFF when reading.

Table 65. Mean Registers Layout

Bit Location	Register Description	Attributes
31:0	**FP32 formatted Mean value per plane**	WO

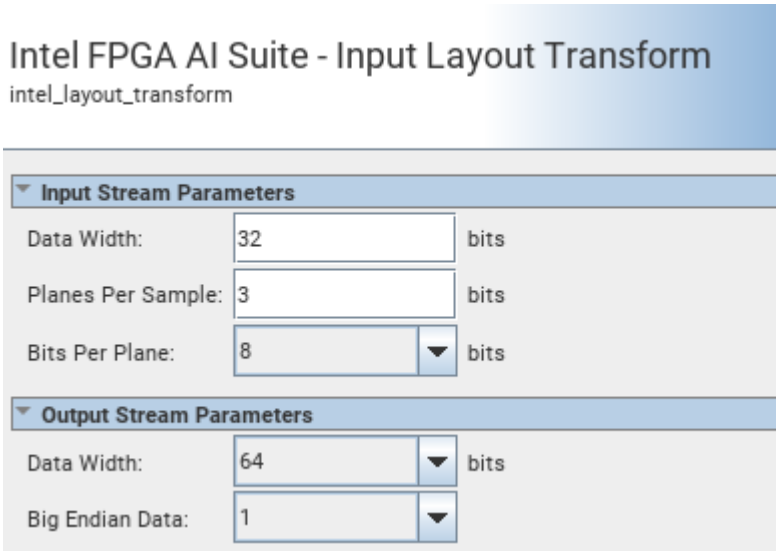
14.3.5.3. Layout Transform Configuration Options

For SoC design examples other than the Agilex 7 S2M design, the example layout transform has a range of parameters to adjust to the data width based on the number of input planes being processed.

A maximum of 16 CSR mean and variance values are supported. The **Planes per sample** field sets this upper threshold.

All output data is in FP16 format which is the expected input format for the FPGA AI Suite.

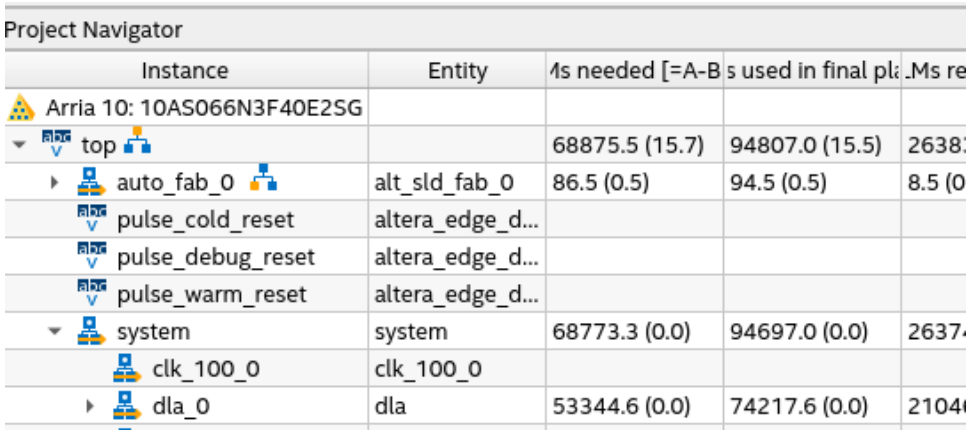
The Input Layout Transform IP is not required for the Agilex 7 S2M design.



14.4. Top Level

After the Quartus Prime project has finished compiling, the design should look similar to the following image in the Quartus Prime Project Navigator:

Figure 47. SoC Design Example Hierarchy



The top-level Verilog file and HPS configuration is derived directly from the GSRD designs located at the Altera FPGA Developer Site (https://altera-fpga.github.io) or RocketBoards.org:

- For more information about the GSRD for the Agilex 5 FPGA E-Series 065B Modular Development Kit, refer to the following URL: https://altera-fpga.github.io/latest/embedded-designs/agilex-5/e-series/modular/gsrd/ug-gsrd-agx5e-modular/.
- For more information about the GSRD for the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit, refer to the following URL: https://altera-fpga.github.io/latest/embedded-designs/agilex-7/i-series/soc/gsrd/ug-gsrd-agx7i-soc/.
- For more information about the GSRD for the Arria 10 SX SoC FPGA Development Kit, refer to the following URL: https://www.rocketboards.org/foswiki/Documentation/arria10SoCGSRD.

The GSRD designs have been modified to include the FPGA AI Suite IP. All unnecessary logic has been removed, which provides a concise design example.

The main FPGA AI Suite SoC design example is contained within a single Platform Designer system, called **system**. Double-click this node in the Quartus Prime Project Navigator to launch Platform Designer.

Related Information

- GSRD for Agilex 5 E-Series Modular Development Kit at Altera FPGA Developer Site
- GSRD for Agilex 7 I-Series Transceiver-SoC DevKit (4x F-Tile) at Altera FPGA Developer Site
- Arria 10 SoC GSRD at RocketBoards.org

14.4.1. Clock Domains

There are three main clocks within this design. All the clocks are considered asynchronous to each other. The SDC file provided has the clocking constraints for this design.

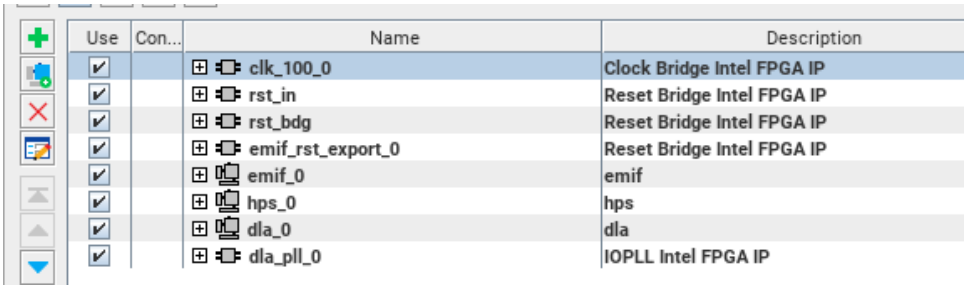
The design clocks are as follows:

Table 66. SDC Clock Domains for SoC Design Example

Clock	Clock Description	Agilex 5 Design Clock Frequency	Agilex 7 Design Clock Frequency	Arria 10 Design Clock Frequency
Board clock	This clock is used for all mSGDMA infrastructure and CPU CSR interfaces. The HPS AXI interfaces all run off this clock.	100 MHz	100 MHz	100 MHz
DLA clock	This clock is used only by the FPGA AI Suite IP. It feeds the `dla_clk` pin and is used inside FPGA AI Suite IP PE array.	200 MHz	400 MHz	200 MHz
DDR clock	This is used for the DDR controller and interconnect between the DLA and DDR. This interface is used by the DLA to transfer workloads back and forth to system memory.	200 MHz	333 MHz	266 MHz

14.5. The SoC Design Example Platform Designer System

At the center of the SoC design example is the Platform Designer system.



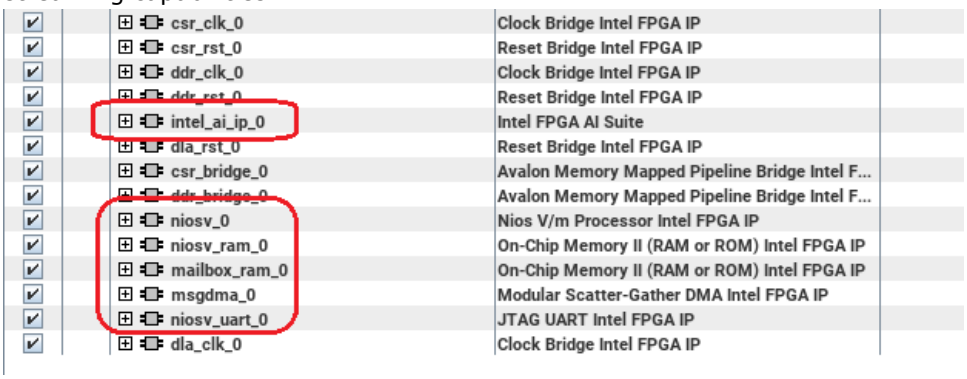
In Platform Designer, the SoC design example is separated into three hierarchical layers, the:

- **emif_0** : This layer contains the FPGA DDR4 External Memory Interface
- **dla_0** : This layer contains all the DLA IP and infrastructure IP
- **hps_0** : This layer contains all the ARM-HPS, ARM-EMIF and infrastructure IP for the ARM

The division of hierarchy demonstrates the sections of the design that are relevant to the solution. For example, if you want to target another board with a different external memory interface, you need to edit only the **emif_0** layer.

14.5.1. The dla_0 Platform Designer Layer (`dla.qsys`)

The **dla_0** layer contains the FPGA AI Suite IP and the Nios V subsystem to provide streaming capabilities.

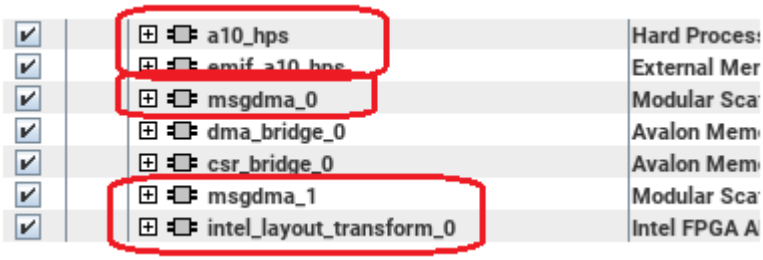


When incorporating the FPGA AI Suite IP into a custom design, you can use the `dla.qsys` file as a starting point for the new design.

14.5.2. The hps_0 Platform Designer Layer (`hps.qys`)

The **hps_0** layer contains the HPS, an mSGDMA instance (`msgdma_0`) for the FPGA AI Suite runtime, and an mSGDMA instance (`msgdma_1`) for the streaming generation app (S2M variant only).

The example layout transform is also located here and can be replaced by your version.



14.6. Fabric EMIF Design Component

The design provides a 266MHz DDR4-64Bit Avalon-based memory controller. This EMIF is used solely by the DLA.

The FPGA AI Suite IP memory interface is configured to be 512 bits wide. The EMIF interface is setup to complement this configuration.

14.7. PLL Configuration

The FPGA AI Suite IP is designed to operate at high f_{MAX} rates in Altera FPGA devices. The SoC design example provides an IOPLL that provides the IP with a fast clock.

In the design example the external board 100 MHz reference clock is fed into the PLL and a 200 MHz output clock is produced. This clock feeds the FPGA AI Suite IP directly.

You can remove or alter this PLL if you want to profile different performance curves of the system.

The FPGA AI Suite `dla_benchmark` application has no runtime method to dynamically determine the PLL operating frequency in the SOC design. The frequency of 200 MHz has been set as a constant in this application source code. If you alter the PLL frequency, you must also alter the `dla_benchmark` application to match the new clock frequency. This matching ensures that the benchmark metrics accurately reflect the performance.

See `dla_mmd_get_coredla_clock_freq` in `$COREDLA_WORK/runtime/coredla_device/mmd/hps_platform/acl_hps.cpp` and change the return value accordingly.

14.8. Yocto Build and Runtime Linux Environment

The Linux runtime and build environment is based on the Yocto build system. Yocto uses a build model based on the concept of layers and recipes.

The Yocto layers and recipes for FPGA AI Suite SoC design example are in `$COREDLA_ROOT/hps/ed4/yocto/`. The recipes extend the standard Golden System Reference Design (GSRD) that is used as the basis for the SoC design example system. For further details on setting up Yocto for SoC devices see the GSRD documentation.

The rest of this section describes key recipes in the `metal-intel-fpga-coredla` layer.

Related Information

- https://yoctoproject.org
- GSRD for Agilex 5 E-Series Modular Development Kit at Altera FPGA Developer Site
- GSRD for Agilex 7 I-Series Transceiver-SoC DevKit (4x F-Tile) at Altera FPGA Developer Site
- Arria 10 SoC GSRD at RocketBoards.org

14.8.1. Yocto Recipe: `recipes-core/images/coredla-image.bb`

This Yocto recipe customizes the image used on the SoC design example.

The `IMAGE_INSTALL:append` section defines extra packages for the FPGA AI Suite SoC design example. In particular, the `msgdma-userio`, `uio-devices`, and `kernel-modules` recipes are enabled to support the UIO and mSGDMA. The `WKS_FILES:*` section specifies which definition is used for building the SD card image.

14.8.2. Yocto Recipe: `recipes-bsp/u-boot/u-boot-socfpga_%.bbappend`

This Yocto recipe appends to the `meta-intel-fpga/recipes-bsp` recipe and enables the FPGA to SDRAM bridge, if it is required for the device target. This bridge is not required for Arria 10 designs.

On devices that require the bridge, the bridge allows mSGDMA to access the HPS SDRAM. This access exposes the full HPS SDRAM to the FPGA device.

14.8.3. Yocto Recipe: `recipes-drivers/msgdma-userio/msgdma-userio.bb`

This Yocto recipe enables an out-of-kernel (userspace) build of the `msgdma-userio` character driver used for transfer data to and from the HPS to the FPGA DDR memory. This driver calls into the DMA_ENGINE API exposed by the `altera-msgdma` kernel driver. The kernel module `altera-msgdma.ko` is enabled in `recipe-kernel/linux` kernel configuration file: `recipes-kernel/linux/files/enable-coredla-mod.cfg`.

Device tree settings are set to assign base address, IRQ for the `altera-msgdma` and associated to the `msgdma-userio` driver. These can be found in `recipes-kernel/linux/files/coredla-dts.patch`.

14.8.4. Yocto Recipe: `recipes-drivers/uio-devices/uio-devices.bb`

This Yocto recipe installs a service which starts up the uio drivers within the system.

The `uio_pdrv_genirq` driver provides user mode access to mapping and umapping CSR registers in the FPGA AI Suite IP.

The kernel modules `uio.ko` and `uio_pdev_genirq.ko` are enabled in the `recipe-kernel/linux` kernel configuration file: `recipes-kernel/linux/files/enable-mod.cfg`.

The device tree settings are set to assign base addresses and IRQs for the FPGA AI Suite IP, the stream controller, and the layout transform module. These can be found in `recipes-kernel/linux/files/coredla-dts.patch`.

14.8.5. Yocto Recipe: `recipes-kernel/linux/linux-socfpga-lts_%.bbappend`

This Yocto recipe applies the `0001-altera-msgdma.patch`, which does the following fixes:

- Set the FPGA DDR `src` and `dest` addresses to allow memory to and from the device to work correctly.
- Fixes the calculation of the number of descriptors used for a transfer on an Arria 10 device.

For Agilex 7 devices, the `agilex-dts.patch` patch enables necessary drivers in the device tree.

For Arria 10 devices, the `coredla-dts.patch` patch enables necessary drivers in the device tree.

This recipe also includes `enable-coredla-mod.cfg`, which is the kernel configuration file to enable `altera-msgdma driver`, `uio`, and `uio_pdrv_genirq` drivers.

14.8.6. Yocto Recipe: `recipes-support/devmem2/devmem2_2.0.bb`

This Yocto recipe downloads, compiles, and installs The `devmem2` utility from https://github.com/radii/devmem2 for use in debugging designs. The `devmem2` utility is a simple program to read/write from/to any memory location.

14.8.7. Yocto Recipe: `wic`

This Yocto recipe contains four files that define the layout of the SD card image. Each file defines the layout for one of the following devices: Agilex 5, Agilex 7, Arria 10, and Stratix 10 (not currently supported by the SoC design example). The partitions are as follows:

vfat	Storage for the Linux kernel, device tree, FPGA image, and u-boot
ext4	Root file system
raw	Arria 10 only. A custom raw partition labeled “a2”. This is used for the first-stage boot loader.

14.9. SoC Design Example MMD Layer Hardware Interaction Library

The FPGA AI Suite runtime uses the MMD layer to interact with the memory-mapped device. In the SoC design example, the MMD layer communicates via Linux kernel drivers.

Runtime communication with CSR registers for the stream controller and for the FPGA AI Suite IP happens via the UIO driver. See The Userspace I/O HOWTO for more information on the UIO communication model.

FPGA AI Suite IP graph weights and instructions (from the AOT file) are transferred from host DDR memory to EMIF DDR memory (allocated to the FPGA FPGA AI Suite IP) via the mSGDMA-USERIO driver (a custom kernel driver, see Yocto Recipe: recipes-drivers/msgdma-userio/msgdma-userio.bb on page 311). This driver is also used to send microcode and images. Lastly, inference results are transferred into host DDR via the mSGDMA-USERIO driver.

The source files for the library are in `runtime/coredla_device/mmd/hps_platform/`. The files contain classes for managing and accessing the FPGA AI Suite IP, the stream controller, the layout transform module, and DMA by UIO and MSGDMA-USERIO drivers. The remainder of this section describes the key classes.

Related Information

FPGA AI Suite Runtime on page 46

14.9.1. MMD Layer Hardware Interaction Library Class `mmd_device`

This class has the following responsibilities:

- Acquire the FPGA AI Suite IP, the stream controller (S2M only) and the mSGDMA
- Register interrupt callback for the FPGA AI Suite IP
- Provide read/write CSR and DDR functionality to the FPGA AI Suite runtime
- Linux device discovery.

The class attempts discovery of the following UIO devices, each from the `/sys/class/uio/ui*/` namespace.

coredla0	This represents the FPGA AI Suite IP CSR registers.
stream_controller0	If present (S2M only), this represents the Stream Controller CSR registers.
layout_transform0	If present (S2M only), this represents the Layout Transform CSR registers. Note that this device is not directly controlled from the runtime.

The class also attempts to discover the following mSGDMA-USERIO devices.

/dev/msgdma_coredla0	This represents the mSGDMA used to transfer weights, instructions, microcode, and, during M2M operation, image data.
/dev/msgdma_stream0	This represents the mSGDMA used to transfer images to the Layout Transform during the S2M mode of operation.

14.9.2. MMD Layer Hardware Interaction Library Class `uio_device`

The Linux device tree is used for UIO devices. The following responsibilities are assumed by this file:

- Acquire and maps/unmap the FPGA CSRs to user mode, using the `sysfs` entries for UIO (`/sys/class/uio/uio*/`).
- Acquire and register a callback for the interrupt from the FPGA AI Suite IP.
- Provide `read_block()` and `write_block()` functions for accessing the CSRs.

14.9.3. MMD Layer Hardware Interaction Library Class `dma_device`

The Linux device tree is used for mSGDMA-USERIO devices. These devices provide a Linux character device for simple read/write access to/from the FPGA EMIF via `fseek()`, `fread()`, and `fwrite()`.

The following responsibilities are assumed by this class:

- Acquire the mSGDMA-USERIO driver interface.
- Provide `read_block()` and `write_block()` functions for transfers from and to the FPGA AI Suite assigned DDR memory

14.10. FPGA AI Suite SoC Design Example Run Process

This section describes the steps to run the demonstration application and perform accelerated inference using the SoC design example.

14.10.1. Exporting Trained Graphs from Source Frameworks

Before running any demonstration application, you must convert the trained model to the OpenVINO intermediate representation (IR) format (`.xml`/`.bin`) with the OpenVINO Model Converter.

For details on creating the `.xml`/`.bin` files, refer to Preparing a Model for the PCIe Design Example on page 130, which describes how to create `.xml`/`.bin`) files for ResNet50.

The rest of this guide assumes that the same file locations and file names are used as in Preparing OpenVINO Model Zoo for the PCIe Design Example on page 130.

The `stream_image_app` used for the S2M variant of the SoC design example assumes that images are 224x224. For details, refer to The image_streaming_app Application on page 330.

14.10.2. Compiling Exported Graphs Through the FPGA AI Suite

The network as described in the `.xml` and `.bin` files (created by the Model Converter) is compiled for a specific FPGA AI Suite architecture file by using the FPGA AI Suite compiler.

The FPGA AI Suite compiler compiles the network and exports it to a `.bin` file with the format required by the OpenVINO Inference Engine. For instructions on how to compile the `.xml` and `.bin` files into AOT file suitable for use with the FPGA AI Suite IP, refer to Compiling the Graphs for the SoC Design Example on page 127

This `.bin` file created by the compiler contains the compiled network parameters for all the target devices (FPGA, CPU, or both) along with the weights and biases. The inference application imports this file at runtime.

The FPGA AI Suite compiler can also compile the graph and provide estimated area or performance metrics for a given architecture file or produce an optimized architecture file.

For the demonstration SD card, the FPGA bitstream has been built using one of the following IP architecture configuration files, so the architecture file for your development kit for compiling the OpenVINO™ Model:

- Agilex 5 FPGA E-Series 065B Modular Development Kit

  ```
  AGX5_Performance.arch
  ```

- Agilex 7 FPGA I-Series Transceiver-SoC Development Kit

  ```
  AGX7_Performance_LayoutTransform.arch
  ```

- Arria 10 SX SoC FPGA Development Kit

  ```
  A10_Performance.arch
  ```

For more details about the FPGA AI Suite compiler, refer to the Compiling Your Model with the FPGA AI Suite Compiler on page 193.

14.11. FPGA AI Suite SoC Design Example Build Process

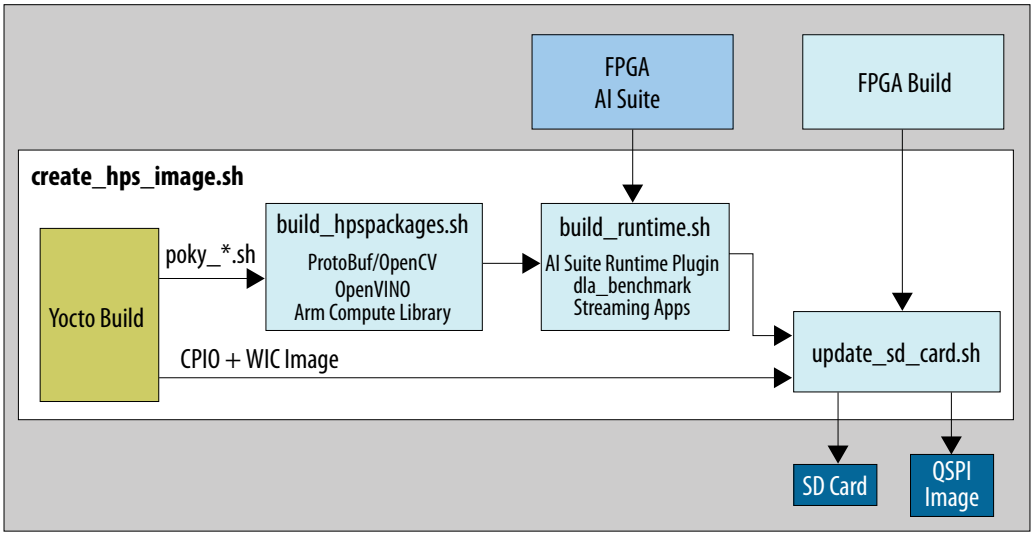
The SoC design example is built around a complete software and hardware solution.

The main building stages are:

- Build a Linux Distribution for a supported SoC FPGA development kit using a cross-compilation flow on a Linux system.
- Build the SoC design example Quartus Prime project
- Combine the FPGA and SoC Linux application and kernel onto an SD card

The following diagram illustrates the overall build process.

Figure 48. FPGA AI Suite SoC Design Example Build Process



14.11.1. Building the Quartus Prime Project

This design example includes prepackaged bitstreams but you can also use the build script provided to build bitstreams with custom architectures or recreate the original bitstreams, subject to IP license limitations.

The Quartus Prime project consists of the FPGA AI Suite IP as well as IP to interface with the HPS and, in the case of the S2M variant, additional flow control IP.

Building the Bitstream and Configuring Your Board for the FPGA AI Suite SoC Design Example on page 101 shows how to use a specific architecture configuration file for the FPGA AI Suite IP, but you can use any other device-appropriate configuration file instead.

14.11.1.1. Quartus Prime Build Flow

All FPGA AI Suite design examples are built by running the `dla_build_example_design.py` command.

This command generates an FPGA AI Suite IP from the provided architecture file, creates an Quartus Prime build directory, builds the Quartus Prime project, and produces a bitstream.

For more information about this command, refer to The dla_build_example_design Command on page 39

You can build the following SoC design example variants with the `dla_build_example_design.py` command:

Table 67. SoC Design Example Variant

Design Example Variant Identifier	Description	Layout Transform
agx5_soc_m2m	Builds a memory-to-memory (M2M) design for the Agilex 5 FPGA E-Series 065B Modular Development Kit	External demonstration transform as described in The Layout Transform IP as an Application-Specific Block in the SoC Design Example on page 303.
agx5_soc_s2m	Builds a memory-to-memory (M2M) design for the Agilex 5 FPGA E-Series 065B Modular Development Kit	External demonstration transform as described in The Layout Transform IP as an Application-Specific Block in the SoC Design Example on page 303.
agx7_soc_m2m	Builds a memory-to-memory (M2M) design for the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit	External demonstration transform as described in The Layout Transform IP as an Application-Specific Block in the SoC Design Example on page 303.
agx7_soc_s2m	Builds a streaming-to-memory (S2M) design for the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit	FPGA AI Suite IP input layout transform must be enabled in the architecture file. For information about the input layout transform, refer to Transforming the Layout of Input Data on page 230.
a10_soc_m2m	Builds a memory-to-memory (M2M) design for the Arria 10 SX SoC FPGA Development Kit	External demonstration transform as described in The Layout Transform IP as an Application-Specific Block in the SoC Design Example on page 303.
a10_soc_s2m	Builds a streaming-to-memory (S2M) design for the Arria 10 SX SoC FPGA Development Kit	External demonstration transform as described in The Layout Transform IP as an Application-Specific Block in the SoC Design Example on page 303.

An example of building the Arria 10 S2M variant with the `A10_Performance` architecture is as follows:

```
dla_build_example_design.py build \
   --output-dir $COREDLA_WORK/a10_perf_bitstream \
   -n 1 \
   a10_soc_s2m \
   $COREDLA_ROOT/example_architectures/A10_Performance.arch
```

After the design is built, the output products (`.sof` or `.rbf` files) must be combined with the SoC Linux system in order to be used. This is done in one of the steps in the `create_hps_image.sh` script.

For Agilex 5 and Agilex 7, the `.sof` file is combined with a `u-boot-spl-dtb.hex` file to create either a bootable `.sof` file or a `.jic` file that can program the flash memory.

For Arria 10, the `.rbf` files are added to the `.wic` image so that the FPGA device can be programmed from the SD card.

If you attempt to reprogram a running Linux system with a new `.sof` file, the Linux system crashes and the reprogramming results in an unpredictable outcome.

The FPGA device is programmed by booting the Linux system on the SoC via the SD card (for Arria 10) or by programming over JTAG (for Agilex 5 and Agilex 7), which then boots the Linux system from the SD card. For details about creating a functional solution by combining the build `.rbf` files with the SD card image or by creating the bootable `.sof` or `.jic` files, refer to Building the Bootable SD Card Image (.wic) on page 319.

14.11.1.1.1. Build Synchronization of FPGA with Software

For a system to function correctly, the release version of each FPGA AI Suite component, including the compiler, the runtime, and the FPGA AI Suite IP, must match.

In addition, the AOT file created by the FPGA AI Suite `dla_compiler` command must target the same architecture (`.arch`) file as the FPGA AI Suite IP.

When Quartus Prime compiles the FPGA AI Suite IP, it generates a build-hash that is embedded into the IP. The runtime software checks this build-hash during runtime and if the hashes do not match then the application aborts and displays a mismatch error.

The FPGA AI Suite SoC design example is always built with only one instance of the FPGA AI Suite IP.

14.11.1.2. Build Script Options

The options available in the `dla_build_example_design.py` script are described in The dla_build_example_design Command on page 39.

14.11.1.3. Build Directory

The `dla_build_example_design.py` command creates a Quartus Prime build in the `hw` folder in the directory that you specify with the `--output-dir` command option. The project is named `top.qpf`. You can open this project in Quartus Prime software to review the build logs.

Use the `dla_build_example_design.py` command to rebuild parts of the design example if you alter the design.

The build directory contains the following additional script file:

- `build_stream_controller.sh`: This script creates the Nios V `.hex` file. This file holds the compiled Nios software that is embedded into the Nios subsystem.

14.11.1.3.1. The `build_stream_controller.sh` Script

The `build_stream_controller.sh` script builds the Nios V application, and then generates a `.hex` file. The `.hex` file is embedded into the Nios V RAM by the Quartus Prime project. This file must be called `stream_controller.hex` and it must reside alongside the Quartus Prime project files (`top.qpf`).

This script has three command line options that must be entered in order:

```
build_stream_controller <quartus-project-file> <qsys system> <output-file>
```

This script is called as part of the `create_project.bash`, but you can call it manually if you modify the Nios V source code and need a new `.hex` file. The build script is typically called with the same options.

```
build_stream_controller.sh top.qpf qsys/dla.qsys stream_controller.hex
```

14.11.2. Building the Bootable SD Card Image (`.wic`)

To create a bootable SD Card image (`.wic` file), the following components must be built:

- Yocto Image
- Yocto SDK Toolchain
- Arm Cross-Compiled OpenVINO
- Arm Cross-Compiled FPGA AI Suite Runtime Plugin
- Arm Cross-Compiled Demonstration Applications
- FPGA `.sof`/`.rbf` files

The `create_hps_image.sh` script performs the complete build process and combines all the necessary components into an SD card image that can be written to an SD card. For steps required to write an SD card image to an SD card, refer to Writing the SoC Design Example SD Card Image (.wic) to an SD Card on page 109.

The SD card image is assembled using Yocto (https://yoctoproject.org).

You must have a build system that meets the minimum build requirements. For details, refer to https://docs.yoctoproject.org/5.0.5/ref-manual/system-requirements.html#supported-linux-distributions.

The commands to install the required packages are shown in Installing HPS Disk Image Build Prerequisites on page 104.

To perform the build, run the following commands:

```
cd $COREDLA_WORK/runtime
./create_hps_image.sh \
   -f <bitstream_directory>/hw/output_files \
   -o <output_directory> \
   -u \
   -m <FPGA_target>
```

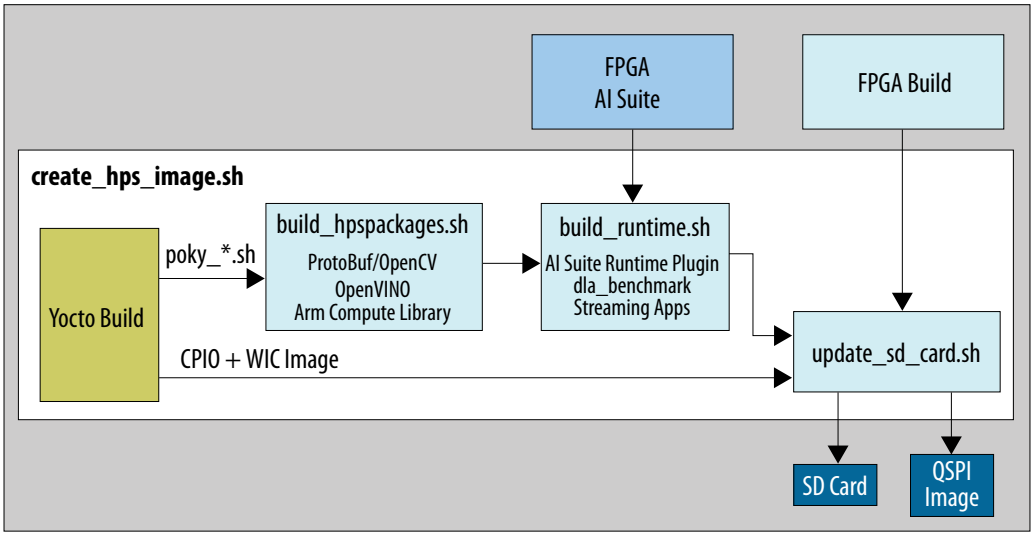
where *<FPGA_target>* is `agilex5_mk_a5e065bb32aes1`, `agilex7_dk_si_agi027fa`, or `arria10`.

The `create_hps_image.sh` script performs the following steps:

1. Build the Yocto boot SD card image and Yocto SDK toolchain on page 320
2. Build the HPS Packages on page 321
3. Build the runtime on page 321
4. Update the SD card image on page 321

The following diagram illustrates the overall build process performed by the `create_hps_image.sh` script:

Figure 49. FPGA AI Suite SoC Design Example Build Process



Build the Yocto boot SD card image and Yocto SDK toolchain

The FPGA AI Suite Soc design example uses the Yocto Project Poky Distribution.

The Yocto images are based on Golden System Reference Designs, which you can find at the following URL: https://altera-fpga.github.io/.

To customize the Yocto Poky distribution, modify the recipes found in layer `$COREDLA_ROOT/hps/ed4/yocto/meta-intel-coredla`.

More details can be found in Yocto Build and Runtime Linux Environment on page 310.

The defined Yocto Image recipe is `coredla-image` and can be found in `$COREDLA_ROOT/hps/ed4/yocto/meta-intel-coredla/recipes-image/coredla-image.bb`.

A Yocto SDK is also built as part of the build and this SDK is used in subsequent build steps to cross-compile the software for the Arm HPS subsystem:

- **Agilex 5**:

  `$COREDLA_WORK/runtime/build_Yocto/build/tmp/deploy/sdk/poky-glibc-x86_64-coredla-image-armv8a-agilex5_mk_a5e065bb32aes1-toolchain-4.2.3.sh`

  Agilex 7:

  `$COREDLA_WORK/runtime/build_Yocto/build/tmp/deploy/sdk/poky-glibc-x86_64-coredla-image-armv8a-agilex7_dk_si_agi027fa-toolchain-4.2.3.sh`

- **Arria 10**:

  `$COREDLA_WORK/runtime/build_Yocto/build/tmp/deploy/sdk/poky-glibc-x86_64-lbs-image-poky-cortexa9t2hf-neon-arria10-toolchain-4.1.2.sh`

The SD card image (WIC file) is in the following location:

- **Agilex 5**:

  `$COREDLA_WORK/runtime/build_Yocto/build/tmp/deploy/images/agilex5_mk_a5e065bb32aes1/*`

- **Agilex 7**:

  `$COREDLA_WORK/runtime/build_Yocto/build/tmp/deploy/images/agilex7_dk_si_agi027fa/*`

- **Arria 10**:

  `$COREDLA_WORK/runtime/build_Yocto/build/tmp/deploy/images/arria10/*`

By default, the `create_hps_image.sh` script builds Yocto from scratch. However, if a prebuilt Yocto build folder is available, you can specify the prebuilt Yocto folder via the `-y` option as follows:

```
./create_hps_image.sh \
   -y <prebuilt_Yocto_directory>
   -f <bitstream_directory>/hw/output_files \
   -o <output_directory>
   -u
   -m <FPGA_target>
```

where *<FPGA_target>* is `agilex5_mk_a5e065bb32aes1`, `agilex7_dk_si_agi027fa`, or `arria10`.

This `-y` option loads the Yocto SDK from *<PREBUILT_YOCTO_DIR>*`/build/tmp/deploy/sdk/` and the `.wic` image from *<PREBUILT_YOCTO_DIR>*`/build/tmp/deploy/images/`*<FPGA_target>*`/` without rerunning a Yocto build.

Build the HPS Packages

The HPS packages are built by the `build_hpspackages.sh` script.

This script cross-compiles OpenCV, OpenVINO, and the Arm-based OpenVINO runtime plugin.

Build the runtime

The runtimes are built by the `build_runtime.sh` script.

This script cross-compiles the OpenVINO FPGA AI Suite runtime plugin and demonstration applications for the SoC devices.

Update the SD card image

If you specify the `-u` and `-f` options of the `create_hps_image.sh` command, the SD card image is updated by the `update_sd_card.sh` script.

The `update_sd_card.sh` script takes the output products from the previous build steps and builds a bootable SD Card image.

The software binaries are installed to the Ext4 partition under the `/home/root/app` directory.

For Arria 10, the `.rbf` files are used to create an RTL `fit_spl_fpga.itb` file is copied to the Fat32 partition.

For Agilex 5 and Agilex 7, the `.sof` file is combined with a `u-boot-spl-dtb.hex` file to create a bootable `.sof` file and a flashable `.jic` file.

For the commands that create the software binaries, review the `create_<FPGA_target>_fpga.sh` script, where <*FPGA_target*> is `agilex5_mk_a5e065bb32aes1`, `agilex7_dk_si_agi027fa`, or `arria10`. You can also run the commands found in the script manually, if needed.

You can skip updating the SD card while building the rest of the SoC Example Design by omitting the `-f` and `-u` options:

```
./create_hps_image.sh \
   -o <output_directory> \
   -m <FPGA_target>
```

where <*FPGA_target*> is `agilex5_mk_a5e065bb32aes1`, `agilex7_dk_si_agi027fa`, or `arria10`.

When you skip updating the SD card image, you can build bitstreams and an HPS image (Yocto, HPS packages, FPGA AI Suite runtime) concurrently. You can update the SD card image (`.wic` file) image after all the files are ready:

```
./create_hps_image.sh \
   -y ./build_Yocto
   -f <bitstream_directory>/hw/output_files \
   -o <output_directory> \
   -u \
   -m <FPGA_target>
```

where <*FPGA_target*> is `agilex5_mk_a5e065bb32aes1`, `agilex7_dk_si_agi027fa`, or `arria10`.

During regular development you might want to build the runtime software binaries and copy them over to the board app directory manually, without building the bitstream, updating the `.wic` image, or rebuilding Yocto:

```
./create_hps_image.sh \
   -y ./build_Yocto
   -o <output_directory> \
   -m <FPGA_target>
```

where <*FPGA_target*> is `agilex5_mk_a5e065bb32aes1`, `agilex7_dk_si_agi027fa`, or `arria10`.

15. Using FPGA AI Suite in Video Applications

A typical use case of the FPGA AI Suite IP is to run inferences on live input data. For instance, live data can come from a video source such as an HDMI IP core and stream to the FPGA AI Suite IP to perform image classification on each frame. You use the SoC design example S2M demonstration as an example.

For simplicity, the S2M demonstration only simulates a live video source. The streaming demonstration consists of the following applications that run on the target SoC device:

- **`streaming_inference_app`**

  This application loads and runs a network and captures the results.

- **`image_streaming_app`**

  This application loads bitmap files from a folder on the SD card and continuously sends the images to the EMIF, simulating a running video source

The images are passed through a layout transform IP that maps the incoming images from their frame buffer encoding to the layout required by the FPGA AI Suite IP.

There is a module called the stream controller that runs on a Nios V microcontroller that controls the scheduling of the source images to the FPGA AI Suite IP.

The `streaming_inference_app` application creates OpenVINO inference requests. Each inference request is allocated memory on the EMIF for input and output buffers. This information is sent to the stream controller when the inference requests are submitted for asynchronous execution.

In its running state, the stream controller waits for input buffers to arrive from the `image_streaming_app` application. When the buffer arrives, the stream controller programs the FPGA AI Suite IP with the details of the received input buffer, which triggers the FPGA AI Suite IP to run an inference.

When an inference is complete, a completion count register is incremented within the FPGA AI Suite IP CSRs. This counter is monitored by the currently executing inference request in the `streaming_inference_app` application, and is marked as complete when the increment is detected. The output buffer is then fetched from the EMIF and the FPGA AI Suite IP portion of the inference is now complete.

Depending on the model used, there might be further processing of the output by the OpenVINO HETERO plugin and OpenVINO Arm CPU plugin. After the complete network has finished processing, a callback is made to the application to indicate the inference is complete.

The application performs some post processing on the buffer to generate the results and then resubmits the same inference request back to OpenVINO, which lets the stream controller use the same input/output memory block again.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

15.1. Nios Subsystem

The stream controller module runs autonomously in a Nios V microcontroller. An interface to the module is created by the FPGA AI Suite OpenVINO plugin when an "external streaming" flag is enabled by the inference application. At startup, the interface checks that the stream controller is present by sending a ping message and waiting for a response.

The following sections describe details of the various messages that are sent between the plugin and the stream controller, along with their packet structure.

15.1.1. Stream Controller Communication Protocol

The FPGA AI Suite OpenVINO plugin running on the HPS system includes a `coredla-device` component which in turn has a stream controller interface if the "external streaming" flag is enabled by the inference application. This stream controller interface manages the communications from the HPS end to the stream controller microcode module running on the Nios V microcontroller.

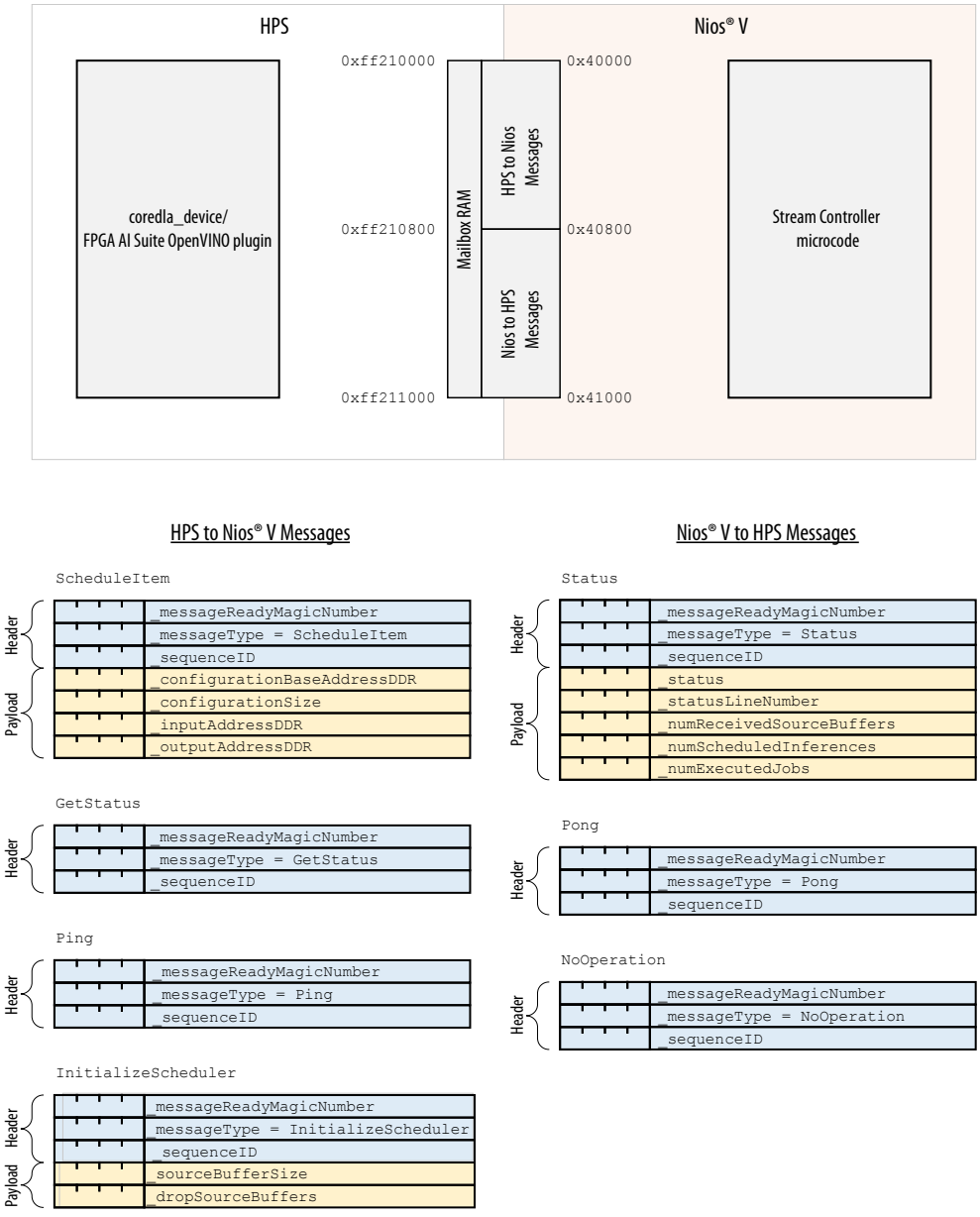
Messages are sent between the HPS and the Nios V microcontroller using the mailbox RAM which is shared between the two. In the HPS, this RAM is at physical address 0xff210000, and in the Nios V microcontroller, it is at address 0x40000. The RAM is 4K bytes. The lower 2K is used to send messages from the HPS to the Nios V microcontroller, and the upper 2K is used to send messages from the Nios V microcontroller to the HPS.

Message flow is always initiated from the HPS end, and the Nios V microcontroller always responds with a message. Therefore, after sending any message the HPS end waits until it receives a reply message. This can contain payload data (for example, status information) or just a "no operation" message with no payload.

Each message has a 3 x uint32_t header, which consists of a `messageReadyMagicNumber` field, a `messageType` field, and a `sequenceID` field. This header is followed by a payload, the size of which depends on the `messageType`. The `messageReadyMagicNumber` field is set to the value of `0x55225522` when the message is ready to be received

When a message is to be sent, all of the buffer apart from the `messageReadyMagicNumber` is first written to the mailbox RAM. The `sequenceID` increments by 1 with every message sent. Then the `messageReadyMagicNumber` is written. The sending end then waits for the value of `messageReadyMagicNumber` to change to the value of the `sequenceID`. This is set by the stream controller microcode module and indicates that the message has been received and processed by the receiving end.

Figure 50. Stream Controller Mailbox RAM and Message Packets



15.1.2. Buffer Flow in Streaming Mode using Nios V Software Scheduler

15.1.2.1. Review of M2M mode

To explain how the buffers are managed in streaming mode, it can help to review the existing flow for M2M mode.

The inference application loads source images from `.bmp` files into memory allocated from its heap. These buffers are 224x224x3 uint8_t samples (150528 bytes). During the load, the BGR channels are rearranged from interleaved channels into planes.

OpenVINO inference requests are created by the application using the inference engine. These inference requests allocate buffers in the on-board EMIF memory. The size of each of these buffers is the size of the input buffer plus the size of the output buffer. The input buffer size depends FPGA AI Suite IP parameters (specified in the `.arch` file) for which the graph was compiled.

The BGR planar image buffers are attached as input blobs to these OpenVINO inference requests, which are then scheduled for execution.

Preprocessing Steps

In M2M mode, the preprocessing steps are performed in software.

- The samples are converted to 32-bit floating point, and the mean G, B and R values of the imagenet dataset are subtracted from each sample accordingly.
- The samples are converted to 16-bit floating point.
- A layout transform then maps these samples into a larger buffer which has padding, in the layout expected by the FPGA AI Suite.

Inference Execution Steps

- The transformed image is written directly to board memory at its allocated address.
- The FPGA AI Suite IP CSR registers are programmed to schedule the inference.
- The FPGA AI SuiteOpenVINO plugin monitors the completion count register (located on the FPGA AI Suite IP), either by polling or receiving an interrupt, and waits until the count increments.
- The results buffer (2048 bytes) is read directly from the EMIF on the board to HPS memory.

Postprocessing Steps

- The samples in the results buffer (1001 16-bit floating point values) are converted to 32-bit floating point.
- The inference application receives these buffers, sorts them, and collects the top five results.

15.1.2.2. External Streaming Mode Buffer Flow

External streaming mode is enabled in the inference application by setting the configuration value `DLIA_CONFIG_KEY(EXTERNAL_STREAMING)` to `CONFIG_VALUE(YES)` in the OpenVINO FPGA plugin.

In streaming mode, the inference application does not handle any of the input buffers. It still must create inference requests, which allocate the input and output buffers in the EMIF memory as before, but no input blobs are attached to the inference requests.

When the inference request is executed, there are no preprocessing steps required, since they do not have any input blobs.

Inference Execution Steps

- Instead of writing a source buffer directly to its allocated address, a `ScheduleItem` command is sent to the Nios V stream controller which contains details of the input buffer EMIF address.
- The FPGA AI Suite IP CSR registers are **not** programmed by the plugin.
- The plugin waits for the completion count register to increment as before.
- The results buffer is read directly from the board as before.

Postprocessing Steps

- The samples in the results buffer (1001 16-bit floating point values) are converted to 32-bit floating point.
- The inference application receives these buffers, sorts them, and collects the top five results.
- The same inference request is rescheduled with the inference engine.

15.1.2.3. Nios V Stream Controller State Machine Buffer Flow

When the network is loaded into the `coredla_device`, if external streaming has been enabled, a connection to the Nios V processor is created and an `InitializeScheduler` message is sent. This message resets the stream controller and sets the size of the raw input buffers and the drop/receive ratio of buffers from the input stream.

The inference application queries the plugin for the number of inference requests to create. When scheduled with the inference engine, these send `ScheduleItem` commands to the stream controller, and a corresponding `CoreDlaJobItem` is created. The `CoreDlaJobItem` keeps details of the input buffer address and size and has flags to indicate if it has a source buffer and to indicate if it has been scheduled for inference on the FPGA AI Suite IP. The `CoreDlaJobItem` instances are handled as if they are in a circular buffer.

When the Nios V stream controller has received a `ScheduleItem` command from all of the inference requests and created a `CoreDlaJobItem` instance for each of them, it changes to a running state, which arms the mSGDMA stream to receive buffers, and sets a pointer `pFillingImageJob` that identifies which of the buffers is the next to be filled.

It then enters a loop, waiting for two types of event:

- A buffer is received through the mSGDMA, which is detected by a callback from an ISR.
- A message is received from the HPS.

New Buffer Received

The `pFillingImageJob` pointer is marked as now having a buffer.

If the next job in the circular buffer does not have a buffer, the `pFillingImageJob` pointer is moved on and the mSGDMA is armed again to receive the next buffer at the address of this next job.

If it does have a buffer, the FPGA AI Suite IP cannot keep up with the input buffer rate, so the `pFillingImageJob` does not move and the mSGDMA is armed to capture the next buffer at the same address. This means that the previous input buffer is dropped and is not processed by the FPGA AI Suite IP.

Buffers that have not been dropped can now be scheduled for inference on the FPGA AI Suite IP provided that the IP has fewer than two jobs in its pipeline.

Scheduling a job for execution means programming the CSR registers with the configuration address, the configuration size, and the input buffers address in DDR memory. This programming also sets the flag on the job so the controller knows that the job has been scheduled.

Message Received

If the message is a `ScheduleItem` message type then an inference request has been scheduled by the inference application.

This request happens only if a previous inference request has been completed and rescheduled. The number of jobs in the FPGA AI Suite IP pipeline has decreased by 1, so another job can potentially be scheduled for inference execution, providing it has an input buffer assigned.

If there are no jobs available with valid input buffers, then the FPGA AI Suite IP is processing buffers faster than they are being received by the mSGDMA stream, and consequently all input buffers are processed (that is, none are dropped).

15.2. Building the Stream Controller Module

The stream controller is built as part of the steps described in Installing HPS Disk Image Build Prerequisites on page 104. For system development that extends the FPGA AI Suite SoC design example, you might want to compile the stream controller module independently.

The stream controller module source code can be found in the distribution, in the `runtime/coredla_device/stream_controller/` directory.

There is a script `build.sh` in the source code directory that builds a binary `.hex` file. This file is then used by Quartus Prime when building the firmware to embed the microcode module.

The script should be run from a Nios V command shell, which is part of Quartus Prime. It requires a Quartus Prime project file and a Quartus Prime `.qsys` file. For this design example, the project file is `top.qpf`, and the Platform Designer file is `dla.qsys`.

An example command to build the stream controller module is as follows:

```
./build.sh top.qpf dla.qsys stream_controller.hex
```

15.3. Building the Streaming Demonstration Applications

The two streaming demonstration applications, `streaming_inference_app` and `image_streaming_app`, are built as part of the runtime and are included on the SD card image for the target device in the directory `/home/root/app/`.

15.4. Running the Streaming Demonstration

You need two terminals connected to the target device, one for each of the streaming applications.

One can be a serial terminal, and the other can be an SSH connection from a desktop PC. For details, refer to Running the S2M Mode Demonstration Application in the SoC Design Example on page 165.

15.4.1. The `streaming_inference_app` Application

The `streaming_inference_app` application is an OpenVINO-based application. It loads a given precompiled ResNet50 network, then creates inference requests that are executed asynchronously by the FPGA AI Suite IP.

The resulting tensors are captured from the EMIF using the mSGDMA controller. The postprocessing required in the software involves converting the output tensors to floating point, assigning the values to the appropriate image classification, sorting the results, and selecting the top 5 classification results.

For each inference, the result is displayed on the terminal, and the results for each inference up to the 1000th one are logged in a `results.txt` file in the application folder.

The application depends on the following shared libraries . The system build adds these libraries to the directory `/home/root/app` on the SD card image, along with the application binary and a `plugins.xml` file that defines the plugins available to OpenVINO.

- `libhps_platform_mmd.so`
- `libngraph.so`
- `libinference_engine.so`
- `libinference_engine_transformations.so`
- `libcoreDLAHeteroPlugin.so`
- `libcoreDlaRuntimePlugin.so`

You also need a compiled network binary file and an `.arch` file (which describes the FPGA AI Suite IP parameterization) to run inferences. These have been copied to the `/home/root/resnet-50-tf` directory.

For example, a ResNet50 model compiled for an Arria 10 might have the following files:

- `RN50_Performance_no_folding.bin`
- `A10_Performance.arch`

Before running the application, set the `LD_LIBRARY_PATH` shell environment variable to define the location of the shared libraries:

```
root@arria10-1ac87246f24f:~# cd /home/root/app
root@arria10-1ac87246f24f:~# export LD_LIBRARY_PATH=.
```

Use the `--help` of the `streaming_inference_app` command to display the command usage:

```
# ./streaming_inference_app -help
Usage:
        streaming_inference_app -model=<model> -arch=<arch> -device=<device>

Where:
        <model>    is the compiled model binary file, eg /home/root/resnet-50-tf/
RN50_Performance_no_folding.bin
        <arch>     is the architecture file, eg /home/root/resnet-50-tf/
A10_Performance.arch
        <device>   is the OpenVINO device ID, eg HETERO:FPGA or HETERO:FPGA,CPU
```

Start the streaming inference app with a command like this:

```
# ./streaming_inference_app \
-model=/home/root/resnet-50-tf/RN50_Performance_no_folding.bin \
-arch=/home/root/resnet-50-tf/A10_Performance.arch \
-device=HETERO:FPGA
```

The distribution includes a shell script utility called `run_inference_stream.sh` which calls this command above.

For Agilex 5 and Arria 10, the layout transform IP core does not support folding on the input buffer. For streaming, you must use models that have been compiled by the `dla_compiler` command with the `--ffolding-option=0` command line option specified.

For Agilex 7, folding is support on the input buffer.

15.4.2. The `image_streaming_app` Application

The `image_streaming_app` application loads images from the SD card, programs the layout transform IP, and then transfers the buffers via a streaming mSGDMA interface to the device. The buffers are sent at regular intervals at a frequency set by one of the command line options. Buffers are continually sent until the program is stopped with Ctrl+C.

The `image_streaming_app` application works only with `.bmp` files with dimensions of 224x224 pixels. The `.bmp` files can be either 24 or 32 bits per pixel format. If they are 24 bits, the buffers are padded to make them 32 bits per pixel. This format is expected by the input of the layout transform IP .

The command usage from the `-help` command option is as follows:

```
root@agilex7:~/app# ./image_streaming_app --help
Usage:
 image_streaming_app [Options]

Options:
-images_folder=folder     Location of bitmap files. Defaults to working folder.
-image=path               Location of a single bitmap file for single inference.
-send=n                   Number of images to stream. Default is 1 if -image is
set, otherwise infinite.
-rate=n                   Rate to stream images, in Hz. n is an integer. Default
is 30.
-width=n                  Image width in pixels, default = 224
-height=n                 Image height in pixels, default = 224
-c_vector=n               C vector size, default = 32
-blue_variance=n          Blue variance, default = 1.0
```

```
-green_variance=n          Green variance, default = 1.0
-red_variance=n            Red variance, default = 1.0
-blue_shift=n              Blue shift, default = -103.94
-green_shift=n             Green shift, default -116.78
-red_shift=n               Red shift, default = -123.68
```

The distribution includes a shell script utility called `run_image_stream.sh` that calls the `image_streaming_app` command with a rate of 50 Hz and default layout transform settings.

Once both applications are running, the `streaming_inference_app` application outputs the results to the terminal and a `results.txt` file.

Example output from the `streaming_inference_app` application is as follows:

```
root@agilex7:~/app# ./run_inference_stream.sh
Runtime version check is enabled.
[ INFO ] Architecture used to compile the imported model: AGX7_Performance
Using licensed IP
Read hash from bitstream ROM...
Read build version string from bitstream ROM...
Read arch name string from bitstream ROM...
Runtime arch check is enabled. Check started...
Runtime arch check passed.
Runtime build version check is enabled. Check started...
Runtime build version check passed.
Ready to start image input stream.
1 - class ID 776, score = 58.4
2 - class ID 968, score = 90.7
3 - class ID 769, score = 97.8
4 - class ID 769, score = 97.8
5 - class ID 872, score = 99.8
6 - class ID 954, score = 94.4
7 - class ID 954, score = 94.4
8 - class ID 776, score = 58.4
9 - class ID 872, score = 99.8
10 - class ID 968, score = 90.7
11 - class ID 776, score = 58.4
12 - class ID 968, score = 90.7
13 - class ID 769, score = 97.8
14 - class ID 769, score = 97.8
15 - class ID 872, score = 99.8
16 - class ID 954, score = 94.4
17 - class ID 954, score = 94.4
18 - class ID 776, score = 58.4
19 - class ID 872, score = 99.8
20 - class ID 968, score = 90.7
^C
Ctrl+C detected. Shutting down application
```

The resulting `results.txt` file from the example is as follows:

```
root@agilex7:~/app# cat results.txt
Result: image[1]
1. class ID 776, score = 58.4
2. class ID 683, score = 27.6
3. class ID 513, score = 10.2
4. class ID 432, score = 1.8
5. class ID 558, score = 1.6

Result: image[2]
1. class ID 968, score = 90.7
2. class ID 901, score = 1.1
3. class ID 868, score = 1.0
4. class ID 899, score = 1.0
5. class ID 725, score = 0.9
```

```
Result: image[3]
1. class ID 769, score = 97.8
2. class ID 845, score = 0.5
3. class ID 587, score = 0.4
4. class ID 798, score = 0.1
5. class ID 618, score = 0.1

Result: image[4]
1. class ID 769, score = 97.8
2. class ID 845, score = 0.5
3. class ID 587, score = 0.4
4. class ID 798, score = 0.1
5. class ID 618, score = 0.1
```

16. Using the FPGA AI Suite IP with High Bandwidth Memory on Stratix 10 MX and Agilex 7 M-Series Devices

The following sections describe how to use high bandwidth memory (HBM) with the FPGA AI Suite IP on Stratix 10 MX and Agilex 7 M-Series devices that have in-package HBM.

Development kits with Stratix 10 MX and Agilex 7 M-Series are available from Altera and third-party board vendors.

Related Information

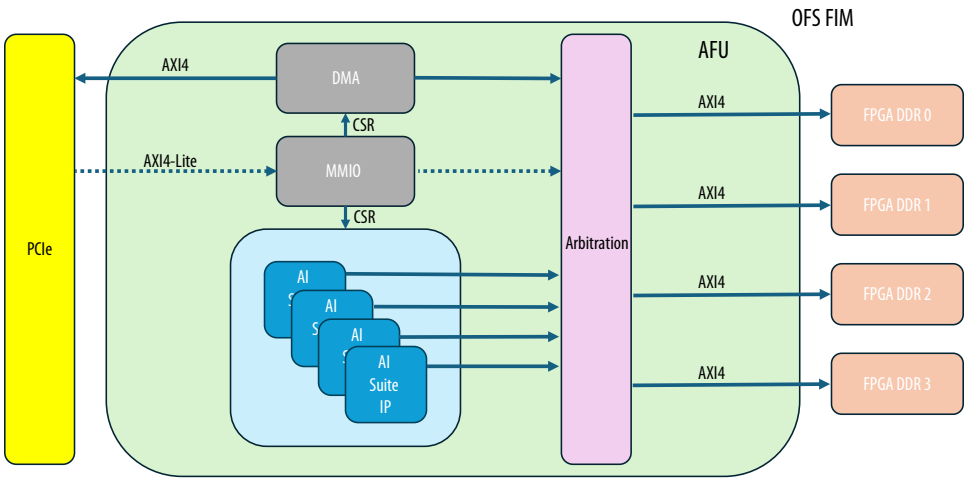
- Stratix 10 MX FPGA Development Kit
- Agilex 7 FPGA M-Series Development Kit - HBM2e Edition (3x F-Tile & 1x R-Tile)

16.1. Current FPGA AI Suite IP Implementation With DDR

The FPGA AI Suite IP, in its current implementation, has a 512-bit wide data and a 32-bit wide address bus and therefore can address a maximum of 4GB of external memory. The current FPGA AI Suite design examples all assume that the required memory is external on the PCB, either soldered on the PCB or as DDR4 modules in PCB DIMM slots.

The following figure shows an example of a system with four FPGA AI Suite IP instances connected to four external DDR memories with four separate AXI4 buses.

Figure 51. System with Four FPGA AI Suite IP Instances Connected to Four External DDR Memories



© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

16.2. HBM Connection Considerations

The HBM memory controller exposes 32 AXI4 pseudo channels for both stacks of HBM (that is, 16 pseudo channels with 256-bit data width for each stack). The 16 pseudo channels for a stack address 8 channels of 2 GB address space each per layer in the stack. This configuration allows for a very large bandwidth. However, the current implementation of the FPGA AI Suite IP, with only one AXI4 channel per IP, limits how much of the available bandwidth can be used by the FPGA AI Suite IP instances.

To take advantage of HBM with the FPGA AI Suite IP, consider the following aspects of your FPGA AI Suite IP memory requirements:

- Memory Size and Bandwidth Considerations
- Memory Data Width Considerations

16.2.1. Memory Size and Bandwidth Considerations

The first consideration when using HBM with the FPGA AI Suite IP is the 2 GB address space limitation created by the structure of the HBM memory. Evaluate whether 2 GB is sufficient memory for your FPGA AI Suite IP instances.

Consider the following techniques to meet the memory size and bandwidth requirements of your FPGA AI Suite IP instances in HBM:

- Direct Mapping of an FPGA AI Suite IP Instance to an HBM Channel on page 334
- Channel Stitching on page 334
- Channel Stitching and Interleaving on page 335

Direct Mapping of an FPGA AI Suite IP Instance to an HBM Channel

If the 2 GB size per channel is enough for each of your FPGA AI Suite IP instances, then the simplest (and most straightforward) path is to use only one pseudo channel per FPGA AI Suite IP instance and leave the second pseudo channel on the same layer and the other HBM channels dangling.

Benefits:

- Easiest implementation, no additional logic required except for data width considerations. For details about data width considerations, refer to Memory Data Width Considerations on page 336.

Limitations:

- Only 2 GB of memory is available for each FPGA AI Suite IP instance (instead of the maximum addressable of 4GB).
- Only can use of one HBM channel (or, more precisely, only one pseudo channel), which is half as wide (256 bit vs 512 bit) as the corresponding DDR4 channel in the current FPGA AI Suite IP implementation.

Channel Stitching

If the 2 GB size per channel does not meet your requirements for the total off-die memory per FPGA AI Suite IP instance, you must develop additional logic to point your FPGA AI Suite IP memory accesses toward the HBM memory controller (such an address decoding scheme).

An address decoding scheme can allow you to use two independent channels that each address 2 GB of HBM memory together to provide a 4 GB memory area for a single IP instance. In such a case, you are not making use of the parallel bandwidth of both channels but limiting the system to use only one or the other channel.

You could implement this scheme in Platform Designer by creating a system with the following features:

- An input pipeline stage that connects to the AXI4 output of the FPGA AI Suite IP instance with an address width that addresses the full 4 GB of memory.
- Two output pipeline stages, each with an address width that addresses 2 GB of memory each. The output stages are parameterized to use either the lower or upper 2 GB of the total 4 GB of memory. The outputs of each of these two output pipeline stages are connected to a separate channel of the HBM memory controller.

The following figure shows how a channel stitched memory scheme might look like:

Figure 52. Channel Stitching in Platform Designer



Benefits:

- Full 4 GB addressable memory space of FPGA AI Suite IP is available.

Limitations:

- Requires you to create additional address decoding logic to spread the 4 GB addressable memory space of the FPGA AI Suite IP to two 2 GB HBM channels.
- No bandwidth benefits compared to the simple approach in Direct Mapping of an FPGA AI Suite IP Instance to an HBM Channel on page 334.

Channel Stitching and Interleaving

Expanding on top of channel stitching, using the available double bandwidth requires additional interleaving logic in addition to the simple address decoding logic described earlier.

For an example of this interleaving logic, review the OpenCL Memory Bank Divider implementation described in *Intel FPGA SDK for OpenCL Pro Edition: Custom Platform Toolkit User Guide*. While the OpenCL implementation is based on an Avalon memory-mapped interface, the logic and techniques used can be applied the AXI4 interface using in the FPGA AI Suite IP.

Benefits:

- Full 4 GB addressable memory space of FPGA AI Suite IP is available.
- Ideally bandwidth is increased as two HBM channels are used by interleaving the traffic from the FPGA AI Suite IP instance to the two respective HBM layers.

Limitations:

- Requires you to create additional interleaving and address decoding logic to spread the 4 GB addressable memory space of the FPGA AI Suite IP to two 2 GB HBM channels and handle interleaving at memory page granularity level.

16.2.2. Memory Data Width Considerations

The other consideration is the 256-bit wide data path that is provided as an AXI4 bus by the HBM memory controllers. Two pseudo channels are exposed for each layer of the HBM memory, which are ideally used simultaneously to achieve highest possible bandwidth per HBM channel.

This 256-bit wide data path is only half of the 512-bit wide data path that the current FPGA AI Suite IP implementation expects and therefore is a limiting factor in terms of bandwidth.

From an implementation perspective, you have the following options to connect the 512-bit data width FPGA AI Suite IP instance to the 256-bit data width HBM pseudo channel AXI4 interface:

- Configure the FPGA AI Suite IP `.arch` file to use only a 256-bit wide data bus to external memory.

  This configuration does not require any additional adaptation logic and is the easiest to implement, but this configuration also decreases the bandwidth to external memory compared to the original 512-bit wide data bus.

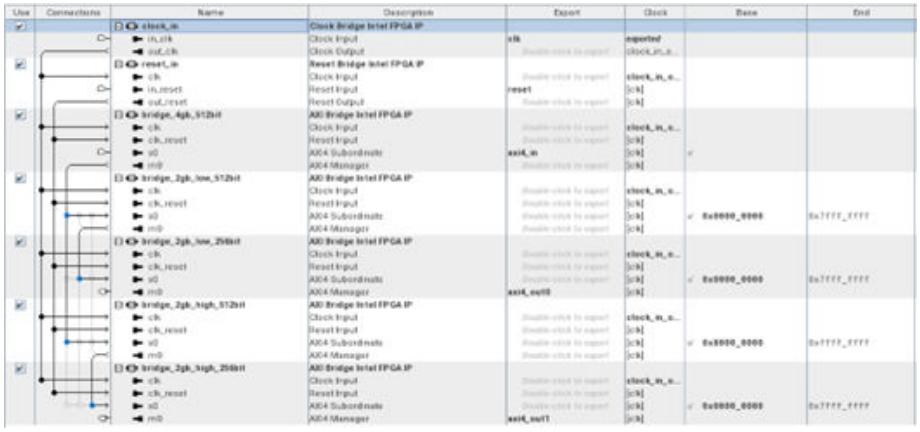
  To use this option, change the `ddr_data_bytes` value in the `.arch` files from 64 to 32.

- Use a width adapter and create two subsequent 256-bit accesses to the HBM memory controller for each 512-bit read/write by the FPGA AI Suite IP.

  This configuration decreases the read/write performance of the original 512-bit data path system to roughly half of the original performance.

  The following figure shows an example in Platform Designer of both channel stitching and width adaptation along with the automatically-generated arbitration and width-adapatation logic.

Figure 53. Channel Stitching and Width Adaptation 512-to-256 Bit



- Use a width adaptation and reordering buffer to divide the 512-bit read/write accesses of the FPGA AI Suite IP into 256-bit read/write accesses to both pseudo channels of the HBM memory controller channel.

  This configuration ensures that one single HBM layer is fully utilized but requires you to create extra logic to not only do the width stitching but also to reorder the read data coming back from both HBM pseudo channels.

  This configuration avoids the memory bandwidth loss of the other configurations.

16.2.3. Conclusion

Ideally, one or a combination of the earlier-mentioned implementation suggestions results in an optimal trade-off between bandwidth, additional logic, and development effort.

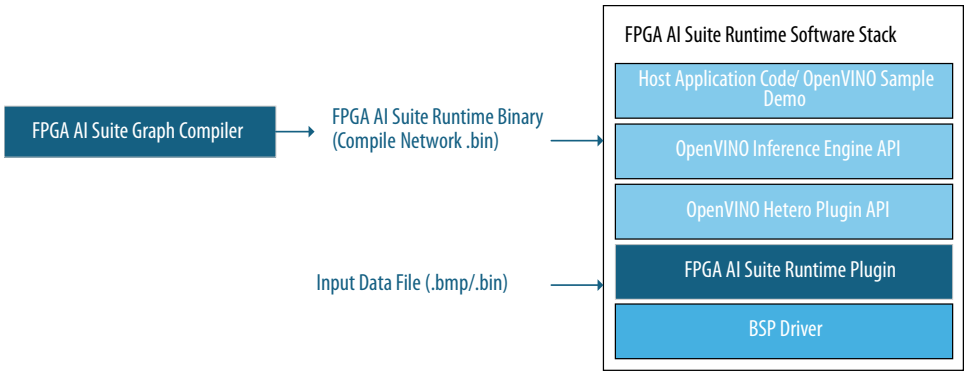
The best performance with highest logic overhead can be achieved with an interleaved channel stitching approach with an additional width adaptation logic that spreads out the two 256-bit memory accesses resulting from a 512-bit transfer to the two HBM pseudo channels on one layer of the HBM stack.

The lowest logic overhead is to use one pseudo channel of one layer in an HBM stack for each FPGA AI Suite instance and configure the FPGA AI Suite IP to use only a 256-bit wide data bus by modifying the underlying `.arch` file.

17. Developing Software Applications with the FPGA AI Suite

17.1. Understanding the FPGA AI Suite Runtime Software Stack

After familiarizing with the compile time software stack, you will recall that the ML graph of your choice as described in the .xml and .bin files (created by the Model Converter) is compiled for a specific FPGA AI Suite architecture file by using the FPGA AI Suite compiler. The output of the `dla_compiler` contains the FPGA AI Suite Runtime Binary (.bin file). This .bin file created by the compiler contains the compiled network parameters for all the target devices (FPGA, CPU, or both) along with the weights and biases. Since it uses the same .bin format as required by the OpenVINO Inference Engine, your inference application can import this file at runtime and load the compiled network onto your target devices for inference.

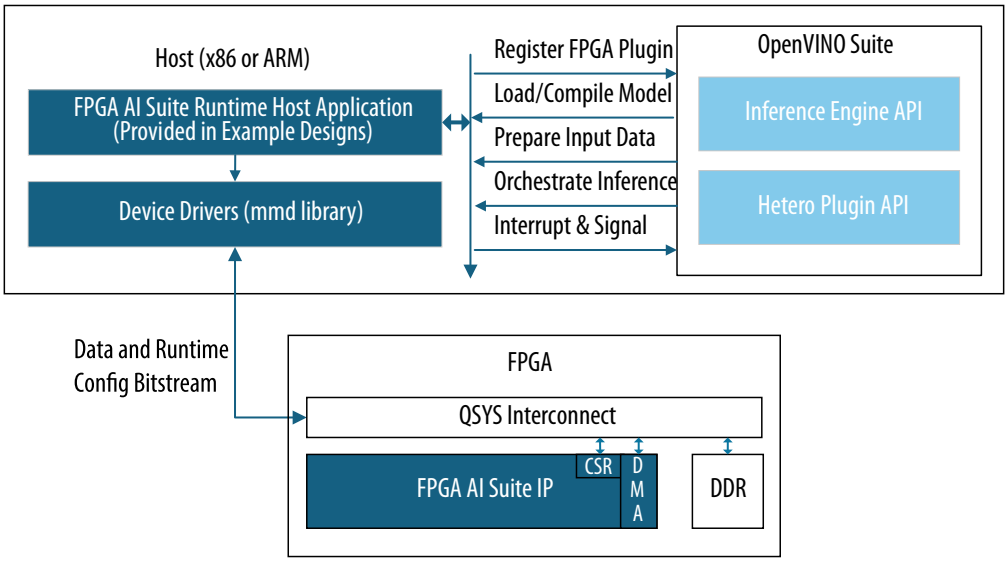


In the $COREDLA_WORK/runtime folder, you will find the build_runtime.sh script to build runtime for the system architecture of your choice. Review the following sections to decide on which system architecture suits your application requirements:

- Using FPGA AI Suite as a PCIe-Attach Platform on page 275
- Using FPGA AI Suite in Hostless DDR-Free Mode on page 283
- Using the FPGA AI Suite in Hostless JTAG Mode on page 294
- Using FPGA AI Suite as an Embedded Platform on page 297

The interaction between the OpenVINO FPGA Runtime Plugin, FPGA AI Suite Runtime Plugin, Host, and FPGA device can be captured in the following illustration.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.



For a detailed explanation on the board driver using Open FPGA Stack, refer to the Open FPGA Stack (OFS) Software Developer Journey Guide.

Related Information

- FPGA AI Suite Runtime on page 46
- Memory-Mapped Device (MMD) Driver on page 48
- Open FPGA Stack (OFS) Software Developer Journey Guide
- OpenVINO Plugin Developer Guide

17.2. Using the FPGA AI Suite Software Emulation

If you do not have an FPGA board installed and you want to evaluate the accuracy of your machine learning model against the architecture file for the FPGA AI Suite IP of your choice, you can still run inference on the software emulation model for fast software development iterations.

The software emulation model is a C++ software model of the FPGA AI Suite IP that is bit-accurate(*). The emulation models the numeric details of the IP (including the behavior of the block floating point arithmetic when enabled), relatively low-level transactions, and the processing delays of modules.

The emulation of the FPGA AI Suite IP is accessible through the OpenVINO plugin interface. This emulation models the numeric details of the IP, including the behavior of the block floating point arithmetic (when used).

The OpenVINO emulation plugin is enabled in the `$COREDLA_ROOT/reference/plugins_emulation.xml` plugins file. Because it uses the OpenVINO plugin architecture, it works with both the OpenVINO Python API and the C++ API. Because the emulation executes on the CPU and does not benefit from the FPGA acceleration, the emulation is much slower than inference on the FPGA. Typical inference times for a single image with ResNet50 are on the order of minutes of time. The inference speed varies dramatically depending on the architecture configuration and the graph.

To use the software emulation model:

1. Build the runtime with the following commands:

```
cd $COREDLA_WORK/runtime

rm -rf build_Release

./build_runtime.sh -target_emulation
```

2. Run inference with the `-niter=1` and `-nireq=1` options (because the software model is reasonably slow) with the following commands:

```
modeldir=$COREDLA_WORK/demo/models/public

imagedir=$COREDLA_WORK/demo/sample_images

curarch=$COREDLA_ROOT/example_architectures/AGX7_Performance.arch

gnd=$imagedir/TF_ground_truth.txt

cd $COREDLA_WORK/runtime/build_Release/dla_benchmark

./dla_benchmark \
   -b 1  `# Run only a single image` \
   -niter 1  `# Run only a single image` \
   -nireq 1  `# Running emulator: so -nireq=1` \
   -m $modeldir/resnet-50-tf/FP32/resnet-50-tf.xml \
    `# Same as when running on hardware - specify the graph` \
   -d HETERO:FPGA,CPU \
    `# Same as when running on hardware - \
    use FPGA if possible, fallback to CPU` \
   -i $imagedir  \
   `# Same as when running on hardware - specify image directory` \
   -arch_file $curarch \
   `# Same as when running on hardware - specify .arch file` \
   -dump_output \
   `# Dump output result.txt file` \
   -plugins emulation \
   `# Use the software emulator` \
   -groundtruth_loc $gnd
   `# Location of the ground truth file for $imagedir`
```

When the ML model is sensitive to a high precision input, you can choose to bootstrap the precision of the initial layers in the ML graph. Less commonly, some graphs may benefit from higher precision for intermediate or final layers.

You must manually modify the OpenVINO IR to use this feature.

Use Mixed Precision to Improve Accuracy

When the ML model is sensitive to a high precision input, you can bootstrap the precision of the initial layers in the ML graph. You must manually modify the OpenVINO IR to use the mixed precision feature.

17.3. Running Inference on the FPGA AI Suite IP Without the OpenVINO Runtime

Typically, the FPGA AI Suite runtime OpenVINO plugin provisions the control-and-status registers (CSR) and memory needed to run inference. If you want to run inference on the FPGA AI Suite IP without using the OpenVINO Inference Engine, you can use the ahead-of-time (AOT) splitter utility (`dla_aot_splitter`) to extract the raw data that is provisioned in memory.

An example system where you might not want to use the OpenVINO Inference Engine is when you have an embedded soft processor that manages a FPGA AI Suite IP instance in the FPGA fabric. The embedded system can provision DDR memory with the files generate by the AOT splitter and configure the IP to use the provisioned data.

Review the source code provided in `dla_aot_splitter_example` to see how to use the files that are generated by the splitter utility in an application that performs inference without using the OpenVINO Inference Engine.

17.3.1. Files Generated by the FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility

The FPGA AI Suite AOT splitter utility converts a model and its associated input or inputs compiled with the `dla_compiler` command into a set of files. The model must target the OpenVINO `HETERO:FPGA` plugin.

The AOT splitter utility generates the following files:

- `input.bin`

  This file is the layout transformed model input (or inputs) intended for direct consumption by the FPGA AI Suite IP.

  The splitter utility also provides `input.mem` as a plain-text representation of `input.bin`. The plain-text file is suitable to be interpreted as C/C++ constants.

- `config.bin`

  This file is the configuration word data stream that the FPGA AI Suite IP uses to execute the model.

  The splitter utility also provide `config.mem` as a plain-text representation of `config.bin`. The plain-text file is suitable to be interpreted as C/C++ constants.

- `filter.bin`

  This file contains the transformed weights of the model.

  The splitter utility also provide `filter.mem` as a plain-text representation of `filter.bin`. The plain-text file is suitable to be interpreted as C/C++ constants.

- `inter_size.mem`

  This file defines the intermediate buffer size. The intermediate buffer is a temporary area of memory that must be allocated but not initialized.

- `output_size.mem`

  This file defines the output buffer size. The output buffer is where the FPGA AI Suite IP writes the results of the output layers of the model.

- `arch_build.bin`

  This file contains the architecture file hash and build version.

  The splitter utility also provide `arch_build.mem` as a plain-text representation of `filter.bin`. The plain-text file is suitable to be interpreted as a C/C++ constant.

17.3.2. Building the FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility

The FPGA AI Suite AOT splitter utility is provided along with the example runtime component. The utility is built when you build the example runtime.

To build the AOT splitter utility:

1. Build the FPGA AI Suite example runtime with the following commands:

```
cd $COREDLA_WORK/runtime
rm -rf build_Release
./build_runtime.sh --target_de10_agilex
```

   When the build finishes, the AOT splitter utility command (`dla_aot_splitter`) is in the following directory:

```
$COREDLA_WORK/runtime/build_Release/dla_aot_splitter/
```

The `dla_aot_splitter` executable generated by building the example runtime remains the same regardless of any board- or device-specific options that you might specify. However, the files generated by the utility are specific to the FPGA device family and FPGA AI Suite architecture that you specify when compiling your inference model with the `dla_compiler` command.

17.3.3. Running the FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility

Before you run the FPGA AI Suite AOT splitter utility, you must have the following files ready:

- A model compiled by the `dla_compiler` command. The model must target the `HETERO:FPGA` format and the output format must be `open_vino_hetero`.

  For example, the following command compiles a model that targets a board with an Agilex 7 device to a file called `RN50_Performance_b1.bin`:

```
cd $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32
dla_compiler \
  --march $COREDLA_ROOT/example_architectures/AGX7_Performance.arch \
  --network-file ./resnet-50-tf.xml \
  --foutput-format=open_vino_hetero \
```

```
  --o $COREDLA_WORK/demo/RN50_Performance_b1.bin \
  --batch-size=1 \
  --fanalyze-performance
```

- The input or inputs for model inference.

The AOT splitter utility outputs the artifacts to the current working directory. The command line options must have a compiled model, model inputs, and a path to the splitter utility plugins XML file.

To run the AOT splitter utility:

1. Change directories to where you want the output files from the AOT splitter utility to go.
2. Run the splitter utility command as follows:

```
$COREDLA_WORK/runtime/build_Release/dla_aot_splitter/dla_aot_splitter \
  -cm <full_path_to_compiled_model_file> \
  -i < XXX-delimited list of inference input files> \
  -plugins <full_path_to_plugins_aot_splitter.xml> \
  [-bgr]
```

   The optional `-bgr` command option tells the AOT splitter to reverse the R, G, and B input channels for images before the channels are passed to the model.

   For example:

```
$COREDLA_WORK/runtime/build_Release/dla_aot_splitter/dla_aot_splitter \
  -cm $COREDLA_WORK/demo/RN50_Performance_b1.bin \
  -i $COREDLA_WORK/demo/sample_images/val_00000000.bmp \
  -plugins \
   runtime/dla_aot_splitter/dla_aot_splitter_plugin/plugins_aot_splitter.xml
  -bgr
```

Running the splitter generates the following files:

- `arch_build.mem` and `arch_build.bin`
- `config.mem` and `config.bin`
- `filter.mem` and `filter.bin`
- `input.mem` and `input.bin`
- `inter_size.mem`
- `output_size.mem`

  For a description of the files, refer to Files Generated by the FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility on page 342.

17.3.4. FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility Example Application

To learn more about how to use the files generated by the AOT splitter utility, review the `dla_aot_splitter_example` example application. This example application uses a PCIe interface to interact with either the Terasic DE10-Agilex Development Board.

Building the AOT Splitter Utility Example Application

You build the AOT splitter utility example application with the `build_runtime.sh` script with the `-aot_splitter_example` option.

The CMake build for the example runs the `dla_compiler` command on a model and the `dla_aot_splitter` command on the compiled model and model input.

The example application is built for a specific target board as follows:

- **Terasic DE10-Agilex Development Board**

```
export AOT_SPLITTER_EXAMPLE_MODEL=<path/to/model.xml>

export AOT_SPLITTER_EXAMPLE_INPUT=<path/to/image.bmp>

sh build_runtime.sh -aot_splitter_example -target_de10_agilex
```

Running the AOT Splitter Utility Example Application

Remember: The AOT splitter utility example application supports only the Terasic DE10-Agilex Development Board.

After you build the example application, you can review its source code to learn how to apply the files generated by the splitter to your system.

Run the example application with the following commands:

```
cd $COREDLA_WORK/runtime/build_Release/dla_aot_splitter/

./dla_aot_splitter_example/dla_aot_splitter_example
```

A. FPGA AI Suite Handbook Archives

For the latest and previous versions of this user guide, refer to FPGA AI Suite Handbook. If an FPGA AI Suite software version is not listed, the user guide for the previous software version applies.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

B. FPGA AI Suite Handbook Document Revision History

Document Version	FPGA AI SuiteVersion	Changes
2025.12.08	2025.3.1	• Revised and updated the description of the `--folding-option` FPGA AI Suite compiler option. • Added information about the `enable_layout_transform_writeback` FPGA AI Suite IP parameter. • Added information about the stream-to-external memory input mode of the FPGA AI Suite IP AIX streaming interface. • Increased the unlicensed IP inference limit.
2025.11.10	2025.3	• Initial release. This initial release consolidates content from the following publications: ☐ FPGA AI Suite Getting Started Guide ☐ FPGA AI Suite Compiler Reference Manual ☐ FPGA AI Suite IP Reference Manual ☐ FPGA AI Suite Design Example User Guide ☐ AN 1008: Using the FPGA AI Suite Docker Image ☐ AN 1020: Using the with High Bandwidth Memory on MX and M-Series Devices The revision histories of these guides is included here for completeness. • Added information about the FPGA AI Suite lightweight layout transform. • Added information about block floating-point (BFP). • Added information about mixed precision support for AI models. • Replaced occurrences of "OpenVINO Model Optimizer" with "OpenVINO Model Converter". • Removed Windows Subsystem for Linux 2 (WSL 2) content. If you need to use FPGA AI Suite on a Windows system use the Docker container or a native Windows installation.

B.1. FPGA AI Suite Getting Started Guide Document Revision History

Document Version	FPGA AI SuiteVersion	Changes
2025.04.21	2025.1	• Corrected typos in instructions for updating the CMake version on Ubuntu 20.04 LTS in "Operating System Prerequisites".
2025.04.14	2025.1	• Added `--layout nhwc` option to `mo` command in "Preparing a Model".
2025.04.11	2025.1	• Updated "Operating System Prerequisites" with instructions for updating the CMake version on Ubuntu 20.04 LTS.

continued...

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

Document Version	FPGA AI SuiteVersion	Changes
2025.04.07	2025.1	• Updated supported OpenVINO version to 2024.6 LTS. • Updated the FPGA AI Suite installation path to be under `/opt/altera`. The Intel Distribution of OpenVINO toolkit installation location remains under `/opt/intel`. • Removed "Running the Hostless Memoryless Design Example". This chapter is now part of the *FPGA AI Suite Design Examples User Guide*. • Removed *Installing Intel Threading Building Blocks (TBB)*. This step is no longer required.
2024.12.16	2024.3	• Updated "Installing Quartus Prime Pro Edition Software" to update the Quartus Prime Pro Edition version required by the various FPGA boards supported by the FPGA AI Suite design examples.
2024.11.25	2024.3	• Added description of the JTAG design example for Agilex 5 E-Series devices to "FPGA AI Suite Components".
2024.09.06	2024.2	• Replaced occurrences of "memoryless" with "DDR-free". • Replaced occurrences of "software reference model" with "software emulation model".
2024.07.31	2024.2	• Added "Running the Hostless Memoryless Design Example". • Added "Performing Inference Without an FPGA Board". • Updated supported OpenVINO version to 2023.3 LTS. • Revised "Setting Required Environment Variables". • Removed references to Intel PAC with Arria 10 GX FPGA. This card is no longer supported by FPGA AI Suite. • Removed references to Ubuntu 18. This operating system is no longer supported by FPGA AI Suite. • Replaced references to the `-plugins_xml_file` option with the `-plugins` option. The `-plugins_xml_file` option is deprecated and will be removed in a future release.
2024.03.29	2024.1	• Added support for Ubuntu 22.04.
2024.02.12	2023.3.1	• Updated for additional Agilex 7 support.
2024.02.02	2023.3	• Corrected OpenVINO version in graphic in "FPGA AI Suite Installation Overview".
2023.12.01	2023.3	• Added instructions for a WSL 2 environment to "Building an FPGA Bitstream for the PCIe Example Design".
2023.09.06	2023.2.1	• Updated supported OpenVINO version to 2022.3.1 LTS. • Updated installation instructions for Ubuntu 18 operating systems.
2023.07.17	2023.2	• Corrected instructions in "Installing OpenVINO Toolkit".
2023.07.03	2023.2	• Added "Installing FPGA AI Suite on Windows Subsystem for Linux". • Revised "OpenVINO Toolkit" for OpenVINO 2022.3 LTS. • Renamed "Installing the FPGA AI Suite" to "FPGA AI Suite Installation Overview" to better reflect the contents of that section. • Updated supported OpenVINO version to 2022.3 LTS. • Updated OpenVINO installation paths to `/opt/intel/openvino_2023`. • Updated FPGA AI Suite installation paths to `/opt/intel/fpga_ai_suite_2023.2`. • Changed occurrences of `tools/downloader/downloader.py` to `omz_downloader`. • Changed occurrences of `tools/downloader/converter.py` to `omz_converter`.

continued...

Document Version	FPGA AI SuiteVersion	Changes
2023.04.05	2023.1	• Added "Performing Inference on the i3d Graph". • Renamed the`dlac` command. The FPGA AI Suite compiler command is now `dla_compiler`. • Updated the Intel Agilex product family name to "Intel Agilex® 7."
2022.12.23	2022.2	• Update for the Debian package-based install. • Add walk-through for validating a YOLOv3 model. • Update to account for the new Terasic DE10-Agilex BSP ZIP file. • Renamed `dla_create_and_build_pcie_ed.py` to `dla_build_example_design.py`
2022.04.27	2022.1	• Minor fixes to some file names, including one for the running inference in the AOT flow on the FPGA.
2022.04.12	2022.1	• Notes on installing CMake for CentOS 7. • Clarifications to the Model Zoo and Model Optimizer install instructions.
2022.04.08	2022.1	• Extensively revised installation instructions. • Added installation information for the PCI-based design examples. • Enhanced details for Intel Agilex devices. • Added a quick-start tutorial. • Removed supplementary performance details.
2021.09.10	2021.2	• Added information for initial Intel Agilex device family support.
2021.04.30	2021.1	• Expanded list of tasks performed by the Intel FPGA AI Suite Compiler. • Distinguished more clearly between requirements for the IP and requirements for the Example Designs.
2020.12.04	2020.2	• Renamed the script that sets the Intel FPGA AI Suite environment variables to init_env.sh. It was called coredla_developer_init_env.sh in earlier releases.
2020.10.30	2020.1	• Initial release.

B.2. FPGA AI Suite Compiler Reference Manual Document Revision History

Document Version	FPGA AI SuiteVersion	Changes
2024.04.21	2025.1	• Updated supported OpenVINO version to 2024.6 LTS. • Updated `/opt/intel` paths to `opt/altera`.
2024.12.16	2024.3	• Various minor corrections and typo fixes.
2024.09.06	2024.3	• Updated descriptions for the `--fanalyze-area` compiler option to indicate that the option now generates a `.ptc` file to use with the FPGA Power and Thermal Analyzer to see power usage estimates for an architecture. • Replaced occurrences of “memoryless” with “DDR-free”.
2024.07.31	2024.2	• Revised “Compiler Use Modes”. • Removed “Estimating the Performance of a Partitions of a Graph”. This use mode is no longer supported. • Added “Generating Artifacts for Memoryless Operation”.

continued...

Document Version	FPGA AI SuiteVersion	Changes
		• Revised "Analyzer Tool Options (`dla_compiler`) Command Options" to remove the following command options: ☐ `--est-fps-single-subgraph` ☐ `--est-fps-network-index` ☐ `--est-fps-subgraph-index` • Added "Compiler Inputs and Outputs" chapter and moved the following topics into that chapter: ☐ "FPGA AI Suite Input File Formats" (formerly "Input File Formats for the `dla_compiler` Command"). ☐ "FPGA AI Suite Compiler Graph Export Formats" (formerly "Generation of Output File Formats". ☐ "The Partitioning Table Report". • Added "Model Analysis Reports".
2024.03.29	2024.1	• Added a description of the `--fmin-subgraph-layers` option to "Compilation Options (`dla_compiler` Command Options)". • Updated the description of the `--fanalyze-performance` option in "Analyzer Tool Options (`dla_compiler` Command Options)". • Updated the description of the performance estimator in "Estimating the Performance of a Graph" • Updated the example command in "Generating an Architecture for Highest Performance".
2023.12.01	2023.3	• Maintenance release. No major changes.
2023.09.06	2023.2.1	• Maintenance release. No major changes.
2023.07.03	2023.2	• Updated "Generating an Architecture for Highest Performance". • Updated "Architecture Optimization Options (dla_compiler Command Options)".
2023.04.05	2023.1	• Updated "Reporting (dla_compiler Command Options)". • Renamed the `dlac` command. The FPGA AI Suite compiler command is now `dla_compiler`. • Updated the Intel Agilex product family name to "Intel Agilex 7."
2022.12.23	2022.2	• Added the `--bin-data` option of the `dlac` command. • Removed references to the `--mfamily` option of the `dlac` command. This option is no longer supported. Use the `family` parameter in the architecture description file (`.arch`) instead.
2022.05.26	2022.1.1	• Added note that the architecture optimizer supports only interleaves 1x*N* and *N*x1.
2022.04.27	2022.1	• Removed some command options that were not implemented. • Added Agilex 7 device information where appropriate.
2021.09.10	2021.2	• Maintenance release.
2021.04.30	2021.1	• Improved information formatting and corrected the names of some command options.
2020.12.04	2020.2	• The following `dlac` command options have been renamed: ☐ `--plugins_file` is now `--plugins-file` ☐ `--network_file` is now `--network-file` ☐ `--network_weightings` is now `--network-weightings`
2020.10.30	2020.1	• Initial release.

B.3. FPGA AI Suite IP Reference Manual Document Revision History

<table>
<tr><th>Document Version</th><th>FPGA AI SuiteVersion</th><th>Changes</th></tr>
<tr><td>2025.04.21</td><td>2025.1</td><td>• Added support for Agilex 3 devices.
• Updated supported OpenVINO version to 2024.6 LTS.
• Updated <code>/opt/intel</code> paths to <code>opt/altera</code>.
• Updated “Model Performance” with Version 2025.1 values.
• Corrected some paths for Ahead-of-Time (AOT) Splitter Utility.
• Remove restrictions on <code>num_lanes</code> parameter.</td></tr>
<tr><td>2025.02.24</td><td>2024.3</td><td>• Fixed typos in “Running the FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility” and “FPGA AI Suite Ahead-of-Time (AOT) Splitter Utility Example Application”.</td></tr>
<tr><td>2024.12.16</td><td>2024.3</td><td>• For Parameter: arch_precision in “Parameter Group: Global Parameters”, corrected the number of operations per DSP for Agilex 7 devices with FP13AGX precision
• Fixed minor errors and typos.</td></tr>
<tr><td>2024.11.25</td><td>2024.3</td><td>• Updated “Model Performance” with Version 2024.3 values.
• Many of the <code>layout_transform_params</code> parameters have changed. In most cases, the old parameters represented an exact value to apply while the new parameters represent the maximum value to apply.
<table>
<tr><th>Old Parameter</th><th>New Parameter</th></tr>
<tr><td>channels</td><td>max_channels</td></tr>
<tr><td>feature_height</td><td>max_feature_height</td></tr>
<tr><td>feature_width</td><td>max_feature_width</td></tr>
<tr><td>feature_depth</td><td>max_feature_depth</td></tr>
<tr><td>stride_height</td><td>max_stride_height</td></tr>
<tr><td>stride_width</td><td>max_stride_width</td></tr>
<tr><td>stride_depth</td><td>max_stride_depth</td></tr>
<tr><td>pad_top</td><td>max_pad_top</td></tr>
<tr><td>pad_left</td><td>max_pad_left</td></tr>
<tr><td>pad_depth</td><td>max_pad_depth</td></tr>
</table>
• Added the following new <code>layout_transform_params</code> parameters:
☐ max_filter_width
☐ max_filter_height
☐ max_filter_depth
• Removed the following <code>layout_transform_params</code> parameters:
☐ output_channels
☐ output_height
☐ output_width
☐ output_depth</td></tr>
<tr><td>2024.09.06</td><td>2024.2</td><td>• Replaced occurrences of “memoryless” with “DDR-free”.
• Renamed “FPGA AI Suite Software Reference Model” to “Software Emulation of the FPGA AI Suite IP”.
• Replaced occurrences of “software reference model” with “software emulation model”.</td></tr>
</table>

continued...

Document Version	FPGA AI SuiteVersion	Changes
2024.07.31	2024.2	• Updated "Model Performance" with Version 2024.2 values • Added "FPGA AI Suite Software Reference Model" • Updated "FPGA AI Suite Layer/Primitive Ranges" as follows: ☐ Added Tanh ☐ Revised Max Pool window size range ☐ Revised pool max_windows_height/width valid range • Updated "FPGA AI Suite IP Block Configuration" as follows: ☐ Added `enable_parameter_rom` ☐ Added `enable_tanh` • Updated "Parameter Group: Global Parameters" as follows: ☐ Revised description of `arch_precision` parameter ☐ Added section *Parameters: `output_image_height_max`, `output_image_width_max`, `output_channels_max`* • Updated "Parameter Group: `activation`" to add *Parameter: `activation/enable_tanh`* • Added "Parameter Group: `input_stream_interface`" • Added "Parameter Group: `ouput_stream_interface`" • Added "Feature Input and Output Streaming" • Added "Input Streaming" • Added "Output Streaming" • Added "Memoryless Operation" • Updated"DMA Control Registers" as follows: ☐ Added `IP_reset` register ☐ Added `Activate_streaming` register
2024.03.29	2024.1	• Updated "Model Performance" with Version 2024.1 values • Rebranded from "Intel FPGA AI Suite" to "FPGA AI Suite". • Added updates to support new dedicated layout transform module. • Updated "FPGA AI Suite Layer / Primitive Ranges" as follows: ☐ Updated description of Softmax. ☐ Added Sigmoid. ☐ Added Swish. • Updated "FPGA AI Suite IP Block Configuration" as follows: ☐ Expanded the ranges of the `c_vector`, `pool k_vector`, and `activation k_vector` parameters. ☐ Added the `enable_sigmoid` parameter. ☐ Expanded the range of the `arch_precision` parameter. • Updated "Parameter Group: Global Parameters" with additional information about Agilex 5 device family support. • Added `activation/enable_sigmoid` to "Parameter Group: `activation`". • Added `softmax/max_num_channels` parameter description to "Module: `softmax`".
2023.12.01	2023.3	• Updated "Model Performance" with Version 2023.3 values • Updated `arch_precision` parameter description in "Parameter group: Global Parameters". • Added "Performance Registers". • Added "Debug Network Registers". • Added "PE scale precision" to "FPGA AI Suite IP Block Configuration". • Updated "Parameter Group: Global Parameters" for Agilex 5 support. • Updated "Parameter Group: `pe_array`" for Agilex 5 support.
2023.09.08	2023.2.1	• Added Multilayer Perceptrons (MLPs) to supported models.

continued...

Document Version	FPGA AI SuiteVersion	Changes
2023.07.03	2023.2	• Updated "Model Performance" with Version 2023.2 values and the enabling of non-default Quartus Prime options (register merging). • Added additional information about modifying the internal precision of a graph for performance improvement in "Architecture Description File Format for Instance Parameterization".
2023.04.05	2023.1	• Added ChannelToSpace, DepthToSpace, and PixelShuffle to "FPGA AI Suite Layer / Primitive Ranges". • Added enable_debug to "FPGA AI Suite IP Block Configuration and Interfaces". • Added description of enable_round_clamp activation parameter where needed. • Added "Input Transform Mapping". • Added output transform mapping information to "Output Tensor In-Memory Format". • Renamed the`dlac` command. The FPGA AI Suite compiler command is now `dla_compiler`. • Updated "Model Performance" with Version 2023.1 values. • Updated the Intel Agilex product family name to "Intel Agilex 7."
2023.02.03	2022.2	• Correct the description of the `-bgr` option of the AOT splitter utility.
2022.12.23	2022.2	• Added SqueezeNet to the list of supported models. • Added the `family` architecture description file global parameter. • Removed the `--family` IP generation utility command option.
2022.04.27	2022.1	• Removed references to HLS generation. • Updated descriptions of parameters in the `.arch` file. • Added a Model Performance section.
2021.09.10	2021.2	• Added information for Intel Agilex device support. • Added information for MobileNet v3 support.
2021.04.30	2021.1	• Various corrections and updates.
2020.12.04	2020.2	• Renamed *Intel FPGA AI Suite IP Core* to *Intel FPGA AI Suite IP*".
2020.10.30	2020.1	• Initial release.

B.4. FPGA AI Suite Example Designs User Guide Revision History

Document Version	FPGA AI Suite Version	Changes
2025.04.30	2025.1	• Updated and revised some sections under *Design Example Software Components*: ☐ Added *OpenVINO FPGA Runtime Overview* ☐ Added *FPGA AI Suite Custom Platform* ☐ Added *FPGA AI Suite Runtime MMD API* ☐ Removed *Software Interface to the BSP* ☐ Added *Board Support Package (BSP) Overview* ☐ Added *Terasic DE10-Agilex Development Board BSP Example* ☐ Added *Agilex 7 PCIe Attach OFS-based BSP Example*
2025.04.23	2025.1	• Made various updates to support the changes to the design example utility (`dla_build_example_design.py`). The following new topics were added: ☐ *FPGA AI Suite Design Example Utility* ☐ *The `dla_build_example_design.py` Command* ☐ *Listing Available FPGA AI SuiteDesign Examples* ☐ *Building FPGA AI Suite Design Examples* ☐ *Staging FPGA AI Suite Design Example Builds* ☐ *WSL 2 FPGA AI Suite Design Example Builds* The following topics were removed: ☐ *[PCIE] Build Script* ☐ *[PCIE] Build Script Options* ☐ *[PCIE] Script Flow* The following topics were changed: ☐ *[OFS-PCIE} Compiling the OFS for PCIe Attach Design Example* ☐ *[HL-NO-DDR] Running the Hostless DDR-Free Design Example* ☐ *[HL-JTAG] Building an FPGA Bitstream for the JTAG Design Examples* ☐ *[SOC} Quartus Prime Build Flow* ☐ *[SOC] Build Script Options* • Added information for the Agilex 5 SoC design example, including the following new topics: ☐ *Preparing the Agilex 5 FPGA E-Series 065B Modular Development Kit* ☐ *Confirming the Agilex 5 FPGA E-Series 065B Modular Development Kit Board Setup* ☐ *Programming the Agilex 5 FPGA Device with the JTAG Indirect Configuration (`.jic`) File* ☐ *Programming the Agilex 5 FPGA Device with the SRAM Object File (`.sof`)* ☐ *Connecting the Agilex 5 FPGA E-Series 065B Modular Development Kit to the Host Development System* • Revised some steps for the Agilex 7 SoC design example. • Updated some Quartus Prime System Console script commands. • Added *Quartus Prime System Console Script Options*

continued...

Document Version	FPGA AI Suite Version	Changes
		• Removed *[HL-NO-DDR] Quartus Prime System Console Performance Script* • Added *[HL-NO-DDR] Measuring Performance* • Design example support for the Agilex 5 FPGA E-Series 065B Premium Development Kit has been replaced with support for the Agilex 5 FPGA E-Series 065B Modular Development Kit.
2025.04.22	2024.3	• Fixed a typo in the command found in *Building an FPGA Bitstream for the JTAG Design Examples*.
2025.03.28	2024.3	Initial release. This initial release merges the content from the following publications: • FPGA AI Suite PCIe-based Design Example User Guide • FPGA AI Suite SoC Design Example User Guide The revision histories of these guides is included here for completeness. This initial release adds information about the following design examples: • JTAG Design Example • DDR-Free Hostless Design Example • OFS for PCIe Attach Design Example

B.4.1. FPGA AI Suite PCIe-based Design Example User Guide Document Revision History

Document Version	FPGA AI SuiteVersion	Changes
2024.12.16	2024.3	• Various minor corrections and typo fixes.
2024.11.25	2024.3	• References to "software reference" have been replaced with "software emulator" or "software emulation.". As part of this change, the `--target_reference` option of the `build.runtime.sh` script has been renamed to `--target_emulation`.
2024.07.31	2024.2	• Added "The `dla_benchmark` Performance Metrics". • Added "Interpreting System Throughput and Latency Metrics". • Revised "Additional `dla_benchmark` Options". • Removed references to the Intel PAC with Arria 10 GX FPGA. The PCIe-based design example no longer supports the Intel PAC with Arria 10 GX FPGA. • Replaced references to the `-plugins_xml_file` option with the `-plugins` option. The `-plugins_xml_file` option is deprecated and will be removed in a future release. • Revised "Script Flow". • Revised "Hardware".
2024.03.29	2024.1	• Added new options to "Additional `dla_benchmark` Options". • Added `segmentation_demo` to "Running the Ported OpenVINO Demonstration Applications".
2023.12.01	2023.3	• Added `--wsl` and `--finalize` options to "Build Script Options".
2023.09.06	2023.2.1	• Updated supported OpenVINO version to 2022.3.1 LTS.
2023.07.03	2023.2	• Updated "Software Components". • Renamed "OPAE Driver" to "BSP Driver" and revised the content in the topic. • Updated supported OpenVINO version to 2022.3 LTS.

continued...

Document Version	FPGA AI SuiteVersion	Changes
		• Updated OpenVINO installation paths to `/opt/intel/openvino_2023`. • Updated FPGA AI Suite installation paths to `/opt/intel/fpga_ai_suite_2023.2`. • Changed occurrences of `tools/downloader/downloader.py` to `omz_downloader`. • Changed occurrences of `tools/downloader/converter.py` to `omz_converter`.
2023.04.05	2023.1	• Renamed the `dlac` command. The FPGA AI Suite compiler command is now `dla_compiler`. • Updated the Intel Agilex product family name to "Intel Agilex 7."
2022.12.23	2022.2	• Removed the `-f` build script (`dla_create_and_build_pcie_ed.py`) option. • Renamed the `benchmark_app` to `dla_benchmark`. • Renamed `dla_create_and_build_pcie_ed.py` to `dla_build_example_design.py`
2022.05.26	2022.1.1	• Updated description of how to run the OpenVINO demos.
2022.04.27	2022.1	• Additional updates for Intel Agilex device support. • Updated the locations of graphs from the Getting Started tutorial.
2021.09.10	2021.2	• Added updates for initial Intel Agilex device support.
2021.04.30	2021.1	• Added additional demonstration programs. • Various corrections and updates.
2020.12.04	2020.2	• Initial release.

B.4.2. FPGA AI Suite SoC Design Example User Guide Document Revision History

Document Version	FPGA AI SuiteVersion	Changes
2024.12.16	2024.3	• Added "Yocto Recipe: `recipes-support/devmem2/devmem2_2.0.bb`". • Revised "Yocto Recipe: `recipes-kernel/linux/linux-socfpga-lts_%.bbappend`" (formerly "octo Recipe: `recipes-kernel/linux/linux-socfpga-lts_5.15.bbappend`")
2024.11.25	2024.3	• Updated required Quartus Prime Pro Edition version to Version 24.3. • Updated "Building the Bootable SD Card Image (`.wic`)" to Yocto project version 5.0.5.
2024.07.31	2024.2	• Revised "Agilex 7 FPGA I-Series Transceiver-SoC Development Kit Hardware Requirements". • Updated required Quartus Prime Pro Edition version to Version 24.2.
2024.03.29	2024.1	• Updated the document for Ubuntu 22.04 support. • Updated required Quartus Prime Pro Edition version to Version 23.4.

continued...

Document Version	FPGA AI SuiteVersion	Changes
2024.02.12	2023.3.1	• Updated the document for Agilex 7 FPGA I-Series Transceiver-SoC Development Kit support, including the following new topics: ☐ "Preparing the Intel Agilex 7 FPGA I-Series Transceiver-SoC Development Kit " ☐ "Confirming Intel Agilex 7 FPGA I-Series Transceiver-SoC Development Kit Board Set Up" ☐ "Programming the Intel Agilex 7FPGA Device with the JTAG Indirect Configuration (.jic) File" ☐ "Connecting the Intel Agilex 7 FPGA I-Series Transceiver-SoC Development Kit to the Host Development System"
2023.12.01	2023.3	• Updated required Quartus Prime Pro Edition version to Version 23.3.
2023.09.06	2023.2.1	• Updated supported OpenVINO version to 2022.3.1 LTS.
2023.07.03	2023.2	• Updated supported OpenVINO version to 2022.3 LTS. • Updated OpenVINO installation paths to `/opt/intel/openvino_2023`. • Updated FPGA AI Suite installation paths to `/opt/intel/fpga_ai_suite_2023.2`. • Changed occurrences of `tools/downloader/downloader.py` to `omz_downloader`. • Changed occurrences of `tools/downloader/converter.py` to `omz_converter`.
2023.04.05	2023.1	• Initial release.

B.5. AN 1008: Using the FPGA AI Suite Docker Image Document Revision History

Document Version	FPGA AI Suite Version	Changes
2025.04.25	2025.1	• Updated Docker repository from the Intel repository to the Altera repository. • Added support for Agilex 5 FPGA E-Series 065B Modular Development Kit. • Updated supported OpenVINO and Quartus Prime Pro Edition versions to those supported by FPGA AI Suite Version 2025.1.
2024.12.16	2024.3	• Corrected typos and other minor errors in "Containerized FPGA AI Suite SoC Design Example Quick-Start Tutorial".
2024.08.15	2024.2	• Corrected a command for programming the Agilex 7 FPGA I-Series Transceiver-SoC Development Kit. • Add link for downloading Win32 Disk Imager
2024.08.08	2024.2	• Updated supported OpenVINO and Quartus Prime Pro Edition versions to those supported by FPGA AI Suite Version 2024.2
2024.06.24	2024.1	• Updated "Containerized FPGA AI Suite SoC Design Example Quick-Start Tutorial".
2024.05.08	2024.1	Initial release.

B.6. AN 1020: Using the FPGA AI Suite IP with High Bandwidth Memory on Stratix 10 MX and Agilex 7 M-Series Devices Document Revision History

Document Version	Changes
2025.05.16	Initial release.